



RenkerVault — Das Handbuch

Technik, Sicherheit und Recht

Eine vollständige Erklärung: wie RenkerVault funktioniert, warum es sicher ist, und in welchem rechtlichen Umfeld ("Chat-Kontrolle", Datenschutzrecht) es entstanden ist.

RENKER INDUSTRIES — SECURE COMMS DIVISION

Stand: Prototyp v0.1. Der Rechtsteil beschreibt den Sach- und Rechtsstand nach bestem Wissen zum Zeitpunkt der Erstellung und ersetzt keine Rechtsberatung (siehe Disclaimer, Teil IV, Kapitel 1).

Inhaltsverzeichnis

Teil I — Einführung & Hintergrund

- 1 Vorwort: Warum dieses Buch
- 2 Was ist RenkerVault?
- 3 Die Bedrohungslage: Massenüberwachung und „Chat-Kontrolle“
- 4 Eine kurze Geschichte der Verschlüsselungsdebatte (Crypto Wars)
- 5 Kryptographische Grundlagen für Laien

Teil II — Architektur & Funktionen

- 1 Gesamtarchitektur im Überblick
- 2 Identität, Konten, Geräte
- 3 Nachrichtentypen und Funktionen im Detail
- 4 Gruppen und Kanäle
- 5 Das Einbruchsalarm-System
- 6 Lokale Datenhaltung (At-Rest-Verschlüsselung)
- 7 Multi-Plattform: Web, Desktop, Mobile
- 12a. Barrierefreiheit

Teil III — Kryptographie im Detail: Warum es sicher ist

- 1 Symmetrische und asymmetrische Verschlüsselung
- 2 Elliptische Kurven: X25519 und Ed25519
- 3 Initialer Schlüsselaustausch: X3DH-lite
- 4 Der Double Ratchet Schritt für Schritt
- 5 Gruppenschlüssel: Sender-Keys-lite
- 6 Post-Quantum-Kryptographie und der Hybrid-Handshake
- 7 Warum kein One-Time-Pad, warum kein „Quanten-Zufallsgenerator“
- 8 Authentifizierung: Ed25519-Challenge-Response
- 9 Das Zero-Knowledge-Relay
- 10 Metadatenschutz und Tor Hidden Service
- 11 Threat Model: Wovor RenkerVault schützt — und wovor nicht
- 12 Vergleich mit anderen Messengern
- 13 Bekannte Grenzen und Selbstkritik

Teil IV — Rechtlicher Hintergrund

- 1 Disclaimer und Einordnung
- 2 Grundrechte: Artikel 7 und 8 der EU-Grundrechtecharta
- 3 Geschichte der „Chat-Kontrolle“
- 4 Der Verordnungsentwurf im Detail
- 5 Client-Side-Scanning: Technik und Kritik
- 6 Kritik aus Wissenschaft und Zivilgesellschaft
- 7 Verhandlungsstand und Ausblick
- 8 DSGVO und Ende-zu-Ende-verschlüsselte Messenger
- 9 Die ePrivacy-Richtlinie und ihre Ausnahmeregelung
- 10 Was RenkerVault rechtlich ist — und was nicht
- 11 Politische Handlungsmöglichkeiten für Nutzer

Teil V — Praxis & Anhang 36a. Feature-Matrix nach Plattform

- 1 Installation und Betrieb Schritt für Schritt
- 2 Hosting-Optionen im Vergleich
- 3 Sicherheitschecklisten für Nutzer
- 4 Glossar
- 5 Häufig gestellte Fragen
- 6 Zeitleiste: Von den Crypto Wars zur Chat-Kontrolle
- 7 Quellen und weiterführende Literatur
- 8 Anhang: Kommentierter Quellcode der Kernmodule

44a. Fragen zum Selbststudium 44b. Tabellarischer Überblick aller kryptographischen Parameter

- 1 Abkürzungsverzeichnis

45a. Deutsch-Englisches Fachbegriff-Verzeichnis

- 1 Stichwortverzeichnis

Kurzzusammenfassung der fünf Teile

Für eilige Leserinnen und Leser oder als Gedächtnisstütze vor der Lektüre: eine Zusammenfassung jedes Teils in wenigen Sätzen.

Teil I — Einführung & Hintergrund: RenkerVault entstand als Antwort auf die Sorge vor anlassloser Chat-Überwachung („Chat-Kontrolle“). Das Kapitel erklärt die Bedrohungslage mit echten Zahlen, ordnet sie in die jahrzehntealte Geschichte der „Crypto Wars“ ein und vermittelt die kryptographischen Grundbegriffe (öffentliche/private Schlüssel, symmetrische/asymmetrische Verschlüsselung, Perfect Forward Secrecy), die der Rest des Buches voraussetzt.

Teil II — Architektur & Funktionen: Beschreibt, was die App tatsächlich kann: Konten ohne Klarname, Mehrgeräte-Unterstützung mit Vertrauensmodell, alle Nachrichtenfunktionen (inklusive der bewusst abweichenden Entscheidung gegen automatische Link-Vorschauen), Gruppen und Kanäle, das eingebaute Einbruchsalarm-System, lokale Verschlüsselung und die Mehrfachverwertung derselben Codebasis für Web, Desktop und Mobile.

Teil III — Kryptographie im Detail: Das technische Herzstück. Erklärt Schritt für Schritt, warum und wie RenkerVault sicher ist: elliptische Kurven, der initiale Schlüsselaustausch (X3DH-lite), der Double Ratchet, Gruppenschlüssel, der Post-Quantum-Hybrid-Handshake, ein explizites Threat Model und ein ehrlicher Vergleich mit Signal, Threema, WhatsApp, Telegram und weiteren Messengern — inklusive aller bekannten Grenzen.

Teil IV — Rechtlicher Hintergrund: Ordnet die Chat-Kontrolle-Debatte rechtlich ein: EU-Grundrechte, die Geschichte des Verordnungsvorschlags, die technische Kritik am Client-Side-Scanning, DSGVO und ePrivacy, eine faire Darstellung aller Perspektiven der Debatte, und eine ausdrückliche Einordnung dessen, was RenkerVault rechtlich ist — und was nicht. Mit Disclaimer: keine Rechtsberatung.

Teil V — Praxis & Anhang: Schritt-für-Schritt-Installation, Hosting-Optionen im Vergleich, Sicherheitschecklisten für unterschiedliche Nutzergruppen, ein umfangreiches Glossar, FAQ, eine Zeitleiste, Quellenverzeichnis und — als Vertrauensanker — der tatsächliche, kommentierte Quellcode der kryptographischen Kernmodule.

TEIL I

Einführung & Hintergrund

Für alle verständlich — kein Vorwissen nötig

1. Vorwort: Warum dieses Buch

Dieses Buch beantwortet drei Fragen, die normalerweise in drei getrennten Welten leben: Was macht diese App eigentlich genau? Warum sollte man ihr technisch vertrauen? Und in welchem rechtlichen Umfeld ist sie entstanden — welches Gesetz war der Anlass, und was würde sich ändern, wenn dieses Gesetz käme?

Die meisten Erklärungen zu Verschlüsselungs-Apps trennen diese drei Fragen strikt. Ein technisches Whitepaper erklärt das Protokoll, aber nicht, warum es überhaupt jemanden interessieren sollte. Ein Nachrichtenartikel über „Chat-Kontrolle“ erklärt die Politik, aber nicht, wie eine konkrete Alternative technisch aussehen könnte. Und eine Produktbeschreibung verkauft ein Versprechen, ohne die Mathematik dahinter offenzulegen oder ihre Grenzen zu benennen.

RenkerVault ist ein Prototyp — kein fertiges, extern auditiertes Produkt. Genau deshalb versucht dieses Buch etwas Ungewöhnliches: die drei Ebenen zusammen zu erklären, ohne eine davon zu beschönigen. Wer nur technisch interessiert ist, findet in Teil III jede kryptographische Entscheidung mit Begründung. Wer nur den rechtlichen Hintergrund verstehen will, findet ihn in Teil IV, mit ausdrücklichem Hinweis darauf, dass dies keine Rechtsberatung ist und der Gesetzgebungsprozess sich weiterentwickelt. Wer die App einfach nur benutzen will, findet in Teil II und Teil V alles Nötige.

Ein Wort zur Ehrlichkeit, das sich durch das ganze Buch zieht: Wo RenkerVault etwas nicht kann, steht das explizit da — nicht als Fußnote, sondern als eigener Abschnitt. Ein Sicherheitswerkzeug, das seine eigenen Grenzen verschweigt, ist gefährlicher als eines, das sie offenlegt, weil Nutzer im ersten Fall falsche Annahmen über ihren Schutz treffen. Dieses Prinzip — lieber eine unbequeme Wahrheit als eine bequeme Übertreibung — ist keine Marketingfloskel, sondern zieht sich durch jede einzelne Designentscheidung, die in diesem Buch beschrieben wird.

„Sicherheit durch Architektur, nicht durch Vertrauen in eine Stelle, die jederzeit ausgeweitet oder missbraucht werden könnte.“ Dieser Satz beschreibt in einem Nebensatz das gesamte Motiv hinter RenkerVault — und wird in Teil I, Kapitel 3 ausführlich hergeleitet.

1.1 Wie dieses Buch gelesen werden kann

Die fünf Teile bauen zwar aufeinander auf, sind aber so geschrieben, dass sie auch einzeln und in beliebiger Reihenfolge gelesen werden können — je nach Interesse:

- **Wer nur verstehen möchte, ob RenkerVault vertrauenswürdig ist**, ohne selbst Programmiererfahrung zu haben: Teil I vollständig, dann Kapitel 23 (Threat Model) und Kapitel 25 (Grenzen) aus Teil III.
- **Wer die App tatsächlich installieren und nutzen möchte**: Teil II als Überblick, dann direkt zu Teil V, Kapitel 37 (Installation).
- **Wer sich für die Kryptographie im Detail interessiert** (etwa im Rahmen eines Informatikstudiums): Teil III vollständig, ergänzt um den Code-Anhang in Kapitel 44 und die Selbststudium-Fragen in Kapitel 44a.
- **Wer sich vor allem für den rechtlichen und politischen Hintergrund interessiert**: Teil I, Kapitel 3-4 als Einstieg, dann Teil IV vollständig.
- **Wer das Buch als Nachschlagewerk statt als durchgehende Lektüre nutzen möchte**: Glossar (Kapitel 40), Stichwortverzeichnis (Kapitel 46) und die Rückverfolgbarkeitstabellen (Kapitel 24.6e, 36.2b) bieten direkten Zugriff auf einzelne Themen, ohne das ganze Buch lesen zu müssen.

Querverweise zwischen Kapiteln (durchgehend in der Form „Kapitel X.Y“) sind bewusst häufig gesetzt, damit unabhängig vom gewählten Einstiegspunkt der jeweilige Kontext auffindbar bleibt.

2. Was ist RenkerVault?

RenkerVault ist eine Chat-Anwendung: Text, Bilder, Sprachnachrichten, Gruppen, Broadcast-Kanäle — auf den ersten Blick vergleichbar mit Signal, Threema, Telegram oder WhatsApp. Der entscheidende Unterschied liegt nicht in der Oberfläche, sondern in zwei Eigenschaften, die zusammen das Fundament der App bilden:

Erstens: **Zero-Knowledge-Architektur**. Der Server, der Nachrichten zwischen den Geräten weiterleitet (der „Relay“), sieht zu keinem Zeitpunkt Klartext. Er kennt Konto-Kennungen, öffentliche Schlüssel, Geräte-Metadaten und undurchsichtige, bereits verschlüsselte Datenpakete — aber niemals den Inhalt einer Nachricht. Das ist keine Richtlinie, die der Betreiber einhalten könnte oder auch nicht; es ist eine Eigenschaft der Software selbst. Selbst wenn der Serverbetreiber gezwungen würde, jede gespeicherte Information herauszugeben, enthielte diese Information keine lesbaren Nachrichten.

Zweitens: **ein eingebautes Einbruchsalarm-System**. Die meisten Chat-Apps behandeln Verschlüsselung und Einbruchserkennung als getrennte Themen — Verschlüsselung schützt die Übertragung, ein Passwort schützt das Gerät, und das war's. RenkerVault verbindet beides: Fehlgeschlagene Entsperrversuche lösen einen sichtbaren Alarm aus, neue Geräte müssen manuell bestätigt werden, die lokale

Datenbank wird durch eine Prüfsumme vor Manipulation geschützt, und eine Duress-PIN öffnet im Zwangsfall eine leere Fake-Ansicht statt der echten Daten.

Diese Kombination — Verschlüsselung, die niemand umgehen kann, plus ein System, das aktiv meldet, wenn jemand es versucht — ist der rote Faden, der sich durch die gesamte technische Architektur zieht, die in Teil II und III dieses Buches beschrieben wird.

2.1 Für wen ist RenkerVault gedacht?

RenkerVault richtet sich nicht an eine bestimmte Berufsgruppe oder ein bestimmtes Bedrohungsszenario, sondern an jeden, der aus einem von drei Gründen mehr Kontrolle über die eigene Kommunikation haben möchte:

- **Grundsätzliches Bedürfnis nach Privatsphäre.** Nicht jede private Unterhaltung muss geheim sein, damit sie geschützt gehört — genauso, wie nicht jeder Brief in einem durchsichtigen Umschlag verschickt werden möchte, nur weil sein Inhalt harmlos ist.
- **Sorge vor anlassloser Massenüberwachung.** Wer verstehen möchte, warum diese Sorge keine Verschwörungstheorie, sondern eine durch offizielle EU-Zahlen belegte Realität ist, findet die Belege in Kapitel 3.
- **Technisches Interesse an einer nachvollziehbaren, offen dokumentierten Sicherheitsarchitektur** — RenkerVault verwendet ausschließlich öffentlich auditierte kryptographische Bausteine (siehe Teil III) und erfindet keine eigenen Verschlüsselungsverfahren.

2.1a Unverbindlich ausprobieren: der Demo-Modus

Wer RenkerVault kennenlernen möchte, ohne bereits eine zweite Person zum Testen zu benötigen, findet nach der Erstellung eines Tresors einen integrierten Demo-Modus vor: mehrere vorkonfigurierte, simulierte Gesprächspartner (mit Namen wie „Nadja“ oder „Werkstatt Nord“), die alle innerhalb desselben Browser-Prozesses laufen. Wichtig für das Verständnis: Diese Demo-Kontakte sind **keine** oberflächliche Attrappe mit vorgefertigten Textbausteinen — im Hintergrund läuft für jeden Demo-Kontakt derselbe echte X3DH-lite-Handshake (Kapitel 15) und derselbe echte Double Ratchet (Kapitel 16) wie bei einem echten, über den Relay verbundenen Kontakt. Der einzige Unterschied: Beide Seiten der Unterhaltung laufen im selben Prozess, statt über eine echte Netzwerkverbindung zwischen zwei unabhängigen Geräten verteilt zu sein. Dieser Demo-Modus eignet sich damit hervorragend, um die in Teil III beschriebene Kryptographie live zu beobachten (etwa über die Sicherheitszentrale, die Safety Numbers und Fingerprints auch für Demo-Kontakte anzeigt), ohne dass dafür bereits ein zweites Gerät oder eine zweite Person nötig wäre. Für echte, über den Relay verbundene Kontakte siehe Kapitel 7.4 sowie die Schritt-für-Schritt-Anleitung in Kapitel 37.7.

2.2 Was RenkerVault ausdrücklich nicht ist

Ebenso wichtig wie die Frage, was RenkerVault ist, ist die Klärung, was es nicht sein will:

- Es ist **kein** Werkzeug, das speziell entwickelt wurde, um Strafverfolgung zu behindern. Es verwendet dieselbe Klasse von Technologie wie Signal, Threema oder Session — Werkzeuge, die von Journalisten, Anwälten, Ärzten, Aktivisten und Millionen gewöhnlicher Menschen weltweit genutzt werden, ohne dass daraus automatisch eine kriminelle Absicht folgt.
- Es ist **kein** fertiges, für den produktiven Masseneinsatz freigegebenes Produkt. Es ist ein Prototyp (Version 0.1), dessen Protokoll-Komposition kryptographisch begründet, aber nicht extern auditiert ist (siehe Kapitel 25).
- Es ist **kein** Ersatz für ein umfassendes persönliches Sicherheitskonzept. Verschlüsselung schützt die Übertragung und die Speicherung von Daten — sie schützt nicht vor einem bereits mit Schadsoftware infizierten Gerät, vor Social Engineering oder vor einem Gegenüber, das Screenshots weiterleitet.
- Es ist **kein** Dienst, der personenbezogene Daten zu Werbe- oder Analysezwecken sammelt oder monetarisiert. Es gibt kein Geschäftsmodell, das auf der Auswertung von Nutzerverhalten beruht — ein struktureller Unterschied zu werbefinanzierten Diensten, bei denen ein wirtschaftlicher Anreiz besteht, möglichst viele Daten über Nutzerverhalten zu sammeln.
- Es ist **kein** rechtsfreier Raum. Wie in Teil IV ausführlich dargestellt, operiert RenkerVault innerhalb des geltenden Rechtsrahmens (Kapitel 35) — Ende-zu-Ende-Verschlüsselung ist in der EU nach aktuellem Kenntnisstand nicht verboten oder genehmigungspflichtig (anders als in einigen anderen, in Kapitel 32.3a/32.3b beschriebenen Rechtsordnungen weltweit).

2.2a Was RenkerVault (noch) nicht ist, aber werden könnte

Zur Abrundung der in 2.2 getroffenen Abgrenzungen: RenkerVault ist aktuell **kein** Ersatz für ein vollständiges, unternehmensweites Compliance-System (etwa für regulierte Branchen mit gesetzlichen Aufbewahrungspflichten für Geschäftskommunikation) und **kein** Dienst mit einer im großen Stil geprüften, mehrjährigen Betriebshistorie wie die in Kapitel 24 verglichenen etablierten Messenger. Diese Einschränkungen sind keine grundsätzlichen, unveränderlichen Eigenschaften, sondern spiegeln den in Kapitel 25 beschriebenen aktuellen Prototyp-Status wider — die in Kapitel 24.6c konsolidierte Roadmap zeigt mehrere Wege, wie sich einzelne dieser Einschränkungen in Zukunft adressieren ließen, ohne dass dies bereits als Versprechen zu verstehen wäre.

3. Die Bedrohungslage: Massenüberwachung und „Chat-Kontrolle“

Um zu verstehen, warum RenkerVault in der vorliegenden Form gebaut wurde, muss man zunächst verstehen, wovor es schützen soll. Dieses Kapitel beschreibt die Bedrohungslage aus Sicht eines durchschnittlichen Nutzers, ohne juristische Tiefe — die vollständige rechtliche Einordnung folgt in Teil IV.

3.1 Was heute schon passiert: freiwilliges Scannen

Ein Bericht der Europäischen Kommission aus dem Jahr 2025 über die Umsetzung einer bestehenden, befristeten EU-Ausnahmeregelung (die es großen Anbietern erlaubt, freiwillig nach mutmaßlich illegalen Inhalten zu suchen) enthält Zahlen, die die Größenordnung des bereits heute stattfindenden automatisierten Scannens verdeutlichen: Microsoft verarbeitete demnach 2024 weltweit rund 9,6 Milliarden Inhalte automatisiert; Meta stufte 2023 rund 3,6 Millionen Inhalte als mutmaßlich illegal ein. Die dabei berichteten Fehlerquoten liegen je nach Anbieter und Verfahren zwischen 0,1 % und über 1 %.

Diese Prozentzahlen wirken klein, bis man sie mit der absoluten Größenordnung kombiniert: Bei Milliarden gescannter Inhalte bedeutet selbst eine Fehlerquote von 0,1 %, dass Millionen Inhalte falsch markiert werden — mit potenziellen Folgen wie Kontosperrung, Weitergabe an Strafverfolgungsbehörden oder den Verlust des Zugriffs auf Jahre persönlicher Kommunikation, Fotos und Dokumente, ohne dass ein Fehlverhalten vorlag.

3.1a Warum diese Zahlen eine Untergrenze darstellen

Die in 3.1 genannten Zahlen betreffen ausschließlich die Anbieter, die überhaupt öffentlich über ihre Scanning-Aktivität berichten — die tatsächliche Gesamtzahl automatisiert gescannter Inhalte über alle unter die Interim-Regelung (Kapitel 28.1) fallenden Anbieter hinweg dürfte damit eher eine Unter- als eine Obergrenze darstellen. Zusätzlich betreffen die genannten Fehlerquoten nur die jeweils öffentlich dokumentierte Kategorie (meist bekanntes Material, Kapitel 29.2) — für die technisch fehleranfälligeren Kategorien (neues Material, Grooming-Erkennung) liegen häufig gar keine vergleichbar detaillierten, öffentlich zugänglichen Fehlerquoten vor, was eine vollständige Einschätzung des Gesamtfehlerrisikos zusätzlich erschwert. Diese Unsicherheit selbst ist Teil der in Kapitel 31.1a beschriebenen Kritik: Eine fundierte Verhältnismäßigkeitsprüfung (Kapitel 27.2a) setzt eigentlich eine verlässliche Datengrundlage voraus, die nach Einschätzung mehrerer der in Kapitel 31 genannten Institutionen bislang nicht in ausreichendem Umfang vorliegt.

3.2 Was als Nächstes zur Debatte steht: verpflichtendes Scannen

Die genannte Ausnahmeregelung ist freiwillig und zeitlich befristet. Ein noch nicht final beschlossener Folge-Vorschlag der Europäischen Kommission — umgangssprachlich „Chat-Kontrolle“ genannt, offiziell Teil der geplanten Verordnung zur Verhinderung und

Bekämpfung des sexuellen Missbrauchs von Kindern — würde ein vergleichbares Scannen für **alle** Anbieter verpflichtend machen, ausdrücklich einschließlich Ende-zu-Ende-verschlüsselter Dienste.

3.2a Freiwillig vs. verpflichtend: warum dieser Unterschied entscheidend ist

Der Unterschied zwischen der heutigen freiwilligen Praxis (3.1) und dem diskutierten verpflichtenden Modell (3.2) ist mehr als eine bloße Nuance — er verändert die gesamte rechtliche und technische Bewertungsgrundlage. Ein Anbieter, der freiwillig scannt, kann diese Praxis jederzeit eigenständig beenden, anpassen oder auf einzelne Nutzergruppen beschränken, etwa wenn er selbst zu dem Schluss kommt, dass die Fehlerquote (Kapitel 29.3) ein akzeptables Maß überschreitet. Eine verpflichtende Regelung entzieht dem Anbieter diese Entscheidungsfreiheit: Er müsste scannen, unabhängig von eigener fachlicher Einschätzung, und könnte diese Pflicht nicht einseitig beenden, selbst wenn eigene Erfahrungswerte für eine Anpassung sprächen. Dieser Kontrollverlust auf Anbieterseite ist einer der Gründe, warum Anbieter Ende-zu-Ende-verschlüsselter Dienste (Kapitel 31.4) eine verpflichtende Regelung grundsätzlich anders bewerten als eine freiwillige — bei letzterer bleibt die Möglichkeit einer eigenverantwortlichen Anpassung oder eines Widerrufs erhalten, bei ersterer nicht.

Der entscheidende technische Punkt: Bei einem Dienst, der tatsächlich Ende-zu-Ende-verschlüsselt ist, kann der Anbieter selbst den Inhalt einer Nachricht nicht einsehen — genau das ist ja der Zweck der Verschlüsselung. Eine verpflichtende Inhaltsprüfung ist unter dieser Voraussetzung nur auf eine einzige technische Weise überhaupt möglich: **Client-Side-Scanning**, also die Analyse des Inhalts direkt auf dem Gerät der Nutzerin oder des Nutzers, bevor er verschlüsselt bzw. nachdem er entschlüsselt wurde. Die vollständige technische und rechtliche Einordnung dieses Mechanismus findet sich in Teil IV, Kapitel 30.

3.3 Warum ein struktureller Zugriffspunkt gefährlich ist — unabhängig vom Zweck

Der entscheidende Einwand von Kryptographen und Sicherheitsforschern gegen Client-Side-Scanning richtet sich nicht in erster Linie gegen den Zweck (Kinderschutz ist unbestritten legitim und wichtig), sondern gegen den Mechanismus selbst: Ein System, das auf jedem Gerät Inhalte vor der Verschlüsselung analysiert und diese Analyse an eine externe Stelle meldet, ist ein **struktureller Zugriffspunkt** auf private Kommunikation — unabhängig davon, wie eng der ursprüngliche Anwendungszweck definiert ist.

Diese Sorge ist nicht abstrakt. Historisch wurden für einen engen Zweck geschaffene Überwachungsinfrastrukturen wiederholt auf zusätzliche Zwecke ausgeweitet, sobald sie einmal existierten — sei es durch spätere Gesetzesänderungen, durch Missbrauch

durch autoritäre Regierungen, denen dieselbe Technologie ebenfalls zur Verfügung stünde, oder schlicht durch technische Fehlfunktionen und False Positives in der eingesetzten Erkennungssoftware selbst.

3.3a Das Steuerungsproblem: Wer kontrolliert die Referenzdatenbank?

Ein oft übersehener, aber entscheidender technischer Aspekt des in 3.3 beschriebenen strukturellen Zugriffspunkts betrifft die Frage, wer tatsächlich festlegt, wonach gesucht wird. Ein Client-Side-Scanning-System benötigt zwangsläufig eine Referenzdatenbank (bekannte Bildhashes) oder ein trainiertes Klassifikationsmodell (für neues Material und Grooming, Kapitel 29.2), die von irgendeiner Stelle gepflegt und aktualisiert wird. Diese Stelle besitzt damit faktisch die Macht, zu bestimmen, wonach auf Millionen privater Geräte gesucht wird — eine Aktualisierung dieser Datenbank kann, ohne dass die einzelne Nutzerin oder der einzelne Nutzer dies bemerkt oder beeinflussen kann, den tatsächlichen Anwendungsbereich der Überwachung verändern. Diese Steuerungsfrage ist unabhängig davon, ob die verantwortliche Stelle vertrauenswürdig ist: Selbst eine heute völlig vertrauenswürdige, ausschließlich für Kinderschutz zuständige Institution könnte morgen unter anderer politischer Führung stehen, gehackt werden, oder rechtlich zu einer Erweiterung des Suchauftrags verpflichtet werden (Kapitel 30.3) — die technische Infrastruktur selbst würde all dies anstandslos mittragen, da sie nicht zwischen einer legitimen und einer zweckentfremdeten Aktualisierung unterscheiden kann.

RenkerVault ist bewusst so gebaut, dass ein solcher struktureller Zugriffspunkt gar nicht erst entsteht: Der Relay-Server bekommt zu keinem Zeitpunkt Klartext zu sehen und kann daher auch nicht nachträglich — durch Gesetzesänderung, gerichtliche Anordnung oder Kompromittierung — zu einem Scanner umgebaut werden. Das macht RenkerVault, wie in Kapitel 2 bereits festgehalten, zu keinem Werkzeug „gegen“ Kinderschutz, sondern zu derselben Art von privatsphäre-schützender Architektur wie Signal, Session oder Threema.

3.4 Wer von struktureller Überwachung besonders betroffen wäre

Auch wenn die Debatte oft abstrakt geführt wird, sind die praktischen Konsequenzen konkret und treffen bestimmte Gruppen besonders hart:

- **Journalistinnen und Journalisten**, deren Quellenschutz von vertraulicher Kommunikation abhängt — ein durchsuchbarer Kommunikationskanal gefährdet unmittelbar die Sicherheit von Informanten.
- **Anwältinnen und Anwälte**, für deren Mandantenkommunikation in den meisten Rechtsordnungen ein besonderer Vertraulichkeitsschutz gilt, der durch verpflichtendes Scannen technisch unterlaufen würde.

- **Opfer häuslicher Gewalt und Stalking**, deren Sicherheit oft gerade davon abhängt, dass ihre Kommunikation (etwa mit Beratungsstellen) nicht von Dritten – einschließlich des Täters, sollte er Zugriff auf Systeme erlangen – eingesehen werden kann.
- **Politisch Verfolgte und Angehörige von Minderheiten** in Regionen, in denen bereits heute abweichende Meinungen strafrechtlich verfolgt werden — für sie ist ein struktureller Zugriffspunkt keine abstrakte Sorge, sondern eine unmittelbare Gefahr für Leib und Leben.
- **Jede Person, die grundsätzlich der Auffassung ist**, dass private Kommunikation privat bleiben sollte, unabhängig von einer konkreten, individuellen Bedrohungslage — ein Grundrecht, das im nächsten Kapitel historisch und im weiteren Verlauf des Buches rechtlich eingeordnet wird.

3.5 Ein illustrativer, verallgemeinerter Ablauf eines Fehlalarms

Um die in 3.1 genannten abstrakten Fehlerquoten greifbarer zu machen, ein verallgemeinertes, nicht auf einen realen Einzelfall bezogenes Beispiel, wie ein Fehlalarm in der Praxis ablaufen könnte: Ein automatisiertes Erkennungssystem stuft ein harmloses, von Eltern aufgenommenes Foto ihres eigenen Kindes am Strand fälschlich als potenziell problematisch ein — ein in der Fachliteratur dokumentiertes Fehlerrisiko bei automatisierten Bilderkennungssystemen, die zwischen medizinisch/familiär unbedenklichen und tatsächlich missbräuchlichen Aufnahmen unterscheiden müssen. In der Folge könnte das betroffene Konto automatisiert gesperrt werden, bevor eine menschliche Prüfung stattfindet; der Zugriff auf möglicherweise Jahre an privater Kommunikation, weiteren Fotos und Dokumenten wäre bis zur Klärung blockiert; im ungünstigsten Fall würde der Vorfall automatisch an eine Behörde gemeldet, mit entsprechendem psychischen Belastungspotenzial für die betroffene Familie, bis der Irrtum aufgeklärt ist. Dieses illustrative Beispiel verallgemeinert ein in Fachkreisen wiederholt diskutiertes Fehlerrisiko-Muster, ohne einen konkreten, realen Einzelfall zu behaupten — es zeigt aber, warum die in 3.1 genannten Prozentzahlen keine abstrakte Statistik bleiben, sondern sich in sehr konkrete, belastende Situationen für einzelne, unbeteiligte Menschen übersetzen können.

4. Eine kurze Geschichte der Verschlüsselungsdebatte (Crypto Wars)

Die aktuelle Debatte um Client-Side-Scanning ist kein neues Phänomen, sondern die jüngste Runde eines Konflikts, der seit den 1990er-Jahren immer wieder aufflammt und in der Fachliteratur als „Crypto Wars“ bezeichnet wird.

4.1 Crypto Wars 1.0: Der Clipper Chip (1990er-Jahre)

In den frühen 1990er-Jahren schlug die US-Regierung den sogenannten „Clipper Chip“ vor — einen Verschlüsselungs-Chip für Telefone, der eine eingebaute Hintertür (ein „Key Escrow“-System) enthielt, über die Strafverfolgungsbehörden mit richterlicher Genehmigung Gespräche entschlüsseln konnten. Der Vorschlag scheiterte letztlich an zwei Faktoren: massivem Widerstand aus der Kryptographie-Community, die nachwies, dass ein solches System auch von unautorisierten Dritten ausgenutzt werden konnte, und dem gleichzeitigen Aufkommen frei verfügbarer, starker Verschlüsselungssoftware wie PGP („Pretty Good Privacy“), die eine Kontrolle über Verschlüsselungstechnologie ohnehin zunehmend unmöglich machte.

4.2 Der Export starker Kryptographie als „Waffe“

Bis in die späten 1990er-Jahre wurde starke Verschlüsselungssoftware in den USA rechtlich wie eine Waffe behandelt und unterlag entsprechenden Exportbeschränkungen. Dies führte zu kuriosen Situationen, etwa dem sogenannten „RSA-in-Perl“-T-Shirt, das den kompletten Quellcode eines Verschlüsselungsalgorithmus als bedrucktes Kleidungsstück verbreitete — eine Form des zivilen Ungehorsams, die zeigte, wie schwer sich mathematisches Wissen tatsächlich unter Kontrolle bringen lässt. Die Beschränkungen wurden Ende der 1990er-Jahre schrittweise gelockert.

4.2a Phil Zimmermann und die strafrechtliche Untersuchung gegen PGP

Der in Kapitel 4.2 knapp erwähnte Phil Zimmermann, Entwickler von PGP, wurde nach der Veröffentlichung seiner Software 1991 mehrere Jahre lang Gegenstand strafrechtlicher Ermittlungen der US-Regierung, da die weltweite Verbreitung von PGP über das Internet faktisch einem „Export“ der als Waffe eingestuft Kryptographie-Technologie gleichkam (Kapitel 4.2). Die Ermittlungen wurden 1996 ohne Anklageerhebung eingestellt, unter anderem weil nie eindeutig geklärt werden konnte, wer für die Veröffentlichung im damals noch jungen, dezentralen Internet konkret verantwortlich zu machen wäre, und weil zivilgesellschaftlicher und akademischer Druck gegen die Kriminalisierung reiner Kryptographie-Forschung wuchs. Dieser Fall wird in der Kryptographie-Community bis heute als frühes Beispiel dafür angeführt, dass der Versuch, mathematisches Wissen wie physische Waffentechnologie zu regulieren, an der grundlegend anderen Natur von Information (leicht kopierbar, grenzenlos verbreitbar) scheitert — ein Argument, das in abgewandelter Form auch in der aktuellen Debatte um Client-Side-Scanning (Kapitel 30) wiederkehrt: Sobald eine Analysetechnologie einmal existiert und dokumentiert ist, lässt sie sich ebenso wenig zuverlässig auf den ursprünglich vorgesehenen Anwendungsbereich beschränken wie einst der Quellcode von PGP auf US-amerikanisches Staatsgebiet.

4.2b Bernstein v. United States: Code als Meinungsäußerung

Parallel zur in 4.2 beschriebenen Exportbeschränkung führte der Mathematiker und spätere Erfinder von Curve25519 (Teil III, Kapitel 14.2), Daniel J. Bernstein, in den 1990er-Jahren einen wegweisenden Rechtsstreit gegen die US-Regierung („Bernstein v. United States“). Bernstein wollte den Quellcode eines von ihm entwickelten Verschlüsselungsverfahrens veröffentlichen und akademisch diskutieren, was nach den damaligen Exportkontrollregeln einer Genehmigungspflicht unterlag. Die Gerichte entschieden in mehreren Instanzen zu seinen Gunsten mit der bemerkenswerten Begründung, dass Quellcode als eine Form der durch den ersten Verfassungszusatz geschützten Meinungsäußerung zu behandeln sei — eine Argumentationslinie, die bis heute in Debatten über die Regulierung von Software zitiert wird, weit über den ursprünglichen kryptographischen Kontext hinaus.

4.2c Die Krypto-Debatte in Europa der 1990er-Jahre

Auch außerhalb der USA gab es in den 1990er-Jahren vergleichbare Auseinandersetzungen: Frankreich etwa unterhielt bis 1996 eine der striktesten Kryptographie-Regulierungen Westeuropas, die den Einsatz starker Verschlüsselung durch Privatpersonen grundsätzlich genehmigungspflichtig machte — eine Regelung, die im Zuge der allgemeinen Liberalisierung der Kryptographie-Politik in den späten 1990er-Jahren gelockert wurde. Diese europäische Vorgeschichte zeigt, dass die Spannung zwischen staatlichem Sicherheitsinteresse und ziviler Verschlüsselungsfreiheit auch auf dem europäischen Kontinent keine neue Erscheinung der Chat-Kontrolle-Debatte ist, sondern eine über Jahrzehnte wiederkehrende Konstante europäischer Digitalpolitik.

4.3 Crypto Wars 2.0: Apple vs. FBI (2016)

Im Jahr 2016 forderte das FBI von Apple, eine Software zu entwickeln, die es erlauben würde, die Passcode-Sperre eines beschlagnahmten iPhones zu umgehen (im Zusammenhang mit den Ermittlungen nach einem Terroranschlag in San Bernardino). Apple weigerte sich mit der Begründung, dass eine solche Software — einmal entwickelt — nicht auf einen einzigen Fall beschränkt bleiben könnte und ein Präzedenzfall für den systematischen Zugriff auf jedes iPhone weltweit schaffen würde. Der Fall wurde letztlich gegenstandslos, da das FBI einen alternativen technischen Weg fand, ohne Apples Mitwirkung auf das Gerät zuzugreifen — die grundsätzliche rechtliche Frage blieb ungeklärt.

4.3a Die Parallele zu RenkerVaults Fehlen einer „Passwort vergessen“-Funktion

Der Kern von Apples Weigerung im Fall San Bernardino (4.3) lässt sich direkt mit einer bereits in diesem Buch beschriebenen Design-Entscheidung von RenkerVault verknüpfen: dem bewussten Fehlen einer „Passwort vergessen“-Funktion (Kapitel 37.4). Beide Fälle beruhen auf derselben strukturellen Überlegung: Sobald ein technischer

Mechanismus existiert, der eine Sicherheitsbarriere (Apples Geräteverschlüsselung, RenkerVaults Passphrase-Schutz) für eine autorisierte Stelle umgehbar macht, existiert dieser Mechanismus grundsätzlich — unabhängig davon, wie eng er ursprünglich für einen einzelnen, autorisierten Anwendungsfall gedacht war. Apple argumentierte, eine einmal entwickelte Entsperr-Software ließe sich nicht zuverlässig auf den einen angefragten Fall beschränken; RenkerVault zieht aus demselben Grundgedanken die Konsequenz, eine solche Umgehungsmöglichkeit von vornherein gar nicht erst zu implementieren (Kapitel 21, Zero-Knowledge-Architektur) — der Unterschied liegt darin, dass Apple sich nachträglich gegen den Bau eines bestimmten Werkzeugs entschied, während RenkerVaults Architektur den Bau eines solchen Werkzeugs von Grund auf technisch ausschließt.

4.4 Die aktuelle Runde: Client-Side-Scanning statt Backdoor

Die heutige Debatte unterscheidet sich in einem wichtigen technischen Detail von den vorherigen Runden: Während Clipper Chip und der Apple-FBI-Fall eine klassische „Hintertür“ in der Verschlüsselung selbst oder im Gerätezugriff forderten, zielt Client-Side-Scanning auf einen Punkt **vor** der Verschlüsselung bzw. **nach** der Entschlüsselung. Kryptographisch bleibt die Übertragung selbst dabei tatsächlich ungebrochen — der Zugriffspunkt verlagert sich lediglich an eine andere Stelle der Kette. Führende Kryptographen (unter anderem in einem viel zitierten wissenschaftlichen Papier mit dem Titel „Bugs in our Pockets“ von 2021, verfasst von einer Gruppe international renommierter Sicherheitsforscher) argumentieren, dass dies technisch gesehen keine Lösung des ursprünglichen Zielkonflikts ist, sondern lediglich eine Verlagerung desselben strukturellen Problems.

4.4a Warum diese Verlagerung aus Sicht der Befürworter attraktiv erscheint

Um die Position der Befürworter (Kapitel 25a.1) fair darzustellen: Aus deren Sicht liegt der Reiz von Client-Side-Scanning gerade darin, dass es formal keine „Schwächung“ der Verschlüsselung selbst darstellt — die mathematische Sicherheit der Übertragung (Teil III) bliebe vollständig erhalten, ein Verkaufsargument, mit dem sich manche frühere Vorschläge (etwa der Clipper Chip, Kapitel 4.1) nicht schmücken konnten, da diese die Verschlüsselung selbst unmittelbar betrafen. Aus dieser Perspektive ließe sich argumentieren, ein Anbieter könne wahrheitsgemäß weiterhin behaupten, „Ende-zu-Ende-verschlüsselt“ zu sein, selbst wenn zusätzlich eine Client-Side-Scanning-Komponente installiert wäre. Genau an diesem Punkt setzt die in Kapitel 30.2 dargestellte Kritik der Kryptographie-Community an: Sie argumentiert, dass eine solche Behauptung zwar formal korrekt, aber praktisch irreführend wäre, da der eigentliche Schutzzweck der Verschlüsselung (dass niemand außer den beiden Gesprächspartnern den Inhalt einsehen kann) durch die zusätzliche Scanning-Komponente faktisch unterlaufen würde, unabhängig davon, ob die

Übertragung selbst weiterhin mathematisch korrekt verschlüsselt bliebe.

Die wiederkehrende Lehre aus über 30 Jahren Crypto Wars: Jeder Versuch, einen kontrollierten, nur für „gute“ Akteure zugänglichen Zugriffspunkt in ein Verschlüsselungssystem einzubauen, hat sich entweder als technisch unsicher (weil derselbe Zugriffspunkt auch von böswilligen Akteuren gefunden werden kann) oder als politisch instabil (weil der Zugriffsumfang im Lauf der Zeit ausgeweitet wurde) erwiesen.

4.5a Ein wiederkehrendes rhetorisches Muster: die „Vier Reiter der Infokalypse“

In der Kryptographie-Community hat sich für ein wiederkehrendes rhetorisches Muster in Verschlüsselungsdebatten scherzhaft der Begriff „Four Horsemen of the Infocalypse“ (die vier Reiter der Infokalypse) etabliert — eine Anspielung auf die biblischen apokalyptischen Reiter. Gemeint sind vier Bedrohungsszenarien, die in politischen Debatten um Verschlüsselungsbeschränkungen historisch besonders häufig als Begründung angeführt wurden: organisierte Kriminalität, Terrorismus, Drogenhandel und — in der aktuellen Chat-Kontrolle-Debatte im Zentrum stehend — sexueller Kindesmissbrauch. Der Begriff ist nicht als Verharmlosung dieser tatsächlich schwerwiegenden Probleme gemeint, sondern als Beobachtung eines wiederkehrenden rhetorischen Musters: In nahezu jeder der in diesem Kapitel beschriebenen Debattenrunden seit den 1990er-Jahren wurde jeweils eines (oder mehrere) dieser vier Szenarien als zentrales Argument für eine Schwächung ziviler Verschlüsselung angeführt. Diese Beobachtung ändert nichts an der Legitimität des jeweils konkret verfolgten Ziels (Kapitel 25a.1) — sie mahnt aber zur Wachsamkeit gegenüber der in Kapitel 30.3 beschriebenen Gefahr der Zweckerweiterung: Eine für ein bestimmtes Szenario geschaffene technische Infrastruktur lässt sich erfahrungsgemäß leicht auf die jeweils nächste, ähnlich dringlich vorgetragene Bedrohung ausweiten.

4.5b Die „Five Eyes“ und internationale Geheimdienstkooperation

Ein weiterer, für die historische Einordnung relevanter Aspekt ist die seit dem Zweiten Weltkrieg bestehende nachrichtendienstliche Kooperation zwischen den USA, dem Vereinigten Königreich, Kanada, Australien und Neuseeland, bekannt als „Five Eyes“ (in erweiterter Form auch „Nine Eyes“ oder „Fourteen Eyes“ unter Einbeziehung weiterer Staaten, darunter mehrerer EU-Mitglieder). Diese Kooperation wurde durch die 2013 von Edward Snowden veröffentlichten Dokumente einer breiten Öffentlichkeit bekannt (Kapitel 45, Zeitleiste) und zeigte das Ausmaß, in dem nachrichtendienstliche Massenüberwachung bereits vor der aktuellen Chat-Kontrolle-Debatte stattfand — ein wichtiger Kontext, um zu verstehen, warum Teile der Kryptographie-Community neuen, gesetzlich verankerten Zugriffsmechanismen grundsätzlich skeptisch gegenüberstehen: Die Sorge speist sich nicht aus einem abstrakten Misstrauen, sondern aus der dokumentierten Erfahrung, dass bereits bestehende Überwachungskapazitäten in der Vergangenheit über das öffentlich bekannte Maß hinausgingen.

5. Kryptographische Grundlagen für Laien

Bevor Teil III die kryptographischen Verfahren von RenkerVault im Detail beschreibt, hilft ein kurzer, allgemeinverständlicher Grundkurs — für Leserinnen und Leser, die noch nie mit Verschlüsselung zu tun hatten.

5.1 Das Grundproblem: ein gemeinsames Geheimnis ohne persönliches Treffen

Stellen wir uns zwei Personen vor, Anna und Ben, die eine geheime Nachricht austauschen wollen, sich aber nicht persönlich treffen können, um sich vorab auf einen gemeinsamen Code zu einigen. Jahrhundertlang war das ein ungelöstes Problem: Jedes Verschlüsselungsverfahren brauchte einen Schlüssel, den beide Seiten kennen mussten — aber wie tauscht man diesen Schlüssel sicher aus, wenn man sich nicht treffen kann und jede Nachricht potenziell mitgelesen werden könnte?

5.2 Die Lösung: asymmetrische Kryptographie

In den 1970er-Jahren wurde ein mathematischer Trick entdeckt, der dieses Problem löst: Statt eines einzigen gemeinsamen Schlüssels bekommt jede Person **zwei zusammengehörige Schlüssel** — einen „öffentlichen“ und einen „privaten“. Der öffentliche Schlüssel kann bedenkenlos an jeden weitergegeben werden, sogar öffentlich veröffentlicht; der private Schlüssel bleibt geheim und verlässt nie das eigene Gerät.

Das Bild eines Vorhängeschlosses hilft hier: Stell dir vor, Ben schickt Anna ein offenes Vorhängeschloss (den öffentlichen Schlüssel) per Post — jeder darf dieses Schloss sehen und in die Hand nehmen. Anna steckt ihre Nachricht in eine Kiste, schließt sie mit Bens Schloss ab und schickt die verschlossene Kiste zurück. Jetzt kann **nur noch Ben** die Kiste öffnen, weil nur er den passenden Schlüssel zu seinem eigenen Schloss besitzt — obwohl das Schloss selbst öffentlich unterwegs war und theoretisch jeder es hätte sehen können.

Mathematisch basiert dieser Trick auf sogenannten „Einwegfunktionen“: Rechenoperationen, die in eine Richtung leicht, in die andere Richtung (ohne einen geheimen Zusatzwert) praktisch unmöglich durchzuführen sind. Ein öffentlicher Schlüssel lässt sich aus einem privaten Schlüssel leicht berechnen — der umgekehrte Weg würde selbst mit den leistungsfähigsten heute verfügbaren Computern astronomisch lange dauern.

5.3 Elliptische Kurven — eine besonders effiziente Umsetzung

Es gibt verschiedene mathematische Familien von Einwegfunktionen, die sich für asymmetrische Kryptographie eignen. RenkerVault verwendet eine Familie namens „elliptische Kurven“ (genauer: die spezifischen Kurven X25519 und Ed25519), weil sie

im Vergleich zu älteren Verfahren (etwa RSA) bei gleichem Sicherheitsniveau deutlich kürzere Schlüssel und schnellere Berechnungen ermöglichen — ein praktischer Vorteil, besonders auf Mobilgeräten mit begrenzter Rechenleistung und Akkukapazität. Die mathematischen Details dazu folgen in Teil III, Kapitel 14.

5.4 Symmetrische Verschlüsselung: schnell, aber braucht einen gemeinsamen Schlüssel

Asymmetrische Kryptographie ist mathematisch aufwendig und daher für große Datenmengen (z. B. eine ganze Chat-Nachricht mit Bildanhang) zu langsam. In der Praxis wird sie deshalb nur benutzt, um sich auf einen gemeinsamen, geheimen **symmetrischen** Schlüssel zu einigen — den eigentlichen Nachrichteninhalte verschlüsselt dann ein viel schnelleres symmetrisches Verfahren (bei RenkerVault: AES-256-GCM) mit diesem gemeinsamen Schlüssel. Das Grundproblem („wie einigt man sich sicher auf einen gemeinsamen Schlüssel, ohne sich zu treffen?“) wird also von der asymmetrischen Kryptographie gelöst; die eigentliche Verschlüsselungsarbeit übernimmt danach die schnellere symmetrische Kryptographie. Diese Kombination aus beiden Verfahren nennt man „hybride Verschlüsselung“ und sie ist der Industriestandard, den praktisch jedes moderne Sicherheitsprotokoll verwendet — von HTTPS im Browser bis zu Signal.

5.5 Hash-Funktionen: der digitale Fingerabdruck

Eine dritte kryptographische Grundzutat ist die Hash-Funktion: ein Rechenverfahren, das aus einer beliebig langen Eingabe (ein Wort, ein Buch, eine ganze Datenbank) einen kurzen, festen „Fingerabdruck“ erzeugt. Zwei Eigenschaften machen Hash-Funktionen nützlich: Erstens erzeugt bereits eine winzige Änderung der Eingabe (ein einziges verändertes Zeichen) einen komplett anderen Fingerabdruck — das macht sie ideal, um Manipulation zu erkennen. Zweitens ist es praktisch unmöglich, aus einem gegebenen Fingerabdruck rückwärts auf die ursprüngliche Eingabe zu schließen, oder zwei unterschiedliche Eingaben zu finden, die denselben Fingerabdruck erzeugen. RenkerVault nutzt Hash-Funktionen (konkret: SHA-256) an mehreren Stellen — etwa um zu erkennen, ob die lokale Datenbank manipuliert wurde (Kapitel 11), und als Baustein der Schlüsselableitung (Kapitel 15/16).

5.6 Passwort-Hashing: absichtlich langsam statt schnell

Ein Sonderfall ist das Hashing von Passwörtern: Hier soll die Berechnung nicht schnell, sondern **absichtlich langsam und rechenintensiv** sein, um automatisierte Rateversuche (Brute-Force-Angriffe) unpraktikabel teuer zu machen. RenkerVault verwendet dafür Argon2id, ein Verfahren, das 2015 die „Password Hashing Competition“ gewann und heute breit als Best Practice gilt (siehe Teil II, Kapitel 11 für die konkrete Parametrisierung).

5.6a Eine gute Passphrase wählen: praktische Empfehlung

Da die in Kapitel 7.2 beschriebene Passphrase den gesamten lokalen Datenbestand schützt, lohnt ein praktischer Hinweis, unabhängig von der zugrundeliegenden Argon2id-Absicherung (Kapitel 11.2): Eine lange, aus mehreren zufälligen, aber leicht merkbaren Wörtern zusammengesetzte Passphrase (etwa vier bis fünf per Zufallsverfahren ausgewählte Wörter, ein Ansatz, der auf ein bekanntes Comic des Webcomics „xkcd“ zurückgeht und seither breite Anerkennung in der Sicherheitsforschung fand) bietet in der Praxis oft eine bessere Kombination aus Merkbarekeit und tatsächlicher Sicherheit als ein kurzes, aber kompliziert wirkendes Passwort mit Sonderzeichen. Der Grund: Die tatsächliche Sicherheit eines Passworts gegen Brute-Force-Angriffe hängt maßgeblich von der Gesamtzahl der möglichen Kombinationen ab (der sogenannten Entropie) — vier zufällig gewählte, gebräuchliche Wörter bieten oft mehr Kombinationsmöglichkeiten als ein kurzes, für Menschen schwer merkbare Zeichengewirr, das obendrein häufig vorhersehbaren Mustern folgt (etwa ein großgeschriebener Anfangsbuchstabe und eine angehängte Ziffer). Diese Empfehlung betrifft ausschließlich die Wahl der Passphrase selbst — die zugrundeliegende Argon2id-Berechnung (Kapitel 11.2) bietet unabhängig davon bereits eine zusätzliche Sicherheitsschicht gegen Brute-Force-Versuche.

5.7 Perfect Forward Secrecy — die Idee hinter dem „doppelten Schloss“

Ein letztes Grundkonzept, das in Teil III immer wieder auftaucht, verdient hier eine allgemeinverständliche Einführung: „Perfect Forward Secrecy“ (perfekte vorwärtsgerichtete Geheimhaltung). Die Grundidee: Selbst wenn ein Angreifer irgendwann doch an einen Schlüssel gelangt, sollen damit **nicht** automatisch auch alle vorherigen Nachrichten lesbar werden. Erreicht wird das, indem für jede einzelne Nachricht (oder zumindest für jede kurze Zeitspanne) ein neuer, einmaliger Schlüssel erzeugt und nach Gebrauch sofort und unwiederbringlich verworfen wird. Das konkrete Verfahren, das RenkerVault dafür einsetzt — der „Double Ratchet“ — wird in Teil III, Kapitel 16 Schritt für Schritt hergeleitet.

5.8 Authentizität: Wie weiß ich, dass die Nachricht wirklich von der Person stammt, die sie angeblich geschickt hat?

Verschlüsselung allein beantwortet nur die Frage „kann ein Dritter mitlesen?“ — nicht die Frage „ist der Absender wirklich, wer er zu sein vorgibt?“. Für Letzteres braucht es ein zweites, eng verwandtes, aber eigenständiges Werkzeug: die digitale Signatur. Eine Signatur wird mit dem privaten Schlüssel einer Person erzeugt und lässt sich mit ihrem öffentlichen Schlüssel von jedem prüfen — sie beweist, dass eine Nachricht (oder eine Challenge, siehe Teil III, Kapitel 20) tatsächlich von der Person stammt, die den zugehörigen privaten Schlüssel besitzt, und dass die Nachricht seit der Signierung nicht verändert wurde. RenkerVault trennt bewusst die Schlüssel für Verschlüsselung

(X25519) und für Signaturen (Ed25519) — eine etablierte Praxis, da die Wiederverwendung eines Schlüssels für beide Zwecke in bestimmten Konstellationen theoretische Angriffsflächen öffnen kann.

5.8a Ein Wort zu TLS/HTTPS — der Verschlüsselung, die schon jeder täglich nutzt

Bevor Teil III eigene, RenkerVault-spezifische Verfahren beschreibt, hilft eine Einordnung, dass die grundlegenden Bausteine bereits aus dem Alltag bekannt sind: Jede Webseite, die im Browser mit einem Schloss-Symbol und „https://“ statt „http://“ beginnt, nutzt bereits dieselbe Grundfamilie von Verfahren (asymmetrische Kryptographie für den Schlüsselaustausch, gefolgt von schneller symmetrischer Verschlüsselung, Kapitel 5.4) — dieses Protokoll heißt „Transport Layer Security“ (TLS) und ist der De-facto-Standard für sichere Verbindungen im gesamten Web, von Online-Banking bis E-Mail. Der wesentliche Unterschied zu der in diesem Buch beschriebenen Ende-zu-Ende-Verschlüsselung liegt darin, **wer** am Ende der verschlüsselten Verbindung steht: Bei TLS ist der Endpunkt in der Regel ein Server (etwa der Server einer Bank), der den Inhalt selbst einsehen kann — TLS schützt die Übertragung zwischen dem eigenen Gerät und diesem Server, nicht davor, dass der Server selbst mitliest. Bei Ende-zu-Ende-Verschlüsselung (Kapitel 2) ist der Endpunkt dagegen ein anderes Nutzergerät, und selbst ein zwischengeschalteter Server (der Relay, Kapitel 6.2) kann nicht mitlesen. RenkerVault verwendet, wie in Teil III, Kapitel 21.2a beschrieben, tatsächlich **beide** Schutzschichten gleichzeitig: TLS für die Verbindung zum Relay, und zusätzlich Ende-zu-Ende-Verschlüsselung für den eigentlichen Nachrichteninhalt.

5.9 Ein durchgehendes Beispiel: Annas und Bens erste Nachricht

Um die in den folgenden Kapiteln beschriebenen Verfahren greifbarer zu machen, begleitet dieses Buch ein durchgehendes Beispiel: Anna möchte Ben zum ersten Mal über RenkerVault kontaktieren. Vereinfacht (die vollständige technische Fassung folgt in Teil III) läuft das so ab:

- 1 Ben hat beim Erstellen seines Kontos bereits einen öffentlichen „Prekey“ beim Relay-Server hinterlegt — vergleichbar mit einer öffentlich ausgehängten Telefonnummer, die es Anna erlaubt, ihn zu erreichen, ohne dass er in diesem Moment online sein muss.
- 2 Annas Gerät lädt Bens öffentliche Schlüssel vom Relay herunter und berechnet daraus, kombiniert mit ihren eigenen privaten Schlüsseln, einen gemeinsamen geheimen Ausgangswert — ohne dass dieser Ausgangswert jemals unverschlüsselt über das Netzwerk übertragen wird (das ist die in Abschnitt 5.2 beschriebene asymmetrische Kryptographie in Aktion).

- ³ Aus diesem Ausgangswert leitet Annas Gerät den ersten symmetrischen Nachrichtenschlüssel ab und verschlüsselt ihre erste Nachricht damit.
- ⁴ Der Relay-Server transportiert nur das verschlüsselte Ergebnis plus die für die Zustellung nötigen Metadaten (an wen, von welchem Konto, wann) — niemals den Ausgangswert oder den Klartext.
- ⁵ Ben empfängt das Paket, berechnet aus seinen privaten Schlüsseln und Annas mitgeschickten öffentlichen Werten denselben gemeinsamen Ausgangswert und entschlüsselt die Nachricht.
- ⁶ Ab der zweiten Nachricht greift der in Abschnitt 5.7 beschriebene Double Ratchet: Jede weitere Nachricht verwendet einen neuen, aus dem vorherigen abgeleiteten Schlüssel, der nach Gebrauch verworfen wird.

Dieses Beispiel wird in Teil III mit den tatsächlich verwendeten Formeln und Bezeichnungen (X3DH-lite, HKDF, Double Ratchet) wieder aufgegriffen.

TEIL II

Architektur & Funktionen

Was die App kann, und wie die Teile zusammenspielen

5.9a Vertrauensanker: warum man Bibliotheken überhaupt vertrauen kann

Ein aufmerksamer Leser könnte an dieser Stelle einwenden: Wenn RenkerVault selbst keine eigene Kryptographie erfinden soll (ein Prinzip, das in Teil III immer wieder betont wird), worauf gründet dann das Vertrauen in die verwendeten Bibliotheken überhaupt? Die Antwort liegt in einem mehrstufigen Vertrauensmodell, das der klassischen Wissenschaft ähnelt: Ein einzelner Algorithmus (etwa AES oder X25519) wird nicht deshalb als sicher angesehen, weil eine einzelne Autorität dies behauptet, sondern weil er über Jahre hinweg von unabhängigen Forschungsgruppen weltweit untersucht wurde, ohne dass praktikable Angriffe gefunden wurden — ein Prozess, der in der Kryptographie „Kryptoanalyse“ genannt wird und strukturell dem wissenschaftlichen Peer-Review ähnelt. Je länger ein Verfahren öffentlich untersucht wurde, ohne gebrochen zu werden, desto höher das Vertrauen, das die Fachgemeinschaft ihm entgegenbringt — AES etwa ist seit über zwei Jahrzehnten öffentlich verfügbar und wurde in dieser Zeit von tausenden Forschern weltweit untersucht. Dieses Prinzip — Sicherheit durch öffentliche, langjährige Überprüfbarkeit statt durch geheime, unüberprüfbare Verfahren — nennt sich „Kerckhoffs' Prinzip“ (nach dem niederländischen Kryptographen Auguste Kerckhoffs, der es bereits 1883 formulierte): Ein Verschlüsselungssystem soll auch dann sicher bleiben, wenn alles über das System bekannt ist außer dem geheimen Schlüssel selbst. Jede in diesem Buch beschriebene Komponente (Teil III) folgt diesem über 140 Jahre alten Grundsatz.

6. Gesamtarchitektur im Überblick

RenkerVault besteht aus zwei unabhängigen, getrennt entwickelten und getrennt betriebenen Codebasen: einem **Client** (die App, die Nutzerinnen und Nutzer tatsächlich installieren bzw. im Browser öffnen) und einem **Relay-Server** (dem Vermittlungsdienst, der Nachrichten zwischen den Geräten weiterleitet). Diese Trennung ist keine zufällige Design-Entscheidung, sondern die Grundvoraussetzung für das in Kapitel 2 beschriebene Zero-Knowledge-Prinzip: Alles, was Vertraulichkeit erfordert (Schlüsselerzeugung, Verschlüsselung, Entschlüsselung), passiert ausschließlich auf dem Client — der Relay bekommt nur das fertige, bereits verschlüsselte Ergebnis zu Gesicht.

6.1 Die drei Schichten des Clients

Der Client selbst ist intern in drei klar getrennte Schichten gegliedert:

- **Kryptographie-Schicht** (`crypto/`): Enthält jede einzelne kryptographische Operation der App — Schlüsselerzeugung, Verschlüsselung, Entschlüsselung, Passwort-Hashing. Diese Schicht kennt keine Chat-Oberfläche, keine Netzwerkverbindung; sie nimmt Klartext und Schlüssel entgegen und gibt Chiffretext zurück, oder umgekehrt.
- **Netzwerk-Schicht** (`net/`): Verwaltet die WebSocket-Verbindung zum Relay, die Authentifizierung, den Nachrichtenversand/-empfang und die Verwaltung der laufenden Verschlüsselungssitzungen pro Kontakt bzw. Gruppe. Diese Schicht ruft die Kryptographie-Schicht auf, implementiert aber selbst keine kryptographischen Verfahren.
- **Oberflächen-Schicht** (`ui/`): Die eigentliche, sichtbare Anwendung — Chat-Fenster, Kontaktliste, Sicherheitszentrale, Einstellungen. Diese Schicht kennt weder Schlüssel noch Rohdaten der Verschlüsselung; sie erhält von der Netzwerk-Schicht bereits entschlüsselten Klartext zur Anzeige und gibt eingegebenen Klartext zur Verschlüsselung weiter.

Diese Dreiteilung hat einen handfesten Sicherheitsvorteil: Ein Fehler in der Oberflächen-Schicht (z. B. ein Darstellungsbug) kann grundsätzlich nicht dazu führen, dass ein privater Schlüssel offengelegt wird, weil die Oberflächen-Schicht schlicht keinen Zugriff auf Schlüsselmaterial hat. Die Angriffsfläche für kryptographische Fehler bleibt auf den vergleichsweise kleinen, überschaubaren `crypto/`-Ordner konzentriert.

6.1a Das Prinzip der geringsten Rechte, angewendet auf Software-Architektur

Die in 6.1 beschriebene Dreiteilung ist eine Anwendung eines der ältesten und am breitesten anerkannten Prinzipien der IT-Sicherheit: des „Prinzips der geringsten Rechte“ (Principle of Least Privilege), das ursprünglich aus der Betriebssystem- und Zugriffskontrollforschung der 1970er-Jahre stammt (u. a. formuliert von Jerome Saltzer und Michael Schroeder in ihrem einflussreichen 1975 veröffentlichten Aufsatz „The Protection of Information in Computer Systems“). Die Grundidee: Jede Komponente eines Systems sollte nur genau die Rechte und Zugriffsmöglichkeiten erhalten, die sie für ihre Aufgabe tatsächlich benötigt — nicht mehr. Übertragen auf RenkerVaults Client-Architektur bedeutet dies: Die Oberflächen-Schicht (`ui/`) benötigt zur Erfüllung ihrer Aufgabe (Anzeige und Eingabe von Klartext) keinerlei Zugriff auf private Schlüssel — sie erhält diesen Zugriff daher schlicht nicht, unabhängig davon, ob ein Entwickler ihn versehentlich nutzen könnte. Dasselbe Prinzip findet sich auf Serverebene in der systemd-Sandbox-Härtung (Kapitel 44.6c: `NoNewPrivileges`, minimales `CapabilityBoundingSet`) wieder — der Relay-Prozess erhält nur die Betriebssystemrechte, die er für seine eng definierte Aufgabe (Netzwerkverbindungen

entgegennehmen, Envelopes weiterleiten) tatsächlich benötigt.

6.2 Der Relay-Server: bewusst „dumm“ gehalten

Der Relay-Server ist absichtlich auf ein Minimum an Funktionalität reduziert. Er:

- verwaltet Konto-Kennungen und die dazugehörigen öffentlichen Schlüssel,
- verwaltet eine Liste der Geräte, die zu einem Konto gehören, samt Online-Status,
- nimmt verschlüsselte Datenpakete („Envelopes“, siehe Kapitel 8 und Teil III, Kapitel 21) entgegen und leitet sie an die Zielgeräte weiter — oder speichert sie kurzzeitig zwischen, falls das Zielgerät gerade offline ist,
- prüft die Authentizität eines Clients über ein passwortloses Challenge-Response-Verfahren (Teil III, Kapitel 20),
- erkennt und blockiert offensichtlichen Missbrauch (zu viele gleichzeitige Verbindungen von derselben IP-Adresse, zu viele fehlgeschlagene Authentifizierungsversuche).

Was der Relay **nicht** tut, ist ebenso wichtig: Er entschlüsselt niemals ein Envelope, er speichert keine Kontaktlisten (jedes Gerät verwaltet seine eigenen Kontakte lokal und verschlüsselt), und er betreibt keine inhaltliche Analyse der transportierten Daten.

6.2a Der Relay als „dummes Rohr“: eine Analogie aus der Netzwerktechnik

Das Konzept eines bewusst funktionsarmen, nur weiterleitenden zentralen Knotenpunkts ist in der Netzwerktechnik unter dem Begriff „Dumb Pipe“ (dummes Rohr) etabliert und keine RenkerVault-spezifische Erfindung: Klassische Internet-Router etwa leiten Datenpakete nach dem sogenannten „Ende-zu-Ende-Prinzip“ (End-to-End Principle, ein aus den 1980er-Jahren stammendes Grundprinzip der Internet-Architektur) weiter, ohne den Inhalt der Pakete zu interpretieren — alle Intelligenz über die Bedeutung der übertragenen Daten liegt bei den Endpunkten, nicht beim Netzwerk selbst. RenkerVaults Relay-Server (6.2) überträgt dieses jahrzehntealte Architekturprinzip konsequent auf die Anwendungsebene eines Chat-Dienstes: Der Relay verhält sich zu den Envelopes wie ein Internet-Router zu IP-Paketen — er leitet weiter, ohne zu verstehen, was er weiterleitet. Diese Analogie hilft auch, den in Kapitel 30 beschriebenen Konflikt um Client-Side-Scanning einzuordnen: Eine verpflichtende Inhaltsanalyse würde dieses seit Jahrzehnten etablierte Ende-zu-Ende-Prinzip der Internet-Architektur an einer neuen Stelle (dem Endgerät selbst, statt dem Netzwerk) durchbrechen, nicht es lediglich fortschreiben.

6.3 Ein Beispiel-Ablauf: Anna schickt Ben eine Nachricht

[Anna: Client]	[Relay-Server]	[Ben: Client]
1. verschlüsselt Nachricht lokal mit Double Ratchet		

```
2. sendet Envelope -----> 3. prueft Auth (Ed25519),  
                               schlaegt Ben-Geraet nach  
                               4. leitet Envelope weiter ---> 5. entschluesselt  
                               ODER: Ben offline?           lokal, zeigt an  
                               -> Envelope in Warteschlange  
                               bis Ben sich verbindet
```

Kein Schritt in diesem Ablauf erfordert, dass der Relay-Server den Inhalt der Nachricht kennt — er trifft alle Routing-Entscheidungen ausschließlich auf Basis von Metadaten (Konto-Kennungen, Geräte-IDs), die für die Zustellung unvermeidlich sichtbar sein müssen (mehr dazu in Kapitel 21).

6.3a Warum WebSocket statt klassischem HTTP-Polling

Die Verbindung zwischen Client und Relay nutzt das WebSocket-Protokoll statt wiederholter klassischer HTTP-Anfragen. Der Unterschied ist praktisch bedeutsam: Bei klassischem HTTP müsste der Client in regelmäßigen Abständen selbst aktiv nachfragen („Gibt es neue Nachrichten?“), was entweder unnötige Verzögerung bei der Zustellung (bei seltenem Nachfragen) oder unnötigen Ressourcenverbrauch durch häufige, meist ergebnislose Anfragen (bei häufigem Nachfragen) bedeutet. WebSocket etabliert stattdessen eine dauerhafte, bidirektionale Verbindung, über die der Relay neue Envelopes aktiv an den Client „pushen“ kann, sobald sie eintreffen — ohne dass der Client aktiv nachfragen muss. Dieselbe dauerhafte Verbindung wird auch für die in Kapitel 8.6 beschriebene Präsenzanzeige und für sofortige Sicherheitsbenachrichtigungen (Kapitel 10.2/10.3) genutzt, die ohne spürbare Verzögerung beim jeweils anderen Gerät ankommen sollen.

6.3b Fehlerbehandlung: was passiert, wenn ein Envelope beschädigt ankommt?

Ein robustes System muss nicht nur den Erfolgsfall, sondern auch Fehlerfälle sauber behandeln. Kommt bei einem Client ein Envelope an, dessen Chiffretext beschädigt ist (etwa durch einen Netzwerkfehler oder einen Manipulationsversuch), schlägt die AES-GCM-Entschlüsselung kontrolliert fehl (Kapitel 13.1) — die Anwendung zeigt in diesem Fall eine Fehlermeldung statt eines verstümmelten Klartexts an und verwirft die betroffene Nachricht, statt sie fälschlich als „leer“ oder „beschädigt, aber lesbar“ darzustellen. Dieses Verhalten ist eine direkte Konsequenz der in Kapitel 13.1 beschriebenen eingebauten Authentizitätsprüfung von AES-GCM: Eine manipulierte oder beschädigte Nachricht lässt sich technisch nicht von einer gültigen Nachricht mit zufällig anderem Inhalt unterscheiden, wenn die Integritätsprüfung fehlschlägt — das System behandelt daher grundsätzlich jeden Integritätsfehler gleich, unabhängig davon, ob die Ursache ein harmloser Übertragungsfehler oder ein gezielter Angriff war.

7. Identität, Konten, Geräte

7.1 Kontoerstellung ohne Klarname

Ein RenkerVault-Konto entsteht rein lokal auf dem Gerät und besteht im Kern aus einer zufällig erzeugten Konto-Kennung und drei kryptographischen Schlüsselpaaren (Identity Key, Signed Prekey, Signing Key — siehe Teil III, Kapitel 14 für die Details). Es gibt bewusst kein Registrierungsformular, keine Telefonnummer, keine E-Mail-Adresse, keine Klarnamenpflicht. Der Relay-Server erfährt bei der ersten Verbindung lediglich die neu erzeugte Konto-Kennung und die zugehörigen öffentlichen Schlüssel — er hat zu keinem Zeitpunkt die Möglichkeit, ein Konto mit einer realen Identität zu verknüpfen, es sei denn, die Nutzerin oder der Nutzer teilt diese Information freiwillig innerhalb eines Chats mit.

7.1a Abgrenzung zu Identifizierungspflichten in anderen Branchen

Diese bewusste Klarnamenlosigkeit unterscheidet RenkerVault von Branchen, in denen eine Identifizierungspflicht („Know Your Customer“, KYC) gesetzlich vorgeschrieben ist — etwa im Bankwesen, wo Geldwäschegesetze eine Identitätsprüfung bei Kontoeröffnung verlangen. Der Unterschied liegt im jeweiligen Regelungszweck: KYC-Pflichten im Finanzsektor zielen auf die Rückverfolgbarkeit von Geldflüssen zur Bekämpfung von Geldwäsche und Terrorismusfinanzierung — ein Bereich, in dem der Gesetzgeber die Abwägung bereits zugunsten verpflichtender Identifizierung getroffen hat. Für Kommunikationsdienste besteht nach aktuell geltendem EU-Recht keine vergleichbare generelle Klarnamenpflicht (Kapitel 33/34) — RenkerVault bewegt sich damit innerhalb des geltenden Rechtsrahmens für Kommunikationsdienste, nicht in einer rechtlichen Grauzone, auch wenn Branchen mit anderem Regelungszweck (wie das Bankwesen) andere Anforderungen kennen. Sollte sich der Rechtsrahmen für Kommunikationsdienste künftig ändern (Kapitel 29.5 zur Altersverifikation zeigt, dass entsprechende Debatten bereits geführt werden), wäre dies gesondert zu bewerten.

7.2 Die Passphrase: Schutz der lokalen Daten, nicht der Übertragung

Beim Anlegen des Kontos wird zusätzlich eine Passphrase festgelegt. Wichtig für das Verständnis: Diese Passphrase schützt **nicht** die Übertragung von Nachrichten (das übernehmen die in Kapitel 7.1 genannten kryptographischen Schlüssel) — sie schützt die **lokale Speicherung** aller Daten auf dem Gerät (siehe Kapitel 11). Aus der Passphrase wird über Argon2id ein Schlüssel abgeleitet, der wiederum den eigentlichen Master-Schlüssel der lokalen Datenbank entschlüsselt. Die Passphrase selbst verlässt das Gerät nie — auch nicht in gehashter Form.

7.2a Der Onboarding-Ablauf im Detail

Um den in 7.1/7.2 beschriebenen Ablauf konkret nachzuvollziehen, die Abfolge der Bildschirme beim allerersten Start:

- 1 **Begrüßungsbildschirm:** Kurze Erklärung, dass gleich ein neuer, lokaler Tresor angelegt wird — keine Registrierung, kein Server-Kontakt an dieser Stelle.
- 2 **Schlüsselerzeugung (im Hintergrund):** Die drei in Kapitel 14 beschriebenen Schlüsselpaare sowie das ML-KEM-Schlüsselpaar (Kapitel 18) werden lokal erzeugt, spürbar innerhalb weniger Sekunden.
- 3 **Passphrase festlegen:** Eingabe und Bestätigung der Passphrase (Kapitel 7.2), mit einer Sichtbarkeits-Umschaltfunktion, um Tippfehler zu vermeiden, sowie einer Stärkeanzeige.
- 4 **Optional: Duress-PIN festlegen:** Wer möchte, kann sofort eine separate Notfall-PIN einrichten (Kapitel 10.5) — dieser Schritt lässt sich auch später nachholen.
- 5 **Vault-Erzeugung:** Der in Kapitel 11.1 beschriebene Master-Key wird erzeugt und mit dem aus der Passphrase abgeleiteten Schlüssel verschlüsselt gespeichert.
- 6 **Relay-Verbindung (optional an dieser Stelle):** Erst jetzt, nach Abschluss der rein lokalen Schritte 1-5, nimmt die App überhaupt Kontakt zu einem Relay-Server auf (Kapitel 20.2) — bewusst als letzter, nicht zwingend sofort nötiger Schritt, um zu verdeutlichen, dass die grundlegende Kontoerzeugung keinen Netzwerkzugriff voraussetzt.

Dieser Ablauf macht die in Kapitel 2.1 beschriebene Zero-Knowledge-Eigenschaft bereits beim allerersten Start erfahrbar: Ein Konto existiert vollständig, bevor überhaupt eine Verbindung zu irgendeinem Server aufgebaut wurde.

7.3 Mehrgeräte-Unterstützung und das Vertrauensmodell neuer Geräte

Ein Konto kann von mehreren Geräten aus genutzt werden. Damit ein zusätzliches Gerät nicht automatisch vollen Zugriff erhält (etwa falls ein Angreifer die Konto-Kennung und ein gestohlenes zweites Gerät kombiniert), gilt: Jedes neu hinzukommende Gerät ist zunächst als „**untrusted**“ (nicht vertrauenswürdig) markiert und erhält keine Nachrichten-Warteschlange, bis es von einem bereits bestehenden, vertrauten Gerät manuell bestätigt wird. Zusätzlich löst eine bereits bekannte Geräte-Kennung, die sich plötzlich mit einem anderen kryptographischen Schlüssel meldet, einen sofortigen **KEY_MISMATCH**-Alarm aus — ein starkes Indiz für einen Impersonationsversuch (siehe auch Kapitel 10).

7.3a Wie sich Mehrgeräte-Unterstützung auf die Kryptographie auswirkt

Ein technisch relevanter Punkt, der über die reine Vertrauensverwaltung hinausgeht: Jedes Gerät eines Kontos besitzt sein eigenes, unabhängiges Schlüsselmaterial (Kapitel 14) und damit auch seine eigenen, unabhängigen Double-Ratchet-Sitzungen (Kapitel 16) zu jedem Kontakt. Das bedeutet praktisch: Wenn Anna sowohl ihr Smartphone als auch ihren Desktop-Computer nutzt (Kapitel 12), unterhält sie mit Ben tatsächlich zwei

getrennte, kryptographisch unabhängige Sitzungen — eine pro Gerät. Eine Nachricht, die Ben an Anna schickt, muss folglich separat für jedes ihrer aktiven Geräte verschlüsselt und zugestellt werden. Dieser Mechanismus ist eine bewusste Vereinfachung gegenüber komplexeren Mehrgeräte-Synchronisationsprotokollen (wie sie etwa Signal für die eigene Mehrgeräte-Unterstützung entwickelt hat), die eine geräteübergreifend konsistente, einzige Sitzung pro Kontaktpaar anstreben — RenkerVaults einfacherer Ansatz erkaufte sich dadurch geringere Implementierungskomplexität auf Kosten einer geringfügig höheren Anzahl an zu verwaltenden Sitzungen bei aktiver Mehrgerätenutzung.

7.4 Kontakte hinzufügen: Lookup statt Adressbuch-Upload

Um einen Kontakt hinzuzufügen, tauschen zwei Personen ihre Konto-Kennung über einen selbst gewählten Kanal aus (mündlich, per QR-Code, über einen anderen Dienst). Die App fragt daraufhin beim Relay ausschließlich die öffentlichen Schlüssel dieser einen, bereits bekannten Konto-Kennung ab (ein „Lookup“) — im Gegensatz zu Diensten, die das komplette Adressbuch eines Geräts zum Abgleich an einen Server hochladen. Damit erfährt der Relay niemals das vollständige soziale Netzwerk einer Nutzerin oder eines Nutzers, sondern immer nur einzelne, gezielt angefragte Verbindungen.

7.4a Warum Adressbuch-Abgleich ein bekanntes Datenschutzproblem ist

Der in 7.4 beschriebene Kontrast zum klassischen Adressbuch-Abgleich verdient eine genauere Einordnung, da dieses Muster bei vielen verbreiteten Messengern seit Jahren Gegenstand datenschutzrechtlicher Kritik ist: Lädt eine App das komplette Telefon-Adressbuch zum serverseitigen Abgleich hoch, erfährt der Serverbetreiber dabei nicht nur, wer selbst die App nutzt, sondern auch die Telefonnummern und teils weitere Kontaktdaten **aller Personen im Adressbuch**, unabhängig davon, ob diese Personen selbst jemals einer Datenverarbeitung durch den betreffenden Dienst zugestimmt haben. Dieses Muster wurde in der Vergangenheit wiederholt datenschutzrechtlich beanstandet, da es faktisch personenbezogene Daten Dritter verarbeitet, die selbst nicht Vertragspartei des jeweiligen Dienstes sind — ein Spannungsfeld zum in Kapitel 33.1 beschriebenen Grundsatz der Datenminimierung. RenkerVaults gezielter Lookup-Mechanismus (7.4) umgeht dieses Problem strukturell: Der Relay erfährt niemals Informationen über Personen, die nicht selbst aktiv als Kontakt gesucht werden, und insbesondere nichts über Personen, die RenkerVault gar nicht verwenden.

7.5 Safety Numbers: Vertrauen verifizieren statt blind annehmen

Da der initiale Schlüsselaustausch (Kapitel 15) asynchron und ohne persönliches Treffen abläuft, bleibt theoretisch die Möglichkeit eines sogenannten

„Man-in-the-Middle“-Angriffs: Ein Angreifer mit Kontrolle über den Relay-Server könnte versuchen, Anna eigene öffentliche Schlüssel unterzuschieben, die vorgeben, Bens Schlüssel zu sein. Um das auszuschließen, berechnet die App aus den beteiligten öffentlichen Schlüsseln eine sogenannte „Safety Number“ — eine 60-stellige Zahlenfolge, die auch als QR-Code dargestellt wird. Vergleichen beide Seiten diese Zahl über einen unabhängigen Kanal (persönliches Treffen, Videoanruf) und stimmt sie überein, ist ein Man-in-the-Middle-Angriff praktisch ausgeschlossen, da der Angreifer dann für beide Seiten unterschiedliche, nicht übereinstimmende Schlüssel hätte einschleusen müssen.

7.5a Was passiert, wenn sich eine Safety Number ändert?

Ändert sich der Identity Key eines Kontakts (etwa weil er die App auf einem neuen Gerät komplett neu eingerichtet hat, statt ein bestehendes Gerät zu bestätigen, Kapitel 7.3), ändert sich zwangsläufig auch die aus diesem Schlüssel berechnete Safety Number (Kapitel 7.5) — die Oberflächen-Schicht (Kapitel 6.1) zeigt in diesem Fall eine deutliche Warnung an, dass sich der Sicherheitsstatus des Kontakts geändert hat und eine erneute Verifikation empfohlen wird, bevor weiterhin sensible Inhalte ausgetauscht werden. Diese Warnung ist bewusst nicht automatisch unterdrückbar: Sie erscheint bei jeder Änderung erneut, auch wenn der Kontakt zuvor bereits als „verifiziert“ markiert war, da eine einmalige Verifikation naturgemäß nur für die zum Verifikationszeitpunkt gültigen Schlüssel Aussagekraft besitzt.

7.5b Warum die QR-Code-Darstellung mehr als nur Komfort ist

Die in Kapitel 7.5 erwähnte QR-Code-Darstellung der Safety Number ist nicht nur eine bequemere Alternative zum Ablesen und Vergleichen von 60 Ziffern von Hand — sie reduziert auch eine reale Fehlerquelle. Beim manuellen Vergleich langer Ziffernfolgen unterlaufen Menschen erfahrungsgemäß Lesefehler, insbesondere bei einer so großen Ziffernmenge, die zudem oft unter Zeitdruck oder in einer für einen persönlichen Vergleich unpraktischen Umgebung (etwa ein kurzes Treffen) verglichen wird. Ein QR-Code-Scan reduziert diese Fehlerquelle auf ein rein technisches Gleichheitsproblem: Entweder die beiden gescannten Codes stimmen exakt überein, oder das Gerät meldet eine Abweichung — ohne dass eine Person selbst 60 Ziffern paarweise vergleichen und dabei möglicherweise einen einzelnen abweichenden Ziffernblock übersehen müsste. Diese Praxisüberlegung ist ein Beispiel dafür, wie Sicherheitsmechanismen (die kryptographisch korrekte Safety Number, Kapitel 7.5) durch eine durchdachte Umsetzung in der Oberflächen-Schicht (Kapitel 6.1) tatsächlich zuverlässig genutzt werden, statt in der Praxis an menschlicher Fehleranfälligkeit zu scheitern.

8. Nachrichtentypen und Funktionen im Detail

8.1 Textnachrichten und Grundfunktionen

Textnachrichten bilden die Basis; alle Ende-zu-Ende-Verschlüsselungsgarantien aus Teil III gelten für sie vollumfänglich. Zusätzlich unterstützt RenkerVault:

- **Antworten/Zitieren** — eine Nachricht bezieht sich sichtbar auf eine vorherige.
- **Bearbeiten** — nachträgliche Korrektur einer bereits gesendeten Nachricht, mit sichtbarer „bearbeitet“-Markierung.
- **Löschen** — Entfernen einer Nachricht aus dem eigenen wie dem Verlauf der Gegenseite.
- **Weiterleiten** — Versand einer bereits erhaltenen Nachricht an einen anderen Chat.
- **Emoji-Reaktionen** — kurze, nonverbale Rückmeldung auf einzelne Nachrichten.
- **@Erwähnungen** — gezieltes Ansprechen einzelner Mitglieder in Gruppen.
- **Verschwindende Nachrichten** — ein einstellbarer Timer, nach dessen Ablauf eine Nachricht automatisch aus dem lokalen Verlauf entfernt wird.

Technisch sind Antworten, Bearbeitungen, Löschungen, Weiterleitungen und Reaktionen keine separaten Server-Mechanismen, sondern eigene Anwendungs-Ereignistypen, die wie jede normale Nachricht über die bestehende, Ende-zu-Ende-verschlüsselte Sitzung übertragen werden (siehe SECURITY.md des Projekts für die genaue Aufschlüsselung, welcher Anteil davon als Metadaten und welcher als verschlüsselte Nutzlast läuft).

8.1c Verschwindende Nachrichten: lokale Löschung, kein serverseitiger Zwang

Ein technisches Detail zur in 8.1 genannten Funktion „Verschwindende Nachrichten“ verdient Erwähnung: Der Timer und die anschließende Löschung laufen ausschließlich lokal auf dem jeweiligen Empfängergerät ab — der Relay-Server (Kapitel 21) erzwingt zu keinem Zeitpunkt eine Löschung und könnte dies aufgrund der Zero-Knowledge-Architektur auch technisch gar nicht, da er den Nachrichteninhalte ohnehin nie im Klartext sieht. Das bedeutet praktisch: Die Funktion verhindert zuverlässig, dass eine Nachricht nach Ablauf der eingestellten Zeit noch im **regulären Chatverlauf** der App sichtbar ist — sie verhindert nicht, dass eine Gesprächspartnerin die Nachricht vor Ablauf des Timers auf andere Weise gesichert hat (etwa durch einen Screenshot, dieselbe grundsätzliche Grenze wie in Kapitel 23.3 beschrieben). Verschwindende Nachrichten sind damit ein Werkzeug gegen das versehentliche Anhäufen eines langen, potenziell sensiblen Chatverlaufs — kein technischer Mechanismus, der eine Weitergabe durch den Empfänger selbst verhindern könnte.

8.2 Medien: Bilder, Dateien, Sprachnachrichten

Bild- und Dateianhänge sowie Sprachnachrichten (aufgenommen direkt über das Mikrofon des Geräts) werden vor dem Versand ebenso Ende-zu-Ende-verschlüsselt wie Textnachrichten. Bild- und Audio-Inhalte werden zusätzlich direkt im Chat-Verlauf als Vorschau angezeigt, ohne dass ein externer Dienst zur Anzeige eingebunden wird — ein bewusster Unterschied zu Diensten, die etwa Link-Vorschaubilder über einen externen Server nachladen (siehe 8.4).

8.2a Nachrichtentypen im Überblick

Typ (, Kapitel 44.5a)	Verschlüsselt?	Inline-Vorschau	Groessenbegrenzung
text	Ja, AES-256-GCM	Ja (Text direkt)	Praktisch unbegrenzt (Textlänge)
file mit Bild-MIME-Type	Ja, AES-256-GCM	Ja (Bild-Thumbnail)	Kapitel 44b: max. Envelope-Groesse
file mit Audio-MIME-Type (Sprachnachricht)	Ja, AES-256-GCM	Ja (Player-Steuerelement)	Kapitel 44b: max. Envelope-Groesse
file sonstiger Typ	Ja, AES-256-GCM	Nein (Download-Link)	Kapitel 44b: max. Envelope-Groesse
group-key	Ja, ueber 1:1-Sitzung (Kapitel 17.2)	N/A (Systemnachricht)	Fest (32 Byte Schluessel)
system	Ja	Als Hinweistext	Kurz
edit/delete/reaction/presence	Ja (Anwendungs-Events, Kapitel 8.1)	Aktualisiert bestehende Anzeige	Kurz

Diese Tabelle macht sichtbar, dass **jeder** Nachrichtentyp ohne Ausnahme verschlüsselt wird — es gibt keinen Envelope-Typ, der Klartext unverschlüsselt überträgt, selbst reine Steuersignale wie Präsenz-Updates (Kapitel 8.6) nicht.

8.3 Lesebestätigungen und Tippindikator: standardmäßig deaktiviert

Sowohl die Anzeige „gelesen“ als auch ein „tippt gerade...“-Indikator sind implementiert, aber **standardmäßig ausgeschaltet** (Opt-in). Diese Entscheidung folgt demselben Prinzip wie in Kapitel 2 beschrieben: Eine Funktion, die zusätzliche Metadaten über das eigene Verhalten preisgibt (wann genau eine Nachricht gelesen wurde, wann man beginnt zu tippen), sollte eine bewusste Entscheidung der Nutzerin oder des Nutzers sein und nicht ein unsichtbarer Standardzustand.

8.3a Das soziale Dilemma symmetrischer Funktionen

Eine praktische Eigenheit von Lesebestätigungen, die über die reine Datenschutzfrage hinausgeht: Bei den meisten Diensten, die Lesebestätigungen anbieten, gilt eine „Alles oder Nichts“-Regel — wer selbst sehen möchte, ob die eigene Nachricht gelesen wurde, muss in der Regel auch den eigenen Lesestatus für andere sichtbar machen.

RenkerVault übernimmt dieses Prinzip der Symmetrie (Kapitel 8.3), aus demselben

Fairness-Gedanken, der auch anderen Diensten zugrunde liegt: Eine einseitige Sichtbarkeit (ich sehe, wann du liest, aber du siehst nicht, wann ich lese) würde ein Informationsungleichgewicht zwischen den Gesprächspartnern schaffen, das leicht zu Missverständnissen oder einem Gefühl der Überwachung führen kann. Da die Funktion aber, wie beschrieben, standardmäßig für beide Seiten deaktiviert ist, stellt sich diese Symmetriefrage in der Praxis erst, wenn beide Gesprächspartner sie bewusst gemeinsam aktivieren.

8.4 Klickbare Links ohne automatischen Vorschau-Abruf

Eine bewusste, vom Marktstandard abweichende Design-Entscheidung: Links in Nachrichten werden klickbar gemacht, aber **niemals automatisch abgerufen**, um eine Vorschau zu generieren. Der Grund: Ein automatischer Abruf einer verlinkten Seite würde die IP-Adresse und den ungefähren Lesezeitpunkt an den Betreiber der verlinkten Seite verraten — unabhängig davon, wie gut die Chat-Übertragung selbst Ende-zu-Ende-verschlüsselt ist. Diese Art von Leck wird in der Fachliteratur oft übersehen, weil sie formal „außerhalb“ der eigentlichen Verschlüsselung liegt, praktisch aber genauso viel über das Kommunikationsverhalten verraten kann.

8.5 Chat-Organisation

Chats lassen sich anpinnen (oben in der Liste fixieren), stummschalten (keine Benachrichtigungen) und archivieren (aus der Hauptübersicht ausblenden, ohne sie zu löschen). Eine Suche durchsucht sowohl Chatnamen als auch Nachrichteninhalte — ausschließlich lokal auf dem eigenen Gerät, da der Relay-Server, wie in Kapitel 6 beschrieben, gar keinen Zugriff auf den Klartext hat, den er durchsuchen könnte.

8.5a Warum lokale Suche keinen unverschlüsselten Suchindex voraussetzt

Eine naheliegende technische Frage: Muss für eine schnelle Volltextsuche nicht ein separater, unverschlüsselter Suchindex angelegt werden, der wiederum ein zusätzliches Datenleck darstellen könnte? RenkerVault umgeht dieses Problem, indem die Suche direkt über den bereits im Arbeitsspeicher vorliegenden, entschlüsselten Zustand läuft (Kapitel 11.1: Der Master-Key liegt bei entsperrtem Vault im RAM, der komplette Zustand liegt dort ohnehin in entschlüsselter Form vor, um überhaupt angezeigt werden zu können) — es entsteht kein zusätzlicher, dauerhaft auf der Platte gespeicherter Klartext-Suchindex, der über die ohnehin verschlüsselt gespeicherte Datenbank (Kapitel 11) hinausginge. Der Kompromiss dieses Ansatzes: Bei sehr großen Chatverläufen kann eine Volltextsuche über den gesamten, unindizierten Zustand spürbar langsamer sein als eine Suche über einen dedizierten, vorab aufgebauten Index — ein bewusst in Kauf genommener Nachteil zugunsten der Vermeidung eines zusätzlichen, potenziell unverschlüsselten Datenbestands.

8.6 Präsenz: online / zuletzt gesehen

Ob ein Kontakt gerade online ist oder wann er zuletzt aktiv war, wird als „Best-Effort“-Funktion direkt zwischen den beteiligten Geräten ausgetauscht — der Relay-Server selbst führt keine zentrale Präsenzliste. Das bedeutet auch: Schließt eine Nutzerin die App abrupt (z. B. durch Absturz oder Stromausfall), kann kein zuverlässiges „Offline“-Signal mehr zugestellt werden — ein bewusst in Kauf genommener Kompromiss zugunsten der Architektur, die keine zentrale, serverseitige Beobachtungsstelle für das Online-Verhalten aller Nutzer schafft.

8.6a Warum Präsenz kein Widerspruch zur Zero-Knowledge-Architektur ist

Auf den ersten Blick könnte die Präsenzanzeige wie ein Fremdkörper in einer ansonsten konsequent auf Datensparsamkeit ausgelegten Architektur wirken — tatsächlich fügt sie sich nahtlos ein, weil sie architektonisch genauso aufgebaut ist wie jede andere Nachricht: Ein „Ich bin online“-Signal ist technisch nichts anderes als ein weiterer Envelope-Typ (`kind: 'presence'`, Kapitel 8.2a), der wie jede andere 1:1-Nachricht über die bestehende Ratchet-Sitzung Ende-zu-Ende-verschlüsselt und über den Relay weitergeleitet wird (Kapitel 21.1). Der Relay sieht dabei genau dasselbe wie bei jeder anderen Nachricht: dass zwei Konto-Kennungen miteinander kommunizieren, aber nicht, dass es sich inhaltlich um ein Präsenz-Update handelt. Erst der jeweils empfangende Client entschlüsselt das Envelope und interpretiert dessen Inhalt als Statusänderung. Präsenz ist damit kein Sonderfall, der die Zero-Knowledge-Architektur durchbricht, sondern lediglich eine weitere Anwendung derselben, bereits etablierten Verschlüsselungsinfrastruktur.

8.7 „Sitzung verbrennen“: sofortige, unwiderrufliche Löschung

Ein zusätzlicher, bewusst auffällig gestalteter Knopf erlaubt es, einen kompletten Chat mit einem doppelten Klick (zur Bestätigung) sofort und unwiderruflich zu löschen. Bei echten 1:1-Kontakten wird dabei nicht nur der Nachrichtenverlauf entfernt, sondern auch die zugrundeliegende Verschlüsselungssitzung und der Kontakt selbst — ein späterer erneuter Kontakt erfordert einen komplett neuen Schlüsselaustausch (Kapitel 15), es bleibt buchstäblich nichts von der alten Sitzung übrig, an das ein späterer Angreifer anknüpfen könnte. Diese Funktion orientiert sich am „ephemeral by design“-Prinzip anonymer Werkzeuge wie OnionShare: Nach dem Verbrennen existiert kein Beweis mehr, dass die Konversation stattgefunden hat.

8.7a Die doppelte Bestätigung als bewusstes UI-Muster

Die Zwei-Klick-Bestätigung der Burn-Funktion (8.7) ist eine bewusste Designentscheidung gegen zwei entgegengesetzte Fehlerarten: Ein einzelner, sofort auslösender Klick würde das Risiko versehentlicher, nicht rückgängig machbarer Löschungen (etwa durch einen Fehltipp) unnötig erhöhen; ein klassischer

Bestätigungsdialog (ein Pop-up-Fenster mit „Wirklich löschen? Ja/Nein“) lässt sich dagegen erfahrungsgemäß leicht unreflektiert wegklicken, da Nutzerinnen und Nutzer solche Dialoge aus anderen Anwendungen gewohnheitsmäßig schnell bestätigen, ohne den Inhalt bewusst zu lesen. Die gewählte Lösung — ein zeitlich begrenztes Bestätigungsfenster von wenigen Sekunden direkt am ursprünglichen Knopf, mit sichtbar verändertem Erscheinungsbild (Warnfarbe, veränderter Text) — verlangt eine bewusste zweite Handlung an derselben Stelle, ohne die Reibung eines separaten Dialogfensters, und folgt damit demselben Prinzip konsistenter Warnfarben-Verwendung, das in Kapitel 42a für das Alarmsystem beschrieben wurde.

8.1a Ein Envelope-Beispiel für eine Emoji-Reaktion

Um die in 8.1 beschriebenen Anwendungs-Events konkreter zu machen: Reagiert Ben mit einem  auf Annas Nachricht, entsteht (vereinfacht, vor der Verschlüsselung) ein Envelope mit `kind: 'reaction'`, das über dieselbe bestehende Ratchet-Sitzung läuft wie eine normale Textnachricht:

```
{
  kind: 'reaction',
  targetMsgId: 'msg_annas_nachricht_id',
  emoji: 'thumbsup',
  reactionOp: 'add',
  chatId: 'chat_mit_anna', chatKind: 'direct',
  ... // header, ct wie bei jeder anderen 1:1-Nachricht
}
```

Der Relay behandelt dieses Envelope, wie in Teil III, Kapitel 21.1 beschrieben, technisch nicht anders als eine normale Textnachricht — er leitet es anhand von `chatId` weiter, ohne `targetMsgId` oder `emoji` inhaltlich auszuwerten. Erst Annas Client interpretiert nach der Entschlüsselung, dass es sich um eine Reaktion auf eine bestimmte, bereits vorhandene Nachricht handelt, und aktualisiert die lokale Anzeige entsprechend.

8.1b Nachrichtenreihenfolge bei gleichzeitigem Versand

Eine praktische Frage, die bei jedem Chat-System auftritt: Was passiert, wenn Anna und Ben nahezu gleichzeitig Nachrichten senden, bevor die jeweils andere Nachricht angekommen ist? RenkerVault löst dies nicht durch eine global synchronisierte Uhr (die zwischen unterschiedlichen Geräten ohnehin nie perfekt übereinstimmt), sondern durch die Kombination aus dem in Kapitel 16.4 beschriebenen Double-Ratchet-Zähler (`n`, die Position innerhalb der jeweiligen Sendekette) und einem lokalen Zeitstempel (`ts`) je Envelope. Jedes Gerät kann dadurch aus dem Header einer eingehenden Nachricht ableiten, an welcher Position innerhalb der Konversation sie steht, unabhängig von möglichen kleinen Abweichungen der Systemuhren beider Geräte. Bei zwei nahezu zeitgleich gesendeten, voneinander unabhängigen Nachrichten (beide Seiten senden, ohne die Nachricht der Gegenseite bereits empfangen zu haben) werden beide Nachrichten unabhängig verarbeitet und in der Oberfläche in der Reihenfolge angezeigt,

in der sie lokal eintreffen — ein in praktisch jedem asynchronen Chat-System übliches, unvermeidbares Verhalten, das keine Sicherheitseigenschaft betrifft, sondern rein die Darstellungsreihenfolge.

9. Gruppen und Kanäle

9.1 Gruppen: gemeinsamer Schlüssel statt einzelner Sitzungen

Für Gruppen wäre es theoretisch möglich, für jedes Mitgliederpaar eine eigene Double-Ratchet-Sitzung zu unterhalten — bei n Mitgliedern wären das $n \cdot (n-1)/2$ parallele Sitzungen, was mit wachsender Gruppengröße schnell unpraktikabel wird. RenkerVault verwendet stattdessen einen Ansatz, der in der Fachliteratur „Sender Keys“ genannt wird: Ein gemeinsamer, zufällig erzeugter Gruppenschlüssel wird einmalig über die bereits bestehenden, individuellen 1:1-Sitzungen an jedes Mitglied verteilt — die Verteilung selbst ist damit Ende-zu-Ende-verschlüsselt, obwohl die eigentlichen Gruppennachrichten danach mit dem gemeinsamen Schlüssel arbeiten. Die vollständige technische Herleitung folgt in Teil III, Kapitel 17.

9.1a Ein Zahlenbeispiel zur Skalierung

Um die in 9.1 genannte Formel $n \cdot (n-1)/2$ konkret einzuordnen: Bei einer kleinen Familiengruppe von 5 Mitgliedern wären das $5 \cdot 4/2 = 10$ parallele Sitzungen — noch überschaubar. Bei einer mittelgroßen Gruppe von 50 Mitgliedern (etwa ein Sportverein) wären es bereits $50 \cdot 49/2 = 1.225$ Sitzungen, und bei einer großen Community von 500 Mitgliedern stiege die Zahl auf $500 \cdot 499/2 = 124.750$ parallele Double-Ratchet-Sitzungen — eine Zahl, die sowohl den Speicherbedarf auf jedem einzelnen Gerät als auch die Anzahl der bei jeder Nachricht individuell zu verschlüsselnden Kopien unpraktikabel in die Höhe treiben würde. Der in 9.1 beschriebene Sender-Keys-Ansatz umgeht dieses quadratische Wachstum vollständig: Unabhängig von der Gruppengröße existiert nur ein einziger, gemeinsamer Gruppenschlüssel pro Epoche (Kapitel 17.2) — die einzige mit der Gruppengröße wachsende Größe ist die Anzahl der einmaligen Schlüsselverteilungen bei einer Epochenerneuerung (Kapitel 17.3), nicht die Anzahl dauerhaft zu unterhaltender Sitzungen.

9.2 Mitgliederänderungen und Schlüsselrotation

Tritt ein neues Mitglied bei oder verlässt jemand die Gruppe (freiwillig oder durch Entfernen), wird automatisch eine neue „Epoche“ mit einem frischen Gruppenschlüssel erzeugt und an alle verbleibenden Mitglieder neu verteilt. Ein entferntes Mitglied besitzt den neuen Schlüssel nicht und kann daher keine ab diesem Zeitpunkt gesendeten Nachrichten mehr lesen — auch wenn es zuvor Zugriff auf den alten Gruppenschlüssel hatte.

9.3 Granulare Admin-Rechte

Innerhalb einer Gruppe lassen sich Rechte granular vergeben: Wer darf senden, wer darf neue Mitglieder einladen, wer darf Mitglieder entfernen, wer darf Nachrichten anpinnen. Diese Rechte werden nicht nur in der Oberfläche durchgesetzt, sondern zusätzlich auf Datenebene geprüft — ein Sicherheits-Review während der Entwicklung deckte auf, dass eine frühere Version die Sendeberechtigung für Broadcast-Kanäle nur in der Oberfläche, nicht aber im zugrundeliegenden Datenfluss erzwang, wodurch ein programmatischer Umweg (etwa über die Weiterleiten-Funktion) die Beschränkung hätte umgehen können. Der Fix bestand darin, die Berechtigungsprüfung an die Stelle zu verlegen, an der Nachrichten tatsächlich verschlüsselt und versendet werden, statt sie nur vor der Anzeige des Sende-Buttons zu prüfen.

9.3a Die allgemeine Lehre aus diesem Fund: „Security at the Boundary“

Dieser während der Entwicklung gefundene und behobene Fehler illustriert ein allgemeines Prinzip sicherer Softwarearchitektur, das über RenkerVault hinaus gilt: Sicherheitsprüfungen (Berechtigungen, Validierung, Rate-Limits) müssen an der Stelle im Datenfluss verankert sein, an der eine Aktion tatsächlich **wirksam** wird — nicht an einer Stelle, die lediglich üblicherweise, aber nicht zwingend durchlaufen wird, bevor die Aktion eintritt. Eine Berechtigungsprüfung, die nur den sichtbaren „Senden“-Button deaktiviert, schützt nicht vor jedem denkbaren Codepfad, der dieselbe zugrundeliegende Sende-Funktion aufruft (etwa Weiterleiten, automatisierte Erinnerungen oder zukünftige, zum Zeitpunkt der ursprünglichen Prüfung noch nicht existierende Funktionen).

Dasselbe Prinzip findet sich an anderen Stellen in RenkerVault bereits konsequent umgesetzt: Die HMAC-Integritätsprüfung des Vaults (Kapitel 11) läuft nicht nur beim Öffnen der Anwendung, sondern bei jedem Zugriff; die Rate-Limits des Relays (Kapitel 10.2) greifen auf Ebene der WebSocket-Verbindung selbst, nicht nur auf Ebene einzelner Oberflächen-Aktionen des Clients, der ohnehin nicht die vertrauenswürdige Instanz für sicherheitsrelevante Durchsetzung sein kann (schließlich könnte ein Angreifer eine modifizierte Client-Version verwenden, die solche Oberflächen-Beschränkungen von vornherein nicht implementiert).

9.4 Broadcast-Kanäle: ein bewusst schwächeres Vertrauensmodell

Kanäle funktionieren wie Gruppen (Owner/Admins senden, Abonnenten empfangen read-only), unterscheiden sich aber in einem wichtigen Punkt im Vertrauensmodell: Bei großen Kanälen mit vielen Abonnenten ist der Kanal-Schlüssel zwangsläufig breit verteilt, und der Sender kennt strukturell die Liste aller Abonnenten. Kanäle sind damit eher als „verschlüsselter Broadcast“ zu verstehen denn als vertrauliche Punkt-zu-Punkt-Kommunikation — dieser Unterschied wird in der App-Dokumentation bewusst nicht verschleiert.

9.4b Warum Verschlüsselung bei Kanälen trotzdem sinnvoll bleibt

Angeichts der in 9.4 beschriebenen breiten Schlüsselverteilung könnte man fragen, ob eine Verschlüsselung bei Broadcast-Kanälen überhaupt einen praktischen Nutzen hat, wenn ohnehin viele Personen den Schlüssel kennen. Die Antwort: Ja, aus zwei Gründen. Erstens schützt die Verschlüsselung weiterhin zuverlässig gegen den **Relay-Betreiber** und jeden externen Netzwerkbeobachter (Kapitel 21) — diese kennen den Kanal-Schlüssel zu keinem Zeitpunkt, unabhängig davon, wie viele Abonnenten ihn kennen. Zweitens bleibt die Verschlüsselung auch gegenüber Außenstehenden wirksam, die zwar wissen, dass ein Kanal existiert, aber selbst nicht abonniert sind und daher den aktuellen Epochenschlüssel nicht besitzen. Die in 9.4 beschriebene Einschränkung betrifft ausschließlich das Vertrauensverhältnis **innerhalb** der Abonentenschaft selbst (jeder Abonnent könnte den Kanalinhalt theoretisch an Dritte weitergeben) — nicht die Schutzwirkung gegenüber Relay und externen Beobachtern, die unverändert der in Teil III beschriebenen vollen Ende-zu-Ende-Verschlüsselung entspricht.

9.4a Rechtevergleich: Gruppen vs. Kanäle im Überblick

Die in Kapitel 9.3/9.4 beschriebenen Berechtigungsmodelle lassen sich tabellarisch gegenüberstellen:

Aktion	Gruppe (normal)	Kanal (Broadcast)
Nachrichten senden	Alle Mitglieder (sofern nicht eingeschränkt)	Nur Owner/Admins
Nachrichten lesen	Alle Mitglieder	Alle Abonnenten
Mitglieder einladen	Owner/Admins (konfigurierbar)	Nur Owner/Admins
Mitglieder entfernen	Owner/Admins (konfigurierbar)	Nur Owner/Admins
Nachrichten anpinnen	Owner/Admins (konfigurierbar)	Nur Owner/Admins
Sender kennt Empfängerliste	Ja (kleine, meist bekannte Runde)	Ja, strukturell (Kapitel 9.4)
Typisches Vertrauensmodell	Vertrauliche Gruppenkommunikation	„Verschlüsselter Broadcast“ (Kapitel 9.4)

Diese Gegenüberstellung macht den in Kapitel 9.4 beschriebenen Unterschied im Vertrauensmodell noch deutlicher sichtbar: Technisch verwenden beide dieselbe Sender-Keys-lite-Verschlüsselung (Kapitel 17), unterscheiden sich aber grundlegend darin, wie viele Personen typischerweise Sende- und Verwaltungsrechte haben.

10. Das Einbruchsalarm-System

10.1 Warum ein eigenes Alarmsystem, wenn die Verschlüsselung schon sicher ist?

Verschlüsselung schützt die Übertragung und Speicherung von Daten — sie schützt nicht davor, dass jemand versucht, sich unbefugten Zugriff auf ein Gerät oder Konto selbst zu verschaffen (etwa durch Diebstahl des Geräts, Erraten der Passphrase oder Zwang). Genau diese Lücke schließt das Einbruchsalarm-System: Es erkennt aktiv Anzeichen eines Einbruchsversuchs und macht sie sofort sichtbar, statt sich ausschließlich auf die kryptographische Unmöglichkeit eines Angriffs zu verlassen.

10.1a Präventive vs. detektive Sicherheitsmaßnahmen

In der klassifizierenden Sprache der IT-Sicherheit lässt sich dieser Unterschied präzise fassen: Man unterscheidet zwischen **präventiven** Maßnahmen, die einen Angriff von vornherein verhindern sollen (etwa die in Teil III beschriebene Verschlüsselung, die einen erfolgreichen Zugriff auf Klartext mathematisch ausschließt), und **detektiven** Maßnahmen, die einen bereits stattfindenden oder versuchten Angriff erkennen und melden, ohne ihn zwangsläufig von vornherein technisch unmöglich zu machen. Die meisten in Teil III beschriebenen RenkerVault-Eigenschaften sind präventiv (Kapitel 24.6e listet sie in der Rückverfolgbarkeitstabelle). Das in diesem Kapitel beschriebene Einbruchsalarm-System ergänzt diese präventiven Maßnahmen um eine detektive Schicht — beide Kategorien schließen sich nicht aus, sondern ergänzen sich: Prävention senkt die Wahrscheinlichkeit eines erfolgreichen Angriffs, Detektion verkürzt die Zeit, bis ein dennoch unternommener Angriffsversuch bemerkt wird, und ermöglicht eine Reaktion (Kapitel 39.5, Vorfall-Reaktionsplan), bevor größerer Schaden entsteht.

10.2 Brute-Force-Erkennung

Nach fünf fehlgeschlagenen Entsperr- bzw. Anmeldeversuchen in kurzer Zeit sperrt sich das Konto für 60 Sekunden, ein deutlich sichtbarer roter Vollbild-Alarm erscheint, und der Vorfall wird im Security-Log protokolliert. Dieselbe Logik ist serverseitig gespiegelt: Der Relay begrenzt die Anzahl der Nachrichten pro Sekunde und Verbindung (Rate-Limiting) und sendet bei einer Sperrung eine Broadcast-Benachrichtigung an alle registrierten Geräte des betroffenen Kontos.

10.2a Warum lokale und serverseitige Brute-Force-Erkennung sich ergänzen, nicht duplizieren

Auf den ersten Blick könnte die doppelte Absicherung (lokal auf dem Gerät und serverseitig im Relay) redundant wirken — tatsächlich schützen beide Ebenen gegen unterschiedliche Angriffsvektoren. Die lokale Erkennung (Kapitel 10.2) greift, wenn jemand versucht, die Passphrase direkt am physischen Gerät zu erraten (Kapitel 11.2 macht dies durch Argon2id zusätzlich rechenintensiv). Die serverseitige Erkennung (Kapitel 20) greift dagegen bei einem gänzlich anderen Angriffsszenario: einem Versuch, sich gegenüber dem Relay als ein bestimmtes Konto auszugeben, ohne physischen Gerätezugriff — etwa durch wiederholtes Erraten oder Fälschen von Signaturen im Challenge-Response-Verfahren (Kapitel 20.2). Da beide

Angriffsszenarien unterschiedliche Angreifer-Voraussetzungen haben (physischer Gerätezugriff vs. reiner Netzwerkzugriff), ergänzen sich die beiden Schutzebenen, anstatt sich zu überschneiden — ein Beispiel für das Prinzip mehrschichtiger Verteidigung („Defense in Depth“, bereits in Kapitel 44.6c im Kontext der systemd-Sandbox erwähnt), bei dem verschiedene Schutzmaßnahmen unterschiedliche Angriffspfade abdecken, statt sich auf eine einzelne, vermeintlich ausreichende Barriere zu verlassen.

10.3 Erkennung neuer und veränderter Geräte

Wie in Kapitel 7.3 beschrieben, muss jedes neue Gerät manuell bestätigt werden, bevor es Zugriff erhält. Zusätzlich löst eine bereits bekannte Geräte-Kennung, die sich plötzlich mit einem anderen kryptographischen Schlüssel meldet, einen **KEY_MISMATCH**-Alarm aus — ein starkes Indiz dafür, dass jemand versucht, sich als bereits vertrautes Gerät auszugeben.

10.3a Der Unterschied zwischen einem neuen Gerät und einem geänderten Schlüssel

Diese beiden in 10.3 beschriebenen Erkennungsmechanismen betreffen zwei unterschiedliche Szenarien, die auseinandergehalten werden sollten. Ein **neues Gerät** (mit einer bislang unbekannten Geräte-Kennung) ist ein erwartbarer, legitimer Vorgang, der lediglich eine manuelle Bestätigung erfordert — etwa wenn eine Nutzerin tatsächlich ein neues Smartphone in Betrieb nimmt. Ein **KEY_MISMATCH** dagegen betrifft eine bereits bekannte, zuvor bestätigte Geräte-Kennung, die sich plötzlich mit einem abweichenden kryptographischen Schlüssel meldet — ein Vorgang, der unter normalen Umständen gar nicht auftreten sollte, da ein Gerät seinen privaten Signing-Key (Kapitel 14.3) nach der Ersteinrichtung nicht von selbst ändert. Ein **KEY_MISMATCH** ist daher ein deutlich stärkeres Alarmsignal als eine bloße Meldung über ein neues Gerät und wird entsprechend prominenter behandelt (Kapitel 39.5, Vorfall-Reaktionsplan Fall B) — die Unterscheidung zwischen beiden Fällen ist ein Beispiel dafür, wie das Einbruchsalarm-System unterschiedliche Anomalietypen unterschiedlich stark gewichtet, statt jede Abweichung pauschal gleich zu behandeln.

10.4 Manipulationserkennung der lokalen Datenbank

Die komplette lokale Datenbank ist mit einem HMAC (Hash-based Message Authentication Code, siehe Teil III, Kapitel 16) gegen Manipulation abgesichert. Diese Prüfung läuft sowohl beim Entsperren als auch bei jedem weiteren Zugriff. Schlägt die Prüfung fehl, wird ein Alarm ausgelöst, und optional lässt sich ein automatischer Lockdown aktivieren, der nur noch den Alarm-Bildschirm anzeigt — die eigentlichen Daten bleiben in diesem Zustand vollständig unzugänglich.

10.4a Was eine Manipulation in der Praxis auslösen könnte

Damit dieser Schutzmechanismus nicht abstrakt bleibt, ein Blick auf realistische Szenarien, die einen solchen Manipulationsalarm auslösen würden: ein Angreifer mit kurzzeitigem physischem Gerätezugriff, der versucht, gezielt einzelne Datensätze in der lokalen Datenbank zu verändern (etwa um eine bestimmte Nachricht unbemerkt zu entfernen, ohne den kompletten Tresor neu zu verschlüsseln); ein fehlerhaftes Backup- oder Synchronisationswerkzeug, das die Datenbankdatei versehentlich unvollständig oder in falscher Reihenfolge wiederherstellt; oder ein Datenträgerfehler (etwa ein beschädigter Speichersektor), der einzelne Bytes der gespeicherten Datei unbemerkt verändert. In allen drei Fällen — böswillig, durch Fremdsoftware verursacht, oder rein technisch bedingt — greift dieselbe HMAC-Prüfung (Kapitel 44.5) und meldet die Abweichung, ohne zwischen Ursachen zu unterscheiden. Das ist eine bewusste Designentscheidung: Aus Sicht des Schutzmechanismus ist nicht relevant, **warum** die Daten von ihrem erwarteten, integren Zustand abweichen, sondern lediglich, **dass** sie es tun — die betroffene Person kann die Ursache anschließend selbst einschätzen (Kapitel 39.5, Vorfall-Reaktionsplan Fall C).

10.5 Duress-PIN: der Zwangsfall

Für den Fall, dass jemand gezwungen wird, sein Gerät zu entsperren (etwa unter Druck oder Nötigung), lässt sich eine separate, geheime Notfall-PIN einrichten. Die Eingabe dieser PIN öffnet eine leere, harmlos aussehende Fake-Ansicht — der echte Tresor mit den tatsächlichen Chats bleibt vollständig verschlossen und unangetastet. Wichtig für das Design: Es wird bewusst **kein** sichtbares Alarm-Ereignis protokolliert, wenn die Duress-PIN verwendet wird — ein Log-Eintrag könnte in genau der Zwangssituation, die die Funktion eigentlich abfedern soll, gegen die betroffene Person verwendet werden oder ihre Notlage verschärfen.

10.5a Die Fake-Ansicht muss plausibel, nicht nur leer sein

Ein Designdetail, das über die reine „leere Ansicht“ hinausgeht: Eine komplett leere App ohne jeden Chat könnte selbst verdächtig wirken, gerade wenn eine zur Entsperrung zwingende Person bereits weiß, dass die betroffene Person die App regelmäßig nutzt. Die Wirksamkeit einer Duress-Funktion hängt letztlich davon ab, dass die Fake-Ansicht in der jeweiligen Zwangssituation plausibel genug wirkt, um weitere Nachfragen zu vermeiden — eine vollkommen leere App bei einer Person, deren Nutzung des Dienstes bereits bekannt ist, könnte diesen Zweck verfehlen. Dies ist eine grundsätzliche, nur teilweise technisch lösbare Grenze jeder Duress-Funktion, nicht nur bei RenkerVault: Je überzeugter der Zwang ausübende Akteur von der Existenz „echter“ Daten ist, desto höher die Anforderung an die Plausibilität der gezeigten Alternative. Nutzerinnen und Nutzer mit besonders hohem Schutzbedarf (Kapitel 39.2) sollten dies bei der Einschätzung berücksichtigen, statt sich blind auf eine technische Lösung für ein grundsätzlich auch soziales und situatives Problem zu verlassen.

10.6 Das Security-Log

Jedes sicherheitsrelevante Ereignis (Sperrungen, neue Geräte, Manipulationsversuche, manuelle Schlüsselerneuerungen) wird mit Zeitstempel in einem lokalen, jederzeit einsehbaren Security-Log festgehalten. Dieses Log ist selbst Teil der verschlüsselten lokalen Datenbank (Kapitel 11) und damit ebenfalls vor unbefugtem Zugriff geschützt.

10.6a Ein durchgespielter Alarm-Vorfall

Um die Zusammenarbeit der in 10.2-10.6 beschriebenen Mechanismen zu illustrieren, ein durchgespieltes Beispiel: Jemand findet ein verlorenes, gesperrtes Smartphone mit installierter RenkerVault-App und versucht, sich Zugang zu verschaffen.

```
Versuch 1-4: Falsche Passphrase eingegeben -> normale Fehlermeldung, kein Alarm
Versuch 5: Fuenfter Fehlversuch innerhalb kurzer Zeit
          -> 60-Sekunden-Lockout aktiviert (Kapitel 10.2)
          -> roter Vollbild-Alarm erscheint
          -> Eintrag im lokalen Security-Log (Kapitel 10.6)
          -> Falls das Geraet online ist: Benachrichtigung an alle
              anderen registrierten Geraete des Kontos (Kapitel 10.2)
Nach Ablauf: Weitere Eingabeversuche moeglich, Zaehler beginnt erneut
```

Meldet sich der rechtmäßige Kontoinhaber währenddessen von einem anderen, bereits vertrauten Gerät aus an, sieht er die Sicherheitsbenachrichtigung sofort (Kapitel 10.3) und kann über die Geräteverwaltung das verlorene Gerät remote abmelden (Kapitel 7.3) — selbst wenn es dem Finder später gelänge, die korrekte Passphrase zu erraten, wäre das Gerät zu diesem Zeitpunkt bereits als nicht mehr vertrauenswürdig markiert und ohne Zugriff auf zukünftige Nachrichten.

10.6b Die Analogie zu klassischen Intrusion-Detection-Systemen

In der traditionellen IT-Sicherheit von Unternehmensnetzwerken ist ein „Intrusion Detection System“ (IDS) eine etablierte, jahrzehntealte Kategorie von Sicherheitswerkzeugen, die Netzwerkverkehr oder Systemaktivität auf verdächtige Muster überwacht und bei Auffälligkeiten Alarm schlägt — üblicherweise eingesetzt zusätzlich zu, nicht anstelle von, präventiven Maßnahmen wie Firewalls oder Verschlüsselung. Das in Kapitel 10 beschriebene Einbruchsalarm-System von RenkerVault überträgt dieses seit Langem in der Unternehmens-IT etablierte Prinzip auf eine Endnutzer-Chat-Anwendung — ein Bereich, in dem vergleichbare, aktive Erkennungsmechanismen bei den in Kapitel 24 verglichenen Messengern bislang unüblich sind, die sich überwiegend auf rein präventive Verschlüsselung ohne begleitende Erkennungsschicht verlassen. Die Übertragung dieses Prinzips von der Unternehmens- auf die Endnutzerebene ist eine der Kerninnovationen, die RenkerVault von reinen Verschlüsselungs-Messengern unterscheidet (siehe auch die zusammenfassende Vergleichstabelle in Kapitel 24.6).

10.6c Warum das Security-Log selbst kein Angriffsziel werden kann

Ein Log, das sicherheitsrelevante Ereignisse aufzeichnet, ist in klassischen IT-Systemen manchmal selbst ein attraktives Angriffsziel: Ein Angreifer, der ein Log manipulieren oder löschen kann, kann damit Spuren seines eigenen Einbruchs verwischen. Bei RenkerVault ist dieses Risiko strukturell gemindert, da das Security-Log (10.6) Teil desselben verschlüsselten und HMAC-integritätsgeschützten Datenbestands ist wie der übrige Vault-Inhalt (Kapitel 11.1): Ein Angreifer, der versuchen würde, einzelne Log-Einträge zu löschen oder zu verändern, ohne den korrekten Master-Key zu besitzen, würde damit zwangsläufig die HMAC-Prüfung des gesamten Datenblobs zum Scheitern bringen (Kapitel 44.5) — was seinerseits einen Manipulationsalarm auslöst (Kapitel 10.4). Der Versuch, das Log zu manipulieren, würde also paradoxerweise genau den Alarm auslösen, den der Angreifer durch die Manipulation eigentlich verhindern wollte.

11. Lokale Datenhaltung (At-Rest-Verschlüsselung)

11.1 Die Schlüsselkette vom Passwort zum Datensatz

Der komplette lokale Zustand der App — Nachrichten, Kontakte, Sitzungsschlüssel, Security-Log — wird verschlüsselt gespeichert, niemals im Klartext. Die Ableitungskette dafür:

```
Passphrase --Argon2id(64 MiB, t=4, p=1)--> Key-Encryption-Key (KEK)
KEK --AES-256-GCM--> entschlüsselt/verschlüsselt einen
                        zufaellig erzeugten Master-Key (32 Byte)
Master-Key --AES-256-GCM--> verschlüsselt den kompletten Vault-Zustand
HKDF(Master-Key, "mac") --> separater MAC-Key
MAC-Key --HMAC-SHA256--> ueber den Daten-Ciphertext (Manipulationserkennung)
```

Die Trennung zwischen KEK und Master-Key hat einen praktischen Vorteil: Möchte man später die Passphrase ändern, muss nur der (kleine) verschlüsselte Master-Key neu verschlüsselt werden — nicht der gesamte, potenziell große Datenbestand.

11.1a Envelope Encryption: ein etabliertes Muster über RenkerVault hinaus

Die in 11.1 beschriebene zweistufige Schlüsselstruktur (ein Schlüssel, der einen zweiten Schlüssel verschlüsselt, der wiederum die eigentlichen Daten schützt) ist als „Envelope Encryption“ (Umschlag-Verschlüsselung) ein etabliertes, weit über RenkerVault hinaus verbreitetes Muster — es findet sich in praktisch identischer Form in den Schlüsselverwaltungsdiensten großer Cloud-Anbieter sowie in Festplattenverschlüsselungssystemen der gängigen Betriebssysteme. Der Name rührt daher, dass der Master-Key wie ein Brief in einem „Umschlag“ (dem KEK-verschlüsselten Wrap) steckt: Um den Brief zu lesen, muss zunächst der Umschlag geöffnet werden, aber der Umschlag selbst lässt sich unabhängig vom Inhalt austauschen. Der praktische Nutzen dieses Musters zeigt sich besonders bei der in 11.1

genannten Passphrase-Änderung: Ohne Envelope Encryption müsste bei einer Passphrase-Änderung der gesamte, potenziell mehrere Megabyte große verschlüsselte Chatverlauf komplett neu verschlüsselt werden — mit Envelope Encryption genügt es, den wenige Dutzend Byte großen, verschlüsselten Master-Key neu zu verpacken, was auch auf einem leistungsschwachen Mobilgerät praktisch verzögerungsfrei geschieht.

11.2 Warum Argon2id und nicht einfach SHA-256?

Ein naiver Ansatz („Passwort einfach hashen“) wäre unsicher, weil moderne Hardware (insbesondere Grafikkarten) einfache Hash-Funktionen milliardenfach pro Sekunde berechnen kann — ein gestohlener, verschlüsselter Datenbestand wäre dann in kurzer Zeit per Brute-Force zu knacken. Argon2id ist absichtlich so konstruiert, dass es viel Arbeitsspeicher benötigt (RenkerVault verwendet 64 MiB) und mehrere Iterationen durchläuft (aktuell 4), was die parallele Berechnung auf spezialisierter Hardware drastisch verteuert, ohne die Entsperrzeit für die legitime Nutzerin oder den legitimen Nutzer spürbar zu verlangsamen.

11.2b Argon2i, Argon2d und Argon2id im Vergleich

Argon2 existiert in drei Varianten, deren Unterschied für ein vollständiges Verständnis der Wahl in RenkerVault relevant ist. Argon2d nutzt einen datenabhängigen Speicherzugriff, der maximale Resistenz gegen GPU-Angriffe bietet, aber dadurch theoretisch anfällig für sogenannte „Timing-Seitenkanalangriffe“ (Kapitel 23.4) sein könnte, da der Speicherzugriffsmuster indirekt vom verarbeiteten Geheimnis abhängt. Argon2i nutzt dagegen einen datenunabhängigen Speicherzugriff, der resistent gegen solche Seitenkanalangriffe ist, dafür aber eine geringere Resistenz gegen spezialisierte Hardware-Angriffe bietet. Argon2id — die von RenkerVault verwendete Variante (Kapitel 44.2) — kombiniert beide Ansätze: Die erste Hälfte der Berechnung nutzt das datenunabhängige Muster von Argon2i (Schutz der anfänglichen, besonders sensiblen Verarbeitungsschritte gegen Seitenkanalangriffe), die zweite Hälfte das datenabhängige Muster von Argon2d (maximale Härte gegen spezialisierte Hardware). Diese hybride Konstruktion ist der Grund, warum Argon2id 2015 als Gewinner der Password Hashing Competition für die meisten praktischen Anwendungsfälle empfohlen wurde und heute den De-facto-Standard darstellt.

11.2a Speicherhärte: warum gerade Arbeitsspeicher der Flaschenhals ist

Der in 11.2 erwähnte hohe Speicherbedarf von Argon2id (64 MiB in RenkerVaults Konfiguration) ist kein Zufall, sondern eine gezielte Designentscheidung der zugrundeliegenden Kryptographie: Spezialisierte Angreifer-Hardware wie GPUs (Grafikkarten) oder gar eigens angefertigte ASICs (anwendungsspezifische Chips) sind bei einfachen, nur rechenintensiven (aber speicherarmen) Hash-Funktionen extrem im Vorteil, weil sie tausende Berechnungen parallel in eigenen Recheneinheiten

durchführen können. Bei einer **speicherintensiven** Funktion wie Argon2id trifft dieser Vorteil auf eine praktische Grenze: Jede parallele Berechnung benötigt gleichzeitig ihre eigenen 64 MiB Arbeitsspeicher, und Arbeitsspeicher lässt sich anders als reine Rechenkerne nicht beliebig vervielfachen, ohne dass die Kosten linear mitwachsen. Ein Angreifer, der Argon2id mit tausendfacher Parallelität knacken wollte, bräuchte entsprechend auch die tausendfache Menge an Arbeitsspeicher — was den Kostenvorteil spezialisierter Hardware gegenüber einem gewöhnlichen Prozessor stark einschränkt, verglichen mit einer rein rechenintensiven, aber speicherarmen Alternative.

11.3 Der Duress-Modus auf Datenebene

Technisch betrachtet ist der in Kapitel 10.5 beschriebene Duress-Modus kein Trick auf Oberflächenebene, sondern auf Datenebene verankert: Die Duress-PIN hasht zu einem separaten Argon2id-Hash, der niemals denselben Master-Key wie die reguläre Passphrase entsperrt. Die Fake-Ansicht zeigt schlicht einen leeren, plausibel wirkenden Zustand, während der echte, verschlüsselte Datenbestand technisch unverändert und unentschlüsselt auf dem Gerät verbleibt.

11.3a Warum eine Duress-Funktion mehr als eine Nischenfunktion ist

Eine Duress-PIN mag auf den ersten Blick wie eine exotische, selten benötigte Funktion wirken — die in Teil I, Kapitel 3.4 genannte Gruppe der Betroffenen häuslicher Gewalt und Stalking-Opfer zeigt jedoch, dass Zwangssituationen, in denen eine Person zur Entsperrung ihres Geräts genötigt wird, keineswegs nur ein theoretisches Randszenario darstellen. In einer solchen Situation bietet eine gewöhnliche, einzelne Passphrase keinen Schutz: Die betroffene Person müsste entweder die Entsperrung verweigern (mit möglicherweise gefährlichen Konsequenzen) oder den vollständigen Zugriff auf ihre echte Kommunikation preisgeben. Die Duress-PIN (Kapitel 10.5) schafft eine dritte Option, die in beiden Fällen zusätzlichen Schutz bietet: eine augenscheinlich normale Entsperrung, die aber keine sensiblen Inhalte offenlegt. Diese Funktion wurde bewusst nicht als optionales Extra, sondern als gleichwertiger Bestandteil des Sicherheitsmodells behandelt (Kapitel 2), da die Zielgruppe, für die sie am wichtigsten ist, oft nicht dieselbe ist wie die Zielgruppe, die eine Chat-App typischerweise selbst öffentlich als „Sicherheitsfunktion“ bewirbt.

12. Multi-Plattform: Web, Desktop, Mobile

12.1 Eine Codebasis, drei Auslieferungswege

RenkerVault ist als React/TypeScript-Anwendung geschrieben und läuft in drei Ausprägungen, ohne dass die eigentliche Kryptographie- oder Netzwerklogik jemals dupliziert werden müsste:

Plattform	Technologie	Kurzbeschreibung
Web	Vite Dev-/Prod-Build	Läuft direkt im Browser, keine Installation nötig
Desktop (Windows)	Tauri (Rust + WebView2)	Native Shell um dieselbe React-App; kleinerer Angriffsvektor als Electron, da kein Node-Kontext im Renderer-Prozess
Mobile (Android)	Capacitor	Wrapped dieselbe Web-Codebasis 1:1 in eine native WebView-App

12.1a Browser-Kompatibilität der verwendeten Web-Standards

Da die Web-Variante (12.1) auf mehreren im Laufe dieses Buches erwähnten, vergleichsweise modernen Web-Standards beruht, folgt eine Einordnung zur Kompatibilität: Die WebCrypto-API (Kapitel 13.1, 20.4a) für AES-GCM ist seit vielen Jahren in allen gängigen, aktuell gepflegten Browsern (Chromium-basierte Browser, Firefox, Safari/WebKit) verfügbar und gilt als stabiler, breit unterstützter Web-Standard. Die [@noble/post-quantum](#)-Bibliothek (Kapitel 18.3) für den ML-KEM-768-Hybrid-Handshake (Kapitel 18) benötigt dagegen keine spezielle Browser-Unterstützung, da sie als reines JavaScript/WebAssembly implementiert ist und nicht auf eine native, browserseitige Post-Quantum-API angewiesen ist — ein Vorteil, da native Post-Quantum-Unterstützung in Browsern zum Zeitpunkt der Erstellung dieses Buches noch nicht flächendeckend standardisiert ist. Für den produktiven Einsatz empfiehlt sich dennoch, wie bei jeder Web-Anwendung, der Einsatz aktuell gehaltener, regelmäßig aktualisierter Browser — ältere, nicht mehr gepflegte Browser-Versionen erhalten auch für bereits etablierte Standards wie WebCrypto keine Sicherheitsupdates mehr.

12.2 Warum Tauri statt Electron?

Sowohl Tauri als auch Electron erlauben es, eine Web-Anwendung als natives Desktop-Programm zu verpacken. Der wesentliche Sicherheitsunterschied: Electron bündelt eine komplette Chromium-Instanz plus eine Node.js-Laufzeitumgebung mit vollem Dateisystem-/Betriebssystemzugriff direkt im Renderer-Prozess der Anwendung — eine Schwachstelle in der angezeigten Web-Oberfläche kann dadurch potenziell weitreichenden Systemzugriff erlangen. Tauri nutzt stattdessen die im Betriebssystem bereits vorhandene WebView-Komponente (unter Windows: WebView2) und ein separates, in Rust geschriebenes Backend, das explizit definierte, eng begrenzte Schnittstellen (sogenannte „Commands“) zur Verfügung stellt — die Angriffsfläche bleibt dadurch strukturell kleiner.

12.2a Ein zusätzlicher Vorteil: kleinere Auslieferungsgröße

Über die reine Sicherheitsbetrachtung hinaus hat die Tauri-Architektur einen praktischen Zusatzvorteil, der ebenfalls aus demselben strukturellen Unterschied folgt: Da Tauri

keine eigene Chromium-Instanz mitliefert, sondern die ohnehin bereits auf dem Betriebssystem vorhandene WebView-Komponente wiederverwendet, fällt die Installationsdatei einer Tauri-Anwendung typischerweise um eine Größenordnung kleiner aus als eine funktional vergleichbare Electron-Anwendung — ein Electron-Installer liegt oft im Bereich von 100 bis 200 MB (da praktisch eine vollständige Browser-Kopie mitgeliefert wird), während eine vergleichbare Tauri-Anwendung oft nur wenige MB benötigt, da lediglich der eigentliche Anwendungscode und das schlanke Rust-Backend enthalten sind. Für den in Kapitel 37.5 beschriebenen Inno-Setup-Installer bedeutet dies eine spürbar schnellere Downloadzeit und einen geringeren Speicherbedarf auf dem Zielsystem — ein für Endnutzer direkt spürbarer, praktischer Vorteil derselben Architektur-Entscheidung, die primär aus Sicherheitsgründen getroffen wurde.

12.3 Warum Capacitor statt einer Zweitimplementierung für Mobile?

Eine native mobile App komplett neu zu schreiben (etwa in Kotlin für Android) würde bedeuten, die komplette kryptographische Logik ein zweites Mal zu implementieren — mit dem doppelten Risiko, zwei unterschiedliche, potenziell inkonsistent gepflegte Implementierungen zu erhalten. Capacitor wrappt stattdessen dieselbe, bereits getestete Web-Codebasis 1:1 in eine native Android-Anwendung (technisch eine native WebView-Hülle um denselben JavaScript-Code) — die Kryptographie-Schicht aus Kapitel 6.1 bleibt dabei buchstäblich identisch.

12.3a Das „Einmal schreiben, überall testen“-Argument

Über die reine Vermeidung doppelter Implementierungsarbeit hinaus hat der Capacitor-Ansatz einen zusätzlichen sicherheitsrelevanten Vorteil: Da dieselbe Kryptographie-Schicht (Kapitel 6.1) auf allen drei Plattformen (Web, Desktop, Mobile) unverändert zum Einsatz kommt, muss auch nur eine einzige Implementierung getestet und im Zweifel extern auditiert werden (Kapitel 25.1a), statt drei separate, plattformspezifische Implementierungen einzeln absichern zu müssen. Diese Eigenschaft wird in der Softwareentwicklung manchmal als „Write Once, Test Everywhere“ bezeichnet — eine gefundene und behobene Schwachstelle in der gemeinsamen Kryptographie-Schicht behebt automatisch dasselbe Problem auf allen drei Plattformen gleichzeitig, ohne dass separate, plattformspezifische Korrekturen koordiniert werden müssten. Bei einer hypothetischen Zweitimplementierung in nativem Kotlin oder Swift bestünde dagegen das Risiko, dass ein und dieselbe Schwachstelle in einer der beiden Implementierungen behoben, in der anderen aber übersehen wird — ein zusätzliches, strukturelles Argument für den in Kapitel 12.1 beschriebenen Multi-Plattform-Ansatz.

12.4 Kompatibilität mit gehärteten Systemen (GrapheneOS)

Die Android-App hat keine Abhängigkeit von Google Play Services oder Firebase (kein Push-Benachrichtigungsdienst, keine Analytics-Software Development Kits) und funktioniert daher unverändert auf de-googelten, sicherheitsgehärteten Systemen wie GrapheneOS. Eine zusätzliche `network_security_config.xml`-Konfiguration erzwingt verschlüsseltes `wss://` für jeden Host außer dem eigenen Gerät (`localhost/127.0.0.1`/dem Android-Emulator-Alias) — unverschlüsseltes `ws://` ist damit strukturell nur auf dem eigenen Gerät überhaupt möglich, nicht nur eine Empfehlung in den Einstellungen.

12.4a Push-Benachrichtigungen: ein bewusst ausgesparter Kompromissbereich

Ein Punkt, der bei mobilen Messengern häufig zu einem schwer auflösbaren Zielkonflikt führt: Damit ein Smartphone auch bei geschlossener App sofort über eine neue Nachricht informieren kann, verwenden die meisten Apps den systemeigenen Push-Benachrichtigungsdienst des Betriebssystems (bei Android in der Regel Firebase Cloud Messaging). Ein solcher Dienst erfordert praktisch immer, dass zumindest ein minimaler Hinweis auf eine eingehende Nachricht („Es ist eine neue Nachricht angekommen“) über die Infrastruktur des Betriebssystemherstellers läuft — ein Metadaten-Leck, das strukturell außerhalb der Kontrolle des Relay-Betreibers liegt. RenkerVault verzichtet in der aktuellen Prototyp-Version bewusst auf eine solche Anbindung (auch das ermöglicht die in Kapitel 12.4 beschriebene GrapheneOS-Kompatibilität ohne Firebase-Abhängigkeit) — mit der praktischen Konsequenz, dass die App eine bestehende Verbindung zum Relay offen halten oder regelmäßig neu aufbauen muss, um neue Nachrichten zu bemerken, was auf mobilen Geräten tendenziell mehr Akku verbraucht als ein systemeigener Push-Mechanismus. Dieser Kompromiss — etwas höherer Akkuverbrauch gegen ein zusätzliches, außerhalb der eigenen Kontrolle liegendes Metadaten-Leck — ist bewusst zugunsten der Privatsphäre gewählt und in der Projektdokumentation offen benannt.

12.4b Backups und Export: ein ungelöstes Spannungsfeld

Ein weiterer, in der aktuellen Prototyp-Version bewusst nicht vollständig gelöster Bereich betrifft Backups: Ein verschlüsseltes Gerätebackup (etwa eine automatische Cloud-Sicherung des Betriebssystems) kann den in Teil II, Kapitel 11 beschriebenen lokalen Vault-Datenbestand mit sichern — bleibt dieses Backup selbst unverschlüsselt oder wird es mit einem vom Nutzer nicht kontrollierten Schlüssel verschlüsselt (wie es bei manchen Cloud-Backup-Diensten großer Betriebssystemhersteller der Fall sein kann), entstünde dadurch ein Umweg um die eigentliche At-Rest-Verschlüsselung. RenkerVault selbst bietet aktuell keine eigene, zusätzlich verschlüsselte Export-/Backup-Funktion an — Nutzerinnen und Nutzer mit hohem Schutzbedarf sollten daher, wie in der Checkliste in Kapitel 39.1 vermerkt, aktiv prüfen, ob automatische Betriebssystem-Backups den eigenen Datenbestand in einer Form sichern, die dem

beabsichtigten Schutzniveau entspricht, und diese gegebenenfalls deaktivieren oder durch eine selbst kontrollierte, verschlüsselte Sicherungsstrategie ersetzen.

12.4c Design und Theme-Presets

Auch wenn dieses Buch primär auf Architektur und Kryptographie fokussiert, sei die Gestaltungsebene kurz erwähnt, da sie funktional mit dem Sicherheitsmodell verknüpft ist (vergleiche auch Kapitel 42a). Die App bietet sechs Theme-Presets (Default, Dark, OLED, Light, Cyber, Minimal) sowie einen frei wählbaren Akzentfarben-Regler. Der OLED-Modus verdient dabei eine technische Anmerkung: Er verwendet ein nahezu reines Schwarz als Hintergrundfarbe, was auf Geräten mit OLED-Bildschirmen (bei denen einzelne Pixel für echtes Schwarz komplett abgeschaltet werden, anders als bei LCD-Hintergrundbeleuchtung) einen messbaren Akkuvorteil bringt — ein kleines, aber für Nutzerinnen und Nutzer mobiler Geräte praktisch relevantes Detail, das über reine Ästhetik hinausgeht.

12.5 Der Windows-Installer

Für die Desktop-Variante existiert ein mit Inno Setup erzeugter Installer, der die Tauri-Anwendung sowie optional eine Kopie des Relay-Servers bündelt (für Nutzerinnen und Nutzer, die einen eigenen Server auf demselben Rechner betreiben möchten), inklusive Start-Menü- und Desktop-Verknüpfung.

12a. Barrierefreiheit

12a.1 Warum Barrierefreiheit auch bei einem Sicherheitswerkzeug zählt

Ein Sicherheitswerkzeug, das nicht barrierefrei bedienbar ist, schließt Menschen mit Behinderungen faktisch von einem grundlegenden Schutzangebot aus — ein Aspekt, der in der Diskussion um Verschlüsselungs-Apps oft zu kurz kommt, obwohl gerade die in Teil I, Kapitel 3.4 genannten besonders schutzbedürftigen Gruppen (etwa Betroffene häuslicher Gewalt) sich mit der allgemeinen Bevölkerung hinsichtlich des Anteils an Menschen mit Behinderungen nicht unterscheiden.

12a.2 Konkrete Umsetzung

Da RenkerVault als React-Webanwendung gebaut ist (Kapitel 12.1), profitiert die Oberflächen-Schicht (Kapitel 6.1) von den etablierten Web-Standards für Barrierefreiheit (WAI-ARIA), die von Screenreadern (Vorlesesoftware für blinde und sehbehinderte Nutzerinnen und Nutzer) interpretiert werden können. Konkret bedeutet dies: interaktive Elemente wie Chat-Einträge sind als solche semantisch ausgezeichnet (etwa über `role="button"` mit zugehörigem Keyboard-Handler, statt sich ausschließlich auf

Mausklicks zu verlassen — eine Korrektur, die während eines internen Sicherheits- und Qualitäts-Reviews vorgenommen wurde, nachdem ein verschachteltes, ungültiges HTML-Element diese Zugänglichkeit zunächst beeinträchtigt hatte), Statusänderungen (etwa ein ausgelöster Alarm, Kapitel 10) werden nicht ausschließlich über Farbe vermittelt, sondern zusätzlich über Text, und die App unterstützt vollständige Tastaturbedienung ohne zwingende Maus- bzw. Touch-Interaktion.

12a.3 Eine bewusste Grenze: der Duress-Modus und Screenreader

Ein interessanter Sonderfall entsteht beim Duress-Modus (Kapitel 10.5): Ein Screenreader, der laut vorliest, was auf dem Bildschirm angezeigt wird, könnte in einer Zwangssituation selbst zu einem Informationsleck werden, wenn er den Unterschied zwischen echter und Fake-Ansicht hörbar kommentieren würde. RenkerVault behandelt beide Ansichten daher bewusst strukturell identisch in ihrer Screenreader-Ansage — es gibt keinen zusätzlichen, nur für die eine oder andere Ansicht spezifischen Hinweistext, der einem mithörenden Dritten verraten könnte, welche der beiden Ansichten gerade aktiv ist.

12a.4 Farbfehlsichtigkeit und die Grenzen reiner Farbcodierung

Über Screenreader-Kompatibilität hinaus berührt Barrierefreiheit auch visuelle Aspekte: Etwa acht Prozent der Männer und ein deutlich kleinerer Anteil der Frauen sind von einer Form der Farbfehlsichtigkeit betroffen (am häufigsten eine Rot-Grün-Schwäche). Da das in Kapitel 42a beschriebene Alarmsystem zentral auf der Farbe Rot als Warnsignal beruht, wäre eine ausschließlich farbbasierte Signalisierung für diesen Bevölkerungsanteil potenziell unwirksam. RenkerVault begegnet dem, indem jeder sicherheitsrelevante Zustand zusätzlich zur Farbe auch über Text, Symbolform (nicht nur Farbe) und im Fall des Vollbild-Alarms (Kapitel 10.2) über die vollständige Bildschirmgestaltung vermittelt wird, nicht über eine punktuelle farbige Markierung allein — eine Umsetzung des etablierten Grundsatzes, dass Information niemals ausschließlich über Farbe kodiert werden sollte, wenn sie sicherheitskritisch ist.

Kryptographie im Detail

Jede Design-Entscheidung mit Begründung — warum RenkerVault sicher ist

13. Symmetrische und asymmetrische Verschlüsselung im Detail

13.0a Blockchiffren und Betriebsarten: der Unterschied zwischen AES und AES-GCM

Bevor AES-256-GCM im Detail behandelt wird, eine wichtige begriffliche Klärung: „AES“ allein bezeichnet nur eine sogenannte „Blockchiffre“ — ein Verfahren, das jeweils einen Datenblock fester Größe (bei AES 128 Bit) verschlüsselt. Eine echte Nachricht ist praktisch immer länger als ein einzelner Block, weshalb eine „Betriebsart“ (englisch „Mode of Operation“) festlegen muss, wie mehrere Blöcke zu einer vollständigen Verschlüsselung kombiniert werden. Hier kommt das „GCM“ in AES-256-GCM ins Spiel (Kapitel 13.1): Es ist eine solche Betriebsart, die AES intern wie eine Stromchiffre nutzt (durch fortlaufende Verschlüsselung eines Zählers, der mit dem Klartext verknüpft wird) und dabei zusätzlich die in Kapitel 13.1 beschriebene Authentizitätsprüfung mitliefert. Andere Betriebsarten (etwa das ältere, in Kapitel 13.1 bereits als weniger sicher erwähnte CBC) kombinieren dieselben AES-Blöcke auf andere Weise, mit jeweils unterschiedlichen Sicherheits- und Performance-Eigenschaften. Diese Unterscheidung zwischen der zugrundeliegenden Blockchiffre (AES) und der darüberliegenden Betriebsart (GCM) ist einer der am häufigsten missverstandenen Aspekte praktischer Kryptographie — die Aussage „wir verwenden AES“ allein sagt noch nichts über die tatsächliche Sicherheit einer Implementierung aus, solange die verwendete Betriebsart nicht mitgenannt wird.

13.1 AES-256-GCM: das Arbeitspferd

Jede tatsächliche Nachricht in RenkerVault wird mit AES-256-GCM verschlüsselt — einem symmetrischen Verfahren, das seit 2001 als „Advanced Encryption Standard“ vom US-amerikanischen National Institute of Standards and Technology (NIST) genormt ist und heute praktisch überall verwendet wird, von Bankdienstleistungen bis zu Betriebssystem-Festplattenverschlüsselung. Die Zahl 256 bezeichnet die Schlüssellänge in Bit — ein Angreifer, der den Schlüssel per reiner Rateversuchsmethode (Brute Force) finden wollte, müsste im schlechtesten Fall 2 hoch

256 Möglichkeiten durchprobieren, eine Zahl, die die Anzahl der Atome im beobachtbaren Universum um viele Größenordnungen übersteigt.

Das Kürzel „GCM“ steht für „Galois/Counter Mode“ — eine Betriebsart, die zwei Eigenschaften gleichzeitig liefert: Vertraulichkeit (der Inhalt ist ohne Schlüssel nicht lesbar) und Authentizität (jede nachträgliche Veränderung des Chiffretexts, und sei es nur ein einziges Bit, wird beim Entschlüsseln zuverlässig erkannt und die Entschlüsselung schlägt kontrolliert fehl, statt stillschweigend falsche Daten zu liefern). Diese eingebaute Manipulationserkennung ist ein wichtiger Unterschied zu älteren Betriebsarten (etwa dem reinen „CBC-Mode“), bei denen Manipulationen am Chiffretext unter Umständen unbemerkt zu einer veränderten, aber scheinbar gültigen Entschlüsselung führen konnten.

13.1a Warum 256 Bit und nicht 128?

AES existiert in mehreren Schlüssellängen-Varianten (128, 192 und 256 Bit). Bereits 128 Bit gilt nach heutigem Kenntnisstand gegen klassische Computer als praktisch unangreifbar per Brute Force. Der Grund, warum RenkerVault dennoch die längere 256-Bit-Variante wählt, liegt in der in Kapitel 18.1 beschriebenen Quantencomputer-Bedrohung: Der Grover-Algorithmus (ein Quantenalgorithmus, der sich von dem in Kapitel 18.1 beschriebenen Shor-Algorithmus unterscheidet) könnte die effektive Sicherheit einer symmetrischen Verschlüsselung theoretisch auf die Quadratwurzel ihrer ursprünglichen Stärke reduzieren — 128 Bit fielen dadurch auf ein effektives Niveau von etwa 64 Bit, was gegen zukünftige Quantencomputer nicht mehr als ausreichend gelten würde. 256 Bit fallen unter derselben Annahme auf ein effektives Niveau von etwa 128 Bit — nach heutigem Kenntnisstand weiterhin praktisch unangreifbar. Die Wahl von AES-256 statt AES-128 ist damit selbst schon eine Form von Post-Quantum-Vorsorge, unabhängig von dem in Kapitel 18 beschriebenen hybriden Schlüsselaustausch, der ein separates, davon zu unterscheidendes Problem (den asymmetrischen Schlüsselaustausch, nicht die symmetrische Verschlüsselung selbst) adressiert.

13.1b Sicherheitsniveaus im Vergleich, tabellarisch

Zur Einordnung, wie unterschiedliche in diesem Buch genannte Verfahren und Schlüssellängen in ein gemeinsames Sicherheitsniveau (angegeben in „Bit Sicherheit“, also der ungefähren Anzahl an Rechenschritten, die ein Angreifer im schlechtesten Fall benötigen würde, als Zweierpotenz) übersetzt werden können:

Verfahren	Schlüssel-/Parametergröße	Klassisches Sicherheitsniveau	Nach Grover/Shor (Quantencomputer)
AES-128	128 Bit	~128 Bit	~64 Bit (Grover)
AES-256	256 Bit	~256 Bit	~128 Bit (Grover)

Verfahren	Schlüssel-/Parametergröße	Klassisches Sicherheitsniveau	Nach Grover/Shor (Quantencomputer)
X25519	256 Bit (32 Byte)	~128 Bit	~0 Bit (Shor bricht es vollständig)
RSA-2048	2048 Bit	~112 Bit	~0 Bit (Shor bricht es vollständig)
ML-KEM-768	siehe Kapitel 44b	~192 Bit (NIST-Kategorie 3)	~192 Bit (gilt als quantenresistent)

Diese Tabelle verdeutlicht auf einen Blick, warum Kapitel 18 einen Hybrid-Ansatz statt eines reinen Ersatzes von X25519 durch ML-KEM wählt: X25519 und RSA fallen unter der Annahme eines ausreichend leistungsfähigen Quantencomputers auf ein Sicherheitsniveau von praktisch null, während ML-KEM (nach heutigem Kenntnisstand) von dieser speziellen Bedrohung unberührt bleibt — die Kombination beider Verfahren (Kapitel 18.4) stellt sicher, dass das resultierende Sicherheitsniveau nie unter das jeweils bessere der beiden Einzelverfahren fällt.

13.2 Nonces: warum derselbe Schlüssel nie zweimal auf dieselbe Weise verwendet werden darf

AES-256-GCM benötigt neben dem Schlüssel einen sogenannten „Nonce“ (number used once) — einen Wert, der für jede einzelne Verschlüsselung mit demselben Schlüssel garantiert einmalig sein muss. Wird versehentlich derselbe Nonce mit demselben Schlüssel zweimal verwendet, bricht die Sicherheit des gesamten Verfahrens in einer für Angreifer praktisch ausnutzbaren Weise zusammen (es lassen sich dann Teile beider Klartexte durch Verknüpfung der beiden Chiffretexte rekonstruieren). RenkerVault begegnet diesem Risiko strukturell, indem für jede einzelne Nachricht im Double Ratchet (Kapitel 16) ohnehin ein komplett neuer, einmaliger symmetrischer Schlüssel erzeugt wird — das Nonce-Wiederverwendungsproblem entsteht in diesem Design gar nicht erst, weil praktisch nie derselbe Schlüssel zweimal zum Einsatz kommt.

13.2a AES-GCM im Vergleich zu ChaCha20-Poly1305

AES-256-GCM ist nicht das einzige etablierte AEAD-Verfahren („Authenticated Encryption with Associated Data“, also Verschlüsselung mit eingebauter Authentizitätsprüfung). Eine häufig genannte Alternative ist ChaCha20-Poly1305, die unter anderem von Signal und WireGuard verwendet wird. Der praktische Unterschied liegt vor allem in der Hardware-Abhängigkeit: AES profitiert auf den meisten modernen Prozessoren von speziellen, in Hardware gegessenen AES-Beschleunigungsbefehlen (AES-NI auf x86, entsprechende Befehle auf ARM), wodurch es dort besonders schnell ist; ChaCha20-Poly1305 wurde dagegen so entworfen, dass es auch ohne solche Spezialbefehle in Software durchgehend schnell und, was sicherheitsrelevant ist, gleichmäßig zeitkonstant bleibt. Da RenkerVault über die WebCrypto-API (Teil III,

Kapitel 13.1) auf plattformoptimiertes AES-GCM zugreift, das auf praktisch allen Zielgeräten (moderne Smartphones, Desktop-Prozessoren) Hardware-Beschleunigung vorfindet, ist AES-256-GCM hier die pragmatischere Wahl, ohne dass dies eine Aussage über eine grundsätzliche Unsicherheit von ChaCha20-Poly1305 impliziert — beide Verfahren gelten nach heutigem Kenntnisstand als gleichermaßen sicher.

13.2b Lieferketten-Sicherheit: Vertrauen in Abhängigkeiten

Eine oft übersehene Dimension von „keine eigene Kryptographie erfinden“ (13.3) ist die Frage, wem man stattdessen vertraut: Jede eingebundene Bibliothek ([@noble/curves](#), [@noble/hashes](#), [@noble/post-quantum](#), [hash-wasm](#)) ist selbst eine Lieferkettenabhängigkeit, deren Kompromittierung (etwa durch einen manipulierten Software-Update oder einen kompromittierten Paket-Registry-Account des Bibliotheksautors) das gesamte darauf aufbauende System gefährden könnte — ein Angriffsmuster, das in der Sicherheitsforschung als „Supply-Chain-Angriff“ bezeichnet wird und in den vergangenen Jahren wiederholt real beobachtet wurde (in verschiedenen Programmiersprachen-Ökosystemen, nicht spezifisch bei den hier verwendeten Bibliotheken). Ein vollständiger Schutz dagegen ist praktisch nicht erreichbar, ohne auf externe Kryptographie-Bibliotheken vollständig zu verzichten — was, wie in 13.3 begründet, seinerseits ein größeres Risiko darstellen würde. Sinnvolle Abmilderungen, die für einen späteren Produktivbetrieb empfohlen werden, umfassen: Versions-Pinning (exakte, nicht automatisch aktualisierte Abhängigkeitsversionen), Prüfsummen-Verifikation beim Installieren von Abhängigkeiten, und die Bevorzugung von Bibliotheken mit mehreren unabhängigen Maintainern statt einer einzelnen Person.

13.3 Warum keine eigene Verschlüsselung erfunden wird

Eine der wichtigsten Design-Entscheidungen in RenkerVault, die sich durch das gesamte Projekt zieht: Es wird an keiner Stelle ein eigenes kryptographisches Primitiv (ein eigener Verschlüsselungsalgorithmus, eine eigene Hash-Funktion) entwickelt. Stattdessen kommen ausschließlich öffentlich auditierte, in unabhängigen Sicherheitsprüfungen untersuchte Bibliotheken zum Einsatz: [@noble/curves](#) und [@noble/hashes](#) (beide von der Kryptographie-Community breit genutzte, spezifikationskonforme JavaScript-Implementierungen), [hash-wasm](#) (für Argon2id) sowie die im Browser bzw. in der Laufzeitumgebung bereits eingebaute WebCrypto-API für AES-GCM.

Der Grund für diese Zurückhaltung ist eine der bekanntesten Faustregeln der angewandten Kryptographie: „Don't roll your own crypto“ (erfinde keine eigene Kryptographie). Selbst erfahrene Softwareentwickler übersehen bei einer selbst geschriebenen Implementierung eines kryptographischen Verfahrens regelmäßig subtile Fehler — etwa in der Behandlung von Randfällen, im Zeitverhalten (das Informationen über geheime Werte preisgeben kann, sogenannte „Timing-Angriffe“), oder in der korrekten Erzeugung von Zufallswerten. Ein etabliertes, von vielen unabhängigen

Expertinnen und Experten über Jahre geprüfetes Verfahren ist praktisch immer sicherer als eine frische Eigenentwicklung, selbst wenn diese auf denselben mathematischen Grundlagen beruht.

13.3a Speichersicherheit: Warum die Implementierungssprache mitzählt

Neben der reinen mathematischen Korrektheit eines kryptographischen Verfahrens spielt auch die Programmiersprache, in der es implementiert wird, eine praktische Sicherheitsrolle. Historisch wurde eine erhebliche Zahl realer Sicherheitslücken in Kryptographie-Bibliotheken nicht durch mathematisch fehlerhafte Algorithmen verursacht, sondern durch Speicherzugriffsfehler auf Implementierungsebene (etwa Pufferüberläufe in C oder C++, bei denen Daten über die eigentlich vorgesehene Speichergrenze hinaus geschrieben oder gelesen werden) — der bekannteste Fall ist vermutlich „Heartbleed“ (2014), eine Schwachstelle in der weit verbreiteten OpenSSL-Bibliothek, die durch einen fehlenden Grenzwert-Check entstand und es erlaubte, beliebigen Arbeitsspeicherinhalt eines Servers auszulesen, obwohl der zugrundeliegende kryptographische Algorithmus selbst nicht fehlerhaft war.

RenkerVault ist vollständig in TypeScript geschrieben, das zur JavaScript-Familie gehört — einer Sprache, die von Haus aus speichersicher ist: Es gibt keine manuelle Zeigerarithmetik, keine Möglichkeit, versehentlich über die Grenzen eines Arrays hinauszuschreiben, und die Speicherverwaltung erfolgt automatisch durch die Laufzeitumgebung (Garbage Collection). Diese Eigenschaft schließt eine ganze Klasse von Implementierungsfehlern von vornherein aus, die in klassischen C/C++-basierten Kryptographie-Bibliotheken historisch wiederholt aufgetreten sind. Der Kompromiss: JavaScript/TypeScript bietet weniger feingranulare Kontrolle über Zeitverhalten auf sehr niedriger Ebene, was bei bestimmten, extrem auf Timing-Angriffe spezialisierten Szenarien theoretisch relevant sein könnte — ein Grund, warum die in Kapitel 13.3 beschriebenen, extern auditierten Bibliotheken bewusst so geschrieben sind, dass sie unabhängig von Compiler-Optimierungen ein konstantes Zeitverhalten anstreben.

13.3b Zur Herkunft und Prüfgeschichte von @noble/curves

Da in Kapitel 13.3 wiederholt von „auditerten Bibliotheken“ die Rede ist, lohnt eine genauere Einordnung, was das im Fall von @noble/curves konkret bedeutet. Die @noble-Bibliotheksfamilie (curves, hashes, post-quantum) wird von Paul Miller entwickelt und zeichnet sich durch mehrere in der Kryptographie-Community geschätzte Eigenschaften aus: Der Code ist bewusst in reinem, dependency-freiem TypeScript gehalten (keine externen Laufzeitabhängigkeiten, die selbst wieder eine Lieferkette darstellen würden, vergleiche Kapitel 13.2b), die Implementierungen sind gegen offizielle Testvektoren der jeweiligen Standards (etwa RFC 7748 für X25519) verifiziert, und die Bibliotheken waren Gegenstand mehrerer bezahlter, unabhängiger Sicherheitsaudits durch spezialisierte Kryptographie-Prüffirmen, deren Berichte

öffentlich einsehbar sind. Diese Kombination — offener Quellcode, standardkonforme Testvektoren, unabhängige bezahlte Audits — entspricht dem in Kapitel 25.1a beschriebenen Idealbild einer geprüften kryptographischen Komponente, auch wenn, wie dort erläutert, die Komposition dieser Bausteine zu RenkerVaults eigenem Protokoll (Kapitel 16.5) davon zu unterscheiden bleibt.

14. Elliptische Kurven: X25519 und Ed25519

14.1 Was eine elliptische Kurve mathematisch ist

Eine elliptische Kurve ist, vereinfacht gesagt, die Menge aller Punkte, die eine bestimmte mathematische Gleichung der Form $y^2 = x^3 + ax + b$ erfüllen, kombiniert mit einer speziell definierten Rechenoperation, mit der man zwei Punkte auf der Kurve „addieren“ kann und wieder einen Punkt auf derselben Kurve erhält. Diese Rechenoperation lässt sich beliebig oft wiederholen (ein Punkt „mit sich selbst addiert“, dann wieder mit dem Ergebnis, und so weiter) — das nennt man „Skalarmultiplikation“.

Der entscheidende kryptographische Trick: Aus einem bekannten Startpunkt und einer bekannten Anzahl von Wiederholungen lässt sich der resultierende Punkt leicht berechnen. Der umgekehrte Weg — aus dem Startpunkt und dem Endpunkt herauszufinden, wie oft die Operation wiederholt wurde — ist nach heutigem mathematischem Kenntnisstand praktisch nicht effizient lösbar, sobald die beteiligten Zahlen groß genug gewählt sind. Dieses Problem heißt „diskreter Logarithmus auf elliptischen Kurven“ und bildet die Sicherheitsgrundlage von X25519 und Ed25519.

14.1a Eine geometrische Veranschaulichung der Punktaddition

Für Leserinnen und Leser, die sich die in 14.1 beschriebene Rechenoperation geometrisch vorstellen möchten: Auf einer klassisch über den reellen Zahlen gezeichneten elliptischen Kurve lässt sich die „Addition“ zweier Punkte A und B geometrisch konstruieren, indem man eine gerade Linie durch A und B zieht, den dritten Schnittpunkt dieser Linie mit der Kurve bestimmt, und diesen Punkt an der x-Achse spiegelt — das gespiegelte Ergebnis ist per Definition A+B. Bei der Addition eines Punktes mit sich selbst (also A+A, der erste Schritt der in 14.1 beschriebenen wiederholten Skalarmultiplikation) wird statt einer Sekante die Tangente an die Kurve im Punkt A verwendet. Diese geometrische Konstruktion ist zwar für die in RenkerVault tatsächlich verwendete Kurve (die über einem endlichen Zahlenkörper statt über reellen Zahlen definiert ist, wo eine anschauliche Zeichnung nicht mehr möglich ist) nicht mehr direkt visualisierbar, vermittelt aber ein intuitives Verständnis dafür, warum diese Rechenoperation überhaupt als eine Art „Addition“ bezeichnet wird und bestimmte algebraische Eigenschaften (Kommutativität, Assoziativität) besitzt.

14.2 X25519: Schlüsselaustausch (Diffie-Hellman auf Curve25519)

X25519 ist die konkrete, 2005/2006 von Daniel J. Bernstein vorgeschlagene und seither breit standardisierte (u. a. in RFC 7748) elliptische Kurve, die RenkerVault für den Schlüsselaustausch verwendet. Der Name „Curve25519“ bezieht sich auf eine Primzahl nahe 2^{255} , die der Kurvgleichung zugrunde liegt. Zwei Eigenschaften machen Curve25519 besonders geeignet für ein Sicherheitsprodukt wie RenkerVault:

- **Geschwindigkeit und kompakte Schlüssel:** Ein X25519-Schlüssel ist nur 32 Byte lang, verglichen mit typischerweise mehreren Hundert Byte bei RSA-Schlüsseln vergleichbarer Sicherheit — ein spürbarer Vorteil für mobile Geräte mit begrenzter Rechenleistung und Bandbreite.
- **Robustheit gegen Implementierungsfehler:** Curve25519 wurde bewusst so konstruiert, dass viele klassische Fehlerquellen bei der Implementierung elliptischer Kurven (etwa ungültige Kurvenpunkte, die bei anderen Kurven eigens abgefangen werden müssen) strukturell praktisch ausgeschlossen sind — ein Entwurfsprinzip, das in der Kryptographie „Misuse Resistance“ genannt wird.

Die eigentliche Diffie-Hellman-Operation (benannt nach Whitfield Diffie und Martin Hellman, den Erfindern des asymmetrischen Schlüsselaustauschprinzips aus Teil I, Kapitel 5.2) auf X25519 funktioniert so: Anna kombiniert ihren eigenen privaten Schlüssel mit Bens öffentlichem Schlüssel und erhält einen gemeinsamen Geheimwert. Ben kombiniert seinen eigenen privaten Schlüssel mit Annas öffentlichem Schlüssel — und erhält, obwohl beide unterschiedliche Eingaben verwenden, **denselben** Geheimwert. Das ist die mathematische Eigenschaft, die den gesamten Schlüsselaustausch in Kapitel 15 ermöglicht.

14.2a Warum Curve25519 einer NIST-Kurve vorgezogen wurde

Neben Curve25519 existieren auch von NIST genormte elliptische Kurven (etwa die sogenannten „P-256“- oder „P-384“-Kurven), die in vielen älteren Sicherheitsstandards verwendet werden. RenkerVault verwendet dennoch Curve25519, aus Gründen, die über die in 14.2 genannte Geschwindigkeit hinausgehen: Die NIST-Kurven basieren auf Parametern, deren genaue Herleitung historisch nicht vollständig transparent veröffentlicht wurde, was in der Kryptographie-Community wiederholt Anlass zu Spekulationen gab, ob die Parameterwahl absichtlich manipuliert worden sein könnte, um eine unentdeckte Schwäche einzubauen (eine Sorge, die durch spätere, öffentlich dokumentierte Fälle absichtlich geschwächter Standardisierungsvorschläge bei anderen kryptographischen Verfahren zusätzlich befeuert wurde). Curve25519 wurde dagegen von Anfang an mit vollständig nachvollziehbarer, öffentlich begründeter Parameterherleitung („nichts up meinen Ärmel“, auf Englisch „nothing up my sleeve“) entwickelt — jeder Interessierte kann nachvollziehen, warum genau diese Zahlen gewählt wurden, ohne auf das Vertrauen in eine nicht näher erläuterte Normierungsentscheidung angewiesen zu sein. Dieses Transparenzprinzip passt unmittelbar zum in Kapitel 5.9a beschriebenen Kerckhoffs'schen Grundsatz öffentlicher Überprüfbarkeit.

14.3 Ed25519: digitale Signaturen

Ed25519 basiert auf derselben zugrundeliegenden Kurve wie X25519, ist aber für eine andere Aufgabe konstruiert: digitale Signaturen (siehe Teil I, Kapitel 5.8). RenkerVault verwendet Ed25519 ausschließlich für die passwortlose Authentifizierung gegenüber dem Relay-Server (Kapitel 20) — bewusst getrennt von den X25519-Schlüsseln, die für die eigentliche Nachrichtenverschlüsselung zuständig sind. Diese Trennung folgt der etablierten kryptographischen Praxis, Schlüssel niemals gleichzeitig für unterschiedliche Zwecke (Verschlüsselung und Signatur) zu verwenden, da bestimmte theoretische Angriffskonstruktionen genau eine solche Wiederverwendung ausnutzen könnten.

14.4a Ein Zahlenbeispiel zum Anfassen (mit kleinen, unsicheren Zahlen)

Die abstrakte Beschreibung aus Abschnitt 14.1 lässt sich mit stark vereinfachten, absichtlich winzigen (und damit kryptographisch völlig unsicheren, rein didaktischen) Zahlen konkret nachvollziehen — nicht auf einer elliptischen Kurve, sondern im noch einfacheren klassischen Diffie-Hellman-Schema, dessen Grundprinzip identisch ist:

Anna und Ben einigen sich öffentlich (es darf jeder mithören) auf zwei Zahlen: eine Primzahl $p = 23$ und eine „Basis“ $g = 5$. Diese beiden Zahlen sind kein Geheimnis.

```
Anna wählt geheime Zufallszahl a = 6 (bleibt bei Anna)
Ben   wählt geheime Zufallszahl b = 15 (bleibt bei Ben)

Anna berechnet und verschickt öffentlich: A = g^a mod p = 5^6 mod 23 = 8
Ben   berechnet und verschickt öffentlich: B = g^b mod p = 5^15 mod 23 = 19

Anna empfangt B=19 und berechnet: B^a mod p = 19^6 mod 23 = 2
Ben   empfangt A=8 und berechnet: A^b mod p = 8^15 mod 23 = 2

-> Beide erhalten denselben Wert 2, OHNE dass a oder b jemals uebertragen wurden.
```

Ein Angreifer, der sowohl p , g als auch die öffentlich übertragenen Werte $A=8$ und $B=19$ mithört, müsste — um den gemeinsamen Wert 2 selbst zu berechnen — entweder a oder b aus A bzw. B zurückrechnen. Bei diesen winzigen Zahlen ist das mit bloßem Ausprobieren in Sekunden möglich (weshalb dieses Beispiel ausdrücklich **nicht** sicher ist); bei den in RenkerVault tatsächlich verwendeten, 32 Byte langen X25519-Schlüsseln (Abschnitt 14.2) ist derselbe Rückrechnungsversuch nach heutigem Kenntnisstand praktisch unmöglich, weil der Zahlenraum astronomisch viel größer ist. Das Prinzip — öffentlicher Austausch von Werten, aus denen beide Seiten unabhängig denselben Geheimwert berechnen, ohne dass ein Mithörer dies ebenfalls könnte — ist in beiden Fällen identisch.

14.4 Warum nicht RSA?

RSA, benannt nach seinen Erfindern Rivest, Shamir und Adleman, ist das historisch älteste breit eingesetzte asymmetrische Verfahren und basiert auf einem anderen

mathematischen Problem (der Schwierigkeit, große Zahlen in ihre Primfaktoren zu zerlegen) als elliptische Kurven. RSA gilt weiterhin als sicher, benötigt aber für ein vergleichbares Sicherheitsniveau deutlich größere Schlüssel (2048 bis 4096 Bit statt 256 Bit bei X25519) und ist rechenintensiver. Für eine Chat-Anwendung, die auf möglichst vielen Geräten inklusive älterer Smartphones flüssig laufen soll, ist die Kombination aus kleineren Schlüsseln und schnelleren Berechnungen bei elliptischen Kurven der praktische Grund für die Wahl von X25519/Ed25519 statt RSA.

14.4b Ein konkretes Man-in-the-Middle-Szenario mit dem Zahlenbeispiel

Um zu verstehen, warum die in Kapitel 7.5 beschriebene Safety Number notwendig ist, lohnt sich ein Blick auf das Zahlenbeispiel aus 14.4a mit einem aktiven Angreifer namens Chris, der sich zwischen Anna und Ben schaltet:

Ohne Absicherung durch Safety Number:

```
Anna berechnet ihren Wert A=8 fuer Ben – schickt ihn ab
Chris faengt A=8 ab, ersetzt ihn durch seinen EIGENEN Wert C=15
Ben erhaelt C=15 (glaubt, es sei Annas Wert) und berechnet C^b mod p
-> Ben und Chris haben nun EINEN gemeinsamen Geheimwert

Ben berechnet seinen Wert B=19 fuer Anna – schickt ihn ab
Chris faengt B=19 ab, ersetzt ihn durch seinen EIGENEN Wert C=15
Anna erhaelt C=15 (glaubt, es sei Bens Wert) und berechnet C^a mod p
-> Anna und Chris haben nun EINEN ANDEREN gemeinsamen Geheimwert

Ergebnis: Chris kann nun jede Nachricht zwischen Anna und Ben mitlesen
UND unbemerkt veraendern, indem er sie jeweils mit dem einen Geheimwert
entschluesselt und mit dem anderen neu verschluesselt.
```

Der entscheidende Punkt: Rein kryptographisch betrachtet läuft in diesem Angriffsszenario alles „korrekt“ ab — jeder einzelne Diffie-Hellman-Schritt für sich ist mathematisch fehlerfrei. Das Problem liegt darin, dass weder Anna noch Ben verifiziert haben, dass die empfangenen Werte tatsächlich von der jeweils anderen Person stammen. Genau diese Lücke schließt die Safety Number (Kapitel 7.5): Da Annas und Bens tatsächliche Safety Number (berechnet aus ihren echten, unverfälschten öffentlichen Schlüsseln) nicht mit der Safety Number übereinstimmen würde, die jeweils mit Chris' eingeschleustem Schlüssel berechnet würde, würde ein manueller Vergleich über einen unabhängigen Kanal diesen Angriff sofort sichtbar machen — Anna und Ben würden zwei unterschiedliche Zahlenfolgen sehen, statt einer übereinstimmenden.

15. Initialer Schlüsselaustausch: X3DH-lite

15.1 Das Ausgangsproblem: Verschlüsseln, ohne dass die Gegenseite online sein muss

Ein zentrales Problem beim ersten Kontakt zweier Nutzer: Anna möchte Ben eine verschlüsselte Nachricht schicken, aber Ben ist gerade nicht online — sein Gerät ist ausgeschaltet oder ohne Internetverbindung. Ein naiver Diffie-Hellman-Austausch (Kapitel 14.2) würde erfordern, dass beide Seiten gleichzeitig online sind, um live Schlüssel auszutauschen. Die Lösung, die Signal unter dem Namen „X3DH“ (Extended Triple Diffie-Hellman) eingeführt hat und die RenkerVault in einer vereinfachten Variante („X3DH-lite“) übernimmt, löst dieses Problem durch vorab veröffentlichte Schlüssel.

15.0a Eine Analogie: der Briefkasten mit vorinstalliertem Schloss

Bevor die formale Herleitung in 15.2 beginnt, eine Alltagsanalogie zum Ausgangsproblem: Man kann sich Bens veröffentlichten Signed Prekey (15.2) wie ein Vorhängeschloss vorstellen, das Ben bereits im Voraus an seinem Briefkasten anbringt und dessen Kombination öffentlich aushängt, während nur er selbst den passenden Schlüssel zum Öffnen besitzt. Möchte Anna Ben etwas schicken, während er verreist ist, kann sie das Vorhängeschloss trotzdem benutzen, um ihre Sendung zu sichern — Ben muss dafür nicht anwesend sein, er braucht das Schloss nur vorab bereitgestellt zu haben. Erst wenn Ben zurückkehrt, öffnet er die Sendung mit seinem eigenen, geheimen Schlüssel. Dieses Bild macht den entscheidenden Unterschied zum klassischen Diffie-Hellman-Austausch (Kapitel 14.4a, wo sowohl Anna als auch Ben gleichzeitig anwesend sein und Werte live austauschen mussten) sofort greifbar: X3DH-lite verschiebt den zeitkritischen Teil des Austauschs vollständig auf die Vorbereitungsphase (das Anbringen des Schlosses), sodass der eigentliche Versand jederzeit, auch bei Abwesenheit der Gegenseite, stattfinden kann.

15.2 Die drei beteiligten Schlüssel

Damit Anna auch ohne Bens Online-Anwesenheit verschlüsseln kann, veröffentlicht Ben vorab zwei öffentliche Schlüssel beim Relay-Server:

- seinen **Identity Key** (Bens langfristige Identität), und
- seinen **Signed Prekey** (ein vorab erzeugter, ebenfalls veröffentlichter Schlüssel, der speziell für den asynchronen Erstkontakt gedacht ist).

Anna erzeugt bei der Kontaktaufnahme zusätzlich einen frischen, nur für diesen einen Vorgang gültigen **Ephemeral Key**. Aus der Kombination dieser drei Schlüsselpaare berechnen beide Seiten unabhängig voneinander denselben gemeinsamen Ausgangswert:

```
SK = HKDF-SHA256( DH(IK_a, IK_b) || DH(EK_a, SPK_b) )
```

```
IK_a  = Identity Key des Initiators (privat) / Identity Key der Gegenseite (öffentlich)
```

```
IK_b  = Identity Key der Gegenseite (privat, beim Empfänger) / (öffentlich, beim Sender)
```

```
EK_a  = frischer Ephemeral Key des Initiators (nur fuer diesen einen Erstkontakt)
```

```
SPK_b = veröffentlichter Signed Prekey der Gegenseite
```

```
DH(x, y) = Diffie-Hellman-Operation zwischen privatem Schlüssel x und öffentlichem Schlüssel y
HKDF-SHA256 = Schlüsselableitungsfunktion (siehe 15.4)
```

Zwei getrennte Diffie-Hellman-Berechnungen fließen also gemeinsam in den finalen Schlüssel ein — ein Prinzip, das im nächsten Abschnitt begründet wird.

15.2a Warum beide Seiten wirklich denselben Wert berechnen

Die in 15.2 gezeigte Formel wirkt zunächst asymmetrisch — Anna und Ben scheinen unterschiedliche Eingaben zu verwenden. Der Grund, warum am Ende dennoch beide denselben Wert **SK** erhalten, liegt in der bereits in Kapitel 14.2 hergeleiteten mathematischen Eigenschaft von Diffie-Hellman: $DH(IK_a_privat, IK_b_öffentlich)$ und $DH(IK_b_privat, IK_a_öffentlich)$ liefern denselben Wert, obwohl die eine Seite ihren eigenen privaten mit dem fremden öffentlichen Schlüssel kombiniert und die andere Seite umgekehrt. Anna berechnet also $DH(ihr_IK_privat, Bens_IK_öffentlich)$ und $DH(ihrem_frischen_EK_privat, Bens_SPK_öffentlich)$; Ben berechnet $DH(seinem_IK_privat, Annas_IK_öffentlich)$ und $DH(seinem_SPK_privat, Annas_mitgeschicktem_EK_öffentlich)$. Da für jedes der beiden Diffie-Hellman-Paare dieselbe „Vertauschungseigenschaft“ gilt, kommen am Ende bei beiden Seiten identische Zwischenwerte heraus, die anschließend durch dieselbe HKDF-Berechnung (Kapitel 15.4) zum selben **SK** verarbeitet werden — ohne dass Anna jemals Bens privaten Schlüssel oder umgekehrt kennen müsste.

15.3 Warum zwei Diffie-Hellman-Operationen statt einer?

Würde nur eine einzige Diffie-Hellman-Berechnung verwendet (etwa nur zwischen den beiden Identity Keys), hätte ein Angreifer, der später einen der beiden langfristigen Identity Keys kompromittiert, theoretisch die Möglichkeit, auch vergangene Sitzungsschlüssel zu rekonstruieren. Durch die Kombination mit einem frischen, nach Gebrauch verworfenen Ephemeral Key wird sichergestellt, dass selbst eine spätere Kompromittierung des langfristigen Identity Keys allein nicht ausreicht, um den ursprünglichen Sitzungsschlüssel zu rekonstruieren — ein erster Baustein der in Teil I, Kapitel 5.7 eingeführten Perfect Forward Secrecy, die im Double Ratchet (Kapitel 16) dann für jede einzelne Folgenachricht fortgesetzt wird.

15.4 HKDF: aus einem Geheimnis mehrere unabhängige Schlüssel ableiten

HKDF (HMAC-based Key Derivation Function) ist ein standardisiertes Verfahren (RFC 5869), das aus einem Eingabe-Geheimnis (hier: die Kombination der beiden Diffie-Hellman-Ergebnisse) einen oder mehrere kryptographisch unabhängige

Ausgabeschlüssel ableitet. „Unabhängig“ bedeutet: Selbst wenn ein Angreifer einen der abgeleiteten Schlüssel kennen würde, ließe sich daraus nicht auf das ursprüngliche Eingabe-Geheimnis oder auf andere, aus demselben Geheimnis abgeleitete Schlüssel zurückschließen. Dieses Prinzip wird im Double Ratchet (Kapitel 16) systematisch wiederverwendet, um aus einem einzigen „Root Key“ fortlaufend neue Nachrichtenschlüssel abzuleiten.

15.5 Bewusster Kompromiss: kein vollständiges X3DH

Das originale X3DH-Protokoll von Signal verwendet zusätzlich sogenannte „One-Time-Prekeys“ — Schlüssel, die nach einmaliger Verwendung verbraucht und nie wieder verwendet werden, wodurch selbst der Signed Prekey nicht zwangsläufig mehrfach eingesetzt werden muss. RenkerVault verzichtet in der aktuellen Prototyp-Version bewusst auf diese zusätzliche Schicht und verwendet stattdessen ausschließlich den wiederverwendbaren Signed Prekey — ein dokumentierter, in SECURITY.md offen benannter Kompromiss zugunsten einfacherer Serverlogik, der die Asynchronitäts-Garantien geringfügig schwächt (bei extrem vielen gleichzeitigen Erstkontakt-Anfragen auf denselben Prekey wäre die Schutzwirkung marginal geringer als bei vollem X3DH mit One-Time-Prekeys).

15.5a HMAC an einem winzigen Beispiel

Um das in Teil I, Kapitel 5.5 abstrakt beschriebene HMAC-Prinzip greifbarer zu machen, bevor es in 16.2 zentral wichtig wird: HMAC kombiniert einen geheimen Schlüssel mit einer Nachricht so, dass nur jemand mit Kenntnis des Schlüssels den korrekten Ausgabewert („Tag“) berechnen kann. Nehmen wir eine stark vereinfachte, nur didaktische Variante mit einer winzigen „Hash-Funktion“, die lediglich die Quersumme aller Zeichencodes einer Zeichenkette modulo 100 berechnet:

```
geheimer Schluessel K = "geheim42"
Nachricht M = "Hallo"

HMAC-artige Operation (stark vereinfacht):
  h1 = einfache_hashfunktion(K + M)      // K und M werden kombiniert
  h2 = einfache_hashfunktion(K + h1)     // Ergebnis wird nochmal mit K gemischt
  -> h2 ist das "Tag"
```

Ändert sich auch nur ein einziges Zeichen in M (z. B. „Hallo“ wird zu „Hallp“), ändert sich h1 komplett unvorhersehbar, und damit auch h2 — ein Angreifer ohne Kenntnis von K kann also weder das korrekte Tag zu einer manipulierten Nachricht berechnen noch aus einem beobachteten Tag auf K zurückschließen. Das in Teil III, Kapitel 44.2 gezeigte echte `hmacSha256` ersetzt die hier zur Veranschaulichung genutzte „einfache Hashfunktion“ durch SHA-256 und arbeitet mit kryptographisch signifikanten Schlüssellängen (32 Byte) statt eines kurzen Textpassworts — das Grundprinzip der Verkettung von Schlüssel und Nachricht bleibt aber identisch.

15.5b Formale Sicherheitsbegriffe: IND-CPA und IND-CCA

Für Leserinnen und Leser mit Informatik-Hintergrund, die tiefer in die formale Kryptographie einsteigen möchten, lohnt die Einordnung zweier Standardbegriffe, die in der akademischen Literatur zur Bewertung von Verschlüsselungsverfahren verwendet werden. „IND-CPA“ (Indistinguishability under Chosen-Plaintext Attack) bezeichnet die Sicherheitseigenschaft, dass ein Angreifer, selbst wenn er beliebige selbst gewählte Klartexte verschlüsseln lassen und die Ergebnisse studieren darf, keinerlei Information über einen ihm unbekannten Klartext aus dessen Chiffretext gewinnen kann — er kann also nicht einmal unterscheiden, welche von zwei möglichen Nachrichten tatsächlich verschlüsselt wurde. „IND-CCA“ (Indistinguishability under Chosen-Ciphertext Attack) verschärft diese Anforderung zusätzlich: Der Angreifer darf zusätzlich beliebige selbst gewählte Chiffretexte entschlüsseln lassen (mit Ausnahme des einen Ziel-Chiffretexts) und trotzdem keine verwertbare Information gewinnen.

AES-256-GCM (Kapitel 13.1) erfüllt nach aktuellem Kenntnisstand die stärkere IND-CCA-Eigenschaft, was direkt mit der in Kapitel 13.1 beschriebenen eingebauten Authentizitätsprüfung zusammenhängt: Ein Angreifer, der versucht, einen manipulierten Chiffretext entschlüsseln zu lassen, um daraus Rückschlüsse zu ziehen, scheitert bereits an der Integritätsprüfung, bevor er überhaupt ein verwertbares Ergebnis erhält. Diese formale Eigenschaft ist der mathematisch präzise Hintergrund dessen, was in Kapitel 13.1 informell als „Manipulationserkennung“ beschrieben wurde.

16. Der Double Ratchet Schritt für Schritt

16.1 Die Grundidee: zwei ineinandergreifende „Zahnräder“

Der Name „Double Ratchet“ (doppeltes Sperrrad bzw. Schaltwerk) beschreibt die Funktionsweise bildlich: Zwei ineinandergreifende Mechanismen sorgen dafür, dass sich Schlüssel mit jeder Nachricht fortlaufend und unumkehrbar weiterdrehen — wie ein Sperrrad, das sich nur in eine Richtung bewegen kann und nie zurückspringt.

- Der **Symmetric-Key-Ratchet** dreht sich mit jeder einzelnen gesendeten oder empfangenen Nachricht innerhalb eines Gesprächs weiter.
- Der **Diffie-Hellman-Ratchet** dreht sich seltener, immer dann, wenn eine Antwort der Gegenseite mit einem neuen öffentlichen Schlüssel eintrifft, und „erneuert“ dabei den zugrundeliegenden Root Key komplett.

16.1a Warum die Bezeichnung „Ratchet“ (Sperrrad) treffend gewählt ist

Die Namensgebung ist mehr als ein bildhafter Zufall: Ein mechanisches Sperrrad (etwa in einem Ratschenschlüssel oder einer Analoguhr) erlaubt Bewegung nur in eine

Richtung — ein Zurückdrehen ist konstruktionsbedingt unmöglich, ohne den Mechanismus gewaltsam zu zerstören. Genau diese Eigenschaft der Irreversibilität ist kryptographisch entscheidend: Da jeder neue Chain Key und jeder neue Message Key durch eine Einwegfunktion (HMAC, Kapitel 5.5) aus dem vorherigen abgeleitet wird, gibt es mathematisch keinen Weg, von einem späteren Zustand auf einen früheren zurückzuschließen — der „Ratchet“ kann sich nur vorwärts bewegen, niemals rückwärts. Diese Ein-Wege-Eigenschaft ist die exakte technische Grundlage der in Kapitel 16.2 beschriebenen Perfect Forward Secrecy: Ein Angreifer, der einen aktuellen Schlüsselzustand erbeutet, kann daraus prinzipbedingt keine vorherigen, bereits „weitergedrehten“ Zustände rekonstruieren.

16.2 Der Symmetric-Key-Ratchet im Detail

```
KDF_RK(rootKey, dhOut)    = HKDF-SHA256(dhOut, salt=rootKey) -> (rootKey', chainKey)
KDF_CK(chainKey)          = ( HMAC(chainKey, 0x02), HMAC(chainKey, 0x01) )
                           naechste Kette           Message Key

encrypt(plaintext):      mk = KDF_CK-Schritt;  ct = AES-256-GCM(mk, plaintext, aad=header)
decrypt(msg):            DH-Ratchet-Schritt falls neuer Sender-Public-Key im Header,
                        dann KDF_CK-Schritte bis zur passenden Nachrichten-Nummer
```

Bei jeder einzelnen gesendeten Nachricht wird aus der aktuellen „Chain Key“ genannten Zwischengröße über eine HMAC-Berechnung gleichzeitig zweierlei erzeugt: ein Message Key, mit dem genau diese eine Nachricht verschlüsselt wird, und ein neuer Chain Key für die nächste Nachricht. Der alte Chain Key wird danach sofort und unwiederbringlich verworfen. Da ein Message Key aus einem HMAC über den vorherigen Chain Key berechnet wird und HMAC praktisch nicht umkehrbar ist (Teil I, Kapitel 5.5), lässt sich aus einem bekannten Message Key nicht auf vorherige Message Keys zurückschließen — das ist die technische Umsetzung der in Teil I, Kapitel 5.7 beschriebenen Perfect Forward Secrecy.

16.2a Warum zwei verschiedene HMAC-Konstanten (0x02 und 0x01) verwendet werden

Ein Detail der in 16.2 gezeigten `KDF_CK`-Formel verdient eine genauere Erklärung: Die beiden HMAC-Berechnungen unterscheiden sich ausschließlich durch eine einzige, angehängte Konstante (`0x01` für den Message Key, `0x02` für den nächsten Chain Key). Dieser scheinbar kleine Unterschied ist entscheidend für die Sicherheit der gesamten Konstruktion: Da HMAC eine deterministische Funktion ist (dieselbe Eingabe erzeugt immer dieselbe Ausgabe), müssen die beiden aus demselben Chain Key abgeleiteten Werte durch unterschiedliche Eingaben erzeugt werden — andernfalls wären Message Key und der nächste Chain Key identisch, was die gesamte Ratchet-Konstruktion sicherheitstechnisch wertlos machen würde (ein Angreifer, der einen Message Key kennt, könnte dann automatisch auch den nächsten Chain Key und könnte damit alle zukünftigen Nachrichten entschlüsseln, was der in Kapitel 16.2 beschriebenen Perfect

Forward Secrecy fundamental widersprüche). Die beiden unterschiedlichen Konstanten stellen sicher, dass beide abgeleiteten Werte trotz identischer Eingangsgröße (dem gemeinsamen Chain Key) kryptographisch unabhängig voneinander sind.

16.3 Der Diffie-Hellman-Ratchet und Post-Compromise Security

Der zweite Mechanismus greift immer dann, wenn eine Antwort der Gegenseite eintrifft, die einen neuen, bisher unbekannten öffentlichen Schlüssel im Nachrichten-Header mitbringt. In diesem Fall wird eine neue Diffie-Hellman-Berechnung durchgeführt und daraus ein komplett neuer Root Key (und daraus wiederum ein neuer Chain Key) abgeleitet. Diese Eigenschaft heißt „Post-Compromise Security“: Selbst wenn ein Angreifer zu einem bestimmten Zeitpunkt Zugriff auf den aktuellen Sitzungszustand erlangt hätte, stellt bereits der nächste normale Antwort-Roundtrip der beiden Gesprächspartner automatisch wieder Sicherheit her, ohne dass ein manuelles Eingreifen nötig wäre — die Sitzung „heilt sich selbst“.

16.3a Die Grenze von Post-Compromise Security: passiver Mitlauscher

Ein wichtiger Vorbehalt zur in 16.3 beschriebenen Selbstheilung: Post-Compromise Security schützt gegen einen Angreifer, der zu einem bestimmten Zeitpunkt Zugriff auf den Sitzungszustand hatte, diesen aber danach **nicht mehr** aufrechterhalten kann. Ein Angreifer, der dagegen weiterhin passiv jeden künftigen Diffie-Hellman-Ratchet-Schritt beobachten kann (etwa weil er weiterhin uneingeschränkten Zugriff auf das kompromittierte Gerät selbst hat, nicht nur einen einmaligen Schnappschuss des damaligen Zustands), würde auch jeden neuen Root Key mitberechnen können und die Selbstheilung dadurch faktisch unterlaufen. Post-Compromise Security schützt also gegen einen **einmaligen, punktuellen** Kompromittierungszeitpunkt (etwa einen einmaligen unbefugten Zugriff auf ein kurzzeitig unbeaufsichtigtes Gerät, Kapitel 23.3a Versuch 2), nicht gegen eine **dauerhafte** Kompromittierung des Geräts durch installierte Schadsoftware (Kapitel 23.3) — dieselbe grundlegende Grenze, die in Kapitel 23.3 bereits als „kein Schutz gegen ein bereits kompromittiertes Endgerät“ benannt wurde, gilt folglich auch hier.

16.4 Nachrichten außer der Reihenfolge

Da Nachrichten über ein Netzwerk nicht garantiert in der gesendeten Reihenfolge ankommen, muss der Double Ratchet auch mit „übersprungenen“ Nachrichten umgehen können. RenkerVault speichert dafür bis zu 64 übersprungene Message Keys zwischen — trifft eine verzögerte Nachricht später doch noch ein, kann sie trotzdem mit dem passenden, zwischengespeicherten Schlüssel entschlüsselt werden. Nach erfolgreicher Entschlüsselung wird auch dieser zwischengespeicherte Schlüssel sofort verworfen.

16.4a Ein durchgehendes Beispiel über fünf Nachrichten

Um die Wechselwirkung zwischen Symmetric-Key-Ratchet (16.2) und Diffie-Hellman-Ratchet (16.3) konkret nachzuvollziehen, ein vereinfachter Ablauf zwischen Anna und Ben über fünf Nachrichten hinweg (Schlüssel hier als kurze symbolische Bezeichner statt echter 32-Byte-Werte dargestellt):

```
Ausgangszustand nach X3DH-lite (Kapitel 15): Root-Key RK0, Anna ist Initiator

Nachricht 1 (Anna -> Ben):
  DH-Ratchet-Schritt (Annas erster ausgehender Schritt): RK0 -> RK1, Sendekette CKs1
  Symmetric-Ratchet: CKs1 -> CKs1' liefert Message-Key MK1
  Anna verschlüsselt "Hallo Ben" mit MK1, verwirft MK1 sofort danach

Nachricht 2 (Ben -> Anna, Antwort):
  Ben empfängt Nachricht 1, sieht Annas neuen Ratchet-Public-Key im Header
  -> DH-Ratchet-Schritt bei Ben: RK0 -> RK1 (identisch zu Annas RK1) -> Empfangskette CKr1
  Ben verwendet CKr1 zum Entschlüsseln von Nachricht 1
  Ben antwortet: eigener DH-Ratchet-Schritt RK1 -> RK2, neue Sendekette CKs2 -> Message-Key MK2
  Ben verschlüsselt "Hallo Anna" mit MK2, verwirft MK2 sofort danach

Nachricht 3 (Anna -> Ben, unmittelbar, OHNE auf Bens Antwort zu warten):
  Symmetric-Ratchet innerhalb derselben Kette: CKs1' -> CKs1'' liefert Message-Key MK3
  (Kein neuer DH-Ratchet-Schritt nötig, da Anna noch keine Antwort von Ben hat)

Nachricht 4 (Anna empfängt Bens Nachricht 2):
  Anna sieht Bens neuen Ratchet-Public-Key -> DH-Ratchet-Schritt: RK1 -> RK2 (wie bei Ben)
  Anna entschlüsselt Nachricht 2 mit der neu abgeleiteten Empfangskette

Nachricht 5 (Anna -> Ben, nach Erhalt von Bens Antwort):
  Neuer DH-Ratchet-Schritt bei Anna: RK2 -> RK3, neue Sendekette CKs3 -> Message-Key MK5
```

Auffällig: Nachricht 3 verwendet keinen neuen DH-Ratchet-Schritt (da Anna zu diesem Zeitpunkt noch keine Antwort von Ben erhalten hat) — nur der Symmetric-Key-Ratchet dreht sich weiter. Erst Nachricht 5, nach Erhalt von Bens Antwort, löst wieder einen DH-Ratchet-Schritt aus. Dieses Beispiel zeigt konkret, warum der Double Ratchet aus **zwei** unterschiedlich häufig „drehenden“ Mechanismen besteht (Kapitel 16.1): Der Symmetric-Key-Ratchet dreht sich bei **jeder** Nachricht, der Diffie-Hellman-Ratchet nur bei einem tatsächlichen Antwort-Roundtrip.

16.5 Warum die Komposition nicht extern auditiert ist — und was das praktisch bedeutet

Ein wichtiger, in SECURITY.md offen dokumentierter Punkt: Während die einzelnen kryptographischen Bausteine (X25519, HMAC, HKDF, AES-256-GCM) alle auditierte, weit verbreitete Implementierungen sind, ist die konkrete **Komposition** dieser Bausteine zum Double Ratchet in RenkerVaults [crypto/ratchet.ts](#) eine Implementierung nach der öffentlichen Signal-Spezifikation, die selbst nicht von einer externen,

unabhängigen Prüfstelle auditiert wurde. Der Grund dafür ist pragmatisch: Für das komplette Signal-Protokoll existiert derzeit keine gepflegte, auditierte **Browser**-Bibliothek (die offizielle `libsignal-protocol-javascript`-Bibliothek ist archiviert und wird nicht mehr gepflegt; `@signalapp/libsignal-client` ist ein natives Node-Modul, das sich nicht direkt in einer Browser- oder WebView-Umgebung verwenden lässt). Vor einem echten Produktiveinsatz mit hohem Schutzbedarf wird in `SECURITY.md` ausdrücklich empfohlen, entweder ein externes kryptographisches Audit der Komposition durchführen zu lassen oder `libsignal` nativ (etwa über die Rust-Backend-Anbindung von Tauri) einzubinden.

16.5a Akademische Verifikation des Signal-Protokolls, auf dem RenkerVault aufbaut

Auch wenn die konkrete Komposition in `ratchet.ts` nicht extern auditiert ist (16.5), ist das zugrundeliegende Signal-Protokoll selbst Gegenstand mehrerer unabhängiger akademischer Untersuchungen gewesen. Eine vielzitierte Arbeit von Katriel Cohn-Gordon, Cas Cremers und Kollegen („A Formal Security Analysis of the Signal Messaging Protocol“, erstmals 2016 vorgestellt) unterzog das Signal-Protokoll einer formalen, mathematisch strengen Sicherheitsanalyse und bestätigte im Wesentlichen die beabsichtigten Sicherheitseigenschaften (Perfect Forward Secrecy, Post-Compromise Security, Kapitel 16.2/16.3), identifizierte zugleich aber auch einige subtile Randfälle, die in späteren Protokollversionen berücksichtigt wurden. Diese akademische Vorarbeit ist einer der Gründe, warum RenkerVault sich für eine Komposition **nach** der öffentlichen Signal-Spezifikation entschieden hat (Kapitel 16.5), statt ein eigenes, unerprobtes Protokolldesign zu entwickeln: Auch wenn die konkrete RenkerVault-Implementierung selbst nicht auditiert ist, beruht sie auf einem Protokoll-**Entwurf**, der bereits mehrfach unabhängig akademisch geprüft wurde — ein wichtiger, aber oft übersehener Unterschied zwischen „das Protokolldesign ist geprüft“ und „diese spezifische Implementierung des Designs ist geprüft“.

16.5b Ratchet-Zustandsverlust: was passiert bei App-Neuinstallation?

Ein praktischer Randfall, der bislang nicht behandelt wurde: Was passiert, wenn eine Nutzerin ihr Gerät verliert, ohne zuvor ein Backup des verschlüsselten Vaults (Kapitel 11) angelegt zu haben, und die App auf einem neuen Gerät komplett neu installiert? Da der komplette Ratchet-Zustand (Root Key, Chain Keys, Kapitel 16.2) ausschließlich verschlüsselt im lokalen Vault gespeichert ist (Kapitel 44.4, `toSnapshot()`), geht dieser Zustand beim Totalverlust des Geräts unwiederbringlich verloren — es gibt keine serverseitige Kopie, aus der er sich wiederherstellen ließe (eine direkte Konsequenz der Zero-Knowledge-Architektur, Kapitel 21). Für die betroffene Person bedeutet dies praktisch: Ein neu installiertes RenkerVault erzeugt eine komplett neue Identität (Kapitel 7.1), und alle bestehenden Kontakte müssten über einen erneuten Handshake (Kapitel

15) neu verbunden werden, inklusive einer erneuten Safety-Number-Verifikation (Kapitel 7.5a) — vergleichbar mit einem Mobiltelefonverlust bei Signal oder ähnlichen Diensten. Dies ist keine RenkerVault-spezifische Schwäche, sondern eine unausweichliche Konsequenz jeder Architektur, die private Schlüssel bewusst niemals unverschlüsselt oder gar unverschlüsselt zentral speichert (Kapitel 21) — der Preis für die Zero-Knowledge-Eigenschaft ist, dass auch der eigene Anbieter im Verlustfall nicht helfend eingreifen kann.

17. Gruppenschlüssel: Sender-Keys-lite

17.1 Warum kein Double Ratchet pro Gruppenmitglied

Wie in Kapitel 9.1 bereits angerissen, würde ein separater Double-Ratchet pro Mitgliederpaar bei n Mitgliedern $n \cdot (n-1)/2$ parallele Sitzungen erfordern — bei einer Gruppe von 50 Mitgliedern bereits 1.225 Sitzungen, die alle synchron gehalten werden müssten. RenkerVault verwendet stattdessen einen einzigen, gemeinsamen Gruppenschlüssel pro „Epoche“.

17.2 Erzeugung und Verteilung

Der Owner (oder ein Admin mit entsprechender Berechtigung, siehe Kapitel 9.3) erzeugt einen zufälligen 256-Bit-Gruppenschlüssel und verteilt ihn **individuell** über die bereits bestehenden 1:1-Double-Ratchet-Sitzungen an jedes einzelne Mitglied. Diese Verteilung ist damit selbst Ende-zu-Ende-verschlüsselt — der Relay-Server sieht auch beim Anlegen einer Gruppe niemals den Gruppenschlüssel im Klartext, sondern nur so viele einzeln verschlüsselte Kopien wie es Mitglieder gibt.

```
newGroupEpochKey() -> zufaelliger 256-Bit-Schlüssel (WebCrypto CSPRNG)
fuer jedes Mitglied m:
    encryptDirect(m, groupKey) // ueber die bestehende 1:1-Sitzung mit m
```

17.2a Eine Voraussetzung, die leicht übersehen wird

Die in 17.2 gezeigte Verteilungsschleife setzt voraus, dass bereits mit **jedem** Gruppenmitglied eine bestehende 1:1-Sitzung existiert (Kapitel 15/16) — ein Owner kann folglich nur Personen zu einer Gruppe hinzufügen, mit denen bereits zuvor ein individueller Kontakt (Kapitel 7.4) hergestellt wurde. Diese Voraussetzung ist keine willkürliche Einschränkung, sondern eine direkte Konsequenz des gewählten Verteilungsmechanismus: Ohne eine bereits etablierte, Ende-zu-Ende-verschlüsselte 1:1-Sitzung gäbe es keinen sicheren Kanal, über den der neue Gruppenschlüssel überhaupt vertraulich übermittelt werden könnte. In der Praxis bedeutet dies, dass das Erstellen einer Gruppe implizit voraussetzt, bereits alle vorgesehenen Mitglieder einzeln kontaktiert zu haben — ein zusätzlicher, aus der Architektur folgender Schritt, der bei zentralisierten Diensten mit serverseitig verwalteten Kontaktlisten oft nicht in gleicher

Form erforderlich ist.

17.3 Epochenwechsel bei Mitgliederänderungen

Jede Änderung der Mitgliederliste (Beitritt oder Entfernen) erzeugt automatisch eine neue Epoche mit frischem Gruppenschlüssel, der erneut individuell an alle verbleibenden Mitglieder verteilt wird. Ein entferntes Mitglied besitzt den neuen Epochenschlüssel nicht und kann folglich keine ab diesem Zeitpunkt gesendeten Nachrichten mehr entschlüsseln — auch dann nicht, wenn es zuvor im Besitz eines älteren Gruppenschlüssels war.

17.3a Was ein entferntes Mitglied trotzdem noch weiß

Ein wichtiger, ehrlich zu benennender Grenzfall: Der Epochenwechsel (17.3) verhindert zuverlässig, dass ein entferntes Mitglied **künftige** Nachrichten lesen kann — er ändert aber nichts daran, dass dasselbe Mitglied bereits **vergangene** Nachrichten aus der Zeit seiner Mitgliedschaft gelesen und möglicherweise gespeichert (etwa als Screenshot oder lokale Kopie) haben könnte. Dieser Grenzfall ist keine RenkerVault-spezifische Schwäche, sondern eine grundsätzliche, in Kapitel 23.3 bereits allgemein beschriebene Grenze jeder Verschlüsselungstechnologie: Sobald eine Nachricht bei einem Empfänger entschlüsselt und angezeigt wurde, hat dieser technisch die Möglichkeit, sie in irgendeiner Form dauerhaft festzuhalten, unabhängig davon, was später mit dem Gruppenschlüssel geschieht. Ein „nachträgliches Vergessen“ bereits gesehener Inhalte ist technisch nicht erzwingbar — der Epochenwechsel schützt die **Zukunft** der Gruppenkommunikation, nicht ihre bereits vergangene Vertraulichkeit gegenüber einem ehemaligen, potenziell böswilligen Mitglied.

17.4 Der bewusste Sicherheitskompromiss gegenüber 1:1-Chats

Innerhalb einer Epoche wird jede reguläre Gruppennachricht mit demselben Epochenschlüssel verschlüsselt (nur einmal, das gleiche Chifftrat wird an jedes Mitglied einzeln adressiert, da der Relay kein natives Gruppen-Fan-out beherrscht). Das bedeutet: Anders als bei 1:1-Chats gibt es **innerhalb** einer Epoche keine Perfect Forward Secrecy pro einzelner Nachricht — kompromittiert ein Angreifer den aktuellen Epochenschlüssel, kann er alle Nachrichten dieser einen Epoche lesen, nicht nur eine einzelne. Dieser Kompromiss ist eine bewusste, in SECURITY.md dokumentierte Design-Entscheidung zugunsten der praktischen Handhabbarkeit von Gruppen mit vielen Mitgliedern.

17.4a Praktische Abmilderung: häufigere manuelle Epochenwechsel

Auch wenn RenkerVault keine automatische Rotation des Gruppenschlüssels innerhalb einer stabilen Mitgliederkonstellation vorsieht (17.4), lässt sich der in 17.4 beschriebene Kompromiss praktisch abmildern: Da jede Mitgliederänderung ohnehin einen neuen Epochenschlüssel erzeugt (Kapitel 9.2), verkürzt eine Gruppe mit häufigerer, bewusster

Fluktuation (etwa durch kurzzeitiges Entfernen und erneutes Hinzufügen aller Mitglieder) die Zeitspanne, über die ein einzelner kompromittierter Schlüssel Gültigkeit hätte. Dies ist kein vollwertiger Ersatz für echte Perfect Forward Secrecy auf Nachrichtenebene (wie sie etwa das in Kapitel 17.5 beschriebene MLS-Protokoll bietet), aber eine mit Bordmitteln umsetzbare Risikominderung für Gruppen mit besonders hohem Schutzbedarf (Kapitel 39.2), die noch keine MLS-Migration (Kapitel 24.6c, Roadmap) abwarten möchten.

17.2b Ein durchgespieltes Beispiel: eine Drei-Personen-Gruppe entsteht

Anna gründet eine Gruppe mit Ben und Clara, mit denen sie jeweils bereits eine bestehende 1:1-Sitzung hat (Kapitel 15/16):

1. Anna erzeugt einen zufälligen 256-Bit-Gruppenschlüssel GK1 (Epoche 1)
2. Anna verschlüsselt GK1 mit der bestehenden 1:1-Sitzung zu Ben
-> sendet ueber Bens individuellen Ratchet-Kanal
3. Anna verschlüsselt GK1 (denselben Schlüssel!) mit der bestehenden 1:1-Sitzung zu Clara -> sendet ueber Claras individuellen Ratchet-Kanal
4. Ben UND Clara entschlüsseln je ihre eigene, individuell verschlüsselte Kopie von GK1 ueber ihre jeweilige 1:1-Sitzung mit Anna
5. Ab jetzt: alle drei verwenden GK1 fuer reguläre Gruppennachrichten

Verlässt Clara die Gruppe später, erzeugt Anna (oder ein Admin mit entsprechender Berechtigung, Kapitel 9.3) einen neuen Schlüssel GK2 (Epoche 2) und wiederholt Schritt 2-4 nur noch mit Ben — Clara erhält GK2 nie und kann folglich keine ab diesem Zeitpunkt gesendeten Nachrichten mehr lesen, selbst wenn sie GK1 weiterhin besitzt.

17.5 Ein Ausblick: MLS (Messaging Layer Security) als Alternative

Neben dem in Kapitel 17.1-17.4 beschriebenen „Sender Keys“-Ansatz existiert seit 2023 mit MLS (Messaging Layer Security, standardisiert in RFC 9420 durch die IETF) ein alternativer, speziell für große Gruppen entworfener Ansatz zur Gruppenverschlüsselung. MLS verwendet eine baumartige Schlüsselstruktur (einen sogenannten „Ratchet Tree“), die es erlaubt, bei einer Mitgliederänderung nur einen logarithmischen statt linearen Anteil der Gruppe an der Schlüsselerneuerung zu beteiligen — ein Effizienzvorteil, der bei sehr großen Gruppen (Hunderte oder Tausende Mitglieder) relevant wird. Zusätzlich bietet MLS, anders als das in RenkerVault verwendete Sender-Keys-Verfahren, tatsächliche Perfect Forward Secrecy **innerhalb** einer Epoche für jede einzelne Nachricht, nicht nur beim Epochenwechsel.

RenkerVault verwendet MLS in der aktuellen Prototyp-Version nicht, aus zwei pragmatischen Gründen: Erstens ist die Implementierungskomplexität von MLS deutlich höher als die des in Kapitel 17.2 beschriebenen einfachen Schlüsselverteilungsschemas, was das in Kapitel 13.3 beschriebene Risiko einer selbst komponierten, fehlerbehafteten Implementierung erhöhen würde. Zweitens sind die typischen Gruppengrößen in RenkerVault (private und

Familien-/Freundeskreis-Gruppen) deutlich kleiner als die Größenordnung, bei der die Effizienzvorteile von MLS praktisch relevant würden. Für eine zukünftige Version mit deutlich größeren Gruppen wäre eine Umstellung auf MLS jedoch eine naheliegende, in SECURITY.md als möglicher zukünftiger Schritt vermerkte Weiterentwicklung.

18. Post-Quantum-Kryptographie und der Hybrid-Handshake

18.1 Die Bedrohung durch zukünftige Quantencomputer

Ein ausreichend leistungsfähiger Quantencomputer könnte mit dem sogenannten „Shor-Algorithmus“ sowohl die Faktorisierung großer Zahlen (die Grundlage von RSA) als auch den diskreten Logarithmus auf elliptischen Kurven (die Grundlage von X25519, siehe Kapitel 14) in praktikabler Zeit lösen — beide Verfahren wären damit gebrochen. Nach aktuellem Stand der Technik existieren solche Quantencomputer noch nicht in einer für diesen Zweck ausreichenden Größe. Das allein wäre jedoch kein Grund zur Entwarnung.

18.1a Wie ein Quantencomputer grundsätzlich anders rechnet

Für Leserinnen und Leser ohne physikalischen Hintergrund eine kurze, stark vereinfachte Einordnung, warum Quantencomputer bestimmte Probleme grundsätzlich anders angehen können als klassische Computer: Ein klassischer Computer verarbeitet Information in Bits, die zu jedem Zeitpunkt eindeutig entweder 0 oder 1 sind. Ein Quantencomputer nutzt stattdessen „Qubits“, die sich (vereinfacht gesagt) in einer Überlagerung („Superposition“) mehrerer Zustände gleichzeitig befinden können, bis sie gemessen werden. Für bestimmte mathematische Probleme — insbesondere solche mit einer periodischen, sich wiederholenden Struktur, wie sie sowohl der Faktorisierung großer Zahlen als auch dem diskreten Logarithmus zugrunde liegt — erlaubt diese Überlagerung einem Quantenalgorithmus wie dem in 18.1 genannten Shor-Algorithmus, viele mögliche Lösungswege gewissermaßen gleichzeitig zu erkunden, statt sie wie ein klassischer Computer nacheinander durchprobieren zu müssen. Für andere mathematische Probleme — etwa die dem in Kapitel 18.3 beschriebenen ML-KEM zugrundeliegende Gitterstruktur — ist bislang kein vergleichbarer Quantenvorteil bekannt, weshalb solche Probleme als „post-quantensicher“ gelten. Diese vereinfachte Erklärung lässt viele physikalische Details aus, vermittelt aber die grundlegende Intuition dafür, warum manche mathematischen Probleme durch Quantencomputer bedroht sind und andere (bislang) nicht.

18.2 „Harvest Now, Decrypt Later“ — warum Wartestrategien nicht funktionieren

Ein Angreifer mit ausreichenden Ressourcen (etwa ein staatlicher Nachrichtendienst) kann bereits heute verschlüsselten Datenverkehr systematisch mitschneiden und archivieren, ohne ihn sofort entschlüsseln zu können — und einfach warten, bis ein ausreichend leistungsfähiger Quantencomputer verfügbar wird, um die gesammelten Daten rückwirkend zu entschlüsseln. Diese Strategie heißt „Harvest Now, Decrypt Later“ (jetzt sammeln, später entschlüsseln). Für Kommunikation, deren Vertraulichkeit auch in fünf, zehn oder zwanzig Jahren noch relevant sein könnte, reicht es folglich nicht aus, erst dann auf Post-Quantum-Kryptographie umzustellen, wenn Quantencomputer tatsächlich verfügbar sind — der schützenswerte Zeitpunkt liegt in der Gegenwart, nicht in der Zukunft.

18.2a Für wen HNDL besonders relevant ist

Nicht jede Kommunikation ist gleichermaßen von der in 18.2 beschriebenen Bedrohung betroffen — die Relevanz hängt maßgeblich davon ab, wie lange der Inhalt einer Nachricht überhaupt noch schützenswert bleiben soll. Eine spontane Verabredung zum Mittagessen verliert ihren Vertraulichkeitswert typischerweise binnen weniger Stunden; ein Gespräch über eine Krankheitsdiagnose, eine juristische Strategie oder journalistische Quellen (Kapitel 39.6, Fallbeispiele) kann dagegen auch nach Jahrzehnten noch erhebliche Konsequenzen haben, wenn es offengelegt würde. Für diese zweite Kategorie ist die in Kapitel 18.4 beschriebene Post-Quantum-Absicherung schon heute relevant, selbst wenn praktisch nutzbare, kryptographisch relevante Quantencomputer nach aktuellem Kenntnisstand noch Jahre oder länger entfernt sein könnten — die HNDL-Logik (18.2) macht das genaue Eintrittsdatum dieser Bedrohung für die Schutzentscheidung von heute weitgehend irrelevant: Sobald ein Quantencomputer verfügbar ist, wäre bereits heute aufgezeichneter, nicht post-quantum-geschützter Datenverkehr rückwirkend gefährdet, unabhängig davon, wann genau dieser Zeitpunkt eintritt.

18.3 ML-KEM-768: das eingesetzte Post-Quantum-Verfahren

RenkerVault verwendet ML-KEM-768 (Module-Lattice-based Key-Encapsulation Mechanism), vormals unter dem Namen „Kyber“ bekannt, seit 2024 vom NIST als FIPS 203 offiziell genormt. ML-KEM basiert nicht auf elliptischen Kurven, sondern auf einem gänzlich anderen mathematischen Problem, dem sogenannten „Module Learning With Errors“ (Module-LWE) — vereinfacht gesagt der Schwierigkeit, ein System linearer Gleichungen zu lösen, dem absichtlich kleine, zufällige „Störungen“ (Errors) hinzugefügt wurden. Dieses Problem gilt nach aktuellem Kenntnisstand auch für Quantencomputer als praktisch nicht effizient lösbar, weshalb ML-KEM zu der Familie der „Post-Quantum“- oder „quantenresistenten“ Verfahren zählt. RenkerVault nutzt dafür [@noble/post-quantum](#), dieselbe Bibliotheksfamilie, die bereits für die klassische Kurvenkryptographie verwendet wird (Kapitel 13.3).

18.3a Vom NIST-Wettbewerb zur fertigen Bibliothek

ML-KEM ist keine Eigenentwicklung eines einzelnen Herstellers, sondern das Ergebnis eines mehrjährigen, öffentlich beobachteten Auswahlverfahrens: 2016 rief das NIST einen offenen internationalen Wettbewerb für Post-Quantum-Kryptographie aus, zu dem Forschungsteams aus aller Welt Vorschläge einreichten. Der ursprüngliche Vorschlag „CRYSTALS-Kyber“ (eines von mehreren eingereichten Verfahren) durchlief mehrere Runden öffentlicher Kryptoanalyse durch die internationale Forschungsgemeinschaft, bei denen konkurrierende Forschergruppen aktiv nach Schwachstellen in allen eingereichten Verfahren suchten — ein Vorgehen, das strukturell dem in Kapitel 13.3 beschriebenen Prinzip entspricht, keine unerprobte Eigenentwicklung, sondern ein breit geprüftes, öffentlich diskutiertes Verfahren einzusetzen. 2022 wählte NIST Kyber als eines der ersten zu standardisierenden Verfahren aus, 2024 wurde daraus nach weiterer Detailarbeit die finale Norm FIPS 203 unter dem neuen Namen ML-KEM. Die von RenkerVault verwendete [@noble/post-quantum](#)-Implementierung folgt dieser finalen Spezifikation und wird, wie die übrigen [@noble](#)-Bibliotheken (Kapitel 13.3b), gegen offizielle Testvektoren verifiziert.

18.3b Andere Wettbewerbsteilnehmer und warum Kyber gewann

Im selben NIST-Wettbewerb traten mehrere weitere Post-Quantum-Verfahren mit unterschiedlichen mathematischen Grundlagen an, darunter NTRU (ebenfalls gitterbasiert, ähnlich wie Kyber, aber mit einer anderen konkreten Konstruktion), SABER (ein weiteres gitterbasiertes Verfahren) und Classic McEliece (basierend auf fehlerkorrigierenden Codes statt auf Gittern, eines der mathematisch ältesten Post-Quantum-Konzepte überhaupt, bereits in den 1970er-Jahren vorgeschlagen). NIST wählte Kyber letztlich unter anderem wegen seines besonders günstigen Verhältnisses aus Sicherheit, Geschwindigkeit und kompakter Schlüssel-/Chiffretextgröße (Kapitel 44b: 1184 Byte Public Key, 2400 Byte Secret Key) — Classic McEliece etwa gilt als mathematisch besonders lange und gründlich untersucht, hat aber deutlich größere Schlüssel (üblicherweise mehrere hundert Kilobyte), was es für den in RenkerVault benötigten, häufigen Erstkontakt-Handshake (Kapitel 15) weniger praktikabel machen würde. NIST standardisierte neben ML-KEM zusätzlich einen zweiten, auf anderen mathematischen Annahmen beruhenden Kandidaten (HQC, basierend auf fehlerkorrigierenden Codes) als Ersatzverfahren („Backup“), falls sich in Zukunft eine bisher unentdeckte Schwäche in gitterbasierten Verfahren wie ML-KEM zeigen sollte — eine zusätzliche Absicherung auf Standardisierungsebene, die demselben Vorsichtsprinzip folgt wie der in Kapitel 18.4 beschriebene Hybrid-Ansatz auf Protokollebene.

18.4 Der Hybrid-Ansatz: klassisch UND Post-Quantum gemeinsam

Statt X25519 durch ML-KEM-768 zu ersetzen, kombiniert RenkerVault beide Verfahren im initialen Schlüsselaustausch (Erweiterung von Kapitel 15):

```
SK = HKDF-SHA256( DH( IK_a, IK_b ) || DH( EK_a, SPK_b ) || PQ_secret )
```

```
PQ_secret = ML-KEM-768-Shared-Secret (siehe 18.5)
```

Der entscheidende Vorteil dieser „Hybrid“-Konstruktion: Die Sicherheit des Gesamtsystems hängt nur davon ab, dass **mindestens eines** der beiden zugrundeliegenden mathematischen Probleme ungelöst bleibt. Sollte sich in einigen Jahren herausstellen, dass ML-KEM angreifbar ist (Post-Quantum-Verfahren sind mathematisch jünger und weniger lange untersucht als klassische Kurvenkryptographie), bliebe der Erstkontakt-Handshake durch X25519 weiterhin geschützt. Sollte umgekehrt ein Quantencomputer X25519 brechen, bliebe der Schutz durch ML-KEM bestehen. Dieser Ansatz — unter dem Namen „PQXDH“ — wird in identischer Grundidee auch von Signal produktiv eingesetzt.

18.5 Wie eine Kapselung (Encapsulation) funktioniert

ML-KEM verwendet, anders als der symmetrische Diffie-Hellman-Austausch aus Kapitel 14.2, ein asymmetrisches „Encapsulation“-Verfahren: Der Responder (etwa Ben) veröffentlicht einen ML-KEM-Prekey. Der Initiator (Anna) führt eine `encapsulate()`-Operation mit Bens öffentlichem PQ-Prekey durch — diese Operation erzeugt gleichzeitig zwei Dinge: ein Chifftrat (das an Ben geschickt wird) und ein geheimes Shared Secret (das nur Anna kennt). Ben führt mit seinem privaten PQ-Prekey und dem erhaltenen Chifftrat eine `decapsulate()`-Operation durch und erhält dabei **dasselbe** Shared Secret zurück, ohne dass dieses je unverschlüsselt übertragen wurde.

18.6 Die ehrliche Grenze: nur der Erstkontakt ist Post-Quantum-geschützt

Ein Punkt, der in der Produktdokumentation ausdrücklich nicht verschwiegen wird: **Nur der initiale X3DH-lite-Handshake** ist Post-Quantum-hybrid geschützt. Die fortlaufenden Double-Ratchet-Schritte danach (Kapitel 16) basieren weiterhin ausschließlich auf klassischem X25519-Diffie-Hellman. Der Grund ist kein Versehen, sondern eine bewusste, mit dem heutigen Stand der Technik begründete Grenze: Ein vollständig quantensicherer, fortlaufender Ratchet-Mechanismus ist derzeit kein etablierter, breit erprobter Industriestandard — auch Signal beschränkt seine PQXDH-Erweiterung ausdrücklich auf den initialen Handshake. Praktisch bedeutet das: Der wichtigste, für „Harvest Now, Decrypt Later“ relevante Moment — der erste, vertrauliche gemeinsame Geheimwert zweier Gesprächspartner — ist geschützt; die fortlaufende Konversation profitiert von diesem Schutz indirekt, da jeder nachfolgende Root Key aus dem bereits PQ-geschützten Ausgangswert abgeleitet wird, jedoch nicht von einer bei jedem Schritt erneuerten Post-Quantum-Absicherung.

18.6a Kryptographische Agilität: was passiert, wenn ML-KEM sich doch als schwach herausstellt?

Eine berechnete Frage angesichts der Neuheit von Post-Quantum-Verfahren (ML-KEM wurde erst 2024 final genormt, Kapitel 18.3): Was, wenn sich herausstellt, dass ML-KEM (oder ein anderes der aktuell standardisierten Post-Quantum-Verfahren) doch angreifbar ist? Die Antwort liegt im Konzept der „kryptographischen Agilität“ — der Fähigkeit eines Systems, ein kryptographisches Verfahren auszutauschen, ohne die gesamte Architektur neu entwerfen zu müssen. RenkerVaults Modularisierung (Kapitel 6.1, [crypto/pq.ts](#) als eigenständiges, klar abgegrenztes Modul mit nur vier Funktionen, Kapitel 44.3) ist bewusst so gestaltet, dass ein Austausch des zugrundeliegenden Post-Quantum-Verfahrens (etwa gegen ein zukünftig als sicherer geltendes Alternativverfahren) auf dieses eine Modul beschränkt bliebe, ohne den Double Ratchet (Kapitel 16) oder das Envelope-Format (Kapitel 21.1) verändern zu müssen. Zusätzlich mindert der in Kapitel 18.4 beschriebene Hybrid-Ansatz das Risiko einer plötzlichen, vollständigen Schutzlücke: Selbst falls ML-KEM sich als komplett gebrochen herausstellen sollte, bliebe der klassische X25519-Anteil weiterhin bestehen — ein abrupter Totalausfall des Schutzes ist durch diese Konstruktion strukturell ausgeschlossen, auch wenn die zusätzliche Post-Quantum-Sicherheit in einem solchen Fall entfielen.

18.6b Ein erwogener, aber verworfener Alternativansatz: der „Privacy Adapter“

In der öffentlichen Debatte um anonyme Kommunikation kursiert gelegentlich ein Konzept, das sich vereinfacht als „Privacy Adapter“ beschreiben lässt: Statt eines eigenen Messengers samt eigenem Relay-Server (wie RenkerVault es umsetzt) verschlüsselt dabei ein separates, kleines Gerät (etwa ein Raspberry Pi) Nachrichten bereits **vor** dem Versand über einen bestehenden, verbreiteten Messenger (etwa WhatsApp), der in diesem Modell nur noch als reiner Transportweg für bereits verschlüsselten, für den Messenger-Betreiber bedeutungslosen Text dient. Als Verschlüsselungsverfahren würde dabei häufig ein One-Time-Pad (Kapitel 19.1) vorgeschlagen, mit vorab über einen physischen Datenträger ausgetauschtem Schlüsselmaterial (einem „Codebook“).

Dieser Ansatz wurde bei der Konzeption von RenkerVault bewusst erwogen und aus mehreren Gründen nicht als Kernarchitektur übernommen:

- **Das OTP-Schlüsselaustauschproblem bleibt bestehen** (Kapitel 19.1) — es verlagert sich nur auf ein zusätzliches physisches Gerät, löst sich aber nicht auf.
- **Abhängigkeit von einem fremden, nicht kontrollierten Transportkanal:** Der zugrundeliegende Messenger könnte seine Nachrichtenformat-Anforderungen jederzeit ändern (etwa durch neue Zeichenbegrenzungen oder Inhaltsfilter), was den „Adapter“ ohne eigenes Zutun funktionsunfähig machen könnte.

- **Kein Schutz der Metadaten durch den zugrundeliegenden Dienst:** Der Betreiber des als Transport genutzten Messengers sieht weiterhin, wer mit wem und wann kommuniziert — der „Privacy Adapter“ schützt ausschließlich den Inhalt, nicht die in Kapitel 22.1 beschriebenen Metadaten, und bietet damit in dieser Dimension keinen Vorteil gegenüber RenkerVaults eigenem, direktem Ansatz.
- **Höhere Einstiegshürde:** Ein physisches Zusatzgerät samt manuellem Schlüsselaustausch ist für die meisten Nutzerinnen und Nutzer eine deutlich höhere praktische Hürde als eine gewöhnliche App-Installation.

RenkerVault übernimmt aus diesem Konzept dennoch eine Kernidee in abgewandelter Form: das Prinzip, dass eine zentrale Infrastruktur (bei RenkerVault der eigene Relay, Kapitel 6.2) strukturell nur bedeutungslose, bereits verschlüsselte Daten sehen soll — nur eben durch eine eigenständige Ende-zu-Ende-Architektur erreicht (Kapitel 21) statt durch einen nachträglich vorgeschalteten Verschlüsselungsadapter über einen fremden Dienst.

18.6c Ausblick: Post-Quantum-Signaturen (ML-DSA)

Der in Kapitel 18 beschriebene Hybrid-Handshake schützt den Schlüsselaustausch gegen Quantencomputer — nicht abgedeckt ist dagegen die in Kapitel 14.3/20 beschriebene Ed25519-Signatur, die weiterhin auf klassischer, nicht quantenresistenter Kurvenkryptographie beruht. Analog zu ML-KEM existiert mit ML-DSA (vormals „Dilithium“ genannt, ebenfalls 2024 durch NIST als FIPS 204 genormt) ein vom selben Standardisierungsprozess stammendes Post-Quantum-Signaturverfahren. RenkerVault verwendet ML-DSA in der aktuellen Prototyp-Version nicht, da die primäre HNDL-Bedrohung (Kapitel 18.2) den **Vertraulichkeitsschutz** von heute aufgezeichneten Nachrichten betrifft, nicht die Authentizität einer Anmeldung, die ohnehin nur im Moment der Verbindung relevant ist und nicht rückwirkend „geerntet“ werden kann. Eine Post-Quantum-Absicherung auch der Authentifizierungsschicht wäre dennoch eine sinnvolle künftige Erweiterung und ist in SECURITY.md als möglicher nächster Schritt vermerkt, sobald sich sowohl Bibliotheksunterstützung als auch Best Practices für hybride Signaturverfahren (analog zum in Kapitel 18.4 beschriebenen hybriden KEM-Ansatz) weiter gefestigt haben.

19. Warum kein One-Time-Pad, warum kein „Quanten-Zufallsgenerator“

19.1 One-Time-Pad: perfekt sicher, aber praktisch fragil

Das One-Time-Pad (OTP) ist das einzige Verschlüsselungsverfahren, das mathematisch beweisbar perfekte Geheimhaltung bietet (ein Resultat, das auf Claude Shannon zurückgeht, den Begründer der Informationstheorie) — vorausgesetzt, drei strenge Bedingungen sind erfüllt: Der Schlüssel muss genau so lang sein wie die zu

verschlüsselnde Nachricht, er muss aus echten Zufallswerten bestehen, und er darf **kein einziges Mal** wiederverwendet werden. Für eine Chat-Anwendung mit potenziell unbegrenzt vielen Nachrichten würde das bedeuten, vorab eine riesige Menge an Schlüsselmaterial sicher auszutauschen (etwa persönlich über einen physischen Datenträger) — und schon die kleinste versehentliche Wiederverwendung eines einzelnen Schlüsselblocks würde die gesamte Sicherheit des Systems zunichtemachen. RenkerVault verzichtet bewusst auf eine OTP-Option: Der Double Ratchet aus Kapitel 16 erreicht in der Praxis einen vergleichbaren Schutz (jede Nachricht mit einem eigenen, einmaligen Schlüssel) ohne das strukturelle Schlüsselaustauschproblem eines OTP.

19.1a Das berühmteste historische Beispiel eines OTP-Wiederverwendungsfehlers

Wie fragil die in 19.1 genannte „kein einziges Mal wiederverwendet“-Bedingung in der Praxis tatsächlich ist, zeigt ein historisches Beispiel aus dem Kalten Krieg: Das sowjetische Venona-Projekt der 1940er-Jahre nutzte eigentlich One-Time-Pads für seine diplomatische und nachrichtendienstliche Kommunikation — theoretisch unknackbar, sofern korrekt angewendet. Aufgrund von Produktionsengpässen bei der Herstellung neuen, echt-zufälligen Schlüsselmaterials während des Zweiten Weltkriegs kam es jedoch zu einer teilweisen Wiederverwendung identischer Schlüsselblöcke für unterschiedliche Nachrichten. US-amerikanische und britische Codeknacker nutzten genau diesen einen Fehler über Jahre hinweg, um Teile der eigentlich perfekt gesicherten Kommunikation nachträglich zu entschlüsseln — ein historisches Lehrstück dafür, dass die mathematische Perfektion eines Verfahrens (19.1) wertlos wird, sobald auch nur eine einzige praktische Randbedingung verletzt wird. Dieses Beispiel wird in der Kryptographie-Ausbildung bis heute als Warnung davor angeführt, theoretisch perfekte, aber praktisch fragile Verfahren wie OTP echten, robusteren Alternativen wie dem Double Ratchet (Kapitel 16) vorzuziehen.

19.2 Warum ein „echter“ Quanten-Zufallsgenerator nicht beansprucht wird

Ein echter Quanten-Zufallsgenerator (QRNG) nutzt physikalische Quanteneffekte (etwa das Schrotrauschen einer Photodiode oder den Zerfall radioaktiver Isotope) als Zufallsquelle und benötigt dafür spezialisierte Hardware. Eine im Browser oder auf einem gewöhnlichen Smartphone laufende Software kann einen solchen physikalischen Effekt nicht auslesen — jede Software, die dennoch behauptet, „Quanten-Zufall“ zu erzeugen, ohne auf entsprechende Spezialhardware zuzugreifen, macht eine irreführende Behauptung. RenkerVault verwendet stattdessen konsequent `crypto.getRandomValues()`, einen kryptographisch sicheren Pseudozufallsgenerator (CSPRNG), der in allen relevanten Standards (unter anderem von NIST) als für kryptographische Zwecke geeignet spezifiziert ist. Es gibt nach heutigem Kenntnisstand keinen praktischen Sicherheitsgewinn durch echten

physikalischen Quantenzufall gegenüber einem korrekt implementierten CSPRNG für diesen Einsatzzweck — der entscheidende Unterschied liegt nicht in der „Quantenhaftigkeit“ der Zufallsquelle, sondern darin, dass der Zufallsgenerator überhaupt kryptographisch sicher ist (im Gegensatz etwa zu einfachen, für Simulationen gedachten Zufallsgeneratoren, die für kryptographische Zwecke ungeeignet wären).

19.2a Woher `crypto.getRandomValues()` seine Entropie tatsächlich bezieht

Eine berechtigte Anschlussfrage: Wenn kein physikalischer Quanteneffekt genutzt wird, woher stammt dann die „Zufälligkeit“ des CSPRNG überhaupt? Die Antwort: Betriebssysteme sammeln fortlaufend sogenannte „Entropie“ aus einer Vielzahl unvorhersehbarer physikalischer Ereignisse, die auf jedem Computer ohnehin auftreten — etwa die exakten Zeitabstände zwischen Tastatureingaben, Mausbewegungen, Festplattenzugriffszeiten, Netzwerkpaket-Ankunftszeiten oder elektronisches Hintergrundrauschen bestimmter Hardwarekomponenten. Diese gesammelten, für einen Angreifer praktisch nicht vorhersagbaren Ereignisse fließen in einen internen Entropie-Pool des Betriebssystems ein, aus dem wiederum ein kryptographisch sicherer Pseudozufallszahlengenerator gespeist wird (unter Linux etwa historisch `/dev/urandom`, in modernen Browsern eine vergleichbare, vom Betriebssystem bereitgestellte Quelle). Der Unterschied zu einem echten QRNG (19.2) liegt also nicht darin, dass hier „vorhersagbare“ Zufallszahlen erzeugt würden — die gesammelte physikalische Entropie ist für einen Angreifer ebenfalls praktisch nicht vorhersagbar —, sondern lediglich darin, dass die zugrundeliegende physikalische Quelle nicht explizit quantenmechanischer Natur ist. Für die in Kapitel 44b tabellierten Anwendungsfälle (Schlüsselerzeugung, Nonces, Salts) ist diese Unterscheidung, wie in 19.2 begründet, praktisch irrelevant.

20. Authentifizierung: Ed25519-Challenge-Response

20.1 Warum passwortlos?

Ein klassisches Passwort-basiertes Login-Verfahren würde erfordern, dass zumindest ein Hash des Passworts an den Server übertragen und dort gespeichert wird — mit dem Risiko, dass ein kompromittierter Server diesen Hash offline per Brute-Force angreifbar macht. RenkerVault verwendet stattdessen ein Challenge-Response-Verfahren auf Basis der in Kapitel 14.3 beschriebenen Ed25519-Signaturen, bei dem die geheime Passphrase den Client zu keinem Zeitpunkt verlässt.

20.1a Verwandtschaft zu FIDO2/WebAuthn

Nutzerinnen und Nutzer, die bereits mit passwortlosen Anmeldeverfahren im Web vertraut sind (etwa der Anmeldung per Fingerabdruck oder Sicherheitsschlüssel bei

großen Online-Diensten, technisch als FIDO2/WebAuthn-Standard bekannt), werden im hier beschriebenen Verfahren ein vertrautes Grundprinzip wiedererkennen: Auch FIDO2/WebAuthn basiert im Kern auf digitalen Signaturen über eine vom Server erzeugte Zufalls-Challenge, wobei der private Schlüssel das Gerät nie verlässt — exakt dasselbe Prinzip wie das in Kapitel 20.2 beschriebene RenkerVault-Verfahren. Der Unterschied liegt vor allem im Schlüsselspeicher: FIDO2 nutzt üblicherweise ein spezialisiertes Hardware-Sicherheitsmodul (etwa einen USB-Sicherheitsschlüssel oder einen im Smartphone eingebauten „Secure Enclave“-Chip), während RenkerVault den privaten Signing-Key softwareseitig, verschlüsselt im Vault (Kapitel 11), verwahrt. Diese Ähnlichkeit ist kein Zufall: Beide Ansätze verfolgen dieselbe, in der Fachwelt inzwischen breit anerkannte Erkenntnis, dass Challenge-Response-Signaturen einem klassischen, serverseitig gespeicherten Passwort-Hash in nahezu jeder Hinsicht überlegen sind.

20.2 Der Ablauf im Detail

1. Client verbindet sich mit dem Relay und sendet seine Konto-Kennung
2. Relay erzeugt eine zufällige Nonce (Einmalwert) und sendet sie an den Client
3. Client signiert die Nonce mit seinem privaten Ed25519-Signing-Key
4. Client sendet die Signatur zurück an den Relay
5. Relay prüft die Signatur mit dem öffentlichen Signing-Key des Kontos
 - > gültig: Verbindung authentifiziert
 - > ungültig: Verbindung abgelehnt, Fehlversuch-Zähler erhöht (Kapitel 10.2)

Der entscheidende Punkt: Die Passphrase selbst wird für diesen Vorgang nicht benötigt und nicht übertragen — sie dient ausschließlich dazu, lokal auf dem Gerät den privaten Signing-Key aus der verschlüsselten lokalen Datenbank (Kapitel 11) zu entsperren. Selbst ein vollständiger Datenabzug des Relay-Servers würde also keine Passwort-Hashes offenlegen, aus denen sich Passphrasen erraten ließen — es gibt schlicht keine serverseitig gespeicherte Repräsentation der Passphrase.

20.3 Warum die Nonce frisch und zufällig sein muss

Ein Detail, das die Sicherheit dieses Verfahrens entscheidend mitträgt: Die vom Relay erzeugte Nonce (Schritt 2 in 20.2) muss bei jedem Anmeldeversuch neu und unvorhersehbar sein. Wäre die Nonce vorhersehbar oder würde sie wiederverwendet, könnte ein Angreifer eine einmal beobachtete, gültige Signatur zu einer bestimmten Nonce später erneut einspielen („Replay-Angriff“), ohne den privaten Signing-Key selbst zu kennen. Da der Relay bei jeder Anmeldung eine neue, über den in Teil III, Kapitel 19.2 beschriebenen CSPRNG erzeugte Nonce verwendet, ist eine zu einem früheren Zeitpunkt beobachtete Signatur für jeden weiteren Anmeldeversuch wertlos — sie passt schlicht zu keiner künftigen Nonce mehr.

20.4 Ein durchgespieltes Beispiel mit symbolischen Werten

Zur Veranschaulichung des Ablaufs aus 20.2 mit (stark vereinfachten, symbolischen statt echten kryptographischen) Werten:

```

1. Client -> Relay: "Ich bin Konto ANNA-7f3a"
2. Relay -> Client: Nonce = "x9K2mQ..." (32 zufaellige Byte, nur fuer diesen Versuch
gueltig)
3. Client (lokal): signatur = Ed25519-Sign(Nonce, AnnasPrivaterSigningKey)
4. Client -> Relay: signatur = "8fA1..."
5. Relay (lokal): Ed25519-Verify(Nonce, signatur, AnnasOeffentlicherSigningKey)
    -> true: Anna ist authentifiziert, OHNE dass eine Passphrase
        oder ein Passwort-Hash je uebertragen wurde

```

Ein Angreifer, der Schritt 4 abfängt, kann die dort übertragene Signatur nicht für eine erneute Anmeldung wiederverwenden, da der Relay beim nächsten Versuch (Schritt 2) eine komplett neue Nonce erzeugt, zu der die alte Signatur nicht mehr passt.

20.4a Warum die Browser-Laufzeitumgebung selbst Teil des Vertrauensmodells ist

Ein Aspekt, der in den vorangegangenen Kapiteln implizit vorausgesetzt wurde, verdient eine explizite Erwähnung: RenkerVaults Web-Variante (Kapitel 12.1) verlässt sich für die eigentliche AES-GCM-Operation auf die im Browser eingebaute WebCrypto-API (Kapitel 13.1), nicht auf eine eigene, in JavaScript geschriebene Implementierung. Diese Entscheidung hat einen doppelten Vorteil: Erstens läuft die eigentliche kryptographische Berechnung dadurch in nativem, hochoptimiertem Code des Browser-Herstellers, nicht in interpretiertem oder just-in-time-kompiliertem JavaScript, was sowohl Geschwindigkeits- als auch Sicherheitsvorteile bietet (etwa robusteres Zeitverhalten gegen Timing-Angriffe, Kapitel 13.3a). Zweitens wird damit ein zusätzlicher Vertrauensanker in das Modell eingeführt: Browser-Hersteller wie die Entwickler von Chromium, Firefox oder WebKit unterhalten eigene, große Sicherheitsteams, die WebCrypto-Implementierungen kontinuierlich pflegen und auf Schwachstellen prüfen — ein Vertrauensmodell, das strukturell dem in Kapitel 5.9a beschriebenen Prinzip öffentlicher, langjähriger Überprüfung entspricht, nur auf der Ebene der Laufzeitumgebung statt der Anwendung selbst. Für die Desktop-Variante (Tauri, Kapitel 12.2) gilt eine vergleichbare Überlegung für die dort verwendete systemeigene WebView-Komponente.

21. Das Zero-Knowledge-Relay

21.1 Das Envelope-Format: was der Server sieht

Das Transportformat, das der Client an den Relay sendet, ist bewusst minimal gehalten:

```

interface Envelope {
  ct: string;                // Base64-Chiffretext (AES-256-GCM) - fuer den Rel
y unlesbar
  chatId, chatKind, kind,    // Routing-Metadaten (fuer Relay sichtbar)
  msgId, ts, fromName
  header?: RatchetHeader;    // 1:1: DH-Public-Key, Kettenposition
  epoch?: number;           // Gruppe: aktuelle SchluesSEL-Epoche

```

```
x3dh?: { ephPub, identityPub, pqCt }; // nur bei der allerersten 1:1-Nachricht
replyTo?, forwardedFrom?, targetMsgId?, emoji? // Anwendungs-Events (Kapitel 8.1)
}
```

Alle Felder außerhalb von `ct` sind für den Relay technisch sichtbar, da sie für die Zustellung (Routing) benötigt werden — analog zur Adresse auf einem Briefumschlag, die notwendigerweise lesbar sein muss, damit die Post den Brief zustellen kann, während der Inhalt des Briefs selbst verschlossen bleibt.

21.1a Warum `fromName` als Klartext-Metadatenfeld existiert

Ein Detail des in 21.1 gezeigten Formats verdient besondere Aufmerksamkeit: Das Feld `fromName` überträgt den Anzeigenamen des Absenders unverschlüsselt als Metadatum, nicht als Teil der Chiffretext-Nutzlast `ct`. Diese Design-Entscheidung erlaubt es der Oberflächen-Schicht (Kapitel 6.1) des Empfängers, den Absendernamen sofort anzuzeigen, ohne zunächst die vollständige Entschlüsselung abzuwarten — praktisch relevant etwa für Benachrichtigungsvorschauen. Der Kompromiss dabei: Der Relay-Server sieht dadurch nicht nur, welche Konto-Kennungen miteinander kommunizieren (was ohnehin für das Routing erforderlich wäre, Kapitel 21.1), sondern zusätzlich den von den Nutzern selbst gewählten Anzeigenamen. Da dieser Anzeigename frei wählbar ist und keine Verifikationspflicht besteht (anders als etwa ein amtlicher Name), bleibt der praktische Informationsgewinn für den Relay-Betreiber gering — Nutzerinnen und Nutzer mit besonders hohem Schutzbedarf (Kapitel 39.2) können hier zusätzlich einen bewusst nichtssagenden, nicht mit der eigenen Identität verknüpfbaren Anzeigenamen wählen.

21.2 Was diese Sichtbarkeit für das Vertrauensmodell bedeutet

Wichtig für eine realistische Einschätzung: Antwort-Zitate und der „Weitergeleitet von“-Name laufen aktuell ebenfalls als Metadaten mit, nicht als Teil der verschlüsselten Nutzlast — ein bewusst dokumentierter Kompromiss (siehe auch Kapitel 25). Wer maximale Vertraulichkeit auch für diese Zusatzinformationen benötigt, sollte dies bei der Bewertung des Schutzniveaus berücksichtigen.

21.2a Warum zusätzlich TLS (`wss://`) nötig ist, obwohl der Inhalt schon Ende-zu-Ende-verschlüsselt ist

Eine häufig gestellte Frage: Wenn der Nachrichteninhalt bereits durch den Double Ratchet (Kapitel 16) Ende-zu-Ende-verschlüsselt ist, wozu braucht die Verbindung zum Relay dann zusätzlich TLS (`wss://` statt `ws://`)? Die Antwort liegt in den beiden unterschiedlichen Schutzebenen, die in diesem Buch mehrfach angesprochen wurden: Die Ende-zu-Ende-Verschlüsselung schützt den **Inhalt** einer Nachricht selbst dann, wenn der Relay-Server vollständig kompromittiert wäre. TLS schützt zusätzlich die **Transportverbindung** zwischen Client und Relay auf dem Weg dorthin — ohne TLS

könnte ein Angreifer, der sich zwischen Client und Relay befindet (etwa in einem unsicheren öffentlichen WLAN), zwar weiterhin nicht den eigentlichen Nachrichteninhalt lesen (dieser bleibt durch die Ende-zu-Ende-Verschlüsselung geschützt), aber er könnte die in Kapitel 21.1 beschriebenen Metadaten mitlesen, den initialen Authentifizierungsvorgang (Kapitel 20) beobachten oder die Verbindung stören/manipulieren. TLS und Ende-zu-Ende-Verschlüsselung sind damit keine redundanten, sondern zwei komplementäre Schutzschichten für unterschiedliche Bedrohungsszenarien — der Verzicht auf eine der beiden Schichten würde eine reale Schutzlücke öffnen, selbst wenn die jeweils andere Schicht intakt bleibt.

21.2b Sichtbarkeitsmatrix des Envelope-Formats

Die folgende Tabelle fasst zusammen, welche Envelope-Felder aus Kapitel 21.1 für den Relay sichtbare Metadaten sind und welche innerhalb des Ende-zu-Ende-verschlüsselten Chiffretexts (`ct`) verborgen bleiben:

Feld	Sichtbar fuer Relay?	Grund
<code>ct</code> (Nachrichteninhalt)	Nein	Kernstück der Ende-zu-Ende-Verschlüsselung (Kapitel 16)
<code>chatId</code> , <code>chatKind</code>	Ja	Fuer Routing zwingend erforderlich
<code>msgId</code> , <code>ts</code>	Ja	Zur Zustellungsverfolgung und Sortierung noetig
<code>fromName</code>	Ja	Anzeigenname, vom Sender mitgeschickt (kein technischer Zwang, aber Praxis des Prototyps)
<code>header</code> (Ratchet-Position)	Ja	Fuer den korrekten Double-Ratchet-Schritt beim Empfaenger noetig (Kapitel 16.2)
<code>x3dh</code> (Handshake-Daten)	Ja, nur bei Erstkontakt	Fuer den initialen Schlüsselaustausch zwingend (Kapitel 15)
<code>replyTo</code> (Zitat-Referenz)	Ja	Bewusster Kompromiss, siehe Kapitel 21.2
<code>emoji</code> , <code>reactionOp</code>	Ja	Bewusster Kompromiss, siehe Kapitel 21.2
Eigentlicher Text/Bildinhalt	Nein	Liegt ausschliesslich innerhalb von <code>ct</code>

Diese Tabelle macht sichtbar, was in Kapitel 21.2 in Worten beschrieben wurde: Der Großteil der eigentlichen Kommunikationsinhalte ist geschützt, während Routing-notwendige Felder sowie einige bewusst als Metadaten belassene Zusatzfelder (Zitat-Referenzen, Reaktions-Emoji) für den Relay sichtbar bleiben.

21.3 Persistenz und Betriebsmodell des Relays

In der aktuellen Prototyp-Version hält der Relay-Server seinen Zustand ausschließlich im Arbeitsspeicher (RAM) — Konten und Warteschlangen gehen bei einem Neustart des Servers verloren. Für einen produktiven Dauerbetrieb wäre eine persistente Datenbank (z. B. PostgreSQL) für Metadaten sowie eine persistente, weiterhin verschlüsselte Warteschlange für Offline-Zustellungen erforderlich — dies ist in SECURITY.md als offener Punkt vor einem echten Produktiveinsatz vermerkt.

21.3a Warum die maximale Envelope-Größe (2 MiB) genau so gewählt wurde

Die in Kapitel 44b tabellierte Obergrenze von 2 MiB pro Envelope ist kein willkürlicher Wert, sondern das Ergebnis einer Abwägung zwischen zwei gegenläufigen Interessen. Eine höhere Grenze würde größere Anhänge (etwa längere Sprachnachrichten oder höher aufgelöste Bilder, Kapitel 8.2) ohne zusätzliche Kompression erlauben — eine niedrigere Grenze schützt den Relay-Server (Kapitel 21) davor, dass ein einzelner, möglicherweise böswillig großer Envelope unverhältnismäßig viel Arbeitsspeicher in der in Kapitel 21.3 beschriebenen Offline-Warteschlange belegt, was bei vielen gleichzeitig offline befindlichen Empfängern zu einem Ressourcenerschöpfungs-Risiko werden könnte (verwandt mit der in Kapitel 25.2 beschriebenen DDoS-Grenze, nur auf Ebene einzelner Nachrichten statt einzelner Verbindungen). Der gewählte Wert von 2 MiB berücksichtigt zusätzlich den in Kapitel 44.2 beschriebenen Base64-Kodierungs-Overhead (Base64 vergrößert Binärdaten um etwa ein Drittel gegenüber der Rohgröße) sowie den JSON-Aufschlag des Envelope-Formats selbst (Kapitel 21.1) — die tatsächlich nutzbare Rohdatengröße für einen Anhang liegt entsprechend etwas niedriger als die genannten 2 MiB.

22. Metadatenschutz und Tor Hidden Service

22.1 Die Grenze der Ende-zu-Ende-Verschlüsselung: Metadaten

Ende-zu-Ende-Verschlüsselung schützt zuverlässig den Inhalt einer Kommunikation — sie schützt nicht automatisch die Tatsache, dass überhaupt kommuniziert wurde, mit wem, wann und wie oft. Diese Information nennt man „Metadaten“, und sie kann für sich genommen bereits erhebliche Rückschlüsse zulassen (ein bekanntes Zitat aus der Sicherheitsforschung lautet sinngemäß: „Metadaten sind buchstäblich das, worüber wir Menschen töten“ — gemeint als Verweis darauf, dass Kommunikationsmuster allein, etwa bei der gezielten Auswahl von Angriffszielen durch Drohnenprogramme, historisch als Entscheidungsgrundlage genutzt wurden).

22.1a Das Sozialgraph-Argument

Ein konkretes Beispiel, warum Metadaten allein bereits sensibel sein können, ohne dass ein einziger Nachrichteninhalt bekannt wäre: Bereits aus der bloßen Liste, wer mit wem

wie häufig kommuniziert (der sogenannte „Sozialgraph“), lässt sich mit statistischen Methoden erstaunlich viel ableiten — etwa auf eine mögliche romantische Beziehung schließen (ungewöhnlich häufiger Kontakt zwischen zwei Personen zu späten Abendstunden), auf eine berufliche Hierarchie (wer meldet sich bei wem, wie schnell antwortet wer), oder auf die Zugehörigkeit zu einer bestimmten sozialen oder politischen Gruppe (ein dichtes Netzwerk gegenseitiger Kontakte innerhalb einer bestimmten Personengruppe). Diese Art der Analyse benötigt keinen einzigen entschlüsselten Nachrichteninhalt — sie arbeitet ausschließlich mit den in Kapitel 21.1 beschriebenen, für den Relay ohnehin sichtbaren Routing-Metadaten. Dies ist der zentrale Grund, warum Sicherheitsforscher (Kapitel 22.4a) Metadatenenschutz als eigenständige, von der reinen Inhaltsverschlüsselung zu unterscheidende Schutzdimension behandeln, statt ihn als bloße Ergänzung zu betrachten.

22.2 Was der Relay-Server über Metadaten weiß — und was nicht

Wie in Kapitel 21.1 beschrieben, sieht der Relay-Server zwangsläufig, welche Konto-Kennung mit welcher anderen Konto-Kennung Nachrichten austauscht, und wann. Was er nicht sieht: den Inhalt, die tatsächliche reale Identität hinter einer Konto-Kennung (sofern diese nicht anderweitig preisgegeben wird) und — sofern über Tor betrieben (22.3) — die reale IP-Adresse der verbindenden Geräte.

22.2a Eine vollständige Gegenüberstellung: sichtbar vs. unsichtbar für den Relay

Zur Verdeutlichung eine erschöpfende Gegenüberstellung dessen, was ein Relay-Betreiber im ungünstigsten Fall (voller technischer Zugriff auf die Serverinfrastruktur, aber ohne Tor-Betrieb) tatsächlich einsehen könnte, gegenüber dem, was ihm strukturell verborgen bleibt:

Für den Relay sichtbar	Für den Relay NICHT sichtbar
Konto-Kennungen der Kommunikationspartner	Nachrichteninhalt (Text, Bild, Sprachnachricht)
Ungefährer Zeitpunkt jeder Nachricht	Tatsächliche reale Identität hinter einer Konto-Kennung
Ungefähre Größe eines Envelopes (Kapitel 21.3a)	Passphrase oder daraus abgeleitete Schlüssel (Kapitel 20.1)
Anzeigenname, sofern im Klartext-Metadatenfeld gesetzt (Kapitel 21.1a)	Gruppenmitgliederliste bei Gruppen ohne eigene Teilnahme
Geräte-Anzahl pro Konto (Kapitel 7.3)	Safety Number / Verifikationsstatus (rein lokal, Kapitel 7.5)
IP-Adresse der Verbindung (ohne Tor, Kapitel 22.3)	Inhalt des Security-Logs (verschlüsselt, Kapitel 10.6)

Diese Tabelle fasst zusammen, was in den Kapiteln 21-22 verteilt beschrieben wurde, an einer Stelle zusammen — sie zeigt, dass die Grenze zwischen sichtbar und unsichtbar nicht zufällig verläuft, sondern exakt der Grenze zwischen Routing-Metadaten (die für die Zustellung technisch notwendig sind) und Inhaltsdaten

(die durch Ende-zu-Ende-Verschlüsselung geschützt werden) entspricht.

22.3 Tor Hidden Service als zusätzliche Anonymitätsschicht

Für Nutzerinnen und Nutzer, denen der Schutz des Nachrichteninhalts allein nicht ausreicht, unterstützt der Relay-Server den Betrieb als reiner Tor Hidden Service: Der Server bindet seinen Port ausschließlich an `127.0.0.1` und veröffentlicht sich über das Tor-Netzwerk unter einer `.onion`-Adresse, ohne einen öffentlichen DNS-Namen oder einen von außen erreichbaren Port zu benötigen.

```
HiddenServiceDir /var/lib/tor/renkervault/  
HiddenServicePort 80 127.0.0.1:8787
```

Der praktische Effekt: Weder ein Netzwerkbeobachter (etwa ein Internetdienstanbieter) noch der Server-Hosting-Anbieter selbst kann feststellen, welche realen IP-Adressen sich mit dem Relay verbinden — die gesamte Verbindung läuft über mehrere, von Tor zufällig ausgewählte Vermittlungsknoten, von denen keiner gleichzeitig Ausgangs- und Zielpunkt der Verbindung kennt.

22.3a Warum RenkerVault kein WebRTC verwendet (und damit ein bekanntes IP-Leck vermeidet)

Viele browserbasierte Kommunikationsanwendungen nutzen für Video- oder Sprachanrufe die WebRTC-Technologie, die für eine möglichst direkte, latenzarme Verbindung zwischen zwei Geräten konzipiert ist — mit einer bekannten, in der Sicherheitsforschung wiederholt dokumentierten Nebenwirkung: WebRTC kann unter bestimmten Browser-Konfigurationen die tatsächliche lokale und teils sogar die öffentliche IP-Adresse eines Geräts offenlegen, selbst wenn ein VPN oder Tor (Kapitel 22.3) aktiv genutzt wird, da WebRTC-Verbindungsaufbauten technisch bedingt eigene, vom übrigen Datenverkehr unabhängige Netzwerkpfade nutzen können. RenkerVault implementiert Sprachnachrichten bewusst nicht als Echtzeit-Anruf über WebRTC, sondern als aufgezeichnete, anschließend wie jede andere Datei Ende-zu-Ende-verschlüsselte Sprachnachricht (Kapitel 8.2), die ausschließlich über die bereits beschriebene Relay-Verbindung (Kapitel 6.2) übertragen wird. Dadurch entfällt das strukturelle WebRTC-IP-Leck-Risiko vollständig — ein Vorteil, der sich direkt aus dem Verzicht auf Echtzeit-Anrufe als Funktion ergibt, nicht aus einer zusätzlichen Schutzmaßnahme.

22.4 Was Tor nicht löst

Tor schützt die Netzwerkebene (wer verbindet sich mit wem, auf IP-Ebene) — es ändert nichts an den in Kapitel 21.1 beschriebenen anwendungsseitigen Metadaten, die der Relay auf Protokollebene weiterhin sieht (welche Konto-Kennung mit welcher anderen kommuniziert). Für vollständigen Schutz auch dieser Ebene wären zusätzliche Techniken wie „Sealed Sender“ (bei dem selbst der Absender einer Nachricht dem

Relay gegenüber verborgen bleibt) oder systematisches Nachrichten-Padding und Cover-Traffic (künstlicher, bedeutungsloser Datenverkehr, der reale Kommunikationsmuster verschleiert) nötig — beides ist in der aktuellen Prototyp-Version nicht implementiert und in SECURITY.md als offener Punkt vermerkt.

22.4a Cover Traffic: Rauschen als Tarnung

Eine in Kapitel 22.4 bereits erwähnte, aber genauer zu erläuternde Technik zum Schutz von Metadaten ist „Cover Traffic“ (Tarnverkehr): Zusätzlich zu echten Nachrichten sendet ein Client dabei in regelmäßigen Abständen bedeutungslose, aber von echten Nachrichten äußerlich nicht unterscheidbare Datenpakete an den Relay — mit dem Ziel, dass ein Beobachter (etwa ein Netzwerkbeobachter oder der Relay-Betreiber selbst) allein aus dem Zeitpunkt und der Häufigkeit von Datenverkehr nicht mehr zuverlässig auf tatsächliche Kommunikationsaktivität schließen kann. Ohne Cover Traffic verrät bereits das bloße Muster „Gerät A sendet immer kurz nachdem Gerät B online gegangen ist“ wertvolle Information, selbst wenn der Inhalt vollständig verschlüsselt bleibt. Der Nachteil von Cover Traffic ist ein realer Kompromiss: zusätzlicher Datenverbrauch und Akkuverbrauch für Datenpakete, die keine tatsächliche Kommunikationsfunktion erfüllen — ein Grund, warum diese Technik in der aktuellen Prototyp-Version von RenkerVault (noch) nicht implementiert ist, aber in SECURITY.md als möglicher zukünftiger Baustein für höhere Metadaten-Schutzstufen vermerkt ist.

22.4b Zentralisiert, föderiert oder Zero-Knowledge-Relay: ein Architekturvergleich

Es lohnt sich, die in Kapitel 6.2 beschriebene Relay-Architektur explizit gegen zwei verbreitete Alternativen abzugrenzen. Ein **zentralisierter** Dienst (wie es die meisten kommerziellen Messenger, einschließlich WhatsApp, im Kern sind) betreibt eine einzige, unternehmenseigene Serverinfrastruktur, die sämtliche Metadaten aller Nutzer an einem Ort konzentriert — effizient zu betreiben, aber mit einem einzigen, großen Angriffs- und Begehrlichkeitsziel für Behördenanfragen oder Angriffe. Ein **föderiertes** Modell (wie Matrix, Kapitel 24.5, oder klassisches E-Mail) verteilt Metadaten auf viele unabhängig betriebene Server, die miteinander kommunizieren — dies verteilt das Risiko, bedeutet aber auch, dass Metadaten potenziell über mehrere, unterschiedlich vertrauenswürdige Server hinweg sichtbar werden, je nachdem, mit wem auf welchem Server kommuniziert wird. RenkerVaults **Zero-Knowledge-Relay**-Modell wählt einen dritten Weg: Es kann zentral **oder** von jeder Nutzergruppe selbst betrieben werden (Kapitel 38), sieht aber in beiden Fällen strukturell nie den Nachrichteninhalt — eine Kombination aus der einfachen Betreibbarkeit eines zentralisierten Modells und einem gegenüber echter Zentralisierung reduzierten Vertrauensbedarf gegenüber dem jeweiligen Betreiber.

22.4c Warum maschinelles Lernen die Metadaten-Analyse gefährlicher macht

Die in 22.4a beschriebene Idee von Cover Traffic gewinnt zusätzlich an Bedeutung durch eine Entwicklung außerhalb der reinen Kryptographie: den Fortschritt bei maschinellem Lernen. Traditionelle Verkehrsanalyse („Traffic Analysis“) beschränkte sich historisch auf vergleichsweise einfache statistische Muster (etwa Nachrichtenhäufigkeit oder -zeitpunkt). Moderne, auf maschinellem Lernen basierende Analyseverfahren können jedoch weitaus subtilere Muster in verschlüsseltem Datenverkehr erkennen — etwa aus der Größe und dem Timing einzelner verschlüsselter Pakete Rückschlüsse auf die Art der Kommunikation (Text vs. Sprachnachricht vs. Bild, siehe Kapitel 8.2) ziehen, selbst wenn der Inhalt selbst durch AES-GCM (Kapitel 13.1) vollständig geschützt bleibt. Diese Entwicklung bedeutet: Metadatenschutz ist kein statisches Ziel, sondern muss mit den sich verbessernden Analysefähigkeiten potenzieller Beobachter mitwachsen — ein zusätzliches Argument dafür, warum die in Kapitel 22.4a/24.6c als Roadmap-Punkte genannten Techniken (Cover Traffic, Sealed Sender) langfristig an Bedeutung gewinnen dürften, auch wenn sie im aktuellen Prototyp noch nicht umgesetzt sind. Dieselbe Beobachtung liefert auch ein zusätzliches Argument für den in Teil IV, Kapitel 32.5a erwähnten Zusammenhang zwischen Chatkontrolle-Debatte und KI-Regulierung: Die Fähigkeiten, die eine verpflichtende KI-gestützte Inhaltsanalyse voraussetzen würde, sind eng verwandt mit denselben Fähigkeiten, die auch eine verbesserte Metadaten-Analyse ermöglichen.

23. Threat Model: Wovor RenkerVault schützt — und wovor nicht

23.1 Die Notwendigkeit eines expliziten Threat Models

Jedes Sicherheitswerkzeug schützt gegen bestimmte Angreifer und Angriffsszenarien — niemals gegen alle denkbaren. Ein Werkzeug ohne explizit benanntes „Threat Model“ (Bedrohungsmodell) verleitet Nutzerinnen und Nutzer dazu, unrealistische Annahmen über ihren Schutz zu treffen. Dieses Kapitel benennt für RenkerVault explizit, wogegen es schützt und wogegen nicht.

23.1a Die drei Elemente eines vollständigen Threat Models

In der Sicherheitsforschung wird ein Bedrohungsmodell typischerweise entlang dreier Fragen strukturiert, die auch diesem Kapitel zugrunde liegen: Erstens, **wer** ist der Angreifer (ein neugieriger Bekannter mit kurzzeitigem Gerätezugriff? Ein krimineller Akteur, der auf Diebstahl von Zugangsdaten aus ist? Ein staatlicher Akteur mit weitreichenden technischen und rechtlichen Mitteln)? Zweitens, **was** versucht der Angreifer zu erreichen (Mitlesen einzelner Nachrichten? Vollständige Kompromittierung eines Kontos? Bloße Feststellung, dass zwei Personen überhaupt kommunizieren, unabhängig vom Inhalt)? Drittens, **welche Mittel** stehen dem Angreifer zur Verfügung (rein remote über das Netzwerk? Kurzzeitiger physischer Gerätezugriff? Dauerhafte

Kompromittierung des Geräts durch Schadsoftware)? Die in Kapitel 23.2/23.3 folgende Gegenüberstellung beantwortet implizit alle drei Fragen für jedes genannte Szenario — ein Grund, warum dort jeweils sowohl der Angreifertyp als auch das konkrete Ziel benannt werden, statt pauschal von „Sicherheit“ oder „Unsicherheit“ zu sprechen.

23.2 Wogegen RenkerVault schützt

Angreifer / Szenario	Schutz durch RenkerVault
Ein Netzwerkbeobachter, der den Datenverkehr zum Relay mitliest	Ja — Inhalt ist Ende-zu-Ende-verschlüsselt, sofern zusätzlich TLS/wss aktiv ist (Metadaten bleiben teilweise sichtbar, siehe 22.1)
Der Betreiber des Relay-Servers selbst	Ja, für den Nachrichteninhalt — der Betreiber sieht nur Metadaten, niemals Klartext
Ein Angreifer, der später einen einzelnen Nachrichtenschlüssel kompromittiert	Ja, weitgehend — Perfect Forward Secrecy begrenzt den Schaden auf einzelne Nachrichten (Kapitel 16.2)
Ein Dieb, der ein gesperrtes Gerät physisch in Besitz nimmt	Ja — lokale Datenbank ist verschlüsselt (Kapitel 11), Brute-Force-Schutz aktiv (Kapitel 10.2)
Ein Angreifer, der versucht, sich als bereits bekanntes Gerät auszugeben	Ja — KEY_MISMATCH-Erkennung (Kapitel 10.3)
Ein Angreifer mit einem zukünftigen, ausreichend grossen Quantencomputer	Teilweise — der initiale Handshake ist Post-Quantum-hybrid geschützt (Kapitel 18), der fortlaufende Ratchet nicht vollständig
Erzwungene Geräteentsperrung unter Druck	Teilweise — Duress-PIN zeigt eine Fake-Ansicht (Kapitel 10.5)

23.3 Wogegen RenkerVault nicht schützt

Angreifer / Szenario	Warum kein Schutz
Ein bereits mit Schadsoftware/Keylogger infiziertes Endgerät	Malware auf dem Gerät selbst sieht Eingaben und entschlüsselte Inhalte direkt an der Quelle — keine Ende-zu-Ende-Verschlüsselung kann das verhindern
Ein Gesprächspartner, der Screenshots macht oder Nachrichten weiterleitet	Grundsätzliche Grenze jeder Chat-Verschlüsselung: Sobald eine Nachricht beim Empfänger entschlüsselt angezeigt wird, kann dieser sie beliebig weiterverwenden
Vollständige Anonymität ohne zusätzliche Massnahmen	Ohne Tor (Kapitel 22.3) sieht der Relay die IP-Adresse der Verbindung; ohne Sealed Sender bleiben Kommunikationsmetadaten sichtbar
Verteilte Angriffe (DDoS) aus vielen IP-Adressen gleichzeitig	Das eingebaute Rate-Limiting schützt vor einzelnen Angreifern, nicht vor koordinierten, verteilten Angriffen (siehe Kapitel 25)
Ein kompromittierter Web-Ausliefer-Server (bei der Web-Variante)	Eine böseartig manipulierte Web-App-Auslieferung könnte theoretisch veränderten Code ausliefern — signierte Desktop-/Mobile-Builds (Kapitel 12) reduzieren dieses Risiko
Social Engineering und Phishing	Kein technisches Werkzeug kann verhindern, dass eine Person freiwillig getauscht wird und selbst Informationen preisgibt

23.3a Drei Angriffsversuche, Schritt für Schritt durchgespielt

Um die abstrakten Aussagen aus 23.2/23.3 greifbarer zu machen, folgen drei narrative Durchspielungen realistischer Angriffsversuche und die jeweiligen Gründe, warum sie an einer bestimmten Stelle scheitern (bzw. in einem Fall tatsächlich erfolgreich wären, um die Grenzen ehrlich zu zeigen).

Versuch 1: Ein Angreifer kompromittiert den Relay-Server vollständig. Ein Angreifer verschafft sich vollen administrativen Zugriff auf die Maschine, auf der der Relay-Prozess läuft, und liest den kompletten Arbeitsspeicher sowie alle gespeicherten Daten aus. Ergebnis: Der Angreifer erhält Konto-Kennungen, öffentliche Schlüssel, Geräte-Metadaten und eine Liste noch nicht zugestellter, aber weiterhin Ende-zu-Ende-verschlüsselter Envelopes (Kapitel 21.1). Er erhält **keinen** lesbaren Nachrichteninhalte, da dieser nie unverschlüsselt auf dem Relay vorlag (Kapitel 21). Der Angriff scheitert am eigentlichen Ziel (Klartext), liefert dem Angreifer aber wertvolle Metadaten — ein Ergebnis, das mit dem in Kapitel 25.2 offen benannten Kompromiss übereinstimmt.

Versuch 2: Ein Angreifer stiehlt ein entsperartes, aber kurzzeitig unbeaufsichtigtes Smartphone. Der Angreifer hat physischen Zugriff auf ein Gerät, dessen Vault gerade entsperart ist (Master-Key im Arbeitsspeicher, Kapitel 11.1). In diesem spezifischen Moment **gelingt** der Zugriff auf den Klartext — dies ist eine reale, in Kapitel 23.3 offen benannte Grenze: Ende-zu-Ende-Verschlüsselung schützt die Übertragung und den Ruhezustand, nicht ein aktiv entsperartes, physisch zugängliches Gerät. Die einzige Abmilderung ist eine kurze automatische Sperrzeit des Geräts selbst (außerhalb des Schutzbereichs von RenkerVault, siehe Checkliste in Kapitel 39.4).

Versuch 3: Ein Angreifer versucht, sich als Bens bereits bekanntes Gerät auszugeben, um Annas Nachrichten mitzulesen. Der Angreifer erzeugt ein neues Schlüsselpaar und versucht, sich beim Relay unter Bens Konto-Kennung, aber mit dem gefälschten Schlüssel anzumelden. Da die Authentifizierung über Ed25519-Challenge-Response läuft (Kapitel 20) und der Relay Bens tatsächlichen öffentlichen Signing-Key bereits kennt, schlägt die Signaturprüfung fehl — der Angreifer kommt nicht einmal bis zur Zustellung irgendeines Envelopes. Versucht der Angreifer stattdessen, ein **neues** Gerät zu Bens Konto hinzuzufügen (etwa weil er zusätzlich Bens Passphrase kennt), greift die in Kapitel 7.3 beschriebene Geräteverwaltung: Das neue Gerät gilt zunächst als „untrusted“ und erhält keine Nachrichten-Warteschlange, bis ein bereits bestehendes, vertrautes Gerät es manuell bestätigt — was Ben, sofern er die Bestätigungsanfrage bemerkt, verweigern würde.

23.4 Nebenkanalangriffe: wenn die Hardware selbst zum Informationsleck wird

Eine Angriffskategorie, die in den vorangegangenen Kapiteln bewusst noch nicht behandelt wurde, weil sie eine eigene Betrachtung verdient: sogenannte „Seitenkanal-“

oder „Nebenkanalangriffe“ (Side-Channel Attacks). Diese Angriffe zielen nicht auf eine mathematische Schwäche des Verschlüsselungsverfahrens selbst, sondern auf physikalische Nebeneffekte, die bei dessen Berechnung zwangsläufig entstehen — etwa den exakten Zeitverlauf einer Berechnung, den Stromverbrauch eines Chips während einer kryptographischen Operation, oder sogar akustische Geräusche, die von elektronischen Bauteilen während intensiver Berechnungen abgestrahlt werden.

Timing-Angriffe: Benötigt eine Operation je nach geheimem Schlüsselwert unterschiedlich lange, kann ein Angreifer allein aus der gemessenen Zeitdauer Rückschlüsse auf den Schlüssel ziehen. Der in Kapitel 44.2 gezeigte `constEq`-Funktion begegnet genau dieser Angriffsklasse für Vergleichsoperationen; die zugrundeliegenden `@noble`-Bibliotheken (Kapitel 13.3b) sind ebenfalls mit dem Ziel konstanten Zeitverhaltens entwickelt.

Power-Analyse: Bei physischem Zugriff auf ein Gerät kann die Messung des Stromverbrauchs während einer kryptographischen Operation (etwa mit einem Oszilloskop an einer Chipkarte) unter Umständen Rückschlüsse auf den verarbeiteten Schlüssel erlauben. Dieser Angriffstyp erfordert allerdings direkten physischen Zugriff auf die Hardware während der Operation — ein Bedrohungsszenario, das außerhalb dessen liegt, wovor Softwarearchitektur wie die von RenkerVault grundsätzlich schützen kann (vergleiche Kapitel 23.3, „kein Schutz gegen ein bereits kompromittiertes Endgerät“).

Warum dieses Risiko für RenkerVault praktisch begrenzt ist: Da RenkerVault in einer gewöhnlichen Browser- oder WebView-Laufzeitumgebung läuft (Kapitel 20.4a), nicht auf spezialisierter, physisch angreifbarer Hardware wie einer Chipkarte, sind die praktisch relevantesten Nebenkanalangriffe in diesem Kontext Timing-Angriffe über Software — genau die Kategorie, die durch konstantzeitige Implementierungen der verwendeten Bibliotheken bereits adressiert wird. Power-Analyse- und akustische Angriffe setzen einen Grad an physischem Zugriff voraus, bei dem ohnehin, wie in Kapitel 23.3 festgehalten, keine Software-Verschlüsselung mehr zuverlässig schützen kann.

24. Vergleich mit anderen Messengern

24.0 Warum so viele Messenger zu ähnlichen Lösungen konvergieren

Bevor einzelne Messenger im Detail verglichen werden, ein übergreifender Befund: Auffällig viele der in diesem Kapitel besprochenen Dienste (Signal, WhatsApp, Wire und, mit eigenen Anpassungen, auch RenkerVault) verwenden im Kern denselben Double-Ratchet-Mechanismus (Kapitel 16) oder ein eng verwandtes Verfahren wie MLS (Kapitel 17.5). Dies ist kein Zufall, sondern ein Zeichen dafür, dass sich in der

Kryptographie-Community über die letzten zehn Jahre ein breiter fachlicher Konsens über die grundsätzliche Architektur sicherer Messaging-Protokolle herausgebildet hat — vergleichbar mit der Art, wie sich im Baugewerbe bestimmte statische Grundprinzipien unabhängig vom konkreten Gebäudeentwurf durchgesetzt haben. Unterschiede zwischen den Diensten liegen daher heute weniger im kryptographischen Grundprinzip selbst als in Fragen der Metadaten-Praxis, der Registrierungsanforderungen, der Betreiberarchitektur (zentral vs. föderiert vs. Zero-Knowledge) und der Zusatzfunktionen wie einem Einbruchsalarm-System — genau den Dimensionen, die in den folgenden Abschnitten verglichen werden.

24.1 Signal

Signal gilt in der Sicherheitsforschung weithin als Referenzimplementierung für Ende-zu-Ende-verschlüsselte Messenger und hat das Double-Ratchet-Protokoll sowie X3DH ursprünglich entwickelt und veröffentlicht. Signal wird zudem regelmäßig extern auditiert und verwendet mit **libsignal** eine geprüfte, produktionsreife Implementierung. RenkerVault übernimmt die grundlegenden Protokollideen (siehe Kapitel 15/16), ohne libsignal selbst zu verwenden (Kapitel 16.5 erläutert, warum), und ergänzt zusätzlich ein eigenes Einbruchsalarm-System (Teil II, Kapitel 10), das in Signal in dieser Form nicht existiert.

24.1a Signals Organisationsform als zusätzlicher Vertrauensfaktor

Neben der rein technischen Reputation ist auch Signals Organisationsform bemerkenswert: Der Dienst wird von der gemeinnützigen Signal Foundation betrieben, finanziert überwiegend durch Spenden statt durch ein werbe- oder datenbasiertes Geschäftsmodell. Diese Struktur reduziert einen wirtschaftlichen Anreiz, der bei kommerziellen, werbefinanzierten Messengern strukturell besteht: den Anreiz, möglichst viele nutzbare Daten über das Kommunikationsverhalten zu sammeln. RenkerVault teilt mit Signal diesen fehlenden kommerziellen Datenverwertungs-Anreiz (Kapitel 2.2), verfolgt aber, anders als die Signal Foundation, kein vergleichbares Spendenmodell oder eine formale Organisationsstruktur — als Prototyp-Projekt (Kapitel 25) fehlt RenkerVault schlicht die institutionelle Reife, die Signal über mehr als ein Jahrzehnt aufgebaut hat.

24.2 Threema

Threema verfolgt einen ähnlichen Ansatz wie RenkerVault in Bezug auf Anonymität (kein zwingendes Telefonnummer- oder E-Mail-Erfordernis) und verwendet ebenfalls das NaCl/libsodium-Kryptographie-Ökosystem. Ein Unterschied: Threema ist ein kommerzielles, geschlossenes Produkt eines einzelnen Unternehmens; RenkerVault ist ein offen dokumentierter Prototyp ohne kommerzielles Geschäftsmodell im Hintergrund.

24.2a NaCl/libsodium im Vergleich zu **@noble**

Ein technischer Vergleichspunkt für Leserinnen und Leser mit Informatik-Hintergrund: Threema baut auf NaCl („Networking and Cryptography Library“, gesprochen wie das englische Wort für Salz) bzw. dessen Weiterentwicklung libsodium auf — einer von Daniel J. Bernstein (demselben Kryptographen, der auch Curve25519 entwickelte, Kapitel 14.2a) mitentwickelten C-Bibliothek, die seit vielen Jahren breit auditiert und produktiv im Einsatz ist. RenkerVault verwendet stattdessen, wie in Kapitel 13.3 beschrieben, die **@noble**-Bibliotheksfamilie — eine in reinem TypeScript geschriebene, von den zugrundeliegenden Algorithmen her aber eng verwandte Alternative (beide nutzen letztlich dieselbe Curve25519-Mathematik, Kapitel 14.2). Der Hauptunterschied liegt in der Implementierungssprache: NaCl/libsodium ist in C geschrieben und wird über sogenannte „Bindings“ in andere Sprachen eingebunden, während **@noble** nativ in TypeScript vorliegt — ein Unterschied, der unmittelbar mit der in Kapitel 13.3a beschriebenen Speichersicherheits-Überlegung zusammenhängt (native C-Bindings können zusätzliche, sprachspezifische Fehlerquellen einführen, die eine reine TypeScript-Implementierung strukturell vermeidet).

24.3 WhatsApp

WhatsApp verwendet für die eigentliche Nachrichtenverschlüsselung ebenfalls das Signal-Protokoll (lizenziert von Signal), speichert jedoch weitreichende Metadaten (u. a. zum Zweck der Kontaktvorschläge und Werbeanzeigen-Auslieferung über das Mutterunternehmen Meta) und lädt standardmäßig das komplette Adressbuch zum Abgleich auf Server hoch — ein Unterschied zum in Kapitel 7.4 beschriebenen gezielten Lookup-Verfahren von RenkerVault.

24.3a Ein instruktives Beispiel: gleiche Kryptographie, unterschiedliches Gesamtschutzniveau

Der Fall WhatsApp verdeutlicht einen wichtigen, oft übersehenen Punkt: Zwei Dienste können exakt dieselbe zugrundeliegende Verschlüsselungstechnologie verwenden (hier: das Signal-Protokoll, Kapitel 15/16) und trotzdem ein unterschiedliches **Gesamtschutzniveau** bieten, wenn sich die umgebende Architektur unterscheidet. Der Nachrichteninhalt selbst ist bei WhatsApp ebenso wie bei RenkerVault Ende-zu-Ende-verschlüsselt und für den Betreiber nicht einsehbar — der Unterschied liegt vollständig außerhalb der reinen Kryptographie: im Umfang der erhobenen Metadaten (Adressbuch-Upload, Kapitel 7.4a), im Geschäftsmodell (werbefinanziert vs. ohne Datenverwertungsanreiz, Kapitel 2.2) und in der Auslieferungskontrolle (zentralisierter App-Store-Vertrieb vs. die in Kapitel 12 beschriebene Multi-Plattform-Offenheit). Diese Beobachtung ist ein praktisches Argument dafür, warum die in Kapitel 23.1 geforderte explizite Bedrohungsmodell-Betrachtung sich nicht auf die reine Verschlüsselungstechnologie beschränken darf, sondern das gesamte umgebende System einbeziehen muss.

24.4 Telegram

Ein häufiges Missverständnis: Telegram-Chats sind **standardmäßig nicht** Ende-zu-Ende-verschlüsselt — nur der optionale Modus „Geheime Chats“ verwendet eine eigens von Telegram entwickelte Verschlüsselung (MTProto), die in der Kryptographie-Community wiederholt wegen unkonventioneller, selbst entwickelter Designentscheidungen kritisiert wurde (siehe auch Kapitel 13.3 zum Prinzip „keine eigene Kryptographie erfinden“). Gruppen und Kanäle bei Telegram sind grundsätzlich nicht Ende-zu-Ende-verschlüsselt.

24.4a Warum „Cloud-Chats“ bei Telegram überhaupt attraktiv sind

Interessant an Telegrams Architektur ist, dass die Standardeinstellung „Cloud-Chats“ (ohne Ende-zu-Ende-Verschlüsselung) nicht bloß eine Sicherheitslücke, sondern eine bewusste Designentscheidung mit einem echten Nutzerkomfort-Vorteil ist: Da Telegram den Nachrichteninhalt zentral auf eigenen Servern speichert (statt nur verschlüsselt weiterzuleiten wie in Kapitel 21.3 für RenkerVaults Offline-Warteschlange beschrieben), lässt sich derselbe Chatverlauf nahtlos auf beliebig vielen Geräten synchronisieren, inklusive vollständiger Historie beim Login auf einem komplett neuen Gerät. RenkerVault erkaufte sich die stärkere Zero-Knowledge-Eigenschaft (Kapitel 21) mit genau diesem Kompromiss: Wie in Kapitel 16.5b beschrieben, geht der Chatverlauf bei einem vollständigen Geräteverlust ohne vorheriges Backup unwiederbringlich verloren, da es keine zentrale, unverschlüsselte Kopie gibt, aus der er sich rekonstruieren ließe. Dieser Vergleich zeigt exemplarisch den grundsätzlichen Zielkonflikt zwischen maximaler Bequemlichkeit (zentrale Synchronisation) und maximaler Vertraulichkeit (Zero-Knowledge), der in unterschiedlichen Messengern unterschiedlich aufgelöst wird.

24.5 Matrix/Element

Matrix ist ein dezentrales, föderiertes Protokoll (vergleichbar mit E-Mail: viele unabhängige Server können miteinander kommunizieren) mit optionaler Ende-zu-Ende-Verschlüsselung über das „Olm/Megolm“-Protokoll, das ebenfalls auf Double-Ratchet-Prinzipien aufbaut. Der wesentliche Architekturunterschied zu RenkerVault: Matrix-Server speichern (verschlüsselte) Nachrichten dauerhaft zur späteren Synchronisation zwischen mehreren Geräten, während RenkerVaults Relay Nachrichten nur bis zur erfolgreichen Zustellung zwischenspeichert.

24.5a Föderation als eigene Architekturkategorie

Matrix' Föderationsmodell verdient eine Einordnung in die bereits in Kapitel 22.4b eingeführte Dreiteilung (zentralisiert / föderiert / Zero-Knowledge-Relay): Bei einem föderierten System betreibt potenziell jede Organisation oder Einzelperson einen eigenen Matrix-Server („Homeserver“), die untereinander nach einem gemeinsamen Protokoll kommunizieren — vergleichbar mit unabhängigen E-Mail-Anbietern, die sich gegenseitig Nachrichten zustellen können. Dies verteilt die Kontrolle über die

Infrastruktur auf viele Hände, bedeutet aber auch, dass Metadaten (wer kommuniziert mit wem) potenziell über mehrere, unterschiedlich vertrauenswürdige und unterschiedlich sorgfältig betriebene Server hinweg sichtbar werden, je nachdem, welche Homeserver an einer Konversation beteiligt sind. RenkerVaults Modell (Kapitel 38: zentral oder selbst betrieben, aber ohne Föderation zwischen mehreren unabhängigen Relays) vermeidet dieses spezifische Risiko, indem für eine gegebene Konversation stets nur ein einziger, von den Beteiligten bewusst gewählter Relay zum Einsatz kommt — auf Kosten der bei Matrix möglichen organisatorischen Dezentralisierung über mehrere unabhängige Betreiber hinweg.

24.6a Weitere Messenger im Kurzportrait

Session: Ein Messenger, der bewusst auf Telefonnummern und E-Mail-Adressen verzichtet (ähnlich wie RenkerVault, Kapitel 7.1) und Nachrichten über ein dezentrales Onion-Routing-Netzwerk (angelehnt an Tor, siehe Kapitel 22.3) weiterleitet, um selbst Metadaten weitgehend zu verschleiern. Der Ansatz ist ambitionierter in Bezug auf Metadatenschutz als RenkerVaults aktueller Stand, bei gleichzeitig geringerer Verbreitung und entsprechend kleinerer Sicherheits-Community, die den Code prüft.

Briar: Ein Messenger, der explizit für Aktivisten und Journalisten in Regionen mit eingeschränkter Internetfreiheit entwickelt wurde und zusätzlich zur klassischen Internetverbindung auch eine direkte Bluetooth- oder WLAN-Kommunikation zwischen Geräten ohne Internetzugang unterstützt — ein Ansatz, der in Extremsituationen (etwa bei gezielter Internetabschaltung) funktioniert, wo RenkerVaults Relay-Modell (Kapitel 6.2) auf eine bestehende Internetverbindung angewiesen bleibt.

Apple iMessage: Verwendet eine eigene, nicht öffentlich vollständig spezifizierte Ende-zu-Ende-Verschlüsselung zwischen Apple-Geräten, ergänzt seit einigen Jahren um eine eigene Post-Quantum-Erweiterung („PQ3“). Ein zentraler Unterschied zu RenkerVault: iMessage ist eng an das Apple-Ökosystem gebunden und funktioniert nicht geräteübergreifend mit Nicht-Apple-Geräten in derselben Ende-zu-Ende-verschlüsselten Weise (Nachrichten an Android-Geräte laufen unverschlüsselt über SMS/RCS, sofern kein zusätzliches, separates Verschlüsselungsprotokoll beider Seiten unterstützt wird).

Wire: Ein aus Europa stammender Ende-zu-Ende-verschlüsselter Messenger mit Fokus auf Unternehmenskunden, der ebenfalls auf dem MLS-Protokoll (Kapitel 17.5) für Gruppenverschlüsselung aufbaut — ein praktisches Beispiel dafür, dass die in Kapitel 17.5 diskutierte MLS-Alternative zu Sender Keys bereits produktiv im Einsatz ist.

SimpleX Chat: Ein vergleichsweise junger Messenger, dessen zentrales architektonisches Merkmal es ist, dass Nutzerinnen und Nutzer überhaupt keine dauerhafte, wiederverwendbare Identifikationsnummer besitzen — anders als bei RenkerVault (Kapitel 7.1, das immerhin eine wiederverwendbare Konto-Kennung erzeugt) verwendet SimpleX für jede einzelne Verbindung separate, nicht miteinander

verknüpfbare Adressen. Dieser Ansatz erschwert die Erstellung eines vollständigen Kontaktgraphen selbst für den Betreiber der Infrastruktur zusätzlich, geht aber auf Kosten des Bedienkomforts (kein einfacher, einmaliger Identitätsaustausch wie bei RenkerVaults Konto-Kennung, Kapitel 7.4).

24.6a2 Ein genauerer Blick auf iMessages PQ3 im Vergleich zu RenkerVaults Hybrid-Handshake

Da Post-Quantum-Kryptographie ein zentrales Thema dieses Buches ist (Teil III, Kapitel 18), lohnt ein genauerer Vergleich zwischen Apples „PQ3“ (kurz erwähnt oben) und RenkerVaults Hybrid-Handshake (Kapitel 18.4): Beide Ansätze verfolgen dasselbe Grundprinzip — die Kombination eines klassischen und eines Post-Quantum-Verfahrens, damit die Sicherheit nicht unter das jeweils bessere Einzelverfahren fällt. Ein von Apple öffentlich hervorgehobener Unterschied ist, dass PQ3 nach eigenen Angaben nicht nur den initialen Handshake, sondern auch eine periodische Neuaushandlung während einer laufenden Konversation post-quantensicher gestaltet — ein Schritt über die in Kapitel 18.6 beschriebene RenkerVault-Grenze („nur der Erstkontakt ist geschützt“) hinaus. Da iMessage jedoch, wie oben erwähnt, closed-source ist, lässt sich diese Behauptung nicht unabhängig auf Code-Ebene verifizieren (Kapitel 44.1a), im Gegensatz zu RenkerVaults öffentlich einsehbarer Implementierung. Diese Gegenüberstellung ist ein Beispiel für einen wiederkehrenden Kompromiss in der Kryptographie: ein möglicherweise fortschrittlicheres, aber nicht unabhängig prüfbares Verfahren gegenüber einem konservativeren, aber vollständig transparenten Ansatz.

24.6 Zusammenfassender Vergleich

Messenger	E2E standardmaessig	Kein Telefonnumm er-Zwang	Post-Quantum-Han dshake	Eingebautes Einbru chsalarm-System
RenkerVault	Ja	Ja	Ja (hybrid, Kapitel 18)	Ja
Signal	Ja	Nein (Telefonnummer erforderlich)	Ja (PQXDH)	Nein
Threema	Ja	Ja	Nein (Stand der oeffentlichen Dokumentation)	Nein
WhatsApp	Ja	Nein	Nein	Nein
Telegram	Nein (nur optional, 1:1)	Nein	Nein	Nein
Matrix/Element	Optional	Ja	Nein	Nein
Session	Ja	Ja	Nein	Nein
Briar	Ja	Ja	Nein	Nein
SimpleX Chat	Ja	Ja	Nein	Nein

Messenger	E2E standardmaessig	Kein Telefonnumm er-Zwang	Post-Quantum-Han dshake	Eingebautes Einbru chsalarm-System
Wire	Ja	Nein (Firmenkonto/E-Mail)	Nein	Nein

24.6b Wie diese Architektur tatsächlich getestet wurde

Über die in Kapitel 25.1a beschriebene, noch ausstehende externe Audit-Prüfung hinaus wurde die in diesem Buch beschriebene Funktionalität während der Entwicklung wiederholt end-to-end verifiziert — nicht nur durch isolierte Modultests, sondern durch tatsächliche Browser-Automatisierung: Es wurden echte Nutzeridentitäten angelegt, echte WebSocket-Handshakes gegen einen laufenden Relay-Server durchgeführt, und reale 1:1- sowie Gruppen-Konversationen zwischen zwei vollständig unabhängigen Client-Instanzen (auf unterschiedlichen Ports simuliert) beobachtet — bis hin zur Verifikation, dass eine testweise verschickte Nachricht nach dem in Kapitel 18 beschriebenen Post-Quantum-Hybrid-Handshake korrekt verschlüsselt und beim Empfänger korrekt entschlüsselt wurde. Diese Art von End-to-End-Test unterscheidet sich von einem klassischen automatisierten Unit-Test dadurch, dass er die tatsächliche Interaktion zwischen zwei unabhängigen Instanzen über eine echte Netzwerkverbindung prüft, statt einzelne Funktionen isoliert mit vorgegebenen Eingaben zu testen — beide Testarten ergänzen sich, ersetzen aber, wie in Kapitel 25.1a betont, kein systematisches externes Sicherheitsaudit.

24.6c Roadmap: konsolidierter Überblick geplanter Weiterentwicklungen

Über das gesamte Buch verstreut wurden mehrere mögliche künftige Erweiterungen erwähnt. Diese Übersicht konsolidiert sie an einer Stelle, sortiert nach der jeweils betroffenen Schutzdimension:

Geplante Erweiterung	Betroffene Dimension	Fundstelle
Externes kryptographisches Audit	Vertrauenswürdigkeit der Protokoll-Komposition	Kapitel 25.1a
Volles X3DH mit One-Time-Prekeys	Asynchronitaets-Garantien	Kapitel 15.5
MLS statt Sender-Keys-lite	Effizienz und PFS bei grossen Gruppen	Kapitel 17.5
ML-DSA (Post-Quantum-Signaturen)	Authentifizierungsschicht	Kapitel 18.6c
Sealed Sender	Metadatenchutz (Absenderidentitaet)	Kapitel 22.4
Cover Traffic	Metadatenchutz (Kommunikationsmuster)	Kapitel 22.4a
Persistenter Server-Store (PostgreSQL)	Produktivbetrieb-Stabilitaet	Kapitel 21.3

Geplante Erweiterung	Betroffene Dimension	Fundstelle
Eigene verschluesselte Backup-/Export-Funktion	Schutz gegen unsichere Betriebssystem-Backups	Kapitel 12.4b
DDoS-Schutz vor dem Reverse-Proxy	Verfuegbarkeit gegen verteilte Angriffe	Kapitel 25.2

Diese Liste ist bewusst als Roadmap, nicht als Versprechen zu verstehen — welche dieser Erweiterungen zuerst umgesetzt wird, hängt von der Priorisierung ab, die typischerweise dem tatsächlichen Risiko (welche Lücke wird in der Praxis am ehesten ausgenutzt) und dem Aufwand (wie viel Entwicklungszeit die jeweilige Erweiterung erfordert) folgt.

24.6d Protokoll-Versionierung: wie künftige Änderungen bestehende Sitzungen betreffen würden

Eine praktische Frage, die jede der in 24.6c genannten Erweiterungen aufwirft: Was passiert mit einer bereits laufenden Double-Ratchet-Sitzung (Kapitel 16), wenn sich das zugrundeliegende Protokoll ändert — etwa durch den Wechsel von Sender-Keys-lite zu MLS (Kapitel 17.5) oder durch die Ergänzung von ML-DSA-Signaturen (Kapitel 18.6c)? Der in Kapitel 16.2 gezeigte Ratchet-Snapshot-Mechanismus (`toSnapshot()`/`fromSnapshot()`, Kapitel 44.4) enthält bereits ein Versionsfeld (`v: 1` im Vault-Dateiformat, Kapitel 44.5), das für genau diesen Zweck vorgesehen ist: Eine künftige Protokolländerung könnte bestehende Sitzungen entweder unverändert im alten Format weiterlaufen lassen, bis beide Seiten auf die neue Version aktualisiert haben (ein Verfahren, das in der Netzwerktechnik „graceful degradation“ genannt wird), oder einen einmaligen, expliziten Re-Handshake für alle bestehenden Kontakte erzwingen. Für tiefgreifende Änderungen wie einen Wechsel des Gruppenverschlüsselungsverfahrens wäre Letzteres wahrscheinlicher — ein einmaliger Übergang, der Nutzerinnen und Nutzer sichtbar gemacht werden müsste, ähnlich einer sichtbaren Safety-Number-Änderung (Kapitel 7.5a).

24.6e Sicherheitsanforderungen und ihre Erfüllung: eine Rückverfolgbarkeitstabelle

Als konsolidierende Zusammenfassung von Teil III eine tabellarische Rückverfolgung, welcher konkrete Mechanismus welche abstrakte Sicherheitsanforderung erfüllt — nützlich sowohl als Nachschlagewerk als auch als Nachweis, dass jede in Kapitel 2 versprochene Eigenschaft tatsächlich auf einen konkreten, benennbaren Mechanismus zurückgeführt werden kann:

Anforderung	Erfuellender Mechanismus	Fundstelle
Vertraulichkeit des Nachrichteninhalts	AES-256-GCM ueber Double-Ratchet-Message-Keys	Kapitel 13.1, 16.2

Anforderung	Erfuellender Mechanismus	Fundstelle
Vertraulichkeit trotz spaeterer Schluesselkompromittierung	Perfect Forward Secrecy	Kapitel 16.2
Selbstheilung nach Kompromittierung	Post-Compromise Security (DH-Ratchet)	Kapitel 16.3
Authentizitaet des Absenders (Relay-Login)	Ed25519-Challenge-Response	Kapitel 20
Schutz vor Man-in-the-Middle beim Erstkontakt	Safety-Number-Verifikation	Kapitel 7.5
Manipulationserkennung der Nachricht	AES-GCM-Auth-Tag + Header als AAD	Kapitel 13.1, 44.4
Manipulationserkennung der lokalen Datenbank	HMAC-SHA256 ueber Ciphertext	Kapitel 11.1, 44.5
Schutz der lokalen Daten im Ruhezustand	Argon2id-abgeleiteter Master-Key + AES-GCM	Kapitel 11.1
Schutz gegen Offline-Brute-Force	Speicherhartes Argon2id	Kapitel 11.2a
Schutz gegen Online-Brute-Force	Lockout + Rate-Limiting	Kapitel 10.2, 10.2a
Schutz bei Geraetediebstahl (gesperrt)	At-Rest-Verschluesselung	Kapitel 11
Schutz bei Zwangsentsperrung	Duress-PIN mit Fake-Ansicht	Kapitel 10.5
Schutz gegen Quantencomputer (Handshake)	ML-KEM-768-Hybrid	Kapitel 18
Gruppenschluessel-Erneuerung bei Mitgliederaustritt	Sender-Keys-lite-Epochenwechsel	Kapitel 17.3
Zero-Knowledge des Betreibers	Architektonische Trennung Client/Relay	Kapitel 6, 21

Wo diese Tabelle bewusst Lücken zeigt (etwa fehlende Zeilen für „Metadatenschutz“ oder „Schutz vor kompromittiertem Endgerät“), verweist das direkt auf die in Kapitel 25.2 und 23.3 offen benannten Grenzen — diese Rückverfolgbarkeitstabelle ist damit selbst ein Instrument der in diesem Buch durchgehend verfolgten Ehrlichkeit über Vollständigkeit (Kapitel 1).

25. Bekannte Grenzen und Selbstkritik

25.1 Warum ein eigenes Kapitel für Grenzen

Wie im Vorwort (Kapitel 1) angekündigt, listet dieses Kapitel bewusst und ohne Beschönigung jede bekannte Einschränkung des aktuellen Prototyps auf — in derselben Ausführlichkeit, mit der zuvor die Stärken beschrieben wurden.

25.1a Wie ein externes Audit praktisch ablaufen würde

Da in diesem Kapitel und an mehreren Stellen zuvor auf ein „externes Audit“ als offenen Punkt verwiesen wird, lohnt eine kurze Erläuterung, was ein solches Audit in der Praxis umfassen würde, um die Aussage greifbarer zu machen. Ein typisches kryptographisches Sicherheitsaudit einer Anwendung wie RenkerVault würde typischerweise folgende Schritte umfassen:

- **Code-Review der Protokoll-Komposition:** Eine unabhängige Prüfstelle liest den vollständigen Quellcode der Kryptographie-Schicht (Kapitel 6.1) Zeile für Zeile und vergleicht ihn mit der zugrundeliegenden Spezifikation (hier: der Signal-Double-Ratchet-Spezifikation sowie den PQXDH-Prinzipien).
- **Formale Modellierung (optional, aber zunehmend Standard):** Für besonders kritische Protokollteile (etwa den X3DH-lite-Handshake) lässt sich mit spezialisierten Werkzeugen (z. B. ProVerif oder Tamarin, in der akademischen Kryptographie verbreitete Werkzeuge zur automatisierten Protokollverifikation) mathematisch nachweisen, dass bestimmte Sicherheitseigenschaften (etwa Perfect Forward Secrecy) unter klar definierten Annahmen tatsächlich gelten — nicht nur informell plausibel erscheinen.
- **Fuzzing und automatisiertes Testen:** Automatisierte Werkzeuge erzeugen gezielt fehlerhafte oder unerwartete Eingaben (z. B. absichtlich beschädigte Envelopes), um zu prüfen, ob die Implementierung in solchen Randfällen kontrolliert fehlschlägt, statt unvorhergesehenes Verhalten zu zeigen.
- **Penetrationstest der Serverkomponente:** Eine gezielte, im Vorfeld abgestimmte Angriffssimulation gegen den Relay-Server (Kapitel 21), um Schwachstellen jenseits der reinen Kryptographie zu finden (etwa in der Verbindungsverwaltung oder im Rate-Limiting, Kapitel 10.2).
- **Abschlussbericht mit eingestuften Befunden:** Ergebnis ist typischerweise ein öffentlicher oder zumindest den Entwicklern zugänglicher Bericht, der gefundene Probleme nach Schweregrad einstuft und konkrete Korrektorempfehlungen enthält.

Ein solches Audit ist mit nicht unerheblichem Aufwand (üblicherweise mehrere Wochen spezialisierter Expertenarbeit) verbunden — der Grund, warum es für den aktuellen Prototyp noch nicht durchgeführt wurde, ist folglich in erster Linie eine Frage der Priorisierung und verfügbarer Ressourcen, nicht eine grundsätzliche Ablehnung der Idee.

25.2 Die vollständige Liste

- **Double-Ratchet- und Vault-Komposition nicht extern auditiert** (Kapitel 16.5) — die Bausteine sind es, die konkrete Zusammensetzung nicht.
- **Kein vollständiges X3DH** (Kapitel 15.5) — keine One-Time-Prekeys, geringfügig schwächere Asynchronitäts-Garantien.
- **Nur der Erstkontakt ist Post-Quantum-geschützt** (Kapitel 18.6), nicht der fortlaufende Ratchet.

- **Relay hält Zustand nur im Arbeitsspeicher** (Kapitel 21.3) — kein persistenter Produktivbetrieb ohne zusätzliche Datenbank-Anbindung.
- **Metadaten sind für den Relay sichtbar** (Kapitel 21.1/22.1) — Sealed-Sender-artige Techniken sind nicht implementiert.
- **Antwort-Zitate und Weiterleitungs-Herkunft laufen als Metadaten**, nicht als verschlüsselte Nutzlast (Kapitel 21.2).
- **Web-Auslieferung kann durch einen kompromittierten Server manipuliert werden** — nur die Desktop-/Mobile-Builds mindern dieses Risiko strukturell.
- **Rate-Limiting schützt nicht vor verteilten Angriffen (DDoS)** aus vielen IP-Adressen gleichzeitig — dafür wäre ein vorgelagertes CDN/DDoS-Schutzangebot nötig.
- **Kein Schutz gegen ein bereits kompromittiertes Endgerät** — Malware, die direkt auf dem Gerät läuft, sieht Inhalte unabhängig von jeder Übertragungsverschlüsselung.
- **Argon2id-Parameter sind ein Kompromiss** zwischen Sicherheit und Entsperr-Latenz auf schwächerer Hardware — für einen dedizierten Produktivbetrieb weiter kalibrierbar.

25.3 Was das für Nutzerinnen und Nutzer praktisch bedeutet

Keine der genannten Grenzen macht RenkerVault unsicherer als vergleichbare, bereits produktiv eingesetzte Messenger — die meisten der genannten Kompromisse (kein vollständiges X3DH, Metadaten-Sichtbarkeit für den Server) teilt RenkerVault mit Signal und anderen etablierten Anwendungen. Der Unterschied liegt darin, dass RenkerVault diese Grenzen aktiv benennt, statt sie zu verschweigen — im Einklang mit dem im Vorwort formulierten Grundsatz, dass ein Sicherheitswerkzeug, das seine eigenen Grenzen offenlegt, für Nutzerinnen und Nutzer letztlich vertrauenswürdiger ist als eines, das ein unrealistisches Vollständigkeitsversprechen macht.

Rechtlicher Hintergrund

Warum es dieses Gesetz gibt, was es vorsieht, und wie RenkerVault rechtlich einzuordnen ist

25a. Ein fairer Blick auf alle Seiten der Verschlüsselungsdebatte

Bevor der rechtliche Teil dieses Buches beginnt, lohnt sich ein Zwischenschritt: eine möglichst faire Darstellung der unterschiedlichen legitimen Interessen, die in der Chat-Kontrolle-Debatte aufeinandertreffen — nicht um eine Seite zu bevorzugen (Kapitel 36.3 hält ausdrücklich fest, dass dieses Buch keine abschließende politische Position bezieht), sondern um zu zeigen, dass es sich um einen echten Zielkonflikt zwischen mehreren begründeten Positionen handelt, nicht um einen Konflikt zwischen einer „guten“ und einer „schlechten“ Seite.

25a.1 Die Perspektive von Strafverfolgungsbehörden und Kinderschutzorganisationen

Strafverfolgungsbehörden und auf Kinderschutz spezialisierte Organisationen weisen zu Recht darauf hin, dass Ende-zu-Ende-Verschlüsselung, so wichtig sie für die Privatsphäre ist, auch für Täter einen Kommunikationsraum schafft, der mit klassischen Ermittlungsmethoden (Beschlagnahme, Überwachung mit richterlicher Anordnung) nicht mehr erreichbar ist. Die in Teil I, Kapitel 3.1 genannten Zahlen zu bereits heute aufgedecktem Material zeigen, dass automatisiertes Scannen tatsächlich vermeidbares Leid aufdeckt und zur Identifikation von Opfern beiträgt, die andernfalls möglicherweise unentdeckt geblieben wären. Aus dieser Perspektive erscheint der Verzicht auf jede Form der Inhaltsprüfung bei Ende-zu-Ende-verschlüsselten Diensten als ein bewusstes Inkaufnehmen zusätzlicher, vermeidbarer Fälle von Kindesmissbrauch — ein Vorwurf, den Kritiker der Chat-Kontrolle-Verordnung ernst nehmen müssen, auch wenn sie zu einer anderen Abwägung kommen.

25a.1a EncroChat und Sky ECC: reale Präzedenzfälle für beide Seiten der Debatte

Zwei tatsächlich stattgefundenene, öffentlich dokumentierte Ermittlungsverfahren werden in der Debatte von beiden Seiten häufig angeführt, allerdings mit gegensätzlicher Schlussfolgerung. 2020 gelang es französischen und niederländischen

Strafverfolgungsbehörden, das Netzwerk des Kommunikationsdienstes „EncroChat“ (spezialisierte, verschlüsselte Mobiltelefone, die überwiegend von organisierter Kriminalität genutzt wurden) zu kompromittieren und über mehrere Wochen große Mengen an Kommunikation mitzulesen, was zu tausenden Festnahmen in mehreren europäischen Ländern führte. Ein Jahr später, 2021, gelang Behörden in einem koordinierten internationalen Verfahren ein vergleichbarer Zugriff auf den ähnlich gelagerten Dienst „Sky ECC“.

Befürworter einer stärkeren Zugriffsbefugnis auf verschlüsselte Kommunikation führen diese Fälle als Beleg dafür an, dass entschlossene Ermittlungsarbeit auch gegen verschlüsselte Systeme erfolgreich sein kann und erhebliche kriminelle Aktivität aufgedeckt hat. Kritiker eines generellen Verschlüsselungs-Zugriffsrechts verweisen dagegen gerade auf die technische Vorgehensweise in beiden Fällen: Die Behörden brachen die zugrundeliegende Verschlüsselung selbst nicht kryptographisch, sondern verschafften sich Zugang zur Infrastruktur der jeweiligen Dienstanbieter bzw. brachten kompromittierte Software auf die Endgeräte auf (eine dem in Kapitel 30.1 beschriebenen Client-Side-Scanning strukturell verwandte, aber im Ergebnis viel gezieltere und auf einen konkreten, bereits unter Verdacht stehenden Dienst beschränkte Methode) — nicht durch eine generelle, alle Nutzer aller Dienste betreffende gesetzliche Zugriffspflicht. Diese beiden Fälle werden von Kritikern (Kapitel 30-31) daher gerade als Argument **gegen** eine pauschale Chatkontrolle-Verpflichtung verwendet: Sie zeigten, dass gezielte, auf konkretem Verdacht beruhende Ermittlungsarbeit gegen spezifisch für kriminelle Zwecke beworbene Dienste möglich ist, ohne dass dafür die Verschlüsselung sämtlicher, überwiegend von unbescholtenen Bürgern genutzter Alltags-Messenger geschwächt werden müsste.

25a.2 Die Perspektive der Kryptographie- und Sicherheitsforschung

Wie in Kapitel 30 ausführlich dargestellt, argumentiert die überwiegende Mehrheit der akademischen Kryptographie-Community, dass ein struktureller Zugriffspunkt auf private Kommunikation — unabhängig vom ursprünglichen Zweck — technisch nicht sicher isoliert werden kann und langfristig zu einer generellen Schwächung der digitalen Sicherheitsinfrastruktur führt, von der auch die ursprünglich zu schützende Personengruppe (Kinder, die zunehmend selbst digitale Geräte nutzen) betroffen wäre.

25a.3 Die Perspektive der Anbieter verschlüsselter Dienste

Anbieter wie Signal (Kapitel 31.4) befinden sich in einer Zwickmühle: Sie müssten im Fall einer verpflichtenden Regelung entweder ihre Verschlüsselungsarchitektur grundlegend verändern (mit den in Kapitel 30 beschriebenen Sicherheitsrisiken), den europäischen Markt verlassen, oder gegen geltendes Recht verstoßen — keine dieser Optionen ist aus ihrer Sicht befriedigend, was die vergleichsweise scharfe öffentliche Positionierung mancher Anbieter erklärt.

25a.4 Die Perspektive gewöhnlicher Nutzerinnen und Nutzer

Für die überwiegende Mehrheit der Menschen, die Chat-Apps für alltägliche, unverfängliche Kommunikation nutzen, ist die Debatte oft abstrakt — die unmittelbaren Auswirkungen einer verpflichtenden Chatkontrolle (etwa in Form geringfügig veränderter App-Funktionalität) wären für sie kaum spürbar, während die in Teil I, Kapitel 3.4 beschriebenen besonders betroffenen Gruppen die Konsequenzen deutlich unmittelbarer erleben würden. Diese Diskrepanz zwischen abstrakter Betroffenheit der Mehrheit und konkreter Betroffenheit einzelner Gruppen erschwert eine breite, informierte öffentliche Debatte zusätzlich.

25a.5 Warum eine faire Darstellung nicht bedeutet, keine Position zu haben

Eine faire Darstellung aller Perspektiven bedeutet nicht, dass alle Positionen gleich gut technisch begründet wären — die in Kapitel 29-30 dargestellten technischen Fakten (insbesondere zur strukturellen Unsicherheit von Client-Side-Scanning) sind Gegenstand breiten fachlichen Konsenses unter Kryptographen, nicht bloß eine von mehreren gleichwertigen Meinungen. Eine faire Darstellung bedeutet aber, anzuerkennen, dass der zugrundeliegende **Zweck** (Kinderschutz) auf allen Seiten der Debatte unbestritten legitim ist — der Dissens betrifft die Frage nach dem richtigen **Mittel**, nicht nach dem richtigen Ziel.

26. Disclaimer und Einordnung

26.1 Dies ist keine Rechtsberatung

Bevor dieser Teil des Buches inhaltlich beginnt, ein Punkt, der wichtiger ist als jede einzelne juristische Aussage darin: **Dieser Teil ersetzt keine anwaltliche oder sonstige professionelle Rechtsberatung.** Er beschreibt den Sach- und Rechtsstand nach bestem verfügbaren Wissen zum Zeitpunkt der Erstellung dieses Buches, verfasst von technisch orientierten Autoren, nicht von Juristinnen oder Juristen. Für eine verbindliche rechtliche Einschätzung — insbesondere für unternehmerische, journalistische oder anderweitig professionelle Nutzung von RenkerVault oder vergleichbaren Werkzeugen — ist stets eine spezialisierte Rechtsberatung im jeweiligen Land einzuholen.

26.2 Gesetzgebung ist ein bewegliches Ziel

Ein zweiter, ebenso wichtiger Vorbehalt: Der in diesem Buch beschriebene Gesetzgebungsprozess zur „Chat-Kontrolle“ befand sich zum Zeitpunkt der Erstellung dieses Textes in einem laufenden, mehrjährigen Verhandlungsprozess auf EU-Ebene. Verordnungsentwürfe werden in solchen Prozessen wiederholt verändert, verschoben,

neu verhandelt oder auch vollständig fallengelassen. Alles, was in diesem Teil zum „aktuellen Stand“ geschrieben steht, ist als Momentaufnahme zu verstehen — Leserinnen und Leser sollten sich vor jeder Entscheidung, die auf diesen Informationen beruht, aktiv über den tatsächlich aktuellen Verhandlungsstand informieren (z. B. über die offiziellen Seiten des Europäischen Parlaments, des Rates der Europäischen Union und etablierter Digitalrechte-Organisationen wie European Digital Rights, EDRI).

26.3 Warum ein technisches Handbuch überhaupt ein Rechtskapitel enthält

Wie im Vorwort begründet, ist RenkerVault als direkte Antwort auf eine konkrete politische Debatte entstanden. Ein Verständnis der reinen Technik (Teil II/III) ohne den politischen und rechtlichen Kontext (dieser Teil) würde nur die halbe Geschichte erzählen — man verstünde zwar, **wie** das Werkzeug funktioniert, aber nicht, **warum** es in dieser spezifischen Form gebaut wurde. Dieser Teil versucht, diese Lücke zu schließen, mit dem in 26.1/26.2 genannten Vorbehalt.

26.4 Umfang dieses Teils

Der rechtliche Fokus dieses Buches liegt auf EU-Recht — konkret der geplanten Verordnung zur Bekämpfung sexuellen Kindesmissbrauchs im Internet (umgangssprachlich „Chat-Kontrolle“), der bestehenden ePrivacy-Ausnahmeregelung, sowie der Datenschutz-Grundverordnung (DSGVO), soweit sie für Ende-zu-Ende-verschlüsselte Kommunikationsdienste relevant ist. Dieser Fokus spiegelt den Ursprung des Projekts wider (siehe Teil I, Kapitel 3) und erhebt keinen Anspruch auf eine vollständige internationale Rechtsvergleichung.

26.5 Die philosophischen Wurzeln des Rechts auf Privatsphäre

Bevor der positivrechtliche Rahmen (die konkreten Gesetzestexte) in den folgenden Kapiteln behandelt wird, lohnt ein kurzer Blick auf die ideengeschichtlichen Wurzeln des Konzepts. Häufig als Ausgangspunkt der modernen Debatte genannt wird der 1890 in der „Harvard Law Review“ erschienene Aufsatz „The Right to Privacy“ der US-amerikanischen Juristen Samuel Warren und Louis Brandeis (letzterer später Richter am Obersten Gerichtshof der USA). Anlass des Aufsatzes war die damals neue Verbreitung handlicher Fotoapparate und der Boulevardpresse, die es erstmals technisch einfach machte, private Momente ungefragt öffentlich zu machen — die Autoren argumentierten, dass diese neue technische Möglichkeit ein bis dahin nicht explizit benanntes Recht sichtbar mache: das „Recht, in Ruhe gelassen zu werden“ („the right to be let alone“).

Die Parallele zur heutigen Debatte ist auffällig: Auch 1890 war der Auslöser der rechtlichen Debatte kein abstraktes philosophisches Bedürfnis, sondern eine **neue Technologie** (die Handkamera), die eine zuvor praktisch garantierte, aber nie explizit

kodifizierte Privatsphäre plötzlich verletzlich machte. Die Chat-Kontrolle-Debatte folgt demselben Muster: Digitale Kommunikationstechnologie hat neue Möglichkeiten sowohl für Privatsphäre (Ende-zu-Ende-Verschlüsselung) als auch für deren Verletzung (Client-Side-Scanning) geschaffen, und der Rechtsrahmen versucht, mit dieser technologischen Entwicklung Schritt zu halten — mit ähnlich kontroversen Debatten wie vor über 130 Jahren.

26.5a Von der „berechtigten Erwartung“ zur informationellen Selbstbestimmung

Zwei weitere ideengeschichtliche Wegmarken verdienen Erwähnung. Im US-amerikanischen Verfassungsrecht prägte die Entscheidung „Katz v. United States“ (1967) den bis heute einflussreichen Maßstab der „berechtigten Erwartung von Privatsphäre“ (reasonable expectation of privacy) — die Frage, ob eine Person unter den gegebenen Umständen vernünftigerweise erwarten durfte, nicht überwacht zu werden, unabhängig vom physischen Ort. Diese Denkfigur beeinflusste weltweit nachfolgende Debatten, auch außerhalb des US-Rechtssystems. Im deutschsprachigen Raum prägte das Bundesverfassungsgericht 1983 im „Volkszählungsurteil“ den Begriff der „informationellen Selbstbestimmung“ — das Recht, grundsätzlich selbst über die Preisgabe und Verwendung der eigenen personenbezogenen Daten zu bestimmen. Dieser Gedanke findet sich strukturell in der DSGVO (Kapitel 33.1a: Zweckbindung, Datenminimierung) wieder und zeigt, dass die europäische Datenschutz-Gesetzgebung nicht ad hoc entstand, sondern auf eine jahrzehntelange verfassungsrechtliche Vorarbeit aufbaut, die dem in Kapitel 27.5a beschriebenen deutschen IT-Grundrecht unmittelbar vorausging.

27. Grundrechte: Artikel 7 und 8 der EU-Grundrechtecharta

27.1 Der Text der Grundrechte

Die Charta der Grundrechte der Europäischen Union (GRCh), seit dem Vertrag von Lissabon 2009 rechtsverbindlicher Bestandteil des EU-Primärrechts, enthält zwei für dieses Thema zentrale Artikel:

Artikel 7 GRCh — Achtung des Privat- und Familienlebens: „Jede Person hat das Recht auf Achtung ihres Privat- und Familienlebens, ihrer Wohnung sowie ihrer Kommunikation.“

Artikel 8 GRCh — Schutz personenbezogener Daten: „Jede Person hat das Recht auf Schutz der sie betreffenden personenbezogenen Daten. Diese Daten dürfen nur nach Treu und Glauben für festgelegte Zwecke und mit Einwilligung der betroffenen Person oder auf einer sonstigen gesetzlich geregelten legitimen Grundlage verarbeitet werden.“

Der ausdrückliche Verweis auf „ihrer Kommunikation“ in Artikel 7 ist für die Chat-Kontrolle-Debatte von zentraler Bedeutung: Er erstreckt den Grundrechtsschutz explizit auf digitale Kommunikationsinhalte, nicht nur auf physische Wohnräume oder klassische Korrespondenz.

27.1a Das Verhältnis von Artikel 7 und Artikel 8: zwei verwandte, aber unterschiedliche Schutzgüter

Ein für Informatiker besonders interessanter Punkt: Artikel 7 und Artikel 8 GRCh schützen zwei unterschiedliche, wenn auch eng verwandte Dinge. Artikel 7 schützt die **Vertraulichkeit** einer Kommunikation als solche — unabhängig davon, ob die Inhalte personenbezogene Daten im technischen Sinne der DSGVO darstellen. Artikel 8 schützt dagegen spezifisch **personenbezogene Daten** und ihre Verarbeitung, mit eigenen, in der DSGVO (Kapitel 33) konkretisierten Grundsätzen wie Zweckbindung und Datenminimierung. Für RenkerVault bedeutet diese Unterscheidung: Die Ende-zu-Ende-Verschlüsselung (Teil III) ist in erster Linie eine technische Umsetzung von Artikel 7 (Schutz der Kommunikation als solcher), während die in Kapitel 33.1a beschriebene Datenminimierung (etwa der Verzicht auf Klarnamen, Kapitel 7.1) eher Artikel 8 zuzuordnen ist. Beide Grundrechte wirken bei RenkerVault zusammen, sind aber konzeptionell unterscheidbar — eine Feinheit, die in vereinfachten Debattenbeiträgen (Kapitel 31.4a) oft unter den Tisch fällt, aber für ein präzises rechtliches Verständnis der Chat-Kontrolle-Debatte relevant bleibt.

27.2 Kein absolutes Recht — die Abwägungsklausel

Weder Artikel 7 noch Artikel 8 GRCh gewähren einen absoluten, uneingeschränkten Schutz. Artikel 52 Absatz 1 GRCh erlaubt Einschränkungen dieser Rechte, sofern sie gesetzlich vorgesehen sind, den Wesensgehalt der Rechte achten, einem von der EU anerkannten, dem Gemeinwohl dienenden Ziel entsprechen (wie dem Schutz von Kindern vor sexuellem Missbrauch — unbestreitbar ein solches Ziel) und **verhältnismäßig** sind. Die gesamte rechtliche Debatte um die Chat-Kontrolle-Verordnung dreht sich im Kern um genau diese Verhältnismäßigkeitsfrage: Ist eine anlasslose, alle Nutzer betreffende Inhaltsprüfung ein verhältnismäßiges Mittel zu einem legitimen Zweck, oder geht sie über das erforderliche Maß hinaus?

27.2a Der dreistufige Verhältnismäßigkeitstest im Detail

Die in 27.2 erwähnte Verhältnismäßigkeitsprüfung nach Artikel 52 GRCh lässt sich in der Rechtsprechung des EuGH typischerweise in drei aufeinander aufbauende Prüfungsschritte gliedern, die für das Verständnis der gesamten Chat-Kontrolle-Debatte hilfreich sind:

- 1 **Geeignetheit:** Ist die Maßnahme überhaupt geeignet, das verfolgte Ziel (hier: Bekämpfung sexuellen Kindesmissbrauchs) zu fördern? Bei dieser Prüfungsstufe würde die grundsätzliche Wirksamkeit automatisierter Erkennungsverfahren (Kapitel 29.2) geprüft — trotz der in Kapitel 29.3 beschriebenen Fehlerquoten dürfte diese erste Stufe im Regelfall bejaht werden, da automatisiertes Scannen nachweislich bereits heute Missbrauchsmaterial aufdeckt (Teil I, Kapitel 3.1).
- 2 **Erforderlichkeit:** Gibt es ein milderes, ebenso wirksames Mittel, das die betroffenen Grundrechte weniger stark einschränkt? Genau an dieser Stufe setzt die in Kapitel 31.1a beschriebene Kritik des EDPS/EDPB an — sie argumentiert, dass gezieltere, auf konkreten Verdacht gestützte Ermittlungsmaßnahmen dasselbe Ziel mit geringerem Grundrechtseingriff erreichen könnten.
- 3 **Angemessenheit (Verhältnismäßigkeit im engeren Sinne):** Steht der Grundrechtseingriff in einem angemessenen Verhältnis zum verfolgten Ziel, selbst wenn die Maßnahme geeignet und erforderlich wäre? Hier wird abgewogen, ob der Nutzen (aufgedecktes Missbrauchsmaterial) die Kosten (potenziell alle Nutzer betreffende Überwachung, Chilling Effects nach Kapitel 27.5, Risiko der Zweckerweiterung nach Kapitel 30.3) überwiegt.

Die in Kapitel 27.3/27.3a beschriebene Rechtsprechung zur Vorratsdatenspeicherung scheiterte im Kern an der zweiten und dritten Stufe dieses Tests: Der EuGH befand eine **anlasslose, pauschale** Speicherung sämtlicher Verkehrsdaten als nicht erforderlich bzw. unangemessen, gerade weil sie nicht nach konkretem Verdacht differenzierte. Ob eine finale Fassung der Chat-Kontrolle-Verordnung denselben Test besteht, hängt maßgeblich davon ab, wie eng oder weit die tatsächlich beschlossenen Aufdeckungsanordnungen (Kapitel 29.1) gefasst sein werden — eine Frage, die sich erst mit der finalen, noch nicht vorliegenden Fassung abschließend beantworten lässt (Kapitel 26.2).

27.3 Relevante Rechtsprechung: Digital Rights Ireland und Schrems

Der Europäische Gerichtshof (EuGH) hat sich bereits mehrfach mit vergleichbaren Fragen befasst. Im Urteil „Digital Rights Ireland“ (2014, verbundene Rechtssachen C-293/12 und C-594/12) erklärte der EuGH die damalige EU-Richtlinie zur anlasslosen Vorratsdatenspeicherung von Telekommunikations-Metadaten für ungültig — mit der zentralen Begründung, eine **pauschale, anlasslose** Speicherung der Kommunikationsdaten der gesamten Bevölkerung, ohne Differenzierung nach tatsächlichem Verdacht, verstoße gegen die Grundsätze der Erforderlichkeit und Verhältnismäßigkeit. Diese Argumentationslinie wurde in nachfolgenden Urteilen (u. a. „Tele2 Sverige/Watson“, 2016, sowie mehreren „Schrems“-Urteilen zum Datentransfer in die USA) wiederholt bestätigt und ausgebaut. Kritiker der Chat-Kontrolle-Verordnung argumentieren, dass eine anlasslose, alle Nutzer betreffende Inhaltsanalyse denselben grundrechtlichen Bedenken unterliegt wie die 2014 für ungültig erklärte Vorratsdatenspeicherung — mit dem Unterschied, dass hier nicht nur Metadaten,

sondern der Kommunikationsinhalt selbst betroffen wäre.

27.3a Weitere Rechtsprechung: La Quadrature du Net und die Schrems-Urteile

Über Digital Rights Ireland hinaus hat der EuGH die in Kapitel 27.3 beschriebene Argumentationslinie in mehreren weiteren Entscheidungen vertieft. Im Urteil „La Quadrature du Net u. a.“ (2020, verbundene Rechtssachen C-511/18, C-512/18 und C-520/18) bestätigte der Gerichtshof erneut, dass eine allgemeine und undifferenzierte Vorratsdatenspeicherung grundsätzlich nicht mit dem EU-Recht vereinbar ist, ließ jedoch enger gefasste Ausnahmen bei einer „ernsten Bedrohung der nationalen Sicherheit“ zu — eine Differenzierung, die zeigt, dass der EuGH nicht jede Form der Datenerhebung kategorisch ausschließt, sondern konsequent eine Verhältnismäßigkeitsprüfung im jeweiligen Einzelfall verlangt.

Die beiden „Schrems“-Urteile (Schrems I, 2015, Rechtssache C-362/14, sowie Schrems II, 2020, Rechtssache C-311/18) betrafen zwar einen anderen Sachverhalt — die Rechtmäßigkeit der Datenübermittlung von der EU in die USA angesichts dortiger Überwachungsprogramme —, bekräftigten aber dieselbe grundsätzliche Haltung des Gerichtshofs: Ein Schutzniveau, das faktisch durch weitreichende, wenig kontrollierte staatliche Zugriffsmöglichkeiten auf personenbezogene Daten unterlaufen wird, genügt den Anforderungen der EU-Grundrechtecharta nicht, unabhängig davon, ob der Zugriff durch ein EU-Land oder durch einen Drittstaat erfolgt. Kritiker der Chat-Kontrolle-Verordnung ziehen aus dieser Linie den Schluss, dass auch ein von der EU selbst geschaffener, strukturell angelegter Zugriffsmechanismus auf private Kommunikation derselben strengen verhältnismäßigkeitsrechtlichen Prüfung standhalten müsste wie eine vergleichbare Maßnahme eines Drittstaats.

27.4 Die Europäische Menschenrechtskonvention als zusätzliche Ebene

Neben der EU-Grundrechtecharta gilt für alle EU-Mitgliedstaaten (als Mitglieder des Europarats, einer von der EU getrennten Organisation) zusätzlich die Europäische Menschenrechtskonvention (EMRK), deren Artikel 8 („Recht auf Achtung des Privat- und Familienlebens“) vom Europäischen Gerichtshof für Menschenrechte (EGMR) in Straßburg ausgelegt wird — eine von EuGH und EU-Grundrechtecharta rechtlich getrennte, aber inhaltlich eng verwandte Schutzebene. Der EGMR hat in mehreren Entscheidungen betont, dass Verschlüsselung eine wichtige Rolle beim Schutz der Privatsphäre und der Meinungsfreiheit spielt.

27.4a Der wichtige Unterschied zwischen EuGH und EGMR

Da beide Institutionen ähnlich klingende Namen tragen und beide „europäische“ Gerichte sind, kommt es in Laiendarstellungen häufig zu Verwechslungen, die für ein

präzises Verständnis der Chat-Kontrolle-Debatte aufgeklärt werden sollten. Der Europäische Gerichtshof (EuGH, Kapitel 27.3) mit Sitz in Luxemburg ist das oberste Gericht der Europäischen Union und für die Auslegung von EU-Recht (darunter die Grundrechtecharta) zuständig — seine Urteile binden unmittelbar alle EU-Mitgliedstaaten und EU-Institutionen. Der Europäische Gerichtshof für Menschenrechte (EGMR) mit Sitz in Straßburg ist dagegen kein EU-Organ, sondern das Gericht des Europarats (einer größeren, von der EU getrennten Organisation mit mehr Mitgliedstaaten, darunter etwa auch das Vereinigte Königreich nach dessen EU-Austritt) und wacht über die Einhaltung der EMRK. Beide Gerichte können in ihrer Rechtsprechung zu ähnlichen, aber rechtlich formal getrennten Ergebnissen kommen — für die Chat-Kontrolle-Verordnung als EU-Rechtsakt ist in erster Linie die Rechtsprechung des EuGH unmittelbar bindend, während EGMR-Urteile eine wichtige, aber formal nachgeordnete Auslegungshilfe darstellen, die vom EuGH regelmäßig berücksichtigt wird.

27.5 Warum Privatsphäre ein soziales, nicht nur ein individuelles Gut ist

Ein verbreitetes Missverständnis in der öffentlichen Debatte lautet sinngemäß: „Wer nichts zu verbergen hat, hat auch nichts zu befürchten.“ Dieses Argument übersieht zwei sozialwissenschaftlich gut belegte Zusammenhänge, die über den rein individuellen Nutzen von Privatsphäre hinausgehen.

Der US-amerikanische Rechtswissenschaftler Daniel J. Solove hat in seiner vielzitierten Taxonomie der Privatsphäre (u. a. in seinem Werk „Understanding Privacy“, 2008) argumentiert, dass Privatsphäre kein einzelnes, klar abgrenzbares Recht ist, sondern ein Bündel unterschiedlicher Schutzinteressen — von Schutz vor Informationssammlung über Schutz vor Informationsverarbeitung bis zu Schutz vor Informationsverbreitung. Das „Nichts zu verbergen“-Argument adressiert bestenfalls einen kleinen Ausschnitt dieses Bündels (die Sorge vor der Aufdeckung konkreten Fehlverhaltens), ignoriert aber die übrigen Dimensionen — etwa das Interesse, selbst zu entscheiden, wer Zugriff auf welche Information über einen selbst erhält, unabhängig davon, ob diese Information belastend ist.

Zweitens belegt sozialwissenschaftliche Forschung zu sogenannten „Chilling Effects“ (abschreckenden Wirkungen), dass bereits das bloße **Wissen** um eine mögliche Überwachung das Verhalten von Menschen verändert — unabhängig davon, ob tatsächlich überwacht wird. Menschen, die glauben, beobachtet zu werden, formulieren Suchanfragen vorsichtiger, äußern politische Meinungen zurückhaltender und suchen seltener nach Informationen zu stigmatisierten Themen (etwa Gesundheitsfragen), selbst wenn ihr Verhalten in keiner Weise rechtswidrig ist. Dieser Effekt betrifft damit nicht nur Personen mit tatsächlich „etwas zu verbergen“, sondern die gesamte Gesellschaft in ihrer Fähigkeit zu freier Meinungsbildung und -äußerung — ein Argument, das über die in Kapitel 27.1/27.2 beschriebenen individuellen Grundrechte

hinausgeht und Privatsphäre als Voraussetzung für eine funktionierende demokratische Öffentlichkeit begreift.

27.5a Das deutsche „IT-Grundrecht“ als nationales Beispiel

Über die EU-Ebene hinaus lohnt exemplarisch ein Blick auf ein einzelnes Mitgliedsland, um zu zeigen, wie sich vergleichbare Grundsätze auch auf nationaler Verfassungsebene wiederfinden: Das deutsche Bundesverfassungsgericht leitete 2008 in einer viel beachteten Entscheidung (zur damals umstrittenen „Online-Durchsuchung“ von Computern durch Sicherheitsbehörden) aus dem allgemeinen Persönlichkeitsrecht des Grundgesetzes ein eigenständiges „Grundrecht auf Gewährleistung der Vertraulichkeit und Integrität informationstechnischer Systeme“ ab — in der Rechtswissenschaft meist kurz „IT-Grundrecht“ oder „Computer-Grundrecht“ genannt. Dieses Grundrecht schützt nach der Rechtsprechung des Gerichts die berechtigzte Erwartung, dass die auf einem eigenen informationstechnischen System (worunter ausdrücklich auch Smartphones fallen) gespeicherten oder verarbeiteten Daten vertraulich bleiben, und zwar unabhängig davon, ob es sich um Kommunikationsinhalte im engeren Sinne handelt.

Die Argumentationslinie dieser Entscheidung ist für die Chat-Kontrolle-Debatte aufschlussreich: Das Gericht begründete das neue Grundrecht ausdrücklich damit, dass die zunehmende Bedeutung informationstechnischer Systeme für die Persönlichkeitsentfaltung eine eigenständige, über die klassischen Grundrechte (etwa das Fernmeldegeheimnis aus Artikel 10 GG) hinausgehende Schutzdimension erfordere. Eine auf dem Gerät selbst ansetzende Analyse-Software, wie sie Client-Side-Scanning voraussetzt (Kapitel 30.1), berührt nach dieser Logik nicht nur die Vertraulichkeit der Kommunikation (Artikel 7 GRCh, Kapitel 27.1), sondern potenziell auch die Integrität des Systems selbst — eine zusätzliche verfassungsrechtliche Dimension, die in der deutschen Fachdiskussion um die Chat-Kontrolle-Verordnung wiederholt herangezogen wird, ohne dass damit eine abschließende Aussage über die tatsächliche verfassungsrechtliche Bewertung einer konkreten, noch nicht verabschiedeten EU-Verordnung getroffen werden könnte (Kapitel 26.1).

28. Geschichte der „Chat-Kontrolle“

28.0 Wie EU-Gesetzgebung überhaupt funktioniert (Kurzeinführung)

Um die in diesem und den folgenden Kapiteln beschriebenen Verhandlungsschritte einordnen zu können, hilft ein kurzer Überblick über das sogenannte „ordentliche Gesetzgebungsverfahren“ der EU, nach dem auch die Chat-Kontrolle-Verordnung behandelt wird:

- 1 **Initiativrecht der Kommission:** Anders als in vielen nationalen Parlamenten kann in der EU praktisch nur die Europäische Kommission neue Gesetzesvorschläge einbringen (Kapitel 28.2 beschreibt den konkreten Vorschlag von 2022).
- 2 **Erste Lesung im Europäischen Parlament:** Das direkt gewählte Europäische Parlament berät den Vorschlag, kann ihn in Ausschüssen bearbeiten und Änderungen vorschlagen, bevor es im Plenum über eine eigene Verhandlungsposition abstimmt (Kapitel 28.3 beschreibt die 2023 angenommene Position).
- 3 **Position im Rat der Europäischen Union:** Parallel erarbeitet der Rat — zusammengesetzt aus je einer bzw. einem Vertreter der Regierung jedes Mitgliedstaats, im jeweils fachlich zuständigen Ressort — eine eigene, gemeinsame Verhandlungsposition (eine „allgemeine Ausrichtung“). Für die meisten Politikbereiche reicht dafür eine „qualifizierte Mehrheit“ (vereinfacht: eine Mehrheit der Mitgliedstaaten, die zugleich eine Mehrheit der EU-Gesamtbevölkerung repräsentiert) — anders als bei manchen anderen EU-Themen ist für die Chat-Kontrolle-Verordnung keine Einstimmigkeit erforderlich, was die in Kapitel 28.3 beschriebene, aber bislang nicht erreichte Mehrheitsfindung umso bemerkenswerter macht.
- 4 **Trilog-Verhandlungen:** Sobald sowohl Parlament als auch Rat eine eigene Position beschlossen haben, verhandeln Vertreter beider Institutionen gemeinsam mit der Kommission informell einen Kompromisstext (den „Trilog“). Dieser informelle Charakter wird von Transparenz-Organisationen gelegentlich kritisiert, da Trilog-Sitzungen anders als Plenardebatten nicht öffentlich stattfinden.
- 5 **Formelle Annahme:** Der im Trilog gefundene Kompromiss muss anschließend formell sowohl vom Parlament als auch vom Rat bestätigt werden, bevor er als Verordnung in Kraft tritt.

Der in Kapitel 28.3/28.4 beschriebene, über Jahre andauernde Stillstand lässt sich mit diesem Ablauf präzise verorten: Die Verhandlungen befinden sich seit 2022 im Wesentlichen in Schritt 3 — der Rat hat über Jahre hinweg keine ausreichende qualifizierte Mehrheit für eine gemeinsame Position gefunden, wodurch Schritt 4 (der eigentliche Trilog) bislang nicht in entscheidungsreifer Form beginnen konnte.

28.0a Verordnung vs. Richtlinie: ein wichtiger Rechtsbegriff

Für das Verständnis der folgenden Kapitel hilft eine Begriffsklärung, die im deutschen Sprachgebrauch gelegentlich durcheinanderght: Eine EU-**Verordnung** (wie die geplante Chat-Kontrolle-Verordnung, Kapitel 29, oder die bereits geltende DSGVO, Kapitel 33) gilt unmittelbar und einheitlich in allen Mitgliedstaaten, ohne dass es einer nationalen Umsetzung bedarf. Eine EU-**Richtlinie** (wie die aktuell noch geltende ePrivacy-Richtlinie, Kapitel 34) gibt dagegen nur ein verbindliches Ziel vor, das jeder Mitgliedstaat durch eigene nationale Gesetze innerhalb einer bestimmten Frist umsetzen muss — was in der Praxis zu Unterschieden in der konkreten nationalen Ausgestaltung

führen kann. Dieser Unterschied erklärt, warum die geplante Ablösung der ePrivacy-Richtlinie durch eine ePrivacy-**Verordnung** (Kapitel 34.2) mehr ist als eine bloße Umbenennung: Sie würde die heute noch möglichen nationalen Unterschiede in der Umsetzung beseitigen und stattdessen unmittelbar einheitliches Recht in allen Mitgliedstaaten schaffen — genau wie es bei der Chat-Kontrolle-Verordnung von vornherein als Verordnung, nicht als Richtlinie, vorgesehen ist.

28.1 Der Ausgangspunkt: die „Interim“-Ausnahmeregelung 2021

Im Juli 2021 trat eine zeitlich befristete EU-Verordnung in Kraft (informell als „Chat-Kontrolle 1.0“ oder „Interim-Verordnung“ bezeichnet), die es Anbietern von Kommunikationsdiensten **freiwillig** erlaubte, automatisiert nach bekanntem Material sexuellen Kindesmissbrauchs sowie nach „Grooming“-Anzeichen (Anbahnungsversuchen) zu suchen — eine Ausnahme von der andernfalls strengeren ePrivacy-Richtlinie (siehe Kapitel 34), die eine solche automatisierte Inhaltsanalyse grundsätzlich untersagen würde. Diese Regelung war als Übergangslösung gedacht, bis eine dauerhafte, umfassendere Regelung beschlossen würde.

28.2 Der Kommissionsvorschlag von Mai 2022

Im Mai 2022 veröffentlichte die Europäische Kommission ihren Vorschlag für eine dauerhafte Verordnung („CSA-Verordnung“, von Child Sexual Abuse), die über die freiwillige Interim-Regelung deutlich hinausgeht: Anstelle einer freiwilligen Möglichkeit sollten Anbieter — nach vorheriger behördlicher „Aufdeckungsanordnung“ (Detection Order) im Einzelfall — verpflichtet werden können, systematisch nach bekanntem und neuem Missbrauchsmaterial sowie nach Grooming-Anzeichen zu suchen. Der Vorschlag enthält ausdrücklich keine Ausnahme für Ende-zu-Ende-verschlüsselte Dienste — was in der öffentlichen Debatte den Namen „Chat-Kontrolle 2.0“ bzw. schlicht „Chat-Kontrolle“ etablierte.

28.2a Die für die Kommission zuständige Generaldirektion

Innerhalb der Europäischen Kommission wurde der Vorschlag maßgeblich von der Generaldirektion Migration und Inneres (kurz „DG HOME“) vorbereitet, die für Fragen der inneren Sicherheit und Strafverfolgungsk Kooperation innerhalb der Kommission zuständig ist. Diese organisatorische Zuordnung ist selbst Gegenstand eines gelegentlich vorgebrachten Kritikpunkts: Kritiker argumentieren, dass eine primär sicherheitsbehördlich geprägte Generaldirektion tendenziell stärker die Perspektive der Strafverfolgung (Kapitel 25a.1) als die der Grundrechte und des Datenschutzes gewichten könnte — eine andere Zuordnung, etwa zur für Justiz und Grundrechte zuständigen Generaldirektion, hätte möglicherweise zu einem anderen Ausgangsvorschlag geführt. Diese Beobachtung ist keine Aussage über die fachliche Qualität der beteiligten Kommissionsmitarbeiter, sondern eine allgemeine Beobachtung darüber, wie organisatorische Zuständigkeiten die Ausgangsperspektive eines

Gesetzesvorschlags mitprägen können, bevor er überhaupt in die in Kapitel 28.0 beschriebene interinstitutionelle Verhandlung eintritt.

28.2b Der LIBE-Ausschuss: wo der Vorschlag im Parlament bearbeitet wurde

Innerhalb des Europäischen Parlaments werden Gesetzesvorschläge zunächst in einem fachlich zuständigen Ausschuss beraten, bevor sie ins Plenum zur Abstimmung durch alle Abgeordneten kommen. Für die Chat-Kontrolle-Verordnung war dies der Ausschuss für bürgerliche Freiheiten, Justiz und Inneres (im EU-Jargon unter seinem englischen Kürzel „LIBE“-Ausschuss bekannt, von „Committee on Civil Liberties, Justice and Home Affairs“) — derselbe Ausschuss, der typischerweise auch für Datenschutz- und Grundrechtsfragen zuständig ist, was die inhaltliche Nähe zu den in Kapitel 27 beschriebenen Grundrechtsfragen unterstreicht. Die im November 2023 angenommene Parlamentsposition (28.3) durchlief zunächst eine ausführliche Beratung und Abstimmung innerhalb dieses Ausschusses, bevor sie im Plenum bestätigt wurde — ein für EU-Gesetzgebungsverfahren typischer, mehrstufiger interner Beratungsprozess innerhalb der in Kapitel 28.0 beschriebenen Stufe 2.

28.3 Der Streit im Rat der Europäischen Union

Anders als oft dargestellt, ist die Chat-Kontrolle-Verordnung (Stand der hier verwendeten Quellen) noch nicht final beschlossen. Der Vorschlag muss sowohl vom Europäischen Parlament als auch vom Rat der Europäischen Union (den Regierungen der Mitgliedstaaten) in identischer Fassung angenommen werden. Das Europäische Parlament positionierte sich in seiner Verhandlungsposition (angenommen im November 2023) deutlich zurückhaltender als der ursprüngliche Kommissionsvorschlag: Es sprach sich unter anderem gegen eine verpflichtende, pauschale Chatkontrolle bei Ende-zu-Ende-verschlüsselten Diensten aus und für eine Beschränkung auf gezielte, risikobasierte Maßnahmen. Im Rat der Europäischen Union blieb die Position über mehrere Jahre hinweg gespalten: Mehrere Ratspräsidentschaften legten Kompromissvorschläge vor, die wiederholt an einer nicht erreichten qualifizierten Mehrheit scheiterten, unter anderem weil eine Gruppe von Mitgliedstaaten (zeitweise angeführt von Deutschland, zusammen mit u. a. Österreich, Polen, den Niederlanden und weiteren) eine verpflichtende Aufweichung von Ende-zu-Ende-Verschlüsselung ablehnte.

28.4 Wiederkehrende Kompromissvorschläge

Über die Verhandlungsjahre hinweg wurden mehrere Kompromissvarianten diskutiert, unter anderem:

- eine Beschränkung der verpflichtenden Detection Orders auf unverschlüsselte Dienste, mit einer bloßen „freiwilligen“ Fortführung für verschlüsselte Dienste,

- die Einführung einer verpflichtenden Altersverifikation für App-Stores und bestimmte Dienste als alternativer oder ergänzender Ansatz,
- eine zeitlich befristete Verlängerung der bestehenden freiwilligen Interim-Regelung (Kapitel 28.1) anstelle einer sofortigen Einigung auf die dauerhafte Verordnung.

Diese wiederholten Verlängerungen der Interim-Regelung (zuletzt jeweils um ein bis zwei Jahre) sind selbst ein politisches Signal: Sie zeigen, dass weder der ursprüngliche Kommissionsvorschlag noch eine der Kompromissvarianten bislang eine ausreichende Mehrheit gefunden hat, ohne dass die Grundsatzfrage (verpflichtendes Scannen ja oder nein, inklusive verschlüsselter Dienste ja oder nein) endgültig entschieden wäre.

28.4a Positionen einzelner Mitgliedstaaten im Rat (Momentaufnahme)

Da Entscheidungen im Rat der Europäischen Union von den einzelnen Regierungen der Mitgliedstaaten getroffen werden, lohnt ein Blick auf einige besonders häufig in Berichterstattung und Fachanalysen genannte nationale Positionen — ausdrücklich als Momentaufnahme zu verstehen (Kapitel 26.2), da sich Regierungspositionen mit Wahlen und Koalitionswechseln ändern können:

- **Deutschland:** Über mehrere Legislaturperioden hinweg positionierte sich die Bundesregierung wiederholt gegen eine verpflichtende Chatkontrolle bei Ende-zu-Ende-verschlüsselten Diensten, mit Verweis auf die grundrechtlichen Bedenken aus Kapitel 27 sowie auf die Position mehrerer im Digital- und Rechtsbereich zuständiger Ministerien. Diese Position war jedoch nicht über die gesamte Verhandlungsdauer hinweg unumstritten, da beteiligte Ressorts (etwa mit Zuständigkeit für Inneres bzw. für Justiz und Verbraucherschutz) im Verlauf der Jahre unterschiedliche Schwerpunkte setzten.
- **Frankreich:** Positionierte sich im Verhandlungsverlauf zeitweise tendenziell aufgeschlossener gegenüber weitreichenderen Detection-Order-Befugnissen, unter anderem mit Verweis auf die Notwendigkeit wirksamer Ermittlungsinstrumente für Strafverfolgungsbehörden.
- **Niederlande:** Gehörte zu den Mitgliedstaaten, die wiederholt auf die technischen Bedenken aus Kapitel 30 verwiesen und sich für eine engere, stärker risikobasierte Fassung der Verordnung aussprachen.
- **Polen:** Positionierte sich in mehreren Verhandlungsrunden ebenfalls kritisch gegenüber einer pauschalen, alle Ende-zu-Ende-verschlüsselten Dienste einschließenden Verpflichtung.
- **Österreich:** Vertrat über weite Strecken der Verhandlungen eine Position, die Ende-zu-Ende-Verschlüsselung explizit von einer verpflichtenden Detection-Order-Regelung ausnehmen wollte.

- **Spanien:** Zeigte sich in mehreren Verhandlungsrunden tendenziell aufgeschlossener gegenüber weitreichenderen Erkennungsbefugnissen, ähnlich der französischen Position.
- **Schweden:** Positionierte sich, auch vor dem Hintergrund einer traditionell starken nationalen Datenschutzkultur, wiederholt kritisch gegenüber einer pauschalen Einbeziehung Ende-zu-Ende-verschlüsselter Dienste.
- **Belgien:** Als Sitz der EU-Institutionen und häufiger Ratspräsidentschaftsinhaber spielte Belgien in mehreren Verhandlungsrunden eine vermittelnde Rolle zwischen den unterschiedlichen Lagern, ohne sich dauerhaft eindeutig einer Seite zuzuordnen.

Diese Positionen sind, wie in Kapitel 26.2 betont, keine feststehenden Fakten, sondern ein sich veränderndes politisches Bild — sie werden hier genannt, um die Breite und Vielschichtigkeit der innereuropäischen Debatte zu illustrieren, nicht um eine abschließende Einschätzung der aktuellen Mehrheitsverhältnisse im Rat zu treffen.

28.4b Was bereits ohne die neue Verordnung strafbar ist

Ein in der öffentlichen Debatte gelegentlich untergehender, aber wichtiger Punkt: Sexueller Missbrauch von Kindern und die Herstellung, der Besitz und die Verbreitung entsprechenden Materials sind bereits nach geltendem Strafrecht sämtlicher EU-Mitgliedstaaten und nach internationalen Übereinkommen (etwa der Lanzarote-Konvention des Europarats) durchweg strafbar — die Chat-Kontrolle-Verordnung schafft also keinen neuen Straftatbestand, sondern zielt auf ein zusätzliches **Ermittlungs- und Erkennungsinstrument** für bereits nach geltendem Recht verbotenes Verhalten. Diese Unterscheidung ist für die Debatte wichtig: Die Kontroverse betrifft nicht die Frage, ob die zugrundeliegenden Taten strafbar sein sollen (darüber besteht Einigkeit), sondern ausschließlich die Frage, mit welchen technischen Mitteln entsprechende Inhalte aufgespürt werden dürfen und sollen, und welche Kollateralwirkungen dabei auf die Grundrechte unbeteiligter Dritter in Kauf genommen werden (Kapitel 27). Strafverfolgungsbehörden verfügen bereits heute — auch ohne die neue Verordnung — über etablierte Ermittlungsinstrumente wie Durchsuchungsbeschlüsse, Beschlagnahme von Geräten bei konkretem Verdacht und internationale Rechtshilfeabkommen; die Debatte dreht sich um die Frage, ob und in welchem Umfang diese bereits bestehenden, auf Einzelfallverdacht gestützten Instrumente durch ein zusätzliches, anlassloses Massenerkennungsverfahren ergänzt werden sollten.

28.5 Warum dieses Kapitel bewusst keine abschließende Zeitangabe macht

Angesichts des in Kapitel 26.2 beschriebenen, mehrjährig instabilen Verhandlungsverlaufs verzichtet dieses Buch bewusst darauf, einen finalen „Ausgang“ der Verhandlungen zu behaupten. Was sich über den gesamten bisherigen Verlauf hinweg als stabil erwiesen hat, ist die grundsätzliche Konfliktlinie: auf der einen Seite

das legitime, unbestrittene Ziel des Kinderschutzes; auf der anderen Seite die ebenso begründete Sorge von Kryptographen, Datenschutzbehörden und Zivilgesellschaft vor einem strukturellen, kaum wieder rückgängig zu machenden Überwachungsmechanismus (ausführlich in Kapitel 30/31).

29. Der Verordnungsentwurf im Detail

29.1 Kernelemente des Kommissionsvorschlags

Der ursprüngliche Kommissionsvorschlag von 2022 (Kapitel 28.2) enthält mehrere ineinandergreifende Mechanismen:

- **Risikobewertung:** Anbieter müssen das Risiko einschätzen, dass ihr Dienst zur Verbreitung von Missbrauchsmaterial oder zu Grooming genutzt wird.
- **Aufdeckungsanordnung (Detection Order):** Eine benannte Behörde kann, gestützt auf die Risikobewertung und mit gerichtlicher oder vergleichbarer Genehmigung, einen Anbieter verpflichten, systematisch nach definierten Kategorien von Inhalten zu suchen.
- **Meldepflicht:** Als Missbrauch eingestufte Funde müssen an ein neu zu schaffendes EU-Zentrum gemeldet werden.
- **Altersverifikation:** Für bestimmte Dienste (etwa App-Stores) sollen verpflichtende Altersnachweise eingeführt werden.
- **Entfernungsanordnungen und Zugangssperren:** Für bereits identifiziertes Material sollen Anbieter zur Entfernung verpflichtet werden können; als letztes Mittel sind auch Netzsperren vorgesehen.

29.1a Wie sich der Vorschlag von seinem Vorgänger unterscheidet — eine Kurzformel

Um den Sprung von der in Kapitel 28.1 beschriebenen freiwilligen Interim-Regelung zum in 29.1 aufgeschlüsselten Kommissionsvorschlag auf eine einprägsame Formel zu bringen: „Freiwillig → verpflichtend“, „nur bekanntes Material → auch neues Material und Grooming“, „keine Ausnahme für E2E → ausdrücklich auch E2E-Dienste betroffen“, und „kein eigenes EU-Zentrum → neues, dediziertes EU-Zentrum als zentrale Meldestelle“. Jede dieser vier Verschiebungen für sich genommen würde bereits eine erhebliche Ausweitung bedeuten — in Kombination erklären sie, warum der 2022er-Vorschlag in der öffentlichen Wahrnehmung einen so viel schärferen Bruch darstellt als die bereits seit 2021 laufende, vergleichsweise unauffällige Interim-Regelung.

29.2 Die drei Kategorien gesuchter Inhalte

Der Vorschlag unterscheidet technisch zwischen drei Erkennungsaufgaben, die jeweils unterschiedliche technische Verfahren erfordern:

Kategorie	Erkennungsmethode	Technische Reife
Bekanntes Material (bereits identifizierte Bilder/Videos)	Hash-Abgleich (Perceptual Hashing, z. B. PhotoDNA) gegen eine Referenzdatenbank	Vergleichsweise ausgereift, aber nicht fehlerfrei (siehe 29.3)
Neues, bisher unbekanntes Material	Automatisierte Bilderkennung/KI-Klassifikation	Deutlich fehleranfälligster als Hash-Abgleich
Grooming (textbasierte Anbahnungsversuche)	Automatisierte Textanalyse/KI-Klassifikation von Gesprächsinhalten	Am fehleranfälligsten; erfordert Analyse des gesamten Gesprächsverlaufs, nicht nur einzelner Bilder

Diese Differenzierung ist zentral für die technische Kritik in Kapitel 30: Während Hash-Abgleich gegen bekanntes Material vergleichsweise treffsicher funktioniert (auch wenn er nicht fehlerfrei ist), sind Klassifikationsverfahren für neues Material und insbesondere für Grooming-Erkennung nach dem Stand der Forschung deutlich fehleranfälliger, was bei den in Teil I, Kapitel 3.1 genannten Nutzerzahlen zu erheblichen absoluten Fehlerzahlen führen würde.

29.3 Bekannte Schwächen von Perceptual-Hashing-Verfahren

Selbst das vergleichsweise ausgereifte Perceptual-Hashing-Verfahren (bei dem nicht der exakte Dateinhalt, sondern ein toleranter „Fingerabdruck“ verglichen wird, der auch bei geringfügigen Bildveränderungen noch übereinstimmt) ist in der Sicherheitsforschung wiederholt durch gezielt konstruierte „Kollisionen“ ausgehebelt worden — Fälle, in denen ein harmloses Bild absichtlich so verändert wurde, dass es denselben Hash wie ein in der Referenzdatenbank gelistetes Bild erzeugt, oder umgekehrt ein tatsächlich problematisches Bild minimal verändert wurde, um der Erkennung zu entgehen. Diese technischen Schwächen sind kein Totschlagargument gegen den Einsatz von Hash-Abgleich insgesamt, zeigen aber, dass auch die technisch ausgereifteste der drei genannten Kategorien keine hundertprozentige Zuverlässigkeit bietet.

29.4 Der Lebenszyklus einer Aufdeckungsanordnung — ein durchgespieltes Beispiel

Um den in 29.1 abstrakt beschriebenen Mechanismus konkreter zu machen, ein vereinfachter, illustrativer Ablauf, wie eine Aufdeckungsanordnung nach dem ursprünglichen Kommissionsvorschlag funktionieren könnte (dies ist eine vereinfachte Illustration des Vorschlagsmechanismus, keine Wiedergabe eines bereits geltenden, verbindlichen Verfahrens, siehe Kapitel 26.2):

- Risikobewertung:** Ein Anbieter eines Kommunikationsdienstes führt eine verpflichtende Selbsteinschätzung durch, wie hoch das Risiko ist, dass sein Dienst

zur Verbreitung von Missbrauchsmaterial oder zu Grooming genutzt wird — Faktoren wären etwa die Nutzerzahl, das Vorhandensein von Bild-/Videofunktionen und die Verfügbarkeit von Ende-zu-Ende-Verschlüsselung.

- 2 **Behördliche Prüfung:** Eine benannte nationale Koordinierungsstelle (oder das im Vorschlag vorgesehene EU-Zentrum) prüft diese Risikobewertung.
- 3 **Antrag auf gerichtliche oder vergleichbare Genehmigung:** Bei ausreichend hohem eingeschätztem Risiko kann eine Aufdeckungsanordnung beantragt werden, die von einer Justiz- oder vergleichbaren unabhängigen Behörde genehmigt werden müsste.
- 4 **Umsetzung durch den Anbieter:** Der betroffene Anbieter müsste die angeordnete Erkennungstechnologie implementieren — bei einem Ende-zu-Ende-verschlüsselten Dienst liefe dies zwangsläufig auf eine Form von Client-Side-Scanning hinaus (Kapitel 30.1).
- 5 **Meldung von Funden:** Als Missbrauch eingestufte Inhalte würden an das vorgesehene EU-Zentrum gemeldet, das die Meldung an nationale Strafverfolgungsbehörden weiterleiten würde.
- 6 **Regelmäßige Überprüfung:** Eine Aufdeckungsanordnung wäre nach dem Vorschlag zeitlich befristet und müsste bei fortbestehendem Bedarf erneut beantragt und genehmigt werden.

Der in Kapitel 30 dargestellte technische Kritikpunkt setzt konkret bei Schritt 4 an: Unabhängig davon, wie sorgfältig die Schritte 1-3 rechtlich und verfahrensmäßig ausgestaltet sind, bliebe die technische Umsetzung in Schritt 4 bei einem tatsächlich Ende-zu-Ende-verschlüsselten Dienst notwendigerweise eine Form von Client-Side-Scanning mit den in Kapitel 30.2/30.3 beschriebenen strukturellen Risiken.

29.3a Vorgesehene rechtsstaatliche Kontrollmechanismen

Um die Kritik aus Kapitel 30/31 ausgewogen einzuordnen, sei ausdrücklich erwähnt, dass der Kommissionsvorschlag selbst mehrere Kontrollmechanismen vorsieht, die eine unbegrenzte, willkürliche Anwendung verhindern sollen: Aufdeckungsanordnungen sollen zeitlich befristet und auf bestimmte, im Vorfeld definierte Risikokategorien beschränkt sein; sie bedürfen einer Genehmigung durch eine Justizbehörde oder eine vergleichbare unabhängige Stelle; und betroffene Anbieter sollen die Möglichkeit haben, eine Anordnung rechtlich anzufechten. Kritiker (Kapitel 30-31) bezweifeln nicht das Vorhandensein dieser Kontrollmechanismen als solches, sondern argumentieren, dass sie das darunterliegende technische Grundproblem — die Notwendigkeit von Client-Side-Scanning bei Ende-zu-Ende-verschlüsselten Diensten, sobald überhaupt eine Anordnung ergeht — nicht auflösen können, da selbst eine sorgfältig kontrollierte Anordnung technisch denselben strukturellen Zugriffspunkt voraussetzt (Kapitel 30.1). Diese Unterscheidung — verfahrensrechtliche Kontrolle vs. technische Grundproblematik — ist zentral, um die Debatte nicht als bloßen Vorwurf mangelnder

Sorgfalt der Gesetzgeber misszuverstehen.

29.4a Das vorgesehene EU-Zentrum: Aufgaben und Kritikpunkte

Der Kommissionsvorschlag sieht die Schaffung eines neuen „EU-Zentrums zur Verhinderung und Bekämpfung von sexuellem Missbrauch von Kindern“ vor, das mehrere zentrale Aufgaben übernehmen soll: die Pflege und Verteilung von Referenzdatenbanken bekannten Missbrauchsmaterials an meldepflichtige Anbieter (Kapitel 29.2), die Entgegennahme und Weiterleitung von Verdachtsmeldungen an nationale Strafverfolgungsbehörden, sowie eine unterstützende und koordinierende Funktion gegenüber den nationalen Koordinierungsstellen aus dem in 29.4 beschriebenen Verfahren.

Kritiker verweisen bei dieser vorgesehenen neuen Behörde auf zwei Punkte: Erstens würde eine solche zentrale Stelle selbst zu einem attraktiven Angriffsziel, da sie zwangsläufig eine große Menge sensibler Referenzdaten sowie eingehender Verdachtsmeldungen (die naturgemäß auch False Positives enthalten, Kapitel 29.3) konzentrieren würde — ein strukturelles Risiko, das unabhängig von der Integrität der dort arbeitenden Personen besteht. Zweitens wird die Frage aufgeworfen, wie die Aufsicht über eine derart neue, mit weitreichenden Befugnissen ausgestattete Behörde ausgestaltet werden sollte, um Fehlentwicklungen frühzeitig zu erkennen — eine Frage, die in ähnlicher Form bereits bei der Einrichtung anderer EU-Agenturen diskutiert wurde und keine RenkerVault-spezifische, sondern eine allgemeine Frage der EU-Verwaltungsarchitektur ist.

29.5 Altersverifikation: ein eigenständiges, oft übersehenes Streitthema

Neben der in Kapitel 30 im Zentrum stehenden Debatte um Inhaltsscanning enthält der Verordnungsvorschlag (Kapitel 29.1) auch Elemente einer verpflichtenden Altersverifikation, etwa für App-Stores. Dieser Aspekt wird in der öffentlichen Debatte häufig weniger diskutiert als das Scanning-Thema, wirft aber eigene, davon unabhängige Datenschutzfragen auf: Eine zuverlässige Altersverifikation erfordert in der Praxis meist entweder den Abgleich mit einem amtlichen Ausweisdokument oder biometrische Schätzverfahren (etwa eine Alterseinschätzung anhand eines Gesichtsbildes) — beide Ansätze bringen eigene, von der Chatkontrolle-Scanning-Debatte losgelöste Datenschutzrisiken mit sich: Ausweisdaten-Abgleiche schaffen neue, potenziell missbrauchbare Datenbanken über die Nutzung bestimmter Dienste, während biometrische Schätzverfahren mit eigenen, in der Forschung dokumentierten Genauigkeitsproblemen behaftet sind, die je nach Hauttyp, Geschlecht und Alter unterschiedlich stark ausfallen können. Eine vollständige rechtliche und technische Bewertung von Altersverifikationsverfahren würde den Rahmen dieses Buches sprengen — sie wird hier lediglich als eigenständiger, von der Kernkritik an Client-Side-Scanning zu unterscheidender Diskussionsstrang benannt.

29.6 Netzsperrern als letztes vorgesehenes Mittel

Der in Kapitel 29.1 knapp erwähnte Mechanismus der „Zugangssperren“ verdient eine genauere Betrachtung, da er in der öffentlichen Debatte oft im Schatten der prominenteren Scanning-Frage steht. Der Vorschlag sieht vor, dass Internetanbieter als letztes Mittel verpflichtet werden könnten, den Zugang zu bestimmten, bereits identifizierten Domains oder URLs zu sperren, auf denen Missbrauchsmaterial gehostet wird. Diese Art von Netzsperrern ist technisch von der in Kapitel 30 diskutierten Chatkontrolle klar zu unterscheiden — sie betrifft nicht die private Individualkommunikation über Messenger, sondern öffentlich abrufbare Inhalte auf Webseiten. Kritiker verweisen dennoch auf zwei bekannte technische Schwächen von Netzsperrern im Allgemeinen: Erstens lassen sie sich mit vergleichsweise einfachen Mitteln (etwa alternativen DNS-Servern oder VPN-Diensten) häufig umgehen, was ihre tatsächliche Wirksamkeit gegenüber technisch versierten Tätern begrenzt, während weniger versierte, zufällig auf gesperrte Inhalte stoßende Personen stärker betroffen wären. Zweitens besteht auch hier das in Kapitel 30.3 beschriebene Function-Creep-Risiko: Eine einmal etablierte technische Sperrinfrastruktur bei Internetanbietern ließe sich grundsätzlich auch auf andere, ursprünglich nicht vorgesehene Inhaltskategorien ausweiten.

30. Client-Side-Scanning: Technik und Kritik

30.1 Wie Client-Side-Scanning technisch funktionieren müsste

Wie in Teil I, Kapitel 3.2 bereits angerissen: Bei einem tatsächlich Ende-zu-Ende-verschlüsselten Dienst kann eine verpflichtende Inhaltsprüfung nur an einer von zwei Stellen ansetzen — entweder direkt vor der Verschlüsselung (auf dem Sendegerät) oder direkt nach der Entschlüsselung (auf dem Empfangsgerät). Beide Varianten erfordern, dass zusätzliche Analyse-Software mit weitreichenden Zugriffsrechten auf dem persönlichen Endgerät jeder einzelnen Nutzerin und jedes einzelnen Nutzers läuft — unabhängig davon, ob ein konkreter Verdacht gegen diese Person vorliegt.

30.1a Serverseitiges Scanning als (nicht funktionierende) Alternative

Ein naheliegender Einwand: Warum kann der Relay-Server (Kapitel 21) nicht einfach selbst scannen, statt eine Analyse auf das Endgerät zu verlagern? Die Antwort folgt unmittelbar aus der in Kapitel 21 beschriebenen Zero-Knowledge-Architektur: Der Relay sieht zu keinem Zeitpunkt Klartext, sondern ausschließlich AES-256-GCM-verschlüsselte Envelopes (Kapitel 13.1). Ohne Kenntnis des jeweiligen Message-Keys (der nach Verschlüsselung sofort verworfen wird, Kapitel 16.2) ist eine inhaltliche Analyse durch den Server mathematisch ausgeschlossen, nicht nur organisatorisch untersagt. Genau das ist ja der beabsichtigte Zweck der

Ende-zu-Ende-Verschlüsselung (Kapitel 2.1) — ein Server, der plötzlich doch scannen könnte, wäre per Definition keine echte Ende-zu-Ende-Verschlüsselung mehr, sondern höchstens eine Transportverschlüsselung mit serverseitigem Klartextzugriff (vergleichbar mit dem in Kapitel 5.8a beschriebenen TLS-Modell, bei dem der Server den Inhalt einsehen kann). Diese mathematische Unmöglichkeit ist der Kernpunkt, warum eine verpflichtende Inhaltsprüfung bei einem tatsächlich Ende-zu-Ende-verschlüsselten Dienst zwangsläufig auf dem Endgerät ansetzen müsste (30.1) — es gibt keinen dritten, serverseitigen Mittelweg, ohne die Verschlüsselung selbst durch eine schwächere Alternative zu ersetzen.

30.2 Das zentrale kryptographische Argument gegen Client-Side-Scanning

Das vielzitierte wissenschaftliche Papier „Bugs in our Pockets: The Risks of Client-Side Scanning“ (2021), verfasst von einer Gruppe von mehr als einem Dutzend international renommierter Kryptographen und Sicherheitsforscher (darunter Ross Anderson, Whitfield Diffie — Miterfinder des in Teil III, Kapitel 14.2 beschriebenen Diffie-Hellman-Verfahrens —, Susan Landau und weitere), argumentiert im Kern, dass Client-Side-Scanning die Ende-zu-Ende-Verschlüsselung nicht technisch umgeht, sondern ihren eigentlichen Schutzzweck unterläuft: Selbst wenn die Übertragung mathematisch ungebrochen bleibt, entsteht durch die verpflichtende Analyse-Software auf jedem Gerät ein neuer, systemweiter Angriffspunkt, der von den Autoren als grundsätzlich nicht sicher konstruierbar eingeschätzt wird — unter anderem, weil die Erkennungsdatenbanken und -algorithmen selbst zu Zielen für Angriffe, Manipulation oder Zweckerweiterung werden.

30.3 Das „Function Creep“-Argument (Zweckerweiterung)

Ein wiederkehrendes Argument in der Kritik: Eine einmal etablierte technische Infrastruktur zur Analyse privater Inhalte auf jedem Endgerät lässt sich, sobald sie existiert, potenziell für zusätzliche Zwecke erweitern — durch spätere Gesetzesänderungen (etwa Ausweitung auf Urheberrechtsverletzungen, „Extremismus“-Erkennung oder andere, politisch definierte Kategorien), durch autoritäre Regierungen außerhalb der EU, die dieselbe technische Infrastruktur für die Verfolgung Andersdenkender missbrauchen könnten, oder durch technische Kompromittierung der Erkennungsdatenbank selbst durch Dritte. Diese Sorge ist nicht rein hypothetisch: Referenzdatenbanken für automatisiertes Content-Scanning wurden in der Vergangenheit bereits für andere Zwecke als den ursprünglich vorgesehenen genutzt oder erweitert, in unterschiedlichen Rechtsordnungen weltweit.

30.4a Gedankenexperiment: Wie müsste ein „chatkontrolle-konformes RenkerVault“ aussehen?

Um die technischen Konsequenzen greifbar zu machen, ein Gedankenexperiment: Angenommen, eine finale Chat-Kontrolle-Verordnung würde tatsächlich verpflichtendes Client-Side-Scanning auch für Ende-zu-Ende-verschlüsselte Dienste wie RenkerVault vorschreiben — welche konkreten Architekturänderungen wären nötig, und welche der in diesem Buch beschriebenen Schutzeigenschaften gingen dabei verloren?

- **Vor der Verschlüsselung (Kapitel 16.2, `encrypt()`):** Eine zusätzliche Analyse-Komponente müsste jeden Klartext vor dem Aufruf von `aesGcmEncrypt` durchlaufen und das Ergebnis (Fund ja/nein) an eine externe Stelle melden — der Double Ratchet selbst bliebe zwar kryptographisch unverändert, aber die in Kapitel 2.1 beschriebene Zero-Knowledge-Eigenschaft wäre gebrochen, da der Klartext (wenn auch nur kurzzeitig, vor der Verschlüsselung) einer externen Analyse zugänglich gemacht würde.
- **Die Analyse-Komponente selbst würde zum neuen Angriffsziel:** Jede auf dem Gerät laufende Zusatzsoftware mit Zugriff auf Klartext ist selbst wieder Software mit eigenen, potenziellen Schwachstellen (Kapitel 13.3a) — ein Umstand, der unabhängig von der Qualität von RenkerVaults eigener Kryptographie bestünde.
- **Die in Kapitel 25.2 beschriebene, bereits bestehende Grenze „Web-Auslieferung kann manipuliert werden“ würde sich verschärfen:** Eine verpflichtende Analyse-Komponente müsste zuverlässig auf jedem Gerät laufen und dürfte nicht durch eine modifizierte, die Analyse umgehende Version der App ersetzbar sein — was in der Praxis signierte, zentral kontrollierte Builds und eine Einschränkung selbst kompilierter Open-Source-Varianten (Kapitel 39.7, Selbst-Audit) erfordern würde, ein Eingriff in die Offenheit der Software selbst.

Dieses Gedankenexperiment zeigt konkret, warum die in Kapitel 30.2 zitierte Kryptographen-Position nicht von einer abstrakten Ideologie ausgeht, sondern von einer nachvollziehbaren technischen Kettenreaktion: Jeder einzelne Schritt zur Umsetzung einer verpflichtenden Inhaltsprüfung würde eine der in Teil II und III beschriebenen Schutzeigenschaften direkt betreffen, nicht nur am Rande berühren.

30.4 Falsch-Positive und ihre konkreten Folgen

Wie in Teil I, Kapitel 3.1 anhand der Interim-Regelung bereits mit Zahlen belegt, führt selbst eine kleine prozentuale Fehlerquote bei Milliarden gescannter Inhalte zu einer erheblichen absoluten Zahl falsch markierter, tatsächlich unbeteiligter Personen. Die konkreten Folgen eines solchen Fehlalarms können erheblich sein: automatische Kontosperrung ohne vorherige menschliche Prüfung, Verlust des Zugriffs auf unter Umständen Jahre an persönlicher Kommunikation, Fotos und Dokumenten, sowie im schlimmsten Fall eine — unbegründete — Meldung an Strafverfolgungsbehörden mit den damit verbundenen psychischen und sozialen Belastungen für die betroffene Person, bis der Irrtum aufgeklärt ist.

31. Kritik aus Wissenschaft und Zivilgesellschaft

31.1 Der Europäische Datenschutzbeauftragte (EDPS)

Der Europäische Datenschutzbeauftragte (European Data Protection Supervisor, EDPS) — die unabhängige EU-Institution, die die Einhaltung des Datenschutzrechts durch die EU-Organen selbst überwacht — äußerte sich in einer gemeinsamen Stellungnahme mit dem Europäischen Datenschutzausschuss (EDPB) bereits 2022 kritisch zum ursprünglichen Kommissionsvorschlag und bezeichnete die vorgesehene, potenziell alle Nutzer betreffende Analyse privater Kommunikation als in dieser pauschalen Form nicht mit dem Grundsatz der Verhältnismäßigkeit vereinbar.

31.1a Konkrete Kritikpunkte des EDPS/EDPB im Detail

Die in 31.1 genannte gemeinsame Stellungnahme benannte mehrere konkrete Kritikpunkte, die über eine allgemeine Unverhältnismäßigkeits-Feststellung hinausgehen:

- **Fehlende Differenzierung nach Verdachtsgrad:** Die vorgesehenen Aufdeckungsanordnungen (Kapitel 29.1) würden nicht nur gegen konkret verdächtige Personen, sondern potenziell gegen die gesamte Nutzerschaft eines betroffenen Dienstes greifen — ein struktureller Unterschied zu klassischen strafprozessualen Ermittlungsmaßnahmen, die typischerweise an einen individuellen Anfangsverdacht anknüpfen.
- **Risiko der Chilling Effects:** Bereits das Wissen um eine mögliche automatisierte Analyse der eigenen Kommunikation könnte das Kommunikationsverhalten breiter Bevölkerungsteile verändern, unabhängig von der tatsächlichen Anwendung im Einzelfall (Teil IV, Kapitel 27.5).
- **Fehlerquote und betroffene Grundrechte Dritter:** Wie in Kapitel 29.3 dargestellt, sind auch technisch ausgereifte Erkennungsverfahren nicht fehlerfrei — bei den in Teil I, Kapitel 3.1 genannten Nutzerzahlen bedeutet dies zwangsläufig, dass auch vollkommen unbeteiligte Personen betroffen sein könnten.
- **Fehlende Evaluierung milderer Alternativen:** Die Stellungnahme verwies darauf, dass vor einer derart weitreichenden Maßnahme geprüft werden müsse, ob mildere, ebenso wirksame Mittel (etwa eine bessere Ressourcenausstattung für gezielte Ermittlungen im Einzelfall) zur Verfügung stünden — ein Element, das der Grundsatz der Erforderlichkeit im EU-Recht ausdrücklich verlangt (Kapitel 27.2).

31.2 Der Wissenschaftliche Dienst des Europäischen Parlaments

Auch der interne wissenschaftliche Dienst des Europäischen Parlaments (European Parliamentary Research Service, EPRS) verwies in begleitenden Analysen wiederholt auf die technischen und grundrechtlichen Bedenken der Kryptographie-Community und

stellte die Frage in den Raum, ob mildere, ebenso wirksame Mittel zur Verfügung stünden, die im EU-Recht nach dem Grundsatz der Erforderlichkeit vorrangig zu prüfen wären.

31.2a Die Funktion wissenschaftlicher Dienste im Gesetzgebungsprozess

Für Leserinnen und Leser, die mit der internen Arbeitsweise des Europäischen Parlaments nicht vertraut sind: Der EPRS ist keine externe, unabhängige Institution, sondern eine dem Parlament selbst zugeordnete Verwaltungseinheit, die Abgeordnete aller Fraktionen mit sachlich möglichst neutralen, wissenschaftlich fundierten Hintergrundanalysen zu anstehenden Gesetzesvorhaben versorgen soll — vergleichbar in der Funktion (wenn auch nicht in der institutionellen Anbindung) mit wissenschaftlichen Diensten nationaler Parlamente. Der Umstand, dass eine solche interne, um Neutralität bemühte Instanz die in Kapitel 30 beschriebenen technischen Bedenken aufgriff, wird von Kritikern der Chat-Kontrolle-Verordnung gelegentlich als zusätzliches Gewicht für ihre Position angeführt — eine Einschätzung, die jedoch, wie in Kapitel 26.1 betont, nicht mit einer abschließenden rechtlichen oder politischen Bewertung verwechselt werden sollte. EPRS-Analysen sind Hintergrundmaterial für die Entscheidungsfindung der Abgeordneten, keine bindende Entscheidung selbst.

31.3 Zivilgesellschaftliche Organisationen

Digitalrechte-Organisationen wie European Digital Rights (EDRi), der Chaos Computer Club (CCC) sowie zahlreiche nationale Datenschutzorganisationen positionierten sich über den gesamten Verhandlungsverlauf hinweg wiederholt gegen eine verpflichtende, pauschale Chatkontrolle. Ein wiederkehrendes Argument dieser Organisationen: Kinderschutz und der Erhalt sicherer, Ende-zu-Ende-verschlüsselter Kommunikation seien kein grundsätzlicher Gegensatz — beide Ziele ließen sich durch gezieltere, weniger invasive Maßnahmen (etwa bessere Ressourcenausstattung von Strafverfolgungsbehörden für gezielte Ermittlungen im Einzelfall, statt anlassloser Massenanalyse) gemeinsam verfolgen.

31.3a Kinderschutzorganisationen selbst sind keine homogene Gruppe

Ein differenzierender Punkt, der in vereinfachten Debattenbeiträgen (Kapitel 31.4b) oft untergeht: Auch Organisationen, deren Kernauftrag der Kinderschutz selbst ist, vertreten in der Chat-Kontrolle-Debatte keine einheitliche Position. Während manche Kinderschutzorganisationen die vorgeschlagene Verordnung ausdrücklich unterstützen und auf die in Teil I, Kapitel 3.1 genannten Aufdeckungserfolge verweisen, äußerten sich andere kinderschutzorientierte Stimmen zurückhaltender und verwiesen auf die in Kapitel 30 dargestellten technischen Risiken oder auf die Sorge, dass ein zu weitreichendes, technisch fehleranfälliges System langfristig das öffentliche Vertrauen in

Kinderschutzmaßnahmen insgesamt beschädigen könnte, wenn es wiederholt zu prominenten Fehlalarmen käme. Diese interne Vielfalt innerhalb der Kinderschutz-Community selbst unterstreicht zusätzlich den in Kapitel 25a.5 gezogenen Schluss: Der Dissens verläuft nicht entlang einer einfachen Linie „für Kinderschutz“ gegen „gegen Kinderschutz“, sondern quer durch praktisch jede an der Debatte beteiligte Interessengruppe hindurch, entlang der Frage nach dem richtigen technischen Mittel.

31.4 Die Position von Anbietern verschlüsselter Dienste

Mehrere Anbieter Ende-zu-Ende-verschlüsselter Dienste positionierten sich öffentlich gegen eine verpflichtende Aufweichung ihrer Verschlüsselung im Fall einer Chatkontrolle-Verpflichtung. Öffentlich dokumentiert ist unter anderem eine Erklärung von Signal (Herbst 2025) mit der sinngemäßen Aussage, das Unternehmen werde den europäischen Markt eher verlassen, als eine Hintertür oder ein vergleichbares Client-Side-Scanning-Verfahren in seinen Dienst einzubauen. Eine solche Ankündigung ist als politische Positionierung zu verstehen, nicht als rechtliche Gewissheit über einen tatsächlichen künftigen Rückzug — sie verdeutlicht aber die Tiefe des wahrgenommenen Zielkonflikts aus Sicht der betroffenen Anbieter.

31.4a Mediale Berichterstattung und öffentliche Wahrnehmung

Ein Phänomen, das die öffentliche Debatte zusätzlich erschwert: Die technische Komplexität des Themas (Kapitel 29.2 unterscheidet allein drei technisch unterschiedliche Erkennungsaufgaben) lässt sich in kurzer journalistischer Berichterstattung nur schwer vollständig abbilden, was in der Praxis zu zwei gegenläufigen Vereinfachungstendenzen führt: Manche Berichterstattung reduziert die Debatte auf ein plakatives „Chatkontrolle = Überwachung aller Bürger“, ohne die in Kapitel 29.3a beschriebenen vorgesehenen rechtsstaatlichen Kontrollmechanismen zu erwähnen; andere Berichterstattung wiederum reduziert die Gegenposition auf ein ebenso plakatives „Kryptographen wollen Kinderschutz verhindern“, ohne die in Kapitel 30 dargestellte technische Substanz der fachlichen Kritik wiederzugeben. Diese doppelte Vereinfachungstendenz ist einer der Gründe, warum dieses Buch in Kapitel 25a bewusst Raum für eine ausführlichere, mehrperspektivische Darstellung geschaffen hat, statt die Debatte auf eine der beiden verkürzten Versionen zu reduzieren.

31.4b Wissenschaftliche Fachgesellschaften

Über die in Kapitel 31.3 genannten Digitalrechte-Organisationen hinaus positionierten sich im Verhandlungsverlauf auch etablierte wissenschaftliche Fachgesellschaften kritisch, darunter Zusammenschlüsse europäischer Kryptographie- und IT-Sicherheitsforscher, die in offenen Briefen an EU-Institutionen auf die in Kapitel 30.2 dargestellte fachliche Einschätzung verwiesen. Ein wiederkehrendes Muster dieser Stellungnahmen: Sie kommen in aller Regel nicht von Organisationen, die grundsätzlich

gegen Kinderschutzmaßnahmen argumentieren, sondern explizit von Fachleuten, deren berufliche Reputation auf der korrekten Einschätzung technischer Sicherheitsfragen beruht — ein Umstand, der die in Kapitel 25a.5 betonte Unterscheidung zwischen Zweck und Mittel zusätzlich untermauert.

31.4c Ein historischer Präzedenzfall: der Fall ACTA (2012)

Dass zivilgesellschaftlicher Widerstand EU-Digitalgesetzgebung tatsächlich verändern oder stoppen kann, ist keine rein theoretische Möglichkeit — der bekannteste Präzedenzfall ist das Anti-Counterfeiting Trade Agreement (ACTA), ein internationales Handelsabkommen mit unter anderem urheberrechtsbezogenen Durchsetzungsbestimmungen, das 2012 nach massiven, europaweiten öffentlichen Protesten (unter anderem Straßendemonstrationen in zahlreichen europäischen Städten sowie einer beispiellosen Welle von Bürgerkontakten zu Europaabgeordneten) vom Europäischen Parlament mit großer Mehrheit abgelehnt wurde, nachdem es zuvor bereits von der Kommission vorangetrieben und von mehreren Mitgliedstaaten unterzeichnet worden war. Der Fall ACTA zeigt zweierlei: Erstens, dass die in Kapitel 36 beschriebenen Beteiligungsmöglichkeiten keine bloße Formalie sind, sondern in der Vergangenheit tatsächlich ein Gesetzgebungsverfahren maßgeblich beeinflusst haben. Zweitens, dass gerade digitalpolitische Themen ein überdurchschnittlich hohes Mobilisierungspotenzial in der Zivilgesellschaft besitzen — ein Muster, das sich auch im bisherigen, mehrjährigen Stillstand der Chat-Kontrolle-Verhandlungen (Kapitel 28.3-28.5) zumindest teilweise wiederfindet, auch wenn die Vergleichbarkeit beider Fälle angesichts unterschiedlicher Ausgangslagen begrenzt ist.

32. Verhandlungsstand und Ausblick

32.1 Wie sich der Fortschritt in solchen Verfahren üblicherweise vollzieht

Ein EU-Gesetzgebungsverfahren dieser Art durchläuft typischerweise mehrere Lesungen und einen abschließenden „Trilog“ (informelle Verhandlung zwischen Kommission, Rat und Parlament), bevor eine endgültige, von allen drei Institutionen getragene Fassung entsteht. Bis zu diesem Punkt können sich einzelne Regelungsinhalte — etwa die Frage, ob Ende-zu-Ende-verschlüsselte Dienste einbezogen werden oder nicht — noch erheblich verändern.

32.1a Warum Trilog-Verhandlungen intransparent kritisiert werden

Ein spezifischer, in Kapitel 28.0 bereits angedeuteter Kritikpunkt an Trilog-Verhandlungen verdient eine Vertiefung: Anders als die öffentlichen Plenardebatten und Ausschusssitzungen des Europäischen Parlaments (die in der Regel live übertragen und protokolliert werden) finden Trilog-Sitzungen informell und

ohne verpflichtende Öffentlichkeit statt. Transparenz-Organisationen kritisieren dies regelmäßig, da gerade in dieser Phase oft die entscheidenden inhaltlichen Kompromisse ausgehandelt werden, ohne dass die breite Öffentlichkeit den genauen Verhandlungsverlauf in Echtzeit nachvollziehen kann — die Öffentlichkeit erfährt in der Regel erst das Verhandlungsergebnis, nicht die einzelnen Zwischenschritte und Kompromisslinien, die dorthin geführt haben. Für die Chat-Kontrolle-Verordnung bedeutet dies praktisch: Sollte es tatsächlich zu formellen Trilog-Verhandlungen kommen (was voraussetzt, dass der Rat zuvor die in Kapitel 28.3 beschriebene, bislang fehlende Einigung erzielt), wäre der genaue Verlauf dieser entscheidenden letzten Verhandlungsphase für die interessierte Öffentlichkeit nur eingeschränkt nachvollziehbar — ein zusätzlicher Grund, warum die in Kapitel 31.3 genannten spezialisierten Organisationen eine wichtige Rolle als informelle Informationsvermittler einnehmen.

32.2 Was aufmerksame Beobachtung wert ist

Für Leserinnen und Leser, die den weiteren Verlauf verfolgen möchten, sind insbesondere folgende Indikatoren aufschlussreich: ob eine im Rat erzielte allgemeine Ausrichtung („General Approach“) eine ausdrückliche Ausnahme für Ende-zu-Ende-verschlüsselte Dienste enthält oder nicht; ob die Trilog-Verhandlungen zu einer verpflichtenden oder weiterhin freiwilligen Regelung für solche Dienste führen; und ob die wiederholten Verlängerungen der Interim-Regelung (Kapitel 28.1/28.4) fortgesetzt werden, was auf eine anhaltende politische Pattsituation hindeuten würde.

32.3a Vergleichbare Debatten außerhalb der EU (kurzer Ausblick)

Der Fokus dieses Buches liegt, wie in Kapitel 26.4 begründet, auf EU-Recht. Zur Einordnung, dass die Grundspannung zwischen Kinderschutz und Verschlüsselung kein rein europäisches Phänomen ist, folgt ein knapper, bewusst nicht erschöpfender Überblick über vergleichbare Debatten in anderen Rechtsordnungen:

- **Vereinigtes Königreich — Online Safety Act (2023):** Enthält eine Regelung, die Ofcom (die zuständige Regulierungsbehörde) unter bestimmten Voraussetzungen ermächtigen könnte, von Anbietern den Einsatz „akkreditierter Technologie“ zur Erkennung von Missbrauchsmaterial zu verlangen — auch dies stieß auf Kritik von Anbietern Ende-zu-Ende-verschlüsselter Dienste, die eine faktische Verpflichtung zu client-seitigem Scannen befürchteten.
- **USA — EARN IT Act (mehrfach im US-Kongress eingebracht, Stand der hier verwendeten Quellen nicht final verabschiedet):** Zielt darauf ab, den bestehenden Haftungsschutz von Plattformbetreibern für nutzergenerierte Inhalte (Section 230 des Communications Decency Act) an bestimmte Bedingungen im Bereich Kinderschutz zu knüpfen — Kritiker befürchten einen indirekten Druck auf Anbieter, Verschlüsselung zugunsten von Scanning-Fähigkeit einzuschränken, auch ohne eine explizite Verschlüsselungsverbot-Klausel.

- **USA — CLOUD Act (2018, bereits geltendes Recht):** Erlaubt es US-Behörden unter bestimmten Voraussetzungen, von US-Unternehmen die Herausgabe von Daten zu verlangen, unabhängig davon, wo diese Daten physisch gespeichert sind — ein anderer rechtlicher Mechanismus als Chat-Kontrolle, aber ebenfalls Teil der breiteren Debatte um staatlichen Zugriff auf digitale Kommunikation.
- **Australien — Telecommunications and Other Legislation Amendment (Assistance and Access) Act, kurz TOLA (2018):** Erlaubt es australischen Behörden, von Unternehmen technische Unterstützung bei der Umgehung von Verschlüsselung zu verlangen; das Gesetz war seit seiner Verabschiedung wiederholt Gegenstand von Kritik durch die internationale Kryptographie-Community aus denselben, in Kapitel 30.2 dargestellten Gründen.

Diese internationale Perspektive zeigt: Die in Teil IV beschriebene Debatte ist ein Spezialfall eines global wiederkehrenden Musters (vergleiche auch die historische Einordnung in Teil I, Kapitel 4) — nicht ein isoliertes EU-Phänomen.

32.3b Indien und Brasilien: Verschlüsselungsdebatten im globalen Süden

Über die in 32.3a genannten westlichen Industriestaaten hinaus lohnt ein Blick auf zwei weitere große Demokratien: Indiens 2021 novellierte „Information Technology Rules“ verlangen von großen Messaging-Diensten unter bestimmten Umständen die Fähigkeit, den „ersten Urheber“ einer Nachricht („first originator“) zu identifizieren — eine Anforderung, die von Anbietern Ende-zu-Ende-verschlüsselter Dienste als faktisch unvereinbar mit dem Grundprinzip der Ende-zu-Ende-Verschlüsselung kritisiert wurde und zu rechtlichen Auseinandersetzungen zwischen der indischen Regierung und betroffenen Anbietern führte. Brasilien wiederum hat in der Vergangenheit wiederholt (zeitlich befristete) gerichtliche Sperrungen von WhatsApp verhängt, nachdem der Anbieter richterlichen Anordnungen zur Herausgabe von Chatinhalten nicht habe nachkommen können — ein Umstand, der von WhatsApp damit begründet wurde, aufgrund der eigenen Ende-zu-Ende-Verschlüsselung technisch nicht in der Lage zu sein, die verlangten Klartextinhalte herauszugeben. Beide Beispiele zeigen: Der Grundkonflikt zwischen staatlichem Zugriffswunsch und der mathematischen Eigenschaft echter Ende-zu-Ende-Verschlüsselung, dass selbst der Anbieter den Klartext nicht herausgeben kann, tritt unabhängig vom jeweiligen Rechtssystem und Kontinent in strukturell ähnlicher Form auf.

32.3 Warum RenkerVault unabhängig vom Verhandlungsausgang relevant bleibt

Unabhängig davon, wie die Verhandlungen konkret ausgehen, bleibt die in Teil I, Kapitel 3.3 beschriebene strukturelle Überlegung bestehen: Ein Kommunikationswerkzeug, dessen Architektur so gestaltet ist, dass ein nachträglicher Einbau eines Scanning-Mechanismus technisch nicht möglich ist, ohne die Architektur grundlegend zu

verändern, bietet eine andere Art von Schutzversprechen als ein Werkzeug, bei dem ein Scanning-Mechanismus per Software-Update jederzeit ergänzt werden könnte. RenkerVault (siehe Kapitel 35) macht keinen Anspruch, außerhalb geltenden Rechts zu operieren — es macht lediglich eine Architektur-Entscheidung, die bestimmte, in Zukunft denkbare Verpflichtungen technisch schwerer nachrüstbar macht als bei zentralisierten, closed-source Diensten.

32.4 Verschlüsselung als Menschenrecht: die Position der Vereinten Nationen

Über die EU-Ebene hinaus lohnt ein Blick auf die Vereinten Nationen: Der damalige UN-Sonderberichterstatter für die Förderung und den Schutz des Rechts auf Meinungsfreiheit, David Kaye, legte 2015 einen vielzitierten Bericht vor, der Verschlüsselung und Anonymität im Internet als wesentliche Voraussetzung für die Ausübung des Rechts auf freie Meinungsäußerung nach Artikel 19 des Internationalen Pakts über bürgerliche und politische Rechte einordnete. Der Bericht argumentierte, dass staatliche Beschränkungen von Verschlüsselung strengen Anforderungen an Rechtmäßigkeit, Notwendigkeit und Verhältnismäßigkeit genügen müssten — eine Argumentationslinie, die der in Teil IV, Kapitel 27.2 beschriebenen EU-rechtlichen Prüfung strukturell ähnelt, aber auf einer globalen, nicht EU-spezifischen Ebene ansetzt. Dieser Bericht wird von zivilgesellschaftlichen Organisationen (Kapitel 31.3) regelmäßig als zusätzliches Argument dafür angeführt, dass der Schutz von Verschlüsselung nicht nur ein europäisches Grundrechtsthema, sondern ein international anerkanntes Menschenrechtsanliegen ist.

32.4a Der Unterschied zwischen einer UN-Sonderberichterstattung und bindendem Recht

Zur rechtlichen Einordnung des in 32.4 genannten Berichts: Ein UN-Sonderberichterstatter handelt in unabhängiger, unbezahlter Funktion im Auftrag des UN-Menschenrechtsrats, und die von ihm oder ihr vorgelegten Berichte sind rechtlich nicht bindend — sie stellen eine autoritative Einordnung und Auslegungshilfe dar, keine unmittelbar durchsetzbare Verpflichtung für einzelne Staaten. Der Wert eines solchen Berichts liegt daher weniger in seiner unmittelbaren Rechtswirkung als in seiner Funktion als international anerkannte, unabhängige fachliche Einschätzung, auf die sich nachfolgende rechtliche und politische Debatten (etwa die in Kapitel 27 beschriebene EU-Grundrechtsprüfung) argumentativ stützen können. Diese Unterscheidung zwischen bindendem Recht (Kapitel 26.1-26.2) und autoritativer, aber nicht bindender fachlicher Einschätzung zieht sich durch mehrere in diesem Buch genannte Quellen — auch die in Kapitel 31.1 genannte EDPS/EDPB-Stellungnahme oder der in Kapitel 31.2 genannte EPRS-Bericht sind in diesem Sinne einflussreiche, aber formal nicht bindende Fachmeinungen, keine Gerichtsurteile.

32.5 Interoperabilität als verwandtes, aber getrenntes Digitalpolitik-Thema

Ein weiteres, von der Chat-Kontrolle-Debatte zu unterscheidendes, aber verwandtes EU-Regelungsthema ist die im Digital Markets Act (DMA, seit 2023 anwendbar) vorgesehene Interoperabilitätspflicht für besonders große Messaging-Dienste („Gatekeeper“): Diese sollen unter bestimmten Voraussetzungen verpflichtet werden, ihre Nachrichtenübermittlung für kleinere, konkurrierende Dienste zu öffnen. Diese Regelung wirft eigene, technisch anspruchsvolle Fragen auf, wie Ende-zu-Ende-Verschlüsselung über die Grenzen unterschiedlicher, jeweils eigener Verschlüsselungsprotokolle hinweg aufrechterhalten werden kann — ein Problem, das mit der in Kapitel 24 beschriebenen Konvergenz vieler Dienste auf ähnliche kryptographische Grundprinzipien (Double Ratchet, MLS) potenziell erleichtert wird, aber keineswegs trivial gelöst ist. RenkerVault ist von dieser Regelung mangels der dafür vorausgesetzten Marktgröße nicht betroffen; das Thema wird hier der Vollständigkeit halber erwähnt, um die Chat-Kontrolle-Debatte (Verschlüsselung schwächen) klar von der Interoperabilitätsdebatte (Verschlüsselung über Dienstgrenzen hinweg aufrechterhalten) abzugrenzen — beide werden in der öffentlichen Debatte gelegentlich vermischt, obwohl sie unterschiedliche Ziele verfolgen.

32.5c Warum Interoperabilität die Chat-Kontrolle-Debatte indirekt berührt

Ein Berührungspunkt zwischen beiden Themen verdient dennoch Erwähnung: Sollte die in Kapitel 32.5 beschriebene DMA-Interoperabilitätspflicht tatsächlich breit umgesetzt werden, entstünde ein zusätzliches praktisches Argument gegen eine Schwächung der Verschlüsselung einzelner Dienste (Kapitel 30): Ein interoperables Ökosystem mehrerer, jeweils Ende-zu-Ende-verschlüsselter Messenger könnte Nutzerinnen und Nutzern eine einfachere Ausweichmöglichkeit bieten, sollte ein einzelner Dienst durch eine verpflichtende Chatkontrolle-Regelung zur Schwächung seiner Verschlüsselung gezwungen werden — der Umstieg auf einen anderen, weiterhin voll verschlüsselten, aber interoperablen Dienst würde durch geringere Wechselkosten erleichtert. Diese Wechselwirkung zwischen zwei formal getrennten Regulierungssträngen (Wettbewerbsrecht einerseits, Kinderschutz-/Verschlüsselungsdebatte andererseits) ist ein weiteres Beispiel für das bereits in Kapitel 34.3d beschriebene Muster, bei dem sich ursprünglich unabhängige Digitalpolitik-Vorhaben in ihrer praktischen Wirkung gegenseitig beeinflussen können.

32.5a Berührungspunkt mit dem EU-KI-Gesetz (AI Act)

Ein weiterer, oft übersehener Berührungspunkt: Die in Kapitel 29.2 beschriebenen Erkennungsverfahren für neues Missbrauchsmaterial und für Grooming setzen zwangsläufig auf automatisierte Klassifikationsverfahren (im Kern: maschinelles Lernen bzw. KI-Modelle). Der seit 2024 schrittweise anwendbare EU AI Act (Verordnung über

Künstliche Intelligenz) stuft bestimmte KI-Anwendungen nach ihrem Risiko ein und verlangt für „Hochrisiko“-Anwendungen zusätzliche Anforderungen an Transparenz, Dokumentation und menschliche Aufsicht. Ob und wie die im Rahmen der Chat-Kontrolle-Verordnung eingesetzten Klassifikationsverfahren unter diese Risikokategorien fallen würden, ist eine eigenständige, in der Fachliteratur diskutierte Frage, die über den Rahmen dieses Buches hinausgeht — sie wird hier erwähnt, um zu zeigen, dass die Chat-Kontrolle-Verordnung nicht isoliert von der übrigen, sich parallel entwickelnden EU-Digitalregulierung betrachtet werden kann.

32.5b Die Verordnung über digitale Identitäten (eIDAS 2.0) als weiterer Berührungspunkt

Ein weiteres, paralleles EU-Digitalvorhaben verdient Erwähnung: die überarbeitete eIDAS-Verordnung (electronic IDentification, Authentication and trust Services), die eine EU-weite „Digitale Brieftasche“ (European Digital Identity Wallet) für alle Bürgerinnen und Bürger vorsieht. Diese Digitale Brieftasche ist konzeptionell von der Chat-Kontrolle-Debatte unabhängig, könnte aber praktisch mit der in Kapitel 29.5 diskutierten Altersverifikation zusammenspielen: Ein über die Digitale Brieftasche technisch abgesicherter Altersnachweis könnte perspektivisch eine datensparsamere Alternative zu den in Kapitel 29.5 genannten Ausweisdokumenten-Abgleichen oder biometrischen Schätzverfahren darstellen, da ein Wallet-basierter Nachweis grundsätzlich nur die Tatsache „volljährig: ja/nein“ bestätigen könnte, ohne das konkrete Geburtsdatum oder weitere Ausweisdaten preiszugeben (ein Konzept, das in der Kryptographie „Zero-Knowledge-Proof“ genannt wird — nicht zu verwechseln mit der in Kapitel 2 beschriebenen Zero-Knowledge-**Architektur** von RenkerVault, auch wenn beide Konzepte auf verwandten mathematischen Ideen beruhen). Wie genau eIDAS 2.0 und eine mögliche Altersverifikationspflicht der Chat-Kontrolle-Verordnung zusammenwirken würden, ist zum Zeitpunkt der Erstellung dieses Buches noch nicht abschließend geklärt.

33. DSGVO und Ende-zu-Ende-verschlüsselte Messenger

33.1 Die Grundprinzipien der DSGVO

Die Datenschutz-Grundverordnung (DSGVO, seit Mai 2018 unmittelbar geltendes EU-Recht) beruht auf mehreren Grundprinzipien, von denen zwei für Ende-zu-Ende-verschlüsselte Messenger besonders relevant sind:

- **Datenminimierung (Art. 5 Abs. 1 lit. c DSGVO):** Es dürfen nur die personenbezogenen Daten verarbeitet werden, die für den jeweiligen Zweck tatsächlich erforderlich sind.

- **Privacy by Design und by Default (Art. 25 DSGVO):** Datenschutzfreundliche Voreinstellungen und eine datenschutzfreundliche Systemgestaltung sind bereits bei der Entwicklung, nicht erst nachträglich, zu berücksichtigen.

33.0a Artikel 32 DSGVO: Sicherheit der Verarbeitung

Neben den in 33.1 genannten Prinzipien enthält die DSGVO in Artikel 32 eine ausdrückliche Pflicht zur „Sicherheit der Verarbeitung“: Verantwortliche müssen geeignete technische und organisatorische Maßnahmen treffen, um ein dem Risiko angemessenes Schutzniveau sicherzustellen — der Artikel nennt dabei explizit „die Verschlüsselung personenbezogener Daten“ als Beispiel einer solchen Maßnahme. Diese ausdrückliche gesetzliche Nennung von Verschlüsselung ist bemerkenswert: Sie zeigt, dass der europäische Gesetzgeber Verschlüsselung nicht als Gegensatz zu einer funktionierenden Rechtsordnung begreift, sondern selbst als eines der in der DSGVO ausdrücklich empfohlenen Mittel zur Erfüllung gesetzlicher Schutzpflichten. Diese Beobachtung ist für die in Kapitel 25a beschriebene Debatte bemerkenswert: Dieselbe EU-Rechtsordnung, die in der Chat-Kontrolle-Verordnung (Kapitel 28-29) möglicherweise eine Schwächung von Ende-zu-Ende-Verschlüsselung erwägt, empfiehlt an anderer Stelle (Art. 32 DSGVO) genau diese Technologie als vorbildliche Sicherheitsmaßnahme — ein Spannungsfeld innerhalb des EU-Rechts selbst, das über den in Kapitel 34.3b beschriebenen Abstimmungsbedarf zwischen einzelnen Rechtsakten hinausgeht.

33.1a Die sieben Grundsätze des Artikel 5 DSGVO im Überblick

Artikel 5 DSGVO listet sieben Grundsätze, die jede Verarbeitung personenbezogener Daten in der EU erfüllen muss. Eine vollständige Übersicht hilft, die in 33.1 genannten zwei Prinzipien einzuordnen:

Grundsatz	Kurzbeschreibung	Bezug zu RenkerVault
Rechtmäßigkeit, Verarbeitung nach Treu und Glauben, Transparenz	Verarbeitung braucht eine Rechtsgrundlage und muss nachvollziehbar sein	Relevant fuer Betreiber eines Relays (Kapitel 33.3), weniger fuer die Kernarchitektur selbst
Zweckbindung	Daten duerfen nur fuer festgelegte, legitime Zwecke erhoben werden	Relay erhebt ausschliesslich Routing-Metadaten fuer den Zustellungszweck (Teil III, Kapitel 21)
Datenminimierung	Nur die fuer den Zweck erforderlichen Daten duerfen verarbeitet werden	Zero-Knowledge-Architektur als staerkste Form der Datenminimierung (Teil I, Kapitel 2)
Richtigkeit	Daten muessen sachlich richtig und aktuell sein	Wenig einschlaegig, da der Relay keine inhaltlichen Nutzerdaten fuehrt
Speicherbegrenzung	Daten duerfen nur so lange gespeichert werden wie fuer den Zweck noetig	Offline-Warteschlange wird nach Zustellung geleert (Teil III, Kapitel 21.3)

Grundsatz	Kurzbeschreibung	Bezug zu RenkerVault
Integrität und Vertraulichkeit	Angemessene Sicherheit gegen unbefugte Verarbeitung/Verlust	Ende-zu-Ende-Verschlüsselung und At-Rest-Verschlüsselung (Teil II, Kapitel 11)
Rechenschaftspflicht	Verantwortliche müssen die Einhaltung der Grundsätze nachweisen können	Betrifft in erster Linie Betreiber, die einen eigenen Relay öffentlich anbieten (Kapitel 33.3)

33.1b Warum „Datenminimierung durch Architektur“ mehr ist als eine Compliance-Übung

Ein wichtiger konzeptioneller Punkt: Viele Dienste erreichen Datenminimierung durch organisatorische Richtlinien (etwa interne Löschfristen), die grundsätzlich rückgängig gemacht oder geändert werden könnten. RenkerVaults Zero-Knowledge-Architektur erreicht denselben Effekt stattdessen durch die technische Unmöglichkeit, überhaupt an den geschützten Klartext zu gelangen (Teil I, Kapitel 2.1) — ein Unterschied, der in der Datenschutz-Fachliteratur manchmal als Unterschied zwischen „Privacy by Policy“ (Schutz durch Regeln, die gebrochen oder geändert werden können) und „Privacy by Architecture“ (Schutz durch technische Unmöglichkeit) beschrieben wird. Artikel 25 DSGVO („Privacy by Design und by Default“, Kapitel 33.1) verlangt ausdrücklich, dass Datenschutz technisch und nicht nur organisatorisch abgesichert wird — RenkerVaults Architektur ist ein Beispiel für eine konsequente Umsetzung dieses Grundsatzes.

33.2 Wie RenkerVaults Architektur diese Prinzipien technisch umsetzt

Die in Teil II und III beschriebene Architektur von RenkerVault lässt sich unmittelbar auf diese beiden Prinzipien zurückführen, auch wenn das Projekt selbst kein personenbezogene Daten im großen Stil verarbeitender kommerzieller Dienst ist: Das Zero-Knowledge-Relay (Kapitel 21) verarbeitet strukturell nur das Minimum an Metadaten, das für die Zustellung technisch erforderlich ist (Datenminimierung); Funktionen wie Lesebestätigungen und Präsenzanzeige sind standardmäßig deaktiviert (Kapitel 8.3 — „Privacy by Default“); und das Fehlen einer Registrierungspflicht mit Telefonnummer oder E-Mail (Kapitel 7.1) reduziert von vornherein die Menge an personenbezogenen Daten, die überhaupt anfallen könnte.

33.2a Betroffenenrechte und ihre Grenzen bei Zero-Knowledge-Architekturen

Die DSGVO gewährt betroffenen Personen mehrere Rechte gegenüber Verantwortlichen — unter anderem ein Auskunftsrecht (Artikel 15), ein Recht auf Löschung (Artikel 17) und ein Recht auf Datenübertragbarkeit (Artikel 20). Diese Rechte werfen bei einer Zero-Knowledge-Architektur eine interessante Besonderheit auf: Ein Relay-Betreiber, der wie in Teil III, Kapitel 21 beschrieben strukturell keinen Zugriff auf

den Nachrichteninhalt hat, kann ein Auskunftersuchen über den **Inhalt** einer Kommunikation gar nicht erfüllen — nicht, weil er sich weigert, sondern weil er technisch nicht in der Lage dazu ist. Diese Situation ist datenschutzrechtlich nicht ungewöhnlich (vergleichbar mit einem Postdienstleister, der ebenfalls keine Auskunft über den Briefinhalt geben kann), verdeutlicht aber, dass „Datenschutz durch technische Unmöglichkeit“ (Kapitel 33.1b) in einer Hinsicht sogar über die klassische DSGVO-Compliance hinausgeht: Es gibt schlicht keine Inhaltsdaten beim Betreiber, über die überhaupt Auskunft erteilt werden könnte. Für die beim Relay tatsächlich vorhandenen Metadaten (Kontoexistenz, öffentliche Schlüssel, Geräteliste) bleiben die üblichen Betroffenenrechte dagegen unverändert anwendbar.

33.2b Meldepflichten bei Datenschutzverletzungen (Art. 33/34 DSGVO)

Die DSGVO verpflichtet Verantwortliche, eine Verletzung des Schutzes personenbezogener Daten binnen 72 Stunden an die zuständige Aufsichtsbehörde zu melden (Artikel 33) und unter bestimmten Voraussetzungen zusätzlich die betroffenen Personen selbst zu informieren (Artikel 34) — insbesondere, wenn ein hohes Risiko für deren Rechte und Freiheiten besteht. Für einen selbst betriebenen RenkerVault-Relay (Kapitel 33.3) bedeutet dies konkret: Ein erfolgreicher Angriff auf den Relay-Server, der zur Offenlegung der dort gespeicherten Metadaten führt (Konto-Kennungen, öffentliche Schlüssel, Geräte-Metadaten, siehe Kapitel 21.1), wäre grundsätzlich eine meldepflichtige Datenschutzverletzung, selbst wenn — wie in Kapitel 23.2 dargestellt — keine Nachrichteninhalte kompromittiert wurden, da auch Metadaten personenbezogene Daten im Sinne der DSGVO darstellen können. Ein Betreiber sollte daher, unabhängig vom technischen Schutzniveau des Zero-Knowledge-Prinzips, einen Plan für den Umgang mit einem solchen Vorfall bereithalten (vergleichbar mit dem in Kapitel 39.5 beschriebenen Vorfall-Reaktionsplan für Endnutzer, nur auf Betreiberebene).

33.3 Betreiberpflichten bei Selbst-Hosting eines Relays

Wer einen eigenen RenkerVault-Relay-Server betreibt (Teil V, Kapitel 38) und dabei personenbezogene Daten Dritter verarbeitet (etwa Konto-Metadaten von Kontakten, die nicht die eigene Person sind), wird dadurch unter Umständen selbst zum „Verantwortlichen“ im Sinne der DSGVO mit entsprechenden Pflichten (u. a. eine Rechtsgrundlage für die Verarbeitung, Informationspflichten, technische und organisatorische Schutzmaßnahmen). Dies gilt es bei einer Nutzung über einen rein privaten, persönlichen Rahmen hinaus zu berücksichtigen — im Zweifel ist auch hier eine rechtliche Beratung angeraten (siehe 26.1).

33.3a Datenschutz-Folgenabschätzung (Art. 35 DSGVO)

Für Verarbeitungen, die voraussichtlich ein hohes Risiko für die Rechte und Freiheiten betroffener Personen mit sich bringen, verlangt Artikel 35 DSGVO eine sogenannte

Datenschutz-Folgenabschätzung (Data Protection Impact Assessment, DPIA) — eine vorab durchzuführende, dokumentierte Risikoanalyse. Ob der Betrieb eines RenkerVault-Relays für eine größere Nutzergruppe (etwa Kapitel 39.6a, das Kanzlei-/Praxis-Beispiel) eine solche Pflicht auslöst, hängt vom Einzelfall ab — Faktoren, die für eine DPIA-Pflicht sprechen könnten, wären etwa eine große Zahl betroffener Personen oder die Verarbeitung im Zusammenhang mit besonders schützenswerten Datenkategorien (etwa Gesundheitsdaten in einem Praxis-Kontext). Interessant für die Bewertung: Die in Kapitel 21 beschriebene Zero-Knowledge-Architektur selbst wirkt sich risikomindernd aus, da der größte denkbare Schaden (Offenlegung von Kommunikationsinhalten) durch das Architekturdesign strukturell ausgeschlossen ist — eine DPIA würde sich in der Praxis vor allem auf die verbleibenden Metadaten (Kapitel 21.1) konzentrieren müssen, nicht auf Nachrichteninhalte. Auch hier gilt: Die abschließende Beurteilung, ob und in welcher Form eine DPIA im Einzelfall erforderlich ist, sollte durch rechtliche Beratung erfolgen (Kapitel 26.1).

33.3b Internationale Datenübermittlung (Kapitel V DSGVO)

Wer einen Relay-Server außerhalb der EU hostet (Kapitel 38.4a erwähnt bereits die Serverstandort-Überlegung), muss zusätzlich das in Kapitel V der DSGVO geregelte Schutzniveau für die Übermittlung personenbezogener Daten in Drittstaaten beachten. Die Grundregel: Eine Übermittlung in ein Land außerhalb des Europäischen Wirtschaftsraums ist grundsätzlich nur zulässig, wenn ein angemessenes Datenschutzniveau sichergestellt ist — etwa durch einen Angemessenheitsbeschluss der EU-Kommission für das jeweilige Land, durch von der Kommission genehmigte Standardvertragsklauseln, oder durch andere in der DSGVO vorgesehene Mechanismen. Die in Kapitel 27.3a bereits erwähnten Schrems-Urteile betrafen genau diese Frage im Verhältnis zu den USA und führten dazu, dass ein früherer Angemessenheitsbeschluss („Privacy Shield“) vom EuGH für ungültig erklärt wurde, da die US-amerikanischen Überwachungsbefugnisse (Kapitel 45, Abkürzung „Five Eyes“, Kapitel 4.5b) als nicht mit dem EU-Grundrechtsschutz vereinbar eingestuft wurden. Für den Betrieb eines RenkerVault-Relays außerhalb der EU bedeutet dies praktisch: Der gewählte Serverstandort sollte nicht allein nach Preis oder technischer Ausstattung, sondern auch nach dem dort geltenden Datenschutz- und Überwachungsrecht ausgewählt werden — ein Aspekt, der über die rein technische Zero-Knowledge-Eigenschaft des Relays selbst (Kapitel 21) hinausgeht und die verbleibenden, für den Relay-Standort sichtbaren Metadaten betrifft.

33.3c Das Recht auf Löschung und die „Sitzung verbrennen“-Funktion

Ein interessanter Berührungspunkt zwischen Recht und Technik: Artikel 17 DSGVO gewährt betroffenen Personen unter bestimmten Voraussetzungen ein „Recht auf

Löschung" (auch „Recht auf Vergessenwerden" genannt). Die in Teil II, Kapitel 8.7 beschriebene „Sitzung verbrennen"-Funktion setzt dieses Recht in gewisser Weise bereits technisch um, noch bevor ein förmliches Löschungsersuchen überhaupt gestellt werden müsste: Nutzerinnen und Nutzer können ihre eigenen Daten (den lokalen Chatverlauf, Kapitel 11) jederzeit selbst und sofort unwiderruflich löschen, ohne eine dritte Stelle (den Relay-Betreiber) um Mitwirkung bitten zu müssen — ein struktureller Unterschied zu Diensten, bei denen Nutzerdaten zentral beim Anbieter liegen und ein Löschungsersuchen tatsächlich an diesen gerichtet werden muss. Für die beim Relay selbst vorhandenen Metadaten (Kapitel 21.1) bleibt das klassische DSGVO-Löschungsrecht gegenüber dem jeweiligen Betreiber weiterhin relevant und unverändert anwendbar.

34. Die ePrivacy-Richtlinie und ihre Ausnahmeregelung

34.1 Die ePrivacy-Richtlinie als Spezialregelung

Neben der allgemeinen DSGVO gilt für elektronische Kommunikationsdienste zusätzlich die ePrivacy-Richtlinie (2002, zuletzt wesentlich geändert 2009), die unter anderem die Vertraulichkeit der Kommunikation als eigenständiges Schutzgut absichert — ohne die in Kapitel 28.1 beschriebene, zeitlich befristete Ausnahmeregelung wäre selbst das heute bereits praktizierte freiwillige Scannen großer Anbieter nach dieser Richtlinie rechtlich nicht ohne Weiteres zulässig.

34.0a Warum ePrivacy als „lex specialis" gegenüber der DSGVO gilt

Ein für das Verständnis des Zusammenspiels beider Regelwerke wichtiges Rechtsprinzip: Die ePrivacy-Richtlinie gilt gegenüber der DSGVO als sogenannte „lex specialis" (spezielleres Recht) für den konkreten Bereich elektronischer Kommunikation — wo die ePrivacy-Regelung spezifische Vorgaben enthält, gehen diese den allgemeineren Regelungen der DSGVO vor. Das bedeutet praktisch: Auch wenn eine Datenverarbeitung nach den allgemeinen Grundsätzen der DSGVO (Kapitel 33.1a) zulässig wäre, kann sie dennoch an den spezielleren, strengeren Anforderungen der ePrivacy-Richtlinie scheitern, wenn es um die Vertraulichkeit elektronischer Kommunikation im engeren Sinne geht. Dieses Verhältnis erklärt, warum die in Kapitel 28.1 beschriebene Interim-Ausnahmeregelung überhaupt als eigenständiger Rechtsakt nötig war: Selbst wenn ein Anbieter eine datenschutzrechtliche Rechtsgrundlage nach der DSGVO für automatisiertes Scannen gehabt hätte, hätte ihn dies nicht automatisch von der spezielleren, strengeren Vertraulichkeitsvorgabe der ePrivacy-Richtlinie befreit.

34.2 Der seit Jahren angekündigte Nachfolger: die ePrivacy-Verordnung

Bereits seit 2017 wird auf EU-Ebene an einer ePrivacy-**Verordnung** gearbeitet, die die bisherige Richtlinie ablösen und (als Verordnung, anders als eine Richtlinie) unmittelbar und einheitlich in allen Mitgliedstaaten gelten soll. Dieses Vorhaben verzögerte sich über mehrere Jahre erheblich und wurde im Verhandlungsverlauf wiederholt mit der Chat-Kontrolle-Debatte verknüpft, da beide Regelwerke den Umgang mit der Vertraulichkeit elektronischer Kommunikation betreffen.

34.2a Warum sich zwei eigentlich getrennte Vorhaben gegenseitig blockieren können

Der in 34.2 beschriebene, mehrjährige Stillstand der ePrivacy-Verordnung illustriert ein allgemeines Phänomen komplexer Gesetzgebungsprozesse: Zwei formal unabhängige Vorhaben (die allgemeine Modernisierung des Datenschutzrechts für elektronische Kommunikation einerseits, die spezifische Chat-Kontrolle-Debatte andererseits) können sich gegenseitig faktisch blockieren, sobald Verhandlungsparteien beginnen, eine Einigung im einen Bereich von Zugeständnissen im anderen Bereich abhängig zu machen — auch wenn die beiden Vorhaben rechtlich getrennt behandelt werden. Für Beobachter des Gesetzgebungsprozesses (Kapitel 32.2) bedeutet dies: Der Fortschritt der ePrivacy-Verordnung lässt sich nicht isoliert von der Chat-Kontrolle-Debatte einschätzen, selbst wenn beide formal unterschiedliche Rechtsakte mit unterschiedlichen Zeitplänen sind.

34.3 Das Spannungsfeld zwischen beiden Regelwerken

Die grundsätzliche Spannung: Während die ePrivacy-Richtlinie im Ausgangspunkt die Vertraulichkeit der Kommunikation schützt und automatisiertes Scannen grundsätzlich untersagt, würde eine verpflichtende Chat-Kontrolle-Verordnung (Kapitel 29) genau diesen Schutz für einen eng definierten Zweck durchbrechen. Dieses Spannungsfeld ist kein Zufall, sondern der eigentliche Kern der politischen Auseinandersetzung: Beide Regelwerke verfolgen ein legitimes Ziel (Vertraulichkeit der Kommunikation einerseits, Kinderschutz andererseits), die sich in ihrer jeweils weitestgehenden Auslegung gegenseitig ausschließen.

34.3a Die Geschichte der ePrivacy-Richtlinie seit 2002

Die ursprüngliche ePrivacy-Richtlinie von 2002 entstand in einer Zeit, in der die heute dominierenden Ende-zu-Ende-verschlüsselten Messenger noch nicht existierten — der Anwendungsbereich zielte historisch vor allem auf klassische Telekommunikationsanbieter (Telefon- und SMS-Dienste) ab. Die wesentliche Änderung von 2009 erweiterte unter anderem die Regelungen zu Cookies und elektronischer Werbung, ohne den Grundcharakter der Richtlinie als Ergänzung zur allgemeinen Datenschutzregulierung zu verändern. Erst mit dem Aufkommen internetbasierter Kommunikationsdienste („Over-the-Top“-Dienste wie WhatsApp, Signal oder eben RenkerVault) stellte sich verstärkt die Frage, ob und wie diese Dienste in den

Anwendungsbereich der ePrivacy-Regulierung fallen — eine Klarstellung, die die seit 2017 diskutierte, aber bislang nicht final verabschiedete ePrivacy-Verordnung (Kapitel 34.2) ausdrücklich vorsieht.

34.3b Warum die Verzögerung der ePrivacy-Verordnung die Chat-Kontrolle-Debatte zusätzlich verkompliziert

Da die ePrivacy-Verordnung als eigentlich vorgesehener, moderner Rechtsrahmen für genau die Art von Diensten, um die es in der Chat-Kontrolle-Debatte geht, seit Jahren nicht verabschiedet ist, verhandeln beide Regelungsvorhaben faktisch nebeneinander her, ohne dass ihr Verhältnis zueinander abschließend geklärt wäre. Dies führt in Fachkreisen gelegentlich zu der Sorge, dass eine isolierte Verabschiedung der Chat-Kontrolle-Verordnung ohne eine gleichzeitig abgestimmte ePrivacy-Verordnung zu einem inkonsistenten Gesamtrechtsrahmen führen könnte, in dem zentrale Begriffe (etwa „elektronische Kommunikation“ oder „Vertraulichkeit“) in unterschiedlichen Rechtsakten nicht deckungsgleich verwendet werden.

34.3c App-Stores als zusätzlicher faktischer Regulierungspunkt

Über die in diesem Teil beschriebene Gesetzgebung hinaus üben auch die großen App-Store-Betreiber (Apple App Store, Google Play) durch ihre eigenen Richtlinien einen faktischen Einfluss auf Verschlüsselungs-Apps aus, unabhängig von staatlicher Regulierung: Beide Stores verlangen etwa Offenlegungen zur Datenverarbeitung („Datenschutz-Kennzeichnungen“) und behalten sich vor, Apps aus ihren jeweiligen Angeboten zu entfernen, sollten sie gegen interne Richtlinien verstoßen. Sollte eine verpflichtende Chatkontrolle-Regelung (Kapitel 28-29) tatsächlich in Kraft treten, könnten diese privaten App-Store-Betreiber faktisch zu einem zusätzlichen Durchsetzungsmechanismus werden, indem sie etwa Apps, die eine angeordnete Erkennungstechnologie nicht implementieren, aus ihren Stores entfernen — ein Mechanismus, der neben der eigentlichen gesetzlichen Verpflichtung eine zusätzliche, privatwirtschaftlich vermittelte Durchsetzungsebene darstellen würde. Für die Web- und selbst kompilierte Desktop-/Android-Variante von RenkerVault (Kapitel 12, außerhalb offizieller Stores vertrieben) wäre diese zusätzliche Ebene nicht unmittelbar relevant — ein weiterer praktischer Vorteil der in Kapitel 12.3 beschriebenen Multi-Plattform-Strategie, der über die dort genannten technischen Gründe hinausgeht.

34.3d Das „Sideloading“-Argument im DMA-Kontext

Interessanterweise berührt der bereits in Kapitel 32.5 erwähnte Digital Markets Act auch die in 34.3c beschriebene App-Store-Problematik: Der DMA verpflichtet große „Gatekeeper“-Plattformen (worum die Betriebssysteme der beiden größten mobilen Ökosysteme fallen) unter bestimmten Voraussetzungen dazu, alternative Vertriebswege für Apps zuzulassen („Sideloading“ — die Installation von Apps außerhalb des offiziellen Stores, wie sie in Kapitel 37.6 für die selbst gebaute RenkerVault-APK bereits genutzt

wird). Diese DMA-Vorschrift wurde ursprünglich aus wettbewerbsrechtlichen Erwägungen eingeführt, hat aber einen interessanten Nebeneffekt für die in 34.3c beschriebene Problematik: Je einfacher Sideloadung technisch und rechtlich zugänglich ist, desto weniger Durchgriff hätten App-Store-Betreiber, um eine app-store-vermittelte Durchsetzung einer möglichen Chatkontrolle-Verpflichtung zu erzwingen — zwei ursprünglich unabhängige Regelungsstränge (Wettbewerbsrecht einerseits, Kinderschutz-/Verschlüsselungsdebatte andererseits) berühren sich hier auf eine Weise, die in der öffentlichen Debatte bislang selten explizit gemacht wird.

35. Was RenkerVault rechtlich ist — und was nicht

35.1 Kein Overlay auf bestehende Dienste

Ein wichtiger konzeptioneller Unterschied zu dem in Kapitel 18.6b diskutierten „Privacy Adapter“-Konzept: RenkerVault ist keine nachträgliche Verschlüsselungsschicht, die über einen bestehenden, fremden Messenger gelegt wird, sondern eine eigenständige Anwendung mit eigenem Relay-Server. Diese Unterscheidung ist rechtlich relevant, weil ein „Overlay“-Ansatz unter Umständen die Nutzungsbedingungen des zugrundeliegenden Dienstes verletzen könnte — RenkerVault als eigenständiger Dienst unterliegt stattdessen ausschließlich seinen eigenen, selbst festgelegten Bedingungen und dem allgemein geltenden Recht.

35.1a Digitale Souveränität als zusätzlicher politischer Kontext

Über die individuelle Rechtslage hinaus lässt sich RenkerVaults Architektur-Ansatz auch in eine breitere politische Debatte einordnen: die europäische Diskussion um „digitale Souveränität“ — das Bestreben, kritische digitale Infrastruktur (Cloud-Dienste, Kommunikationsplattformen) weniger abhängig von einzelnen, oft außereuropäischen Anbietern zu gestalten. Ein selbst betreibbarer, offen dokumentierter Relay-Server (Kapitel 38) passt in dieses Bestreben, ohne dass RenkerVault selbst als politisches Projekt zur digitalen Souveränität konzipiert wäre: Die Möglichkeit, den gesamten Kommunikationsdienst — von der Kryptographie (Teil III) bis zum Hosting (Kapitel 38) — vollständig selbst zu kontrollieren, ist ein Nebeneffekt derselben Zero-Knowledge-Architektur (Kapitel 21), die primär aus Datenschutzgründen gewählt wurde, nicht aus industriepolitischen Erwägungen.

35.2 Betreiberverantwortung eines Zero-Knowledge-Relays

Wer einen eigenen Relay-Server betreibt (Teil V, Kapitel 38), stellt einen Vermittlungsdienst („Hosting-Dienst“ im Sinne des EU-Digital Services Act, DSA) bereit. Der DSA differenziert die Verantwortlichkeit von Vermittlungsdiensten grundsätzlich danach, ob und inwieweit der Betreiber tatsächliche Kenntnis von rechtswidrigen Inhalten hat oder haben könnte. Ein Zero-Knowledge-Relay, das strukturell keinen

Zugriff auf Klartext hat (Kapitel 21), befindet sich in einer grundlegend anderen tatsächlichen Ausgangslage als ein Anbieter, der Inhalte einsehen könnte, es aber unterlässt — die konkrete rechtliche Einordnung im jeweiligen Einzelfall ist jedoch, wie in 26.1 betont, keine Frage, die dieses Buch abschließend beantworten kann.

35.2a Der Digital Services Act im Detail: abgestufte Verantwortlichkeit

Der Digital Services Act (DSA, seit 2024 EU-weit anwendbar) unterscheidet zwischen mehreren Kategorien von Diensten mit jeweils unterschiedlichem Pflichtenkatalog: reine Durchleitung („Mere Conduit“), Caching und Hosting. Ein Zero-Knowledge-Relay wie das in Teil III, Kapitel 21 beschriebene, das ausschließlich verschlüsselte Envelopes zwischen Geräten weiterleitet und keine dauerhafte, für den Betreiber einsehbare Speicherung von Inhalten vornimmt, weist strukturelle Ähnlichkeiten zur Kategorie der reinen Durchleitung auf — der Betreiber „übermittelt“ letztlich nur bereits verschlüsselte Datenpakete, ohne sie inhaltlich zu verändern oder auswerten zu können. Der DSA knüpft die Verantwortlichkeit eines Vermittlungsdienstes maßgeblich daran, ob dieser tatsächliche Kenntnis von rechtswidrigen Inhalten hat oder nach den Umständen haben müsste („Notice-and-Action“-Mechanismus) — bei einem Dienst, der technisch überhaupt keinen Zugriff auf den Inhalt hat (Kapitel 21.1), stellt sich diese Frage in einer strukturell anderen Weise als bei einem Dienst, der Inhalte einsehen könnte, es aber unterlässt. Eine abschließende rechtliche Einordnung dieser Frage für einen konkreten Einzelfall bleibt, wie in Kapitel 26.1 betont, Gegenstand individueller Rechtsberatung.

35.3 RenkerVault ist kein Werkzeug „gegen“ ein bestimmtes Gesetz

Wie bereits in Teil I, Kapitel 2.2 und Kapitel 3.3 betont: RenkerVault ist nicht als gezieltes Umgehungswerkzeug gegen eine spezifische, bereits geltende Rechtsvorschrift konzipiert — zum Zeitpunkt der Erstellung dieses Buches existiert die Chat-Kontrolle-Verpflichtung, wie in Kapitel 28 dargestellt, ohnehin noch nicht in verabschiedeter Form. RenkerVault ist stattdessen dieselbe Art von privatsphäre-schützender Architektur wie zahlreiche bereits etablierte, legal weltweit genutzte Messenger (Kapitel 24) — eine Architektur-Entscheidung für den Fall, dass ein struktureller Zugriffspunkt auf private Kommunikation gesetzlich verpflichtend würde, nicht eine Reaktion auf eine bereits existierende, gebrochene Rechtspflicht.

35.3a Der Unterschied zwischen einem Werkzeug und seiner Nutzung

Ein grundsätzlicher rechtlicher und ethischer Gedanke, der über RenkerVault hinausgeht, aber hier explizit festgehalten werden soll: Die Rechtmäßigkeit eines Werkzeugs (etwa eines Verschlüsselungs-Messengers) ist von der Rechtmäßigkeit seiner jeweiligen Nutzung zu unterscheiden — ein Küchenmesser ist nicht deshalb rechtswidrig, weil es theoretisch auch als Waffe missbraucht werden könnte, und ein

verschlüsselter Kommunikationskanal ist nicht deshalb problematisch, weil er theoretisch auch für illegale Zwecke genutzt werden könnte. Diese sogenannte „Dual-Use“-Eigenschaft (Werkzeuge, die sowohl legal als auch illegal genutzt werden können) betrifft praktisch jede leistungsfähige Technologie, von Verschlüsselung über Fahrzeuge bis zu Finanzdienstleistungen — der rechtliche und gesellschaftliche Umgang damit besteht typischerweise darin, konkretes Fehlverhalten zu sanktionieren (Kapitel 28.4b: bereits geltendes Strafrecht), nicht das zugrundeliegende, neutrale Werkzeug selbst zu verbieten. Diese Unterscheidung ist der gedankliche Kern, warum RenkerVault (Kapitel 2.2) explizit nicht als Umgehungswerkzeug, sondern als neutrale, privatsphäre-schützende Infrastruktur verstanden werden möchte.

36. Politische Handlungsmöglichkeiten für Nutzer

36.1 Der Unterschied zwischen technischem und politischem Handeln

Dieses Buch hat bislang vor allem technische Antworten auf eine politische Frage beschrieben. Ein technisches Werkzeug wie RenkerVault kann jedoch die zugrundeliegende politische Debatte nicht ersetzen — es kann höchstens die praktischen Konsequenzen einer bestimmten Entscheidung für den einzelnen Nutzer abfedern. Wer sich mit der in diesem Teil beschriebenen Thematik auseinandergesetzt hat, hat typischerweise auch die Möglichkeit, sich am politischen Prozess selbst zu beteiligen.

36.1a Technik als Verhandlungsmasse, nicht als Endzustand

Ein Gedanke, der die in diesem Buch beschriebene Beziehung zwischen Technik (Teil II/III) und Recht (Teil IV) abschließend einordnet: Technische Architektur-Entscheidungen wie die in RenkerVault umgesetzte Zero-Knowledge-Architektur (Kapitel 21) verändern nicht direkt geltendes Recht — sie verändern die **Ausgangsbedingungen**, unter denen künftige rechtliche und politische Entscheidungen getroffen werden. Eine Gesellschaft, in der die überwiegende Mehrheit der Kommunikation über Zero-Knowledge-Architekturen läuft, steht vor anderen politischen Abwägungsfragen als eine Gesellschaft, in der zentralisierte, für den Betreiber einsehbare Kommunikation die Norm ist — nicht weil das Recht sich geändert hätte, sondern weil sich die technische Realität geändert hat, auf die das Recht reagieren muss. Diese Wechselwirkung zwischen technischer Realität und rechtlicher Regelung ist keine Einbahnstraße: Genauso wie Technik die Rahmenbedingungen für Recht verändert (dieser Absatz), verändert umgekehrt Recht auch die Rahmenbedingungen, unter denen Technik entwickelt wird (Kapitel 30.4a, Gedankenexperiment) — beide Ebenen stehen in einem fortlaufenden, wechselseitigen Verhältnis zueinander, das dieses Buch von Anfang an (Kapitel 1) abzubilden versucht

hat, statt sie als getrennte, unabhängige Themenfelder zu behandeln.

36.2 Konkrete Ansatzpunkte

- **Kontaktaufnahme mit gewählten Vertretern:** Sowohl im Europäischen Parlament als auch in den nationalen Regierungen (die im Rat der EU über die Position des jeweiligen Mitgliedstaats mitentscheiden) lassen sich Abgeordnete und Regierungsvertreter direkt zu ihrer Position zur Chat-Kontrolle-Verordnung befragen.
- **Unterstützung etablierter zivilgesellschaftlicher Organisationen** (Kapitel 31.3), die den Gesetzgebungsprozess fachlich und rechtlich begleiten und dabei häufig aktuellere, tagesgenaue Informationen bereitstellen als ein gedrucktes Buch es könnte.
- **Informierte öffentliche Diskussion:** Die in Teil I, Kapitel 3 und Teil III, Kapitel 30 dargestellten technischen Zusammenhänge sind komplex genug, dass viele öffentliche Debattenbeiträge sie vereinfachen oder verzerren — ein fundiertes eigenes Verständnis (wie es dieses Buch zu vermitteln versucht) ist die Grundvoraussetzung, um an dieser Debatte konstruktiv teilzunehmen, unabhängig von der eigenen letztlichen Position.

36.2a Formelle Beteiligungsmöglichkeiten auf EU-Ebene

Über die informellen, in 36.2 genannten Ansatzpunkte hinaus bietet die EU auch formelle Beteiligungskanäle: Öffentliche Konsultationen der Europäischen Kommission erlauben es grundsätzlich jeder Person und Organisation, im Vorfeld eines Gesetzgebungsvorschlags oder während bestimmter Verfahrensphasen eine schriftliche Stellungnahme einzureichen, die in die offizielle Auswertung einfließt. Zusätzlich lässt sich über die Europäische Bürgerinitiative (sofern die formellen Voraussetzungen, insbesondere eine Mindestzahl an Unterstützern aus mehreren Mitgliedstaaten, erreicht werden) ein Thema direkt auf die politische Agenda der Kommission setzen. Diese Mechanismen ersetzen nicht die in 36.2 genannten direkteren Wege über gewählte Vertreter, bieten aber einen zusätzlichen, strukturierten Kanal für alle, die sich über die reine Meinungsäußerung hinaus formell am Verfahren beteiligen möchten.

36.2b Offene Rechtsfragen im Überblick

Als Ergänzung zur in Kapitel 24.6e gezeigten technischen Rückverfolgbarkeitstabelle eine vergleichbare Zusammenfassung der in Teil IV identifizierten, noch offenen Rechtsfragen — bewusst als „offen“ markiert, nicht als abschließend beantwortet (Kapitel 26.2):

Offene Frage	Warum noch nicht geklärt	Fundstelle
Wird die finale Chat-Kontrolle-Verordnung Ende-zu-Ende-verschlüsselte Dienste einbeziehen?	Rat hat bislang keine ausreichende qualifizierte Mehrheit gefunden	Kapitel 28.3-28.5

Offene Frage	Warum noch nicht geklärt	Fundstelle
Wie wird die Altersverifikationspflicht konkret ausgestaltet?	Abhaengig vom Zusammenspiel mit eIDAS 2.0	Kapitel 29.5, 32.5b
Faellt automatisierte Erkennungssoftware unter den AI Act als Hochrisiko-System?	Eigenstaendige, noch nicht abschliessend geklaerte Rechtsfrage	Kapitel 32.5a
Wann kommt die ePrivacy-Verordnung als Ersatz der Richtlinie?	Verzoegert sich bereits seit 2017	Kapitel 34.2-34.3b
Wie wird die Verantwortlichkeit eines Zero-Knowledge-Relay-Betreibers nach dem DSA konkret eingestuft?	Bislang keine gefestigte Rechtsprechung zu diesem spezifischen Architekturtyp	Kapitel 35.2a
Loest der Betrieb eines RenkerVault-Relays fuer eine groessere Gruppe eine DPIA-Pflicht aus?	Einzelfallabhaengig	Kapitel 33.3a

Diese Tabelle ist als Wegweiser gedacht: Wer eine dieser Fragen für die eigene konkrete Situation abschließend beantwortet haben möchte, findet hier den Verweis auf das relevante Kapitel als Ausgangspunkt für eine vertiefte, im Zweifel fachkundig begleitete Recherche (Kapitel 26.1).

36.3 Ein Schlusswort zu Teil IV

Die in diesem Teil beschriebene Debatte hat kein einfaches, für alle Seiten zufriedenstellendes Ergebnis — sie stellt einen echten Zielkonflikt zwischen zwei jeweils legitimen Anliegen dar. Dieses Buch bezieht an dieser Stelle bewusst keine abschließende politische Position, sondern versucht, die technischen und rechtlichen Fakten so darzustellen, dass Leserinnen und Leser sich eine eigene, informierte Meinung bilden können — im Einklang mit dem im Vorwort formulierten Grundprinzip der Ehrlichkeit über Vollständigkeit.

Praxis & Anhang

Installation, Betrieb, Checklisten, Glossar und Nachschlagewerk

36a. Feature-Matrix nach Plattform

Bevor Teil V mit der praktischen Installation beginnt, eine konsolidierte Übersicht, welche der in Teil II beschriebenen Funktionen auf welcher Plattform verfügbar sind:

Funktion	Web	Desktop (Tauri)	Android (Capacitor)
Text-/Bild-/Dateinachrichten (Kapitel 8.1/8.2)	Ja	Ja	Ja
Sprachnachrichten (Kapitel 8.2)	Ja (Mikrofon-API)	Ja	Ja (RECORD_AUDIO-Berechtigung)
Gruppen/Kanaele (Kapitel 9)	Ja	Ja	Ja
Einbruchsalarm-System (Kapitel 10)	Ja	Ja	Ja
Duress-PIN (Kapitel 10.5)	Ja	Ja	Ja
Tor-Hidden-Service-Relay (Kapitel 38.4)	Ja (als Client)	Ja (als Client)	Ja (als Client)
Push-Benachrichtigungen (Kapitel 12.4a)	Nein (bewusst)	Nein (bewusst)	Nein (bewusst)
Offline-Betrieb des Relay-Servers selbst	Ja (Node.js)	Ja (Node.js, gebündelt)	Nein (nur Client)
GrapheneOS-Kompatibilität (Kapitel 12.4)	N/A	N/A	Ja
Barrierefreiheit/Screenreader (Kapitel 12a)	Ja	Ja (systemeigenes WebView)	Ja (systemeigenes WebView)
Windows-Installer (Kapitel 12.5)	N/A	Ja	N/A

Diese Matrix bestätigt konkret, was in Kapitel 12.1 als Kernversprechen formuliert wurde: Die Kryptographie- und Netzwerk-Schicht (Kapitel 6.1) verhält sich auf allen drei Plattformen identisch — Unterschiede bestehen ausschließlich bei plattformspezifischen Fähigkeiten (etwa dem eigenständigen Serverbetrieb, der auf einem Mobilgerät architektonisch nicht vorgesehen ist).

37. Installation und Betrieb Schritt für Schritt

37.0a Checkliste der Voraussetzungen vor dem ersten Start

- [] Node.js Version 18 oder neuer installiert (`node --version` zur Prüfung)
- [] Ausreichend freier Speicherplatz (mehrere hundert MB für Abhängigkeiten von Client und Server zusammen)
- [] Ein moderner Browser für die Web-Variante (Chromium-, Firefox- oder WebKit-basiert, jeweils aktuelle Version)
- [] Für den Desktop-Build zusätzlich: Rust und die Tauri-CLI (Kapitel 37.5)
- [] Für den Mobile-Build zusätzlich: Android Studio bzw. das Android SDK sowie ein passendes JDK (Kapitel 37.6)
- [] Für einen selbst betriebenen Relay-Server: Zugriff auf einen Linux-Server oder eine lokale Linux-/WSL-Umgebung (Kapitel 38)

37.1 Voraussetzungen

Für den lokalen Betrieb (Ausprobieren, Entwicklung, Selbst-Hosting) werden benötigt: Node.js (Version 18 oder neuer) für Client und Relay-Server, ein moderner Browser (Chromium-, Firefox- oder WebKit-basiert) für die Web-Variante, sowie optional Rust und die Tauri-CLI für den Windows-Desktop-Build bzw. Android Studio/das Android SDK für den Mobile-Build. Keine dieser Komponenten ist proprietär oder kostenpflichtig.

37.2 Relay-Server lokal starten

```
cd server
npm install
node src/index.js
# Relay lauscht per Default auf ws://localhost:8787
```

Der Relay-Server benötigt keine Datenbank-Installation für den lokalen Testbetrieb (Kapitel 21.3 beschreibt die Grenzen dieses In-Memory-Modus für den Produktivbetrieb).

37.3 Client lokal starten (Web-Variante)

```
cd client
npm install
npm run dev
# Vite Dev-Server, standardmaessig unter http://localhost:5173
```

37.4 Ersten Tresor anlegen

Beim ersten Öffnen der App im Browser wird automatisch eine neue Konto-Kennung sowie das in Teil III, Kapitel 14 beschriebene Schlüsselmaterial erzeugt. Es folgt die

Festlegung einer Passphrase (Kapitel 7.2) und optional einer Duress-PIN (Kapitel 10.5). Beide sollten unabhängig voneinander gewählt und sicher verwahrt werden — es gibt bewusst **keine** „Passwort vergessen“-Funktion, da eine solche Funktion zwangsläufig eine Hintertür zum verschlüsselten Datenbestand implizieren würde (siehe die Erläuterung zur Zero-Knowledge-Architektur in Kapitel 21).

37.5 Desktop-Build (Windows, Tauri)

```
cd client
npx tauri build
# Ergebnis liegt in client/src-tauri/target/release/
```

Für den fertigen Installer zusätzlich Inno Setup (ISCC.exe) über das mitgelieferte Skript `installer/RenkerVault.iss` ausführen.

37.6 Mobile-Build (Android, Capacitor)

```
cd client
npm run build
npx cap sync android
cd android
./gradlew assembleDebug
# APK liegt in android/app/build/outputs/apk/debug/
```

Für eine Installation auf einem echten Gerät statt eines Emulators muss zusätzlich „Installation aus unbekannten Quellen“ erlaubt werden, sofern die App nicht über einen offiziellen App-Store vertrieben wird — dies ist bei selbst gebauten APKs, die nicht über Google Play installiert werden, normal und kein Sicherheitsproblem für sich genommen, sofern die APK aus vertrauenswürdiger, selbst kontrollierter Quelle stammt.

37.7 Echte Kontakte anlegen: ein Zwei-Personen-Test

Um RenkerVault mit einer zweiten realen Person zu testen (statt der simulierten Demo-Kontakte), starten beide Seiten je eine eigene Instanz gegen denselben Relay (lokal z. B. über zwei unterschiedliche Ports, `npx vite --port 5174` für die zweite Instanz), legen je einen eigenen Tresor an, und tragen anschließend gegenseitig die eigene Konto-Kennung unter „Kontakte → Echten Kontakt hinzufügen“ ein (Kapitel 7.4). Aus Sicherheitsgründen sollte der Kontakt-Kennungs austausch über einen vom Chat selbst unabhängigen Kanal erfolgen (mündlich, persönliches Treffen), und die Safety Number (Kapitel 7.5) sollte vor dem Austausch sensibler Inhalte verglichen werden.

37.8 Häufige Installationsprobleme und ihre Lösung

Die folgenden Probleme sind während der Entwicklung tatsächlich aufgetreten und werden hier dokumentiert, da sie mit hoher Wahrscheinlichkeit auch anderen Nutzerinnen und Nutzern begegnen können:

„Die Syntax für den Dateinamen... ist falsch" beim Android-Gradle-Build. Ursache: Die Datei `local.properties` verwendet unter Windows Backslashes im Pfad (`sdk.dir=C:\Users\...`), die von Java-„.properties“-Dateien als Escape-Zeichen interpretiert werden und dadurch den Pfad verstümmeln. Lösung: Im Pfad konsequent Forward-Slashes verwenden (`sdk.dir=C:/Users/.../Android/Sdk`), auch unter Windows — Java interpretiert diese korrekt.

Rust/Cargo nach der Installation nicht im aktuellen Terminal gefunden. Ursache: Eine geänderte Systemumgebungsvariable (PATH) wird von bereits laufenden Terminal-Sitzungen nicht automatisch übernommen. Lösung: Terminal neu starten, oder den Pfad für die aktuelle Sitzung manuell voranstellen (z. B. in PowerShell: `$env:Path = "$env:USERPROFILE\.cargo\bin;$env:Path"`).

Inno Setup (ISCC.exe) wird nicht in Program Files (x86) gefunden. Ursache: Neuere Inno-Setup-Installationen legen sich je nach Installationsmodus unter `AppData\Local\Programs` statt im klassischen Programme-Verzeichnis ab. Lösung: Gezielt in `%LOCALAPPDATA%\Programs` nach `Inno Setup 6\ISCC.exe` suchen, statt die klassische Programme-Pfadannahme zu verwenden.

Selbstsigniertes TLS-Zertifikat lässt sich über Git-Bash/MSYS nicht erzeugen. Ursache: MSYS (die Unix-Kompatibilitätsschicht, die Git-Bash unter Windows nutzt) wandelt Pfad-artige Argumente wie `/CN=localhost` automatisch in Windows-Pfade um, was den `openssl`-Befehl mit unsinnigen Parametern aufruft. Lösung: Die Umgebungsvariable `MSYS_NO_PATHCONV=1` vor dem Befehl setzen, um diese automatische Pfad-Konvertierung zu unterdrücken.

Veraltete Web-Assets in einer neu gebauten Android-APK. Ursache: Nach Code-Änderungen wurde `npx cap sync android` ausgeführt, aber zuvor kein neuer `npm run build` durchgeführt — dadurch synchronisiert Capacitor lediglich den alten, bereits vorhandenen Build-Ordner in das Android-Projekt. Lösung: Vor jedem `cap sync` sicherstellen, dass `npm run build` tatsächlich frisch durchgelaufen ist (am jeweiligen Datei-Hash im Build-Ordner überprüfbar).

Client verbindet sich nicht zum selbst gehosteten Relay („Connection refused" oder Dauerzustand „connecting"). Mögliche Ursachen, in der Reihenfolge der Wahrscheinlichkeit zu prüfen: Erstens, der Relay-Prozess läuft nicht oder ist abgestürzt (Kapitel 38.2: `systemctl status renkervault-relay` bei systemd-Betrieb prüfen). Zweitens, eine Firewall blockiert den konfigurierten Port (Kapitel 44b: Standard 8787) — sowohl die Firewall des Servers selbst als auch eine vorgelagerte Cloud-Firewall des Hosting-Anbieters prüfen. Drittens, bei Verwendung eines Reverse-Proxys (Kapitel 38.2): Der Proxy selbst läuft, leitet aber nicht korrekt an `127.0.0.1:8787` weiter — die Caddy- bzw. nginx-Konfiguration auf Tippfehler prüfen. Viertens, ein Zertifikatsproblem bei `wss://` (Kapitel 21.2a): Der Browser blockiert die Verbindung bei einem ungültigen oder abgelaufenen TLS-Zertifikat, sichtbar in der

Entwicklerkonsole des Browsers.

Sprachnachrichten lassen sich auf Android nicht aufnehmen, obwohl die App startet. Ursache: Die in Kapitel 36a tabellierte `RECORD_AUDIO`-Berechtigung wurde vom Nutzer bei der Installation abgelehnt oder nachträglich in den Android-Systemeinstellungen entzogen. Lösung: In den Android-Systemeinstellungen unter „Apps → RenkerVault → Berechtigungen“ die Mikrofon-Berechtigung manuell erteilen.

37.9 Aktualisierung auf eine neue Version

Da RenkerVault (Stand dieses Buches) keinen automatischen Update-Mechanismus über einen offiziellen App-Store bietet (Kapitel 34.3c: bewusst außerhalb der Stores vertrieben), erfolgt eine Aktualisierung manuell. Für die Web-Variante (Kapitel 12.1) genügt es, den aktuellen Code erneut zu bauen (`npm run build`) und auszuliefern — Nutzerinnen und Nutzer erhalten die neue Version automatisch beim nächsten Laden der Seite, ohne eigenes Zutun. Für die Desktop- und Android-Varianten (Kapitel 12.2/12.3) muss dagegen ein neuer Installer bzw. eine neue APK gebaut und manuell verteilt werden, da diese Varianten lokal auf dem Gerät installiert sind. Vor jeder Aktualisierung, die Änderungen an der Kryptographie-Schicht (Kapitel 6.1) betrifft, empfiehlt sich ein Blick in die in Kapitel 24.6d beschriebene Protokoll-Versionierung: Bestehende, bereits verschlüsselte Sitzungen sollten auch nach einem Update ohne manuellen Eingriff weiter funktionieren, sofern die Aktualisierung das zugrundeliegende Vault- und Ratchet-Dateiformat (Kapitel 44.5, `v: 1`) nicht in einer nicht abwärtskompatiblen Weise verändert.

38. Hosting-Optionen im Vergleich

38.1 Übersicht der drei Wege

Weg	Aufwand	Laufender Betrieb	Anonymität	Empfohlen fuer
Gemieteter VPS + Reverse-Proxy (Caddy)	Mittel	24/7, automatisches TLS-Zertifikat	Server-IP oeffentlich bekannt	Dauerhafter Betrieb mit mehreren Nutzern
Eigener PC + Portweiterleitung	Niedrig-Mittel	Nur waehrend PC laeuft	Server-IP oeffentlich bekannt	Kurzes Ausprobieren im Familien-/Freund eskreis
Tor Hidden Service	Mittel-Hoch	24/7 (auf VPS) oder situativ (auf PC)	Server-IP nicht ermittelbar	Maximale Anonymitaet, houhere Bedrohungslage

38.2 Weg 1 im Detail: gemieteter VPS

Ein virtueller Server (VPS) bei einem der zahlreichen Hosting-Anbieter kostet je nach Anbieter und Ausstattung oft nur wenige Euro pro Monat und bietet eine feste IP-Adresse sowie durchgehende Erreichbarkeit. Der empfohlene Aufbau: ein schlanker Linux-Server (z. B. Debian oder Ubuntu), der Relay-Prozess unter einem eigenen, unprivilegierten Systembenutzer via systemd verwaltet (siehe die mitgelieferte `renkervault-relay.service`-Datei mit Sandbox-Härtung: `NoNewPrivileges`, `ProtectSystem=strict`, `ProtectHome`, minimales `CapabilityBoundingSet`), und ein vorgeschalteter Caddy-Reverse-Proxy, der TLS-Zertifikate automatisch über Let's Encrypt bezieht und erneuert, während der eigentliche Relay-Prozess nur auf `127.0.0.1` lauscht und vom Internet aus nicht direkt erreichbar ist.

38.2a Die Konfigurationsdatei im Detail

Der vollständige Inhalt der mitgelieferten `.env.example`-Vorlage, die die in Kapitel 21.3/38 beschriebenen Betriebsparameter zusammenfasst:

```
PORT=8787
HOST=127.0.0.1

# Nur noetig ohne Reverse-Proxy (natives TLS):
# TLS_CERT_FILE=/pfad/zu/fullchain.pem
# TLS_KEY_FILE=/pfad/zu/privkey.pem

# Nur aktivieren, wenn ein VERTRAUENSWUERDIGER Reverse-Proxy
# den X-Forwarded-For-Header setzt (Kapitel 21, Client-IP-Ermittlung):
# TRUST_PROXY=1
```

Die auskommentierten Standardwerte sind bewusst so gewählt, dass ein unbedarft übernommener Standardbetrieb (`HOST=127.0.0.1`, kein natives TLS, kein aktiviertes „Trust Proxy“) automatisch die sicherste Grundkonfiguration ergibt: Der Relay ist ohne explizit vorgeschalteten Reverse-Proxy überhaupt nicht von außen erreichbar, und eine Fälschung der Client-IP über `X-Forwarded-For` (Kapitel 21) ist ohne aktiviertes „Trust Proxy“ wirkungslos. Wer diese Werte ändern möchte, muss dies bewusst und explizit tun — ein Beispiel für das Entwurfsprinzip „sichere Voreinstellung“ („Secure by Default“), das sich durch die gesamte Konfiguration des Projekts zieht (vergleiche Kapitel 8.3, Lesebestätigungen standardmäßig aus).

38.3 Weg 2 im Detail: eigener PC

Der eigene Rechner lässt sich ebenfalls als Relay-Host verwenden, mit spürbaren praktischen Einschränkungen: Der Rechner muss durchgehend eingeschaltet und mit dem Internet verbunden bleiben, und im Heimrouter muss eine Port-Weiterleitung eingerichtet werden, wodurch der Rechner (zumindest auf dem freigegebenen Port) direkt aus dem Internet erreichbar wird — ein zusätzliches Risiko für das gesamte Heimnetzwerk, falls der Relay-Prozess selbst eine unentdeckte Schwachstelle enthält. Ohne feste IP-Adresse ist zusätzlich ein DynDNS-Dienst nötig, damit die Adresse trotz wechselnder Internetanschluss-IP stabil erreichbar bleibt.

38.4 Weg 3 im Detail: Tor Hidden Service

Wie in Teil III, Kapitel 22.3 technisch beschrieben, lässt sich derselbe Relay-Prozess zusätzlich ausschließlich über einen Tor Hidden Service veröffentlichen — ohne öffentliche IP-Adresse und ohne offenen Port. Dieser Weg eignet sich besonders für Nutzergruppen mit erhöhtem Schutzbedarf (Kapitel 39.3), erfordert aber zusätzliches technisches Verständnis für den Betrieb eines Tor-Relays und akzeptiert unter Umständen etwas höhere Latenz durch das mehrstufige Tor-Routing.

38.4a Der Serverstandort als rechtliche Überlegung

Neben den technischen Eigenschaften der drei Hosting-Wege ist der **Rechtsraum**, in dem ein VPS steht, eine oft übersehene Zusatzüberlegung. Ein Server innerhalb der EU unterliegt unmittelbar der DSGVO (Teil IV, Kapitel 33) und den entsprechenden Aufsichtsbehörden der Mitgliedstaaten; ein Server außerhalb der EU kann zusätzlichen, teils abweichenden rechtlichen Zugriffsmöglichkeiten der jeweiligen lokalen Behörden unterliegen (vergleiche die Erwähnung des US-amerikanischen CLOUD Act in Kapitel 32.3a). Da RenkerVaults Zero-Knowledge-Architektur (Kapitel 21) den Inhalt selbst unabhängig vom Serverstandort schützt, betrifft diese Überlegung in erster Linie die dort verbleibenden Metadaten (Kapitel 22.1) — ein Betreiber mit besonders hohem Schutzbedarf für Metadaten sollte den Serverstandort bewusst wählen oder, wie in Kapitel 38.4 beschrieben, ganz auf einen Tor Hidden Service ausweichen, der die Frage des physischen Serverstandorts für die Verbindungsanonymität weitgehend entschärft.

38.4b Ressourcenbedarf realistisch einschätzen

Da der Relay-Server bewusst schlank gehalten ist (Kapitel 6.2) und pro Nutzer nur geringe Mengen an Metadaten sowie kurzzeitig zwischengespeicherte, bereits verschlüsselte Envelopes verwaltet (Kapitel 21.3), ist der Ressourcenbedarf für kleine bis mittlere Nutzergruppen (Familie, Freundeskreis, kleiner Verein) gering — ein VPS mit minimaler Ausstattung reicht in der Regel aus, ohne dass konkrete Preisangaben hier sinnvoll wären, da sich Hosting-Preise laufend ändern und je nach Anbieter und Region stark variieren. Relevanter als der Preis ist die Einschätzung des tatsächlichen Bedarfs: Die in Kapitel 44b tabellierte maximale Envelope-Größe (2 MiB) und Warteschlangenlänge (500 Envelopes pro Nutzer) begrenzen den Ressourcenbedarf strukturell, unabhängig von der tatsächlichen Nutzerzahl, solange diese im für einen Freundes- oder Familienkreis üblichen Rahmen bleibt. Für eine deutlich größere Nutzerbasis (etwa mehrere hundert gleichzeitig aktive Konten) wäre, wie in Kapitel 21.3 und 25.2 beschrieben, eine Umstellung auf einen persistenten Datenbank-Store sowie eine sorgfältigere Kapazitätsplanung angeraten.

38.5 Die Relay-Adresse konfigurieren

Unabhängig vom gewählten Weg wird die endgültige Adresse einmalig in den App-Einstellungen unter „Relay-Server“ eingetragen; alle an einer Konversation

beteiligten Personen müssen dieselbe Adresse verwenden. Wichtig: Sobald es sich nicht mehr um den eigenen, lokalen Rechner handelt, muss die Adresse mit `wss://` (verschlüsseltes WebSocket) statt `ws://` beginnen — die App warnt aktiv, sollte dennoch eine unverschlüsselte Verbindung zu einem entfernten Host konfiguriert werden.

38.5a Auswahlkriterien für einen Hosting-Anbieter, unabhängig vom Preis

Statt konkreter, schnell veraltender Namen oder Preise folgt hier eine Liste allgemeiner Kriterien, die bei der Wahl eines VPS-Anbieters für einen RenkerVault-Relay (Kapitel 38.2) unabhängig von Marktentwicklungen relevant bleiben:

- **Jurisdiktion des Anbieters und des Rechenzentrums** (Kapitel 38.4a) — beide können auseinanderfallen, etwa ein Anbieter mit Hauptsitz in einem Land, aber Rechenzentren in einem anderen.
- **Unterstützung für gängige Linux-Distributionen** mit langfristigem Sicherheitsupdate-Support (Kapitel 39.3).
- **Verfügbarkeit einer Firewall auf Netzwerkebene** vor dem eigentlichen Server, zusätzlich zur in Kapitel 44.6c beschriebenen systemd-Sandbox-Härtung.
- **Transparente, nachvollziehbare Datenschutzbestimmungen des Anbieters selbst** — der Hosting-Anbieter sieht zwar nur Metadaten (Kapitel 21), sollte aber dennoch seine eigenen Datenschutzpraktiken offenlegen.
- **Unterstützung für IPv6** neben IPv4, für zukunftsichere Erreichbarkeit.
- **Möglichkeit eines eigenen, root-artigen Zugriffs**, um die in Kapitel 38.2 beschriebene systemd-Konfiguration vollständig selbst zu kontrollieren, statt auf eine eingeschränkte Verwaltungsoberfläche angewiesen zu sein.

38.6 Betriebsüberwachung ohne Verletzung der Zero-Knowledge-Eigenschaft

Wer einen eigenen Relay betreibt (Kapitel 38), möchte typischerweise ein Mindestmaß an Betriebsüberwachung (Monitoring) einrichten, um Ausfälle oder ungewöhnliches Verhalten frühzeitig zu erkennen — ohne dabei versehentlich die in Kapitel 21 beschriebene Zero-Knowledge-Eigenschaft zu unterlaufen. Sinnvolle, unbedenkliche Überwachungsmetriken umfassen: die Anzahl aktiver WebSocket-Verbindungen, die Größe der Offline-Warteschlangen (Kapitel 21.3, ohne deren Inhalt einzusehen), CPU- und Speicherauslastung des Relay-Prozesses, sowie die Häufigkeit von Rate-Limit-Auslösungen (Kapitel 10.2a) als Indikator für mögliche Angriffsversuche. Ausdrücklich zu vermeiden wäre dagegen jede Form von Logging, das über die in Kapitel 21.1 beschriebenen, für die Zustellung ohnehin nötigen Metadaten hinausgeht — ein Betreiber, der etwa zusätzlich sämtliche Verbindungszeitpunkte jedes einzelnen

Kontos dauerhaft protokolliert, obwohl dies für den Betrieb nicht erforderlich ist, würde damit eigenständig ein zusätzliches Metadaten-Risiko schaffen, das über das hinausgeht, was die Architektur selbst strukturell bereits vorsieht (Kapitel 22.1).

39. Sicherheitschecklisten für Nutzer

39.1 Checkliste: Alltagsnutzung

- ☐ Passphrase gewählt, die nirgendwo sonst wiederverwendet wird, und sicher (z. B. in einem separaten Passwort-Manager) verwahrt.
- ☐ Safety Number mit wichtigen Kontakten über einen unabhängigen Kanal verglichen (Kapitel 7.5).
- ☐ Lesebestätigungen/Tippindikator bewusst ein- oder ausgeschaltet gelassen (Kapitel 8.3), statt den Standard unreflektiert zu übernehmen.
- ☐ Automatische Backups des Geräts (falls vorhanden) berücksichtigen, dass ein unverschlüsseltes Geräte-Backup die in Kapitel 11 beschriebene lokale Verschlüsselung unterlaufen könnte, falls das Backup selbst nicht zusätzlich verschlüsselt ist.
- ☐ Geräteliste (Sicherheitszentrale) gelegentlich auf unbekannte Einträge prüfen (Kapitel 7.3).

39.2 Checkliste: erhöhter Schutzbedarf (Journalismus, Anwaltschaft, Aktivismus)

- ☐ Eigenen Relay-Server (statt eines von Dritten betriebenen) in Erwägung ziehen, um die Metadaten-Sichtbarkeit (Kapitel 21/22) auf eine selbst kontrollierte Stelle zu begrenzen.
- ☐ Tor-Hidden-Service-Betrieb (Kapitel 38.4) für maximale IP-Anonymität in Erwägung ziehen.
- ☐ Duress-PIN einrichten und deren Existenz nicht gegenüber Dritten erwähnen (die Wirksamkeit hängt davon ab, dass ein Angreifer nicht weiß, dass eine solche Funktion existiert).
- ☐ „Sitzung verbrennen“ (Kapitel 8.7) als bewusstes Werkzeug nach Abschluss besonders sensibler Konversationen nutzen.
- ☐ Gerätesicherheit über RenkerVault hinaus prüfen: vollständige Festplattenverschlüsselung des Betriebssystems, aktuelle Sicherheitsupdates, Bildschirmsperre mit kurzer Wartezeit.
- ☐ Bei besonders hoher Bedrohungslage: gehärtetes Betriebssystem (GrapheneOS, Kapitel 12.4) und minimale Zahl zusätzlich installierter Apps in Erwägung ziehen.

- ☐ Verbale Codewörter mit engen Vertrauenspersonen vereinbaren, um sich im Zweifel auch außerhalb von RenkerVault gegenseitig zu verifizieren — eine Vorsichtsmaßnahme, die außerhalb der App liegt, aber angesichts fortschreitender KI-Stimmklonierungstechnik (Kapitel 32.5a: dieselbe technische Entwicklung, die maschinelles Lernen für Metadaten-Analyse und Content-Klassifikation leistungsfähiger macht) zunehmend an Bedeutung gewinnt: Ein Anruf mit vertraut klingender Stimme ist allein kein zuverlässiger Identitätsnachweis mehr.
- ☐ Regelmäßig prüfen, ob die Geräteliste (Kapitel 7.3) weiterhin nur bekannte, selbst genutzte Geräte enthält — gerade bei erhöhtem Schutzbedarf öfter als die für Alltagsnutzung (39.1) empfohlene gelegentliche Prüfung.

39.3 Checkliste: Verwaltung eines eigenen Relay-Servers

- ☐ TLS (wss://) korrekt eingerichtet, entweder über Reverse-Proxy (empfohlen) oder nativ (Kapitel 38.2).
- ☐ systemd-Sandbox-Härtung aus der mitgelieferten Service-Datei übernommen, nicht durch eigene, laxere Konfiguration ersetzt.
- ☐ `TRUST_PROXY` nur gesetzt, wenn tatsächlich ein Reverse-Proxy vorgeschaltet ist (sonst können Client-IP-Header gefälscht werden, Teil III Kapitel 21).
- ☐ Regelmäßige Betriebssystem- und Node.js-Updates eingeplant.
- ☐ Eigene rechtliche Verantwortlichkeiten als Betreiber geprüft (Kapitel 33.3/35.2), insbesondere bei Betrieb für mehr als den rein privaten eigenen Gebrauch.
- ☐ Backup-Strategie für den Fall eines Server-Totalausfalls überlegt (im aktuellen Prototyp-Stand bedeutet ein Neustart des Relay-Prozesses den Verlust nicht zugestellter Nachrichten in der Warteschlange, Kapitel 21.3).

39.4 Checkliste: was RenkerVault NICHT ersetzt

- ☐ Ein separates, umfassendes Konzept für Endgerätesicherheit (Antiviren-/Anti-Malware-Software, Betriebssystem-Updates) — Kapitel 23.3.
- ☐ Vorsicht gegenüber Social-Engineering- und Phishing-Versuchen, die technische Schutzmaßnahmen grundsätzlich umgehen können.
- ☐ Physische Sicherheit des Geräts selbst (ein entsperrtes, unbeaufsichtigtes Gerät kann von jeder anwesenden Person eingesehen werden, unabhängig von jeder Verschlüsselung).
- ☐ Vorsicht bei der Weitergabe von Informationen an Gesprächspartner, die selbst Screenshots erstellen oder Nachrichten weiterleiten könnten (Kapitel 23.3).
- ☐ Absicherung gegen verteilte Angriffe (DDoS) bei Selbst-Hosting eines Relays — dafür wäre ein vorgelagertes CDN/DDoS-Schutzangebot nötig (Kapitel 25.2).

- [] Ein vollständiges externes Sicherheitsaudit der Protokoll-Komposition — die einzelnen Bausteine sind geprüft, die Gesamtkomposition (noch) nicht (Kapitel 25.1a).
- [] Schutz gegen Nebenkanalangriffe bei physischem Zugriff auf spezialisierte Hardware (Kapitel 23.4) — außerhalb des Bedrohungsmodells einer gewöhnlichen Browser-/App-Laufzeitumgebung.

39.5 Vorfall-Reaktionsplan: Was tun bei konkretem Verdacht?

Dieser Abschnitt beschreibt ein strukturiertes Vorgehen für den Fall, dass eines der in Teil II, Kapitel 10 beschriebenen Alarmsignale tatsächlich auftritt.

Fall A: Brute-Force-Alarm (Kapitel 10.2) erscheint, ohne dass man selbst sich mehrfach vertippt hat.

- 1 Ruhe bewahren — die 60-Sekunden-Sperre (Kapitel 10.2) hat bereits automatisch gegriffen, es ist in diesem Moment keine weitere Eingabe möglich oder nötig.
- 2 Nach Ablauf der Sperre: Security-Log (Kapitel 10.6) auf Zeitpunkt und Häufigkeit der Fehlversuche prüfen — stimmen diese mit der eigenen Erinnerung überein, oder deuten sie auf einen fremden Zugriffsversuch hin?
- 3 Geräteliste (Kapitel 7.3) auf unbekannte Einträge prüfen.
- 4 Bei begründetem Verdacht auf einen fremden Zugriffsversuch: Passphrase ändern, sobald der Tresor wieder entsperrt werden kann.

Fall B: **KEY_MISMATCH**-Alarm (Kapitel 10.3) für einen eigenen, bereits bestehenden Kontakt.

- 1 Diesem Kontakt vorerst keine sensiblen Informationen mehr senden, bis der Vorfall geklärt ist.
- 2 Safety Number (Kapitel 7.5) über einen vom Chat komplett unabhängigen Kanal (z. B. persönliches Treffen, Telefonanruf über eine bekannte, unabhängig verifizierte Nummer) neu vergleichen.
- 3 Stimmt die Safety Number nicht mehr überein: Von einem Kompromittierungsversuch (kompromittiertes Gerät des Kontakts, oder ein Angriff auf die Zustellung) ausgehen und den betroffenen Kontakt über einen sicheren Kanal informieren.
- 4 Stimmt die Safety Number überein: Der Alarm deutete vermutlich auf einen legitimen Vorgang hin (z. B. hat der Kontakt sein Gerät gewechselt oder neu eingerichtet) — trotzdem den Vorfall im Security-Log dokumentiert lassen.

Fall C: Manipulationserkennung (Kapitel 10.4) schlägt bei der lokalen Datenbank an.

- 1 Gerät nicht weiterverwenden, bis der Vorfall geklärt ist — ein fehlgeschlagener Integritätscheck deutet auf physischen oder softwareseitigen Zugriff Dritter hin.

- 2 Prüfen, ob das Gerät zwischenzeitlich unbeaufsichtigt oder außerhalb der eigenen Kontrolle war.
- 3 Falls verfügbar: Gerät auf Schadsoftware prüfen (außerhalb des Schutzbereichs von RenkerVault selbst, Kapitel 23.3).
- 4 Datenbestand gilt als potenziell kompromittiert — ein Neuaufbau des Tresors auf einem als sauber bekannten Gerät ist die sicherste Option, sofern die Situation dies zulässt.

39.6a Ein viertes Fallbeispiel: kleines Unternehmen mit Kundenkommunikation

Ein kleines Unternehmen (etwa eine Steuerberatungskanzlei oder eine Arztpraxis) möchte mit Mandanten bzw. Patienten über sensible Angelegenheiten kommunizieren, für die berufsrechtlich eine besondere Vertraulichkeitspflicht gilt. Relevant sind hier mehrere der bereits genannten Bausteine gleichzeitig: ein eigener, professionell betriebener Relay-Server (Kapitel 38.2) mit klarer betrieblicher Verantwortung nach Kapitel 33.3/35.2a, eine dokumentierte interne Richtlinie zur Nutzung (wer darf welche Informationen wie versenden), sowie eine bewusste Abwägung, ob die in Kapitel 25.2 benannten Grenzen (insbesondere die fehlende externe Auditierung) für den konkreten Vertraulichkeitsgrad der jeweiligen Berufsgruppe ausreichend sind — hier ist eine Rücksprache mit der jeweiligen Standesorganisation bzw. rechtliche Beratung (Kapitel 26.1) besonders angeraten, da manche Berufsgruppen eigene, spezifische Anforderungen an die eingesetzte Kommunikationstechnologie stellen.

39.6 Drei anonymisierte Fallbeispiele

Die folgenden drei Fallbeispiele sind zur Veranschaulichung konstruiert (keine realen Personen oder Ereignisse) und zeigen, wie die Checklisten aus 39.1-39.4 in unterschiedlichen Lebenslagen praktisch angewendet werden könnten.

Beispiel 1: Alltägliche Familienkommunikation. Eine Familie nutzt RenkerVault für die alltägliche Absprache (Einkaufslisten, Abholzeiten, private Fotos der Kinder). Relevant sind hier vor allem die Checkliste aus 39.1 (sichere Passphrase, gelegentlicher Blick in die Geräteliste) sowie ein Bewusstsein dafür, dass die stärkste Verschlüsselung nichts nützt, wenn das Familientablet unversperrt herumliegt (39.4) — die höchste Priorität liegt hier auf einfacher, robuster Grundsicherheit, nicht auf den in 39.2 beschriebenen fortgeschrittenen Maßnahmen.

Beispiel 2: Lokaler Vereinsvorstand mit sensiblen Personaldaten. Ein Vereinsvorstand koordiniert über eine RenkerVault-Gruppe Angelegenheiten, die gelegentlich sensible Informationen über Mitglieder enthalten (etwa gesundheitliche Angaben im Zusammenhang mit Veranstaltungsplanung). Hier wird der Betrieb eines eigenen kleinen Relay-Servers (Kapitel 38.2/39.3) relevant, samt der in Kapitel 33.3 beschriebenen datenschutzrechtlichen Verantwortung als Betreiber — der Vorstand

sollte sich vorab bewusst machen, dass er durch den Betrieb eines eigenen Relays selbst zum datenschutzrechtlich Verantwortlichen für die verarbeiteten Metadaten werden kann, auch wenn der Inhalt der Kommunikation durch die Ende-zu-Ende-Verschlüsselung geschützt bleibt.

Beispiel 3: Freie Journalistin mit Quellenschutz-Bedarf. Eine freie Journalistin nutzt RenkerVault für die Kommunikation mit einer Quelle, die anonym bleiben möchte. Hier greift die vollständige Checkliste aus 39.2: eigener, über Tor betriebener Relay (38.4), eingerichtete Duress-PIN (10.5), bewusster Einsatz von „Sitzung verbrennen“ (8.7) nach besonders sensiblen Gesprächen, sowie ergänzende Maßnahmen außerhalb von RenkerVault selbst (vollständige Festplattenverschlüsselung, ein gehärtetes Betriebssystem, siehe 12.4 und 39.4). In diesem Szenario ist zusätzlich der in Kapitel 27.5 beschriebene „Chilling Effect“ besonders relevant: Selbst der bloße Verdacht einer Quelle, überwacht zu werden, kann die Bereitschaft zur Kontaktaufnahme mit einer Journalistin von vornherein verhindern — ein Grund, warum robuste, nachvollziehbare Sicherheit hier keine technische Nebensächlichkeit, sondern eine Voraussetzung für die journalistische Arbeit selbst ist.

39.6b Umgang mit Phishing-Versuchen im Namen von RenkerVault

Ein praktisches, oft unterschätztes Risiko liegt außerhalb der eigentlichen Kryptographie: Betrüger könnten versuchen, sich als „RenkerVault-Support“ oder eine ähnliche vermeintlich offizielle Stelle auszugeben, um Nutzerinnen und Nutzer zur Preisgabe ihrer Passphrase oder zur Installation einer manipulierten, gefälschten Version der App zu bewegen. Da RenkerVault, wie in Kapitel 37.4 erläutert, strukturell **niemals** von einer Nutzerin oder einem Nutzer verlangen würde, die Passphrase außerhalb der eigenen, lokal installierten App einzugeben (es gibt keine „Support-Hotline“, die die Passphrase zur Fehlerbehebung benötigen würde, da selbst der Entwickler keinen technischen Weg hat, mit ihr etwas anzufangen, ohne physischen Zugriff auf das eigene Gerät), ist jede Aufforderung dieser Art ein zuverlässiges Anzeichen für einen Betrugsversuch. Dasselbe gilt für Aufforderungen, eine App-Installationsdatei von einer anderen Quelle als der offiziellen Projekt-Dokumentation oder dem selbst gebauten Quellcode herunterzuladen (Kapitel 37.6 beschreibt, warum eine selbst gebaute APK aus vertrauenswürdiger Quelle unproblematisch, eine App aus unbekannter Drittquelle dagegen ein Risiko ist).

39.7 Eine Anleitung für ein eigenes „Audit-lite“ (für technisch versierte Leserinnen und Leser)

Wer nicht auf ein vollständiges externes Audit (Kapitel 25.1a) warten möchte oder selbst zum Verständnis des Codes beitragen will, kann eine informelle, kleinere Prüfung in Eigenregie durchführen. Folgende Schritte bieten einen strukturierten Einstieg:

- 1 **Abhängigkeiten prüfen:** Mit `npm ls` bzw. einem Werkzeug wie `npm audit` die tatsächlich installierten Versionen von `@noble/curves`, `@noble/curves`, `@noble/curves`, `@noble/curves`, `@noble/curves` und `hash-wasm` gegen die in der Projektdokumentation genannten, geprüften Versionen abgleichen (Kapitel 13.2b).
- 2 **Die Kernmodule lesen:** Die in Kapitel 44 gezeigten Dateien (`primitives.ts`, `pq.ts`, `ratchet.ts`, `vault.ts`, `safety.ts`) sind bewusst kurz gehalten (jeweils unter 250 Zeilen) und lassen sich in wenigen Stunden vollständig durchlesen.
- 3 **Die Spezifikation danebenlegen:** Die öffentliche Signal-Double-Ratchet-Spezifikation (Kapitel 43) neben `ratchet.ts` legen und Formel für Formel vergleichen, ob die Implementierung tatsächlich der beschriebenen Komposition folgt (Kapitel 16.2 liefert dafür bereits eine Gegenüberstellung).
- 4 **Testvektoren nachvollziehen:** Sofern im Projekt-Repository vorhanden, automatisierte Tests ausführen und prüfen, ob sie bekannte Testvektoren der zugrundeliegenden Standards (RFC 7748 für X25519, FIPS 203 für ML-KEM) erfolgreich verifizieren.
- 5 **Eigene Beobachtungen dokumentieren:** Auch ohne einen vollständigen, formalen Auditbericht ist es hilfreich, eigene Beobachtungen (Verständnisfragen, mögliche Unklarheiten) schriftlich festzuhalten und den Entwicklern zurückzumelden — ein informeller, aber wertvoller Beitrag zur in Kapitel 44.6 beschriebenen Transparenz.

Diese Anleitung ersetzt kein professionelles, formales Sicherheitsaudit mit den in Kapitel 25.1a beschriebenen Werkzeugen (ProVerif, Tamarin, professionelles Fuzzing) — sie senkt aber die Einstiegshürde für alle, die sich selbst ein fundiertes Bild machen möchten, statt sich ausschließlich auf die Beschreibung in diesem Buch verlassen zu müssen.

39.8 Checkliste: Migration von einem anderen Messenger

Wer von einem anderen Messenger zu RenkerVault wechselt, sollte einige Besonderheiten beachten, die sich von gewohnten Abläufen unterscheiden:

- [] **Keine automatische Kontaktübernahme:** Anders als bei manchen Diensten (Kapitel 7.4) gibt es keinen automatischen Adressbuch-Abgleich — jeder Kontakt muss einzeln über die Konto-Kennung hinzugefügt werden, was zwar etwas mehr Aufwand bedeutet, aber die in Kapitel 7.4 beschriebene Datensparsamkeit erst ermöglicht.
- [] **Kein Chatverlauf-Import:** Nachrichten aus einem anderen Dienst lassen sich aus kryptographischen Gründen nicht automatisch importieren — jede Konversation beginnt mit einem frischen Handshake (Kapitel 15).
- [] **Kontakte über den neuen Kanal informieren:** Da es keine Telefonnummer-basierte automatische Erkennung gibt (Kapitel 7.1), müssen bestehende Kontakte aktiv über die neue Konto-Kennung informiert werden —

idealerweise über einen vom neuen Dienst unabhängigen Kanal, um gleichzeitig einen ersten informellen Abgleich zur späteren Safety-Number-Verifikation zu ermöglichen (Kapitel 7.5).

- [] **Übergangsphase einplanen:** Es ist üblich und unproblematisch, den bisherigen Messenger parallel weiterzunutzen, bis alle relevanten Kontakte auf den neuen Kanal umgestiegen sind.

40. Glossar

Advanced Encryption Standard (AES) — Seit 2001 von NIST genormtes symmetrisches Verschlüsselungsverfahren, weltweiter Industriestandard; RenkerVault nutzt die 256-Bit-Variante im GCM-Betriebsmodus (Teil III, Kapitel 13.1).

Argon2id — Passwort-Hashing-Verfahren, Gewinner der Password Hashing Competition 2015; absichtlich speicher- und rechenintensiv konstruiert, um Brute-Force-Angriffe unpraktikabel zu verteuern (Teil II, Kapitel 11.2).

Asymmetrische Kryptographie — Verschlüsselungsprinzip mit zwei zusammengehörigen Schlüsseln (öffentlich und privat), das den sicheren Schlüsselaustausch ohne persönliches Treffen ermöglicht (Teil I, Kapitel 5.2).

Aufdeckungsanordnung (Detection Order) — Im Kommissionsvorschlag zur Chat-Kontrolle-Verordnung vorgesehene behördliche Anordnung, die einen Anbieter zur systematischen Suche nach bestimmten Inhaltskategorien verpflichten würde (Teil IV, Kapitel 29.1).

Brute-Force-Angriff — Systematisches Durchprobieren aller möglichen Schlüssel oder Passwörter, bis der richtige gefunden ist; wird durch ausreichend lange Schlüssel und rechenintensives Passwort-Hashing praktisch unmöglich gemacht.

Capacitor — Framework, das eine bestehende Web-Anwendung unverändert in eine native Android- oder iOS-App einbettet, ohne die zugrundeliegende Logik neu zu implementieren (Teil II, Kapitel 12.3).

Chain Key — Zwischengröße im Double-Ratchet-Algorithmus, aus der bei jeder Nachricht ein neuer Message Key und ein neuer Chain Key abgeleitet werden (Teil III, Kapitel 16.2).

Chat-Kontrolle — Umgangssprachliche Bezeichnung für den Vorschlag einer EU-Verordnung, die eine verpflichtende Analyse von Chat-Inhalten zur Bekämpfung sexuellen Kindesmissbrauchs vorsieht, einschließlich Ende-zu-Ende-verschlüsselter Dienste (Teil IV, Kapitel 28).

Client-Side-Scanning — Analyse von Inhalten direkt auf dem Endgerät, bevor sie verschlüsselt bzw. nachdem sie entschlüsselt wurden — die einzige technisch mögliche

Form der Inhaltsprüfung bei tatsächlich Ende-zu-Ende-verschlüsselten Diensten (Teil IV, Kapitel 30).

Crypto Wars — Sammelbegriff für wiederkehrende politische Auseinandersetzungen um staatlichen Zugriff auf Verschlüsselungstechnologie seit den 1990er-Jahren (Teil I, Kapitel 4).

CSPRNG — Kryptographisch sicherer Pseudozufallsgenerator; erzeugt Zufallswerte, die für kryptographische Zwecke (Schlüsselerzeugung, Nonces) geeignet sind, im Gegensatz zu einfachen, nicht-kryptographischen Zufallsgeneratoren (Teil III, Kapitel 19.2).

Detection Order — Siehe Aufdeckungsanordnung.

DSGVO (Datenschutz-Grundverordnung) — Seit 2018 unmittelbar geltendes EU-Recht zum Schutz personenbezogener Daten (Teil IV, Kapitel 33).

Double Ratchet — Von Signal entwickeltes Protokoll, das bei jeder Nachricht automatisch neue Schlüssel erzeugt und dadurch Perfect Forward Secrecy sowie Post-Compromise Security realisiert (Teil III, Kapitel 16).

Duress-PIN — Separate, geheime PIN, die im Zwangsfall eine leere Fake-Ansicht statt des echten Datenbestands öffnet (Teil II, Kapitel 10.5).

Ed25519 — Elliptische-Kurven-Verfahren speziell für digitale Signaturen, in RenkerVault für die passwortlose Authentifizierung gegenüber dem Relay verwendet (Teil III, Kapitel 14.3).

Einwegfunktion — Mathematische Operation, die in eine Richtung leicht, in die Gegenrichtung ohne geheimen Zusatzwert praktisch nicht durchführbar ist — Grundlage asymmetrischer Kryptographie (Teil I, Kapitel 5.2).

Ende-zu-Ende-Verschlüsselung (E2E) — Nur Sender und Empfänger können den Inhalt einer Nachricht lesen, keine Zwischenstation, auch nicht der Serverbetreiber.

Ephemeral Key — Frisch erzeugter, nur für einen einzigen Vorgang gültiger Schlüssel, der nach Gebrauch verworfen wird — zentraler Baustein von Perfect Forward Secrecy (Teil III, Kapitel 15.2).

ePrivacy-Richtlinie — EU-Richtlinie von 2002 zum Schutz der Vertraulichkeit elektronischer Kommunikation, Grundlage für die Notwendigkeit einer Ausnahmeregelung für freiwilliges Scannen (Teil IV, Kapitel 34).

EuGH (Europäischer Gerichtshof) — Oberstes Gericht der Europäischen Union für Fragen des EU-Rechts, u. a. zuständig für die Auslegung der Grundrechtecharta (Teil IV, Kapitel 27.3).

Function Creep (Zweckerweiterung) — Phänomen, bei dem eine für einen engen Zweck geschaffene Infrastruktur im Zeitverlauf auf zusätzliche Zwecke ausgeweitet wird (Teil IV, Kapitel 30.3).

GRCh (Grundrechtecharta der Europäischen Union) — Seit 2009 rechtsverbindlicher Katalog der Grundrechte in der EU, enthält u. a. das Recht auf Achtung des Privatlebens (Artikel 7) und auf Datenschutz (Artikel 8) (Teil IV, Kapitel 27.1).

Hash-Funktion — Rechenverfahren, das aus beliebig langer Eingabe einen kurzen, festen „Fingerabdruck“ erzeugt; kleinste Änderung der Eingabe erzeugt komplett anderen Fingerabdruck (Teil I, Kapitel 5.5).

Harvest Now, Decrypt Later (HNDL) — Angriffsstrategie: verschlüsselten Datenverkehr heute aufzeichnen und erst entschlüsseln, sobald ein ausreichend starker Quantencomputer verfügbar ist (Teil III, Kapitel 18.2).

HKDF — HMAC-based Key Derivation Function; leitet aus einem Geheimnis mehrere kryptographisch unabhängige Schlüssel ab (Teil III, Kapitel 15.4).

HMAC — Keyed-Hash Message Authentication Code; beweist Integrität und Authentizität von Daten mittels eines geheimen Schlüssels (Teil I, Kapitel 5.5).

Hybrid-Verschlüsselung — Kombination aus asymmetrischer Kryptographie (für den Schlüsselaustausch) und symmetrischer Kryptographie (für die eigentliche Datenverschlüsselung); Industriestandard, u. a. bei HTTPS und Signal (Teil I, Kapitel 5.4).

Key Escrow — Konzept, bei dem eine dritte Stelle (etwa eine Regierungsbehörde) im Bedarfsfall auf Verschlüsselungsschlüssel zugreifen kann; zentrales, gescheitertes Element des Clipper-Chip-Vorschlags der 1990er-Jahre (Teil I, Kapitel 4.1).

Man-in-the-Middle-Angriff — Angriff, bei dem sich ein Dritter unbemerkt zwischen zwei kommunizierende Parteien schaltet; wird durch den Vergleich von Safety Numbers ausgeschlossen (Teil II, Kapitel 7.5).

Metadaten — Informationen über eine Kommunikation (wer, mit wem, wann, wie oft), ohne den eigentlichen Inhalt zu umfassen; nicht automatisch durch Ende-zu-Ende-Verschlüsselung geschützt (Teil III, Kapitel 22.1).

ML-KEM-768 — Vormalig „Kyber“ genannt, seit 2024 als FIPS 203 genormtes Post-Quantum-Verfahren zur Schlüsseleinigung, basierend auf dem Module-LWE-Problem (Teil III, Kapitel 18.3).

Nonce — Ein für jede Verschlüsselung mit demselben Schlüssel garantiert einmaliger Wert; Wiederverwendung bricht die Sicherheit von AES-GCM (Teil III, Kapitel 13.2).

One-Time-Pad (OTP) — Theoretisch perfekt sicheres Verschlüsselungsverfahren, das für jede Nachricht einen gleich langen, echt zufälligen und nur einmal verwendeten Schlüssel benötigt; in der Praxis wegen des Schlüsselaustausch-Aufwands kaum handhabbar (Teil III, Kapitel 19.1).

Perfect Forward Secrecy (PFS) — Eigenschaft, bei der ein kompromittierter Schlüssel nicht die Vertraulichkeit vergangener Nachrichten gefährdet, da für jede Nachricht ein eigener, danach verworfener Schlüssel verwendet wird (Teil I, Kapitel 5.7).

Post-Compromise Security — Eigenschaft, bei der ein Protokoll nach einer Kompromittierung durch neue Schlüsselvereinbarung von selbst wieder Sicherheit herstellt (Teil III, Kapitel 16.3).

Post-Quantum-Kryptographie — Verschlüsselungsverfahren, die auch gegen ausreichend große Quantencomputer als sicher gelten (Teil III, Kapitel 18).

Prekey — Im Voraus veröffentlichter öffentlicher Schlüssel, der einen verschlüsselten Erstkontakt ohne gleichzeitige Online-Anwesenheit der Gegenseite ermöglicht (Teil III, Kapitel 15.2).

Root Key — Ausgangsgröße des Double Ratchet, aus der bei jedem Diffie-Hellman-Ratchet-Schritt ein neuer Root Key und Chain Key abgeleitet werden (Teil III, Kapitel 16.3).

Safety Number — 60-stellige Zahlenfolge (auch als QR-Code darstellbar), die zwei Gesprächspartner über einen unabhängigen Kanal vergleichen können, um einen Man-in-the-Middle-Angriff auszuschließen (Teil II, Kapitel 7.5).

Sender Keys — Effizientes Gruppen-Verschlüsselungsverfahren mit einem gemeinsamen, bei Mitgliederänderungen rotierenden Schlüssel, statt eines Double Ratchet pro Mitgliederpaar (Teil III, Kapitel 17).

Shor-Algorithmus — Quantenalgorithmus, der die Faktorisierung großer Zahlen und den diskreten Logarithmus auf elliptischen Kurven in praktikabler Zeit lösen könnte — die theoretische Grundlage der Post-Quantum-Bedrohung (Teil III, Kapitel 18.1).

Signed Prekey — Siehe Prekey; im Kontext von X3DH-lite der wiederverwendbare, vom Responder veröffentlichte Schlüssel (Teil III, Kapitel 15.2).

Symmetrische Kryptographie — Verschlüsselungsprinzip mit einem einzigen, gemeinsamen Schlüssel für Ver- und Entschlüsselung; schneller als asymmetrische Verfahren, erfordert aber einen bereits etablierten gemeinsamen Schlüssel (Teil I, Kapitel 5.4).

Tauri — Framework für native Desktop-Apps mit Rust-Backend und schlankem, systemeigenem WebView statt gebündeltem Chromium wie bei Electron (Teil II, Kapitel 12.2).

Threat Model (Bedrohungsmodell) — Explizite Benennung, gegen welche Angreifer und Szenarien ein Sicherheitswerkzeug schützt — und gegen welche nicht (Teil III, Kapitel 23).

Tor / Hidden Service — Anonymisierungsnetzwerk, das Verbindungen über mehrere Relais verschlüsselt leitet; ein Hidden Service macht einen Dienst nur über eine „onion“-Adresse erreichbar, ohne öffentliche IP oder Domain (Teil III, Kapitel 22.3).

WebView — Eingebetteter Browser-Baustein, mit dem native Apps Webinhalte anzeigen — Basis sowohl von Tauri als auch von Capacitor (Teil II, Kapitel 12).

X25519 — Elliptische-Kurven-Verfahren für Diffie-Hellman-Schlüsselaustausch, Grundlage der Ende-zu-Ende-Verschlüsselung in RenkerVault (Teil III, Kapitel 14.2).

X3DH / X3DH-lite — Extended Triple Diffie-Hellman; initiales Schlüsselaustauschverfahren, das asynchrone Erstkontakte ohne gleichzeitige Online-Anwesenheit beider Seiten ermöglicht (Teil III, Kapitel 15).

Zero-Knowledge-Architektur — Systemdesign, bei dem eine zentrale Stelle (hier: der Relay-Server) strukturell keinen Zugriff auf den eigentlichen Kommunikationsinhalt hat, unabhängig vom Verhalten oder den Absichten des Betreibers (Teil I, Kapitel 2).

AEAD (Authenticated Encryption with Associated Data) — Klasse von Verschlüsselungsverfahren, die gleichzeitig Vertraulichkeit und Manipulationserkennung bieten, z. B. AES-GCM oder ChaCha20-Poly1305 (Teil III, Kapitel 13.2a).

Chilling Effect (Abschreckende Wirkung) — Sozialwissenschaftlich belegtes Phänomen, bei dem bereits das Wissen um mögliche Überwachung das Verhalten von Menschen verändert, unabhängig davon, ob tatsächlich überwacht wird (Teil IV, Kapitel 27.5).

Digital Services Act (DSA) — Seit 2024 EU-weit anwendbare Verordnung, die Pflichten und Haftungsfragen von Vermittlungsdiensten (u. a. Hosting-Anbietern) regelt (Teil IV, Kapitel 35.2a).

Function Creep — Siehe Zweckerweiterung.

Ordentliches Gesetzgebungsverfahren — Der reguläre Ablauf, nach dem die meisten EU-Verordnungen zustande kommen: Kommissionsvorschlag, Positionen von Parlament und Rat, Trilog-Verhandlung, formelle Annahme (Teil IV, Kapitel 28.0).

Privacy by Design / by Default — In Artikel 25 DSGVO verankertes Prinzip, wonach Datenschutz bereits bei der Systementwicklung und in den Voreinstellungen berücksichtigt werden muss, nicht erst nachträglich (Teil IV, Kapitel 33.1b).

Qualifizierte Mehrheit — Abstimmungsregel im Rat der Europäischen Union, die eine Mehrheit der Mitgliedstaaten UND eine Mehrheit der EU-Gesamtbevölkerung erfordert

(Teil IV, Kapitel 28.0).

Sealed Sender — Technik, bei der selbst die Absenderidentität einer Nachricht dem Relay-Server gegenüber verborgen bleibt; in RenkerVault aktuell nicht implementiert (Teil III, Kapitel 22.4).

Trilog — Informelle Verhandlung zwischen Vertretern der EU-Kommission, des Parlaments und des Rates zur Findung eines Gesetzeskompromisses (Teil IV, Kapitel 28.0).

Zweckerweiterung (Function Creep) — Phänomen, bei dem eine für einen engen Zweck geschaffene Infrastruktur im Zeitverlauf auf zusätzliche Zwecke ausgeweitet wird (Teil IV, Kapitel 30.3).

Cover Traffic — Künstlicher, bedeutungsloser Datenverkehr, der reale Kommunikationsmuster vor einem Beobachter verschleiert; in RenkerVault aktuell nicht implementiert (Teil III, Kapitel 22.4a).

IT-Grundrecht — Vom deutschen Bundesverfassungsgericht 2008 anerkanntes Grundrecht auf Gewährleistung der Vertraulichkeit und Integrität informationstechnischer Systeme (Teil IV, Kapitel 27.5a).

IDS (Intrusion Detection System) — Aus der klassischen Unternehmens-IT-Sicherheit stammende Kategorie von Werkzeugen, die verdächtige Aktivität erkennen und melden; Vorbild für RenkerVaults Einbruchsalarm-System (Teil II, Kapitel 10.6b).

ML-DSA (vormals Dilithium) — Post-Quantum-Signaturverfahren, seit 2024 als FIPS 204 genormt; in RenkerVault aktuell nicht eingesetzt (Teil III, Kapitel 18.6c).

IND-CPA / IND-CCA — Formale kryptographische Sicherheitsbegriffe, die beschreiben, ob ein Verschlüsselungsverfahren selbst unter aktiven Angriffsversuchen keinerlei Information über den Klartext preisgibt (Teil III, Kapitel 15.5b).

DPIA (Datenschutz-Folgenabschätzung) — Nach Art. 35 DSGVO vorgeschriebene, dokumentierte Risikoanalyse für Verarbeitungen mit voraussichtlich hohem Risiko für betroffene Personen (Teil IV, Kapitel 33.3a).

LIBE-Ausschuss — Ausschuss des Europäischen Parlaments für bürgerliche Freiheiten, Justiz und Inneres, zuständig für die Beratung der Chat-Kontrolle-Verordnung (Teil IV, Kapitel 28.2b).

KYC (Know Your Customer) — Im Finanzsektor gesetzlich vorgeschriebene Identifizierungspflicht bei Kontoeröffnung, zu unterscheiden von der Klarnamenlosigkeit bei Kommunikationsdiensten (Teil II, Kapitel 7.1a).

Nebenkanalangriff (Side-Channel Attack) — Angriff, der nicht die Mathematik eines Verfahrens angreift, sondern physikalische Nebeneffekte seiner Berechnung ausnutzt,

etwa Zeitverhalten oder Stromverbrauch (Teil III, Kapitel 23.4).

eIDAS 2.0 / EU Digital Identity Wallet — Geplante EU-weite digitale Brieftasche für Identitätsnachweise, mit möglichem Berührungspunkt zur Altersverifikationsdebatte (Teil IV, Kapitel 32.5b).

Replay-Angriff — Angriff, bei dem eine zuvor beobachtete, gültige Nachricht (z. B. eine Signatur) erneut eingespielt wird; durch frische, einmalige Nonces verhindert (Teil III, Kapitel 20.3).

Blockchiffre — Verschlüsselungsverfahren, das jeweils einen Datenblock fester Größe verarbeitet (bei AES 128 Bit); benötigt eine zusätzliche Betriebsart, um längere Nachrichten zu verschlüsseln (Teil III, Kapitel 13.0a).

Betriebsart (Mode of Operation) — Verfahren, das festlegt, wie eine Blockchiffre auf Nachrichten angewendet wird, die länger als ein einzelner Block sind, z. B. GCM oder CBC (Teil III, Kapitel 13.0a).

Präventive vs. detektive Sicherheitsmaßnahme — Unterscheidung zwischen Maßnahmen, die einen Angriff von vornherein verhindern (präventiv, z. B. Verschlüsselung), und solchen, die einen Angriff erkennen und melden (detektiv, z. B. das Einbruchsalarm-System) (Teil II, Kapitel 10.1a).

Lex specialis — Rechtsgrundsatz, wonach ein spezielleres Gesetz einem allgemeineren Gesetz im Kollisionsfall vorgeht; erklärt das Verhältnis von ePrivacy-Richtlinie zu DSGVO (Teil IV, Kapitel 34.0a).

Dual-Use — Eigenschaft eines Werkzeugs oder einer Technologie, sowohl für legale als auch für illegale Zwecke nutzbar zu sein, ohne dass dies die Rechtmäßigkeit des Werkzeugs selbst berührt (Teil IV, Kapitel 35.3a).

Föderation — Architekturprinzip, bei dem viele unabhängig betriebene Server nach einem gemeinsamen Protokoll miteinander kommunizieren, vergleichbar mit E-Mail (Teil III, Kapitel 22.4b, 24.5a).

EPRS (European Parliamentary Research Service) — Interner wissenschaftlicher Dienst des Europäischen Parlaments, der Abgeordnete mit Hintergrundanalysen zu Gesetzesvorhaben versorgt (Teil IV, Kapitel 31.2/31.2a).

Präventive Kontrolle vs. detektive Kontrolle — Siehe präventive vs. detektive Sicherheitsmaßnahme.

Envelope Encryption (Umschlag-Verschlüsselung) — Zweistufiges Schlüsselschema, bei dem ein Schlüssel einen zweiten, eigentlichen Datenschlüssel verschlüsselt, um Schlüsseländerungen ohne Neuverschlüsselung des gesamten Datenbestands zu ermöglichen (Teil II, Kapitel 11.1a).

Generaldirektion (DG) — Fachabteilung innerhalb der Europäischen Kommission, zuständig für die Vorbereitung von Gesetzesvorschlägen in einem bestimmten Politikbereich (Teil IV, Kapitel 28.2a).

41. Häufig gestellte Fragen

Warum verwendet RenkerVault TypeScript und nicht eine „schnellere“ Sprache wie Rust oder C++? Für die Kryptographie-Schicht (Kapitel 6.1) bietet TypeScript, wie in Kapitel 13.3a beschrieben, wichtige Speichersicherheitsvorteile gegenüber C/C++, während die tatsächlich rechenintensiven Operationen (AES-GCM) über WebCrypto (Kapitel 20.4a) ohnehin in nativem, hochoptimiertem Code laufen, nicht in JavaScript selbst. Der praktische Geschwindigkeitsunterschied für eine Chat-Anwendung (im Gegensatz etwa zu Videospielen oder wissenschaftlichen Simulationen) ist in der Praxis nicht spürbar, während der Sicherheits- und Portabilitätsvorteil (dieselbe Codebasis für Web/Desktop/Mobile, Kapitel 12) erheblich ist.

Was passiert mit meinem Konto, wenn RenkerVault als Projekt eingestellt würde?

Da jedes Gerät seine eigenen Schlüssel lokal erzeugt und verwaltet (Kapitel 7.1) und der Quellcode offen dokumentiert ist (Kapitel 44), bliebe die App auch ohne aktive Weiterentwicklung technisch nutzbar, solange ein Relay-Server (egal von wem betrieben, Kapitel 38) verfügbar ist — anders als bei einem rein zentralisierten, closed-source Dienst (Kapitel 22.4b), bei dem eine Einstellung des Dienstes durch den Betreiber gleichbedeutend mit dem Ende der Nutzbarkeit wäre.

Ist es sicher, RenkerVault auf einem Gerät zu nutzen, das auch für andere Zwecke (Arbeit, Familie) verwendet wird? Das hängt vom Bedrohungsmodell ab (Kapitel 23): RenkerVaults eigene At-Rest-Verschlüsselung (Kapitel 11) schützt den Datenbestand vor anderen Apps auf demselben Gerät. Nicht geschützt ist RenkerVault jedoch vor Schadsoftware, die bereits mit denselben oder höheren Rechten auf demselben Gerät läuft (Kapitel 23.3) — bei sehr hohem Schutzbedarf ist ein dediziertes, nur für sensible Kommunikation genutztes Gerät (Kapitel 39.2) die sicherere Wahl.

Ist RenkerVault sicherer als Signal? Nicht pauschal — Signal wird regelmäßig extern auditiert und verwendet eine produktionsreife, geprüfte Implementierung ([libsignal](#)), während RenkerVaults Protokoll-Komposition (Teil III, Kapitel 16.5) das noch nicht durchlaufen hat. RenkerVault ergänzt dafür ein eigenes Einbruchsalarm-System und optional einen Tor-Hosting-Weg. Wer die höchste, am gründlichsten geprüfte Sicherheit für alltägliche Kommunikation sucht, ist mit Signal aktuell besser bedient; RenkerVault richtet sich eher an Nutzerinnen und Nutzer, die zusätzlich Wert auf das Alarmsystem, Selbst-Hosting-Möglichkeiten oder Interesse an einer offen dokumentierten, nachvollziehbaren Architektur legen.

Kann ich RenkerVault nutzen, ohne einen eigenen Server zu betreiben? Ja — jeder von einer vertrauenswürdigen Stelle betriebene Relay-Server funktioniert, solange alle Gesprächspartner dieselbe Adresse in den Einstellungen eintragen (Kapitel 38.5). Der Betreiber dieses Relays sieht dabei ausschließlich Metadaten, niemals Klartext (Teil III, Kapitel 21).

Was passiert, wenn ich mein Gerät verliere? Ohne die Passphrase (die nirgendwo serverseitig gespeichert ist, Kapitel 20.2) bleibt der lokale Datenbestand verschlüsselt und für einen Finder oder Dieb unlesbar. Ein zweites, bereits vertrautes Gerät kann das verlorene Gerät über die Geräteverwaltung remote abmelden (Teil II, Kapitel 7.3), sodass es keinen Zugriff mehr auf zukünftige Nachrichten erhält.

Ist ein selbst gehosteter Relay-Server rechtlich riskant? Das hängt vom Umfang der Nutzung und der jeweiligen Rechtsordnung ab (Kapitel 33.3/35.2) — für den rein privaten, persönlichen Gebrauch im Familien- und Freundeskreis ist das Risiko nach aktuellem Kenntnisstand gering, für einen darüberhinausgehenden, öffentlich beworbenen Betrieb ist rechtliche Beratung ratsam (siehe den Disclaimer in Kapitel 26.1).

Warum gibt es keine „Passwort vergessen“-Funktion? Weil eine solche Funktion zwangsläufig bedeuten würde, dass eine dritte Stelle (der Relay-Betreiber oder ein Backend-Dienst) in der Lage sein müsste, den verschlüsselten lokalen Datenbestand ohne die Passphrase zu entsperren — das würde die in Kapitel 21 beschriebene Zero-Knowledge-Eigenschaft strukturell durchbrechen (Kapitel 37.4).

Schützt mich RenkerVault vor der Chat-Kontrolle-Verordnung, falls sie beschlossen wird? Das hängt von der konkreten, letztlich beschlossenen Fassung ab (Kapitel 28.4/32) — dieses Buch kann und will diese Frage nicht abschließend beantworten (Kapitel 26.2). RenkerVaults Architektur macht einen nachträglichen Einbau eines Scanning-Mechanismus technisch schwerer nachrüstbar als bei zentralisierten Diensten (Kapitel 32.3), was aber keine rechtliche Aussage über die Anwendbarkeit einer zukünftigen Verpflichtung ersetzt.

Kann der Relay-Betreiber sehen, WEN ich kontaktiere, auch ohne den Inhalt zu sehen? Ja, grundsätzlich (Teil III, Kapitel 21.1/22.1) — es sei denn, zusätzliche Maßnahmen wie ein Tor Hidden Service (Kapitel 22.3) werden ergriffen, die zumindest die IP-Ebene verschleiern. Vollständiger Metadatenchutz (z. B. Sealed Sender) ist im aktuellen Prototyp nicht implementiert (Kapitel 25.2).

Ist RenkerVault Open Source? Die Architektur und sämtliche kryptographischen Design-Entscheidungen sind in diesem Buch sowie in den Projektdokumenten SECURITY.md und README.md offen dokumentiert; die konkrete Lizenzierung des Quellcodes selbst richtet sich nach den Angaben im jeweiligen Projekt-Repository.

Warum gibt es keine Push-Benachrichtigungen wie bei anderen Messengern? Weil die gängigen Push-Dienste (z. B. Firebase Cloud Messaging) strukturell ein zusätzliches Metadaten-Leck an den jeweiligen Betriebssystemhersteller darstellen würden — ein bewusster, in Kapitel 12.4a erläuteter Kompromiss zugunsten der Privatsphäre, der mit etwas höherem Akkuverbrauch erkaufte wird.

Was passiert mit meinen Daten, wenn ich RenkerVault deinstalliere? Der lokal verschlüsselte Datenbestand verbleibt, sofern nicht gesondert gelöscht, als verschlüsselte Datei auf dem Gerät bzw. wird beim Entfernen der App durch das Betriebssystem mitgelöscht (abhängig vom jeweiligen Betriebssystemverhalten) — der Relay-Server speichert dabei ohnehin nur Metadaten und ggf. kurzzeitig nicht zugestellte, weiterhin verschlüsselte Nachrichten (Kapitel 21.3), niemals Klartext.

Kann ich RenkerVault auf mehreren Geräten gleichzeitig nutzen, ohne dass ein Gerät als „führend“ gilt? Ja — das in Kapitel 7.3 beschriebene Mehrgeräte-Modell sieht keine Hierarchie zwischen Geräten vor; jedes bestätigte, vertrauenswürdige Gerät kann unabhängig senden und empfangen.

Ist eine automatische Cloud-Sicherung meines Gerätebackups sicher für meine RenkerVault-Daten? Das hängt vom jeweiligen Backup-Mechanismus des Betriebssystems ab — siehe die ausführliche Erläuterung in Kapitel 12.4b, die empfiehlt, dies aktiv zu prüfen, statt es unreflektiert vorauszusetzen.

Was ist der Unterschied zwischen „Sitzung verbrennen“ und dem einfachen Löschen eines Chats? Ein einfaches Löschen entfernt in vielen Chat-Apps nur die lokale Anzeige, während die Verschlüsselungssitzung selbst bestehen bleibt. „Sitzung verbrennen“ (Kapitel 8.7) geht weiter: Bei echten Kontakten wird zusätzlich der Ratchet-Zustand (Kapitel 16) und der Kontakt selbst gelöscht — ein späterer Kontakt erfordert zwingend einen kompletten neuen Handshake (Kapitel 15).

Warum dauert das Entsperren des Tresors spürbar länger als bei anderen Apps? Das ist beabsichtigt — die in Kapitel 11.2/11.2a beschriebene Argon2id-Berechnung ist absichtlich rechen- und speicherintensiv, um Brute-Force-Angriffe auf ein gestohlenes, verschlüsseltes Backup drastisch zu verteuern. Die spürbare, aber kurze Wartezeit beim Entsperren ist der Preis für diesen Schutz.

Kann RenkerVault von der EU-Chat-Kontrolle-Verordnung selbst betroffen sein, falls sie in Kraft tritt? Das hängt von der finalen Fassung ab (Kapitel 28), die zum jetzigen Zeitpunkt nicht feststeht (Kapitel 26.2) — eine pauschale Aussage wäre unseriös. Kapitel 35.3 ordnet ein, warum RenkerVault kein gezieltes Umgehungswerkzeug gegen ein bestehendes Gesetz ist, sondern eine Architektur-Entscheidung für den Fall einer künftigen Verpflichtung.

Warum wird in diesem Buch so oft auf SECURITY.md verwiesen? Weil SECURITY.md im Projekt-Repository laufend mit dem tatsächlichen Code-Stand

aktualisiert wird, während dieses Buch einen Zeitpunkt-Schnappschuss darstellt (Kapitel 26.2) — im Zweifel gilt SECURITY.md als die aktuellere Quelle.

Ist der Quellcode der @noble-Bibliotheken selbst quelloffen einsehbar? Ja — alle in Kapitel 13.3/13.3b genannten Bibliotheken sind öffentlich einsehbarer Open-Source-Code, was Voraussetzung für die dort beschriebenen unabhängigen Audits und den in Kapitel 39.7 beschriebenen Selbst-Audit-Ansatz ist.

Nutzt RenkerVault WebRTC für Anrufe, und leckt das meine IP-Adresse? Nein — wie in Kapitel 22.3a erläutert, verwendet RenkerVault für Sprachnachrichten bewusst aufgezeichnete, Ende-zu-Ende-verschlüsselte Dateien statt Echtzeit-WebRTC-Anrufe, wodurch das bekannte WebRTC-IP-Leck-Risiko strukturell gar nicht erst entsteht.

Kann ich RenkerVault gleichzeitig mit einem VPN nutzen? Ja, ein VPN und RenkerVault schließen sich nicht aus — ein VPN würde zusätzlich die Verbindung zum Relay-Server über einen weiteren, vom eigentlichen Internetanbieter unabhängigen Vermittlungspunkt leiten, ähnlich im Grundprinzip (wenn auch mit geringerer Anonymitätsgarantie) wie der in Kapitel 22.3 beschriebene Tor Hidden Service. Ein VPN ersetzt nicht die in diesem Buch beschriebene Ende-zu-Ende-Verschlüsselung, sondern ergänzt sie auf der Netzwerkebene.

Was ist, wenn ich sowohl RenkerVault als auch einen anderen Messenger für dieselbe Konversation nutze? Das ist technisch problemlos möglich, verwässert aber das Schutzversprechen: Eine Konversation ist immer nur so vertraulich wie ihr am wenigsten geschütztes Übertragungsglied — nutzt eine Gesprächspartnerin dieselben Informationen zusätzlich über einen weniger geschützten Kanal, gilt für diesen Teil der Kommunikation das dortige, potenziell geringere Schutzniveau.

Was passiert, wenn zwei Personen gleichzeitig versuchen, sich gegenseitig zum ersten Mal zu kontaktieren? Beide Seiten würden unabhängig voneinander einen Handshake als Initiator einleiten (Kapitel 15.2). Da beide Sitzungen auf denselben zugrundeliegenden öffentlichen Schlüsseln beruhen, aber unterschiedliche Ephemeral Keys verwenden, entstehen zwei parallele, kryptographisch unterschiedliche Sitzungen — die Anwendung müsste diesen Randfall durch eine Konvention auflösen (etwa: die Sitzung mit dem lexikographisch kleineren Ephemeral-Key-Wert gewinnt), ein in der Praxis seltenes, aber lösbares Randfall-Szenario asynchroner Handshake-Protokolle.

Werden gelöschte Nachrichten wirklich unwiederbringlich entfernt? Auf Anwendungsebene ja (Kapitel 8.1, 8.7) — eine wichtige technische Einschränkung gilt jedoch für die zugrundeliegende Speichertechnologie: Wie bei den meisten Systemen kann das Betriebssystem oder die Speicherhardware selbst gelöschte Daten unter Umständen noch kurzzeitig physisch vorhalten, bis der Speicherbereich überschrieben wird — ein Umstand, der außerhalb der Kontrolle der Anwendungsebene liegt und bei sehr hohem Schutzbedarf zusätzliche Maßnahmen (volle Festplattenverschlüsselung, Kapitel 39.2) nahelegt.

Kann ein Gerichtsurteil RenkerVault dazu zwingen, eine Hintertür einzubauen? Ein Gericht kann grundsätzlich nur die tatsächlich existierenden, konkret Beteiligten zu einer Handlung verpflichten — bei einem selbst gehosteten, offen dokumentierten Open-Source-Projekt ohne zentrales Unternehmen dahinter (anders als etwa bei Apple im Fall San Bernardino, Kapitel 4.3) fehlt ein einzelner, zur Entwicklung einer Hintertür verpflichtbarer Akteur in vergleichbarer Form. Diese Frage berührt jedoch letztlich ungeklärtes rechtliches Neuland (Kapitel 35, Kapitel 26.1) und ist keine, die dieses Buch abschließend beantworten kann.

Warum verwendet dieses Buch durchgehend „Anna“ und „Ben“ als Beispielpersonen? Diese Namen folgen einer in der Kryptographie-Fachliteratur seit Jahrzehnten etablierten Konvention (oft ergänzt um „Chris“ für einen Angreifer, wie in Kapitel 14.4b verwendet) — sie erleichtern es, komplexe Protokollabläufe mit konkreten, unterscheidbaren Rollen statt abstrakter Buchstaben wie „A“ und „B“ zu beschreiben, ohne reale Personen zu meinen.

42. Zeitleiste: Von den Crypto Wars zur Chat-Kontrolle

Jahr	Ereignis
1991	Erste Veröffentlichung von PGP (Pretty Good Privacy) durch Phil Zimmermann
1993	Vorstellung des Clipper Chip durch die US-Regierung (Teil I, Kapitel 4.1)
1996	Weitgehendes Scheitern des Clipper-Chip-Vorschlags nach öffentlicher Kritik
2001	AES wird von NIST als neuer Verschlüsselungsstandard genormt
2005	Vorstellung von Curve25519 durch Daniel J. Bernstein
2009	Vertrag von Lissabon: EU-Grundrechtecharta wird rechtsverbindlich
2013	Snowden-Enthüllungen zu globaler Massenueberwachung
2014	EuGH-Urteil "Digital Rights Ireland": Vorratsdatenspeicherung fuer unguelteig erklart
2016	Apple vs. FBI: Streit um Entsperrung eines iPhones (Teil I, Kapitel 4.3)
2018	DSGVO tritt EU-weit in Kraft
2021	Inkrafttreten der freiwilligen "Chat-Kontrolle 1.0"-Interim-Regelung; Verfassen von "Bugs in our Pockets"
Mai 2022	Vorschlag der EU-Kommission fuer eine dauerhafte CSA-Verordnung ("Chat-Kontrolle 2.0")

Jahr	Ereignis
Nov. 2023	Verhandlungsposition des Europaeischen Parlaments angenommen
2024	ML-KEM (vormals Kyber) wird als FIPS 203 offiziell durch NIST genormt
Herbst 2025	Oeffentliche Positionierung von Signal zu einem moeglichen Marktrueckzug bei verpflichtendem Client-Side-Scanning
laufend	Fortgesetzte Verhandlungen im Rat der EU; wiederholte Verlaengerung der Interim-Regelung (Teil IV, Kapitel 28.4)

41a. Ergänzende Zeitleisten-Einträge

Zur Vervollständigung der in Kapitel 42 gezeigten Übersicht einige zusätzliche, für den Gesamtkontext relevante Daten: **1975** — Whitfield Diffie und Martin Hellman veröffentlichen ihr grundlegendes Konzept des öffentlichen Schlüsselaustauschs (Kapitel 14.2), das die gesamte moderne asymmetrische Kryptographie begründet. **1983** — Das deutsche Bundesverfassungsgericht prägt im Volkszählungsurteil den Begriff der informationellen Selbstbestimmung (Kapitel 26.5a). **2008** — Das Bundesverfassungsgericht erkennt das IT-Grundrecht an (Kapitel 27.5a). **2012** — Europaweite Proteste führen zur Ablehnung von ACTA durch das Europäische Parlament (Kapitel 31.4c). **2016** — Formale akademische Sicherheitsanalyse des Signal-Protokolls durch Cohn-Gordon, Cremers u. a. (Kapitel 16.5a). **2020/2021** — Ermittlungserfolge gegen EncroChat und Sky ECC (Kapitel 25a.1a). **2023** — Inkrafttreten des Digital Markets Act und des Digital Services Act. Diese ergänzten Einträge zeigen, dass die in Teil IV beschriebene Debatte in ein deutlich dichteres Netz paralleler rechtlicher und technischer Entwicklungen eingebettet ist, als die Kernzeitleiste in Kapitel 42 allein abbilden kann.

42a. Die Bedeutung der Farben und Symbole in RenkerVaults Oberfläche

Auch wenn dieses Buch primär Architektur, Kryptographie und Recht behandelt, verdient ein kurzer Blick auf die Gestaltung der Oberfläche Erwähnung, da sie unmittelbar mit dem Sicherheitsmodell verknüpft ist. Die App verwendet ein dunkles Graphit/Anthrazit-Farbschema mit Smaragdgrün und Cyan als Akzentfarben — eine bewusste Abgrenzung von violett-dominierten Designs anderer, verwandter interner Projekte. Signalrot ist ausschließlich dem Alarmzustand vorbehalten (Teil II, Kapitel 10): Rot auf dem Bildschirm bedeutet in RenkerVault ausnahmslos „hier stimmt etwas nicht“, niemals eine dekorative oder neutrale Farbgebung. Diese Konsistenz ist kein rein ästhetisches Anliegen, sondern eine bewusste Sicherheitsmaßnahme: Nutzerinnen und Nutzer sollen sich darauf verlassen können, dass ein rotes Signal immer und ausschließlich einen tatsächlichen Sicherheitshinweis bedeutet, nie eine gewöhnliche

Statusanzeige — eine Lehre aus der Erkenntnis, dass inflationär eingesetzte Warnfarben in vielen Anwendungen dazu führen, dass Nutzer echte Warnungen mit der Zeit ignorieren lernen („Alert Fatigue“).

43. Quellen und weiterführende Literatur

Dieses Kapitel benennt die Art der Quellen, auf denen die Aussagen dieses Buches beruhen, damit Leserinnen und Leser sie eigenständig nachvollziehen und auf Aktualität prüfen können (siehe Disclaimer, Kapitel 26).

- **Offizielle EU-Dokumente:** Der Verordnungsvorschlag der Europäischen Kommission (2022) sowie die Verhandlungspositionen von Europäischem Parlament und Rat der EU sind über die offiziellen Portale der jeweiligen Institutionen einsehbar und stellen die verbindlichste verfügbare Quelle zum jeweils aktuellen Verhandlungsstand dar.
- **Rechtsprechung des EuGH:** Die Originalurteile (u. a. „Digital Rights Ireland“, „Tele2 Sverige/Watson“) sind über die Datenbank des Gerichtshofs der Europäischen Union frei zugänglich.
- **„Bugs in our Pockets: The Risks of Client-Side Scanning“ (2021):** Das in Teil IV, Kapitel 30.2 zitierte wissenschaftliche Papier ist als Preprint frei zugänglich und enthält eine ausführliche technische Analyse der Risiken von Client-Side-Scanning.
- **Stellungnahmen von EDPS/EDPB:** Die gemeinsame Stellungnahme zum Kommissionsvorschlag (Teil IV, Kapitel 31.1) ist über die Website des Europäischen Datenschutzbeauftragten abrufbar.
- **Zivilgesellschaftliche Analysen:** European Digital Rights (EDRi) und der Chaos Computer Club (CCC) veröffentlichen fortlaufend aktualisierte Analysen zum Verhandlungsstand, oft aktueller als gedruckte oder einmalig erstellte Quellen es sein können.
- **Kryptographische Standards:** RFC 7748 (X25519/Ed25519), RFC 5869 (HKDF), FIPS 203 (ML-KEM) sind über die jeweiligen Standardisierungsgremien (IETF, NIST) frei einsehbar und bilden die technische Referenz für Teil III dieses Buches.
- **Die Signal-Protokoll-Spezifikation:** Die von Signal veröffentlichte technische Dokumentation zu X3DH, Double Ratchet und PQXDH bildet die konzeptionelle Grundlage für Teil III, Kapitel 15/16/18.
- **Projektinterne Dokumentation:** [README.md](#) und [SECURITY.md](#) im RenkerVault-Projekt-Repository selbst, die laufend mit dem tatsächlichen Code-Stand synchron gehalten werden und im Zweifel als aktuellere Quelle als dieses Buch gelten sollten.
- **Akademische Protokollanalysen:** Die in Kapitel 16.5a zitierte formale Sicherheitsanalyse des Signal-Protokolls von Cohn-Gordon, Cremers u. a. ist als akademische Publikation über gängige wissenschaftliche Repositorien auffindbar.

- **NIST-Standardisierungsdokumente:** FIPS 203 (ML-KEM), FIPS 204 (ML-DSA) sowie die begleitenden Auswahlberichte des Post-Quantum-Kryptographie-Wettbewerbs (Kapitel 18.3a/18.3b) sind über die offizielle NIST-Website vollständig dokumentiert, einschließlich der Begründungen für die jeweiligen Auswahlentscheidungen.
- **Gerichtsentscheidungen zu EncroChat/Sky ECC:** Die in Kapitel 25a.1a beschriebenen Verfahren waren Gegenstand zahlreicher nationaler Gerichtsentscheidungen in mehreren EU-Mitgliedstaaten; eine vollständige rechtliche Aufarbeitung würde den Rahmen dieses Buches sprengen und wird in der einschlägigen strafprozessualen Fachliteratur behandelt.
- **Berichte über digitale Verschlüsselungsdebatten außerhalb der EU:** Fachbeiträge zu den in Kapitel 32.3a/32.3b genannten Rechtsordnungen (UK Online Safety Act, US EARN IT Act, indische IT Rules, brasilianische WhatsApp-Sperrungen) finden sich in der jeweiligen nationalen Rechts- und Digitalpolitik-Berichterstattung.

44. Anhang: Kommentierter Quellcode der Kernmodule

44.1 Zweck dieses Anhangs

Die vorangegangenen Kapitel haben jedes kryptographische Verfahren in RenkerVault formelhaft und in Worten beschrieben. Dieser Anhang geht einen Schritt weiter und zeigt den **tatsächlichen, unveränderten Quellcode** der zentralen Kryptographie-Module — damit jede Behauptung dieses Buches nicht nur nachvollzogen, sondern am Code selbst überprüft werden kann. Alle Ausschnitte stammen unverändert aus dem Projektverzeichnis `client/src/crypto/`.

44.1a Eine methodische Anmerkung zur Auswahl der gezeigten Ausschnitte

Die in diesem Anhang gezeigten Code-Ausschnitte wurden nach einem klaren Kriterium ausgewählt: Sie zeigen jeweils die Stelle im Code, an der eine zuvor in Prosa oder Formel beschriebene Sicherheitseigenschaft tatsächlich technisch umgesetzt wird — nicht beliebige, willkürlich herausgegriffene Zeilen. Wo ein Kapitel in Teil III eine bestimmte Formel zeigt (etwa die HKDF-Berechnung in Kapitel 15.2), zeigt der entsprechende Abschnitt in diesem Anhang die exakte Code-Zeile, die diese Formel implementiert, damit Leserinnen und Leser die Übereinstimmung zwischen Beschreibung und tatsächlichem Verhalten selbst nachvollziehen können. Diese Eins-zu-eins-Zuordnung zwischen Prosa-Beschreibung und Code-Beleg ist selbst Ausdruck des im Vorwort (Kapitel 1) formulierten Ehrlichkeitsprinzips: Eine Behauptung über Sicherheit, die sich nicht am tatsächlichen Code verifizieren lässt, bliebe eine bloße

Behauptung.

44.2 `primitives.ts` — die dünne Wrapper-Schicht

Wie in Teil III, Kapitel 13.3 beschrieben, implementiert dieses Modul keine eigene Kryptographie, sondern bindet ausschließlich auditierte Bibliotheken ein:

```
import { x25519, ed25519 } from '@noble/curves/ed25519';
import { hkdf } from '@noble/hashes/hkdf';
import { sha256 } from '@noble/hashes/sha256';
import { hmac } from '@noble/hashes/hmac';
import { argon2id } from 'hash-wasm';

export const rand = (n: number): Uint8Array =>
  crypto.getRandomValues(new Uint8Array(n));
```

Bereits diese ersten Zeilen belegen zwei zuvor beschriebene Design-Entscheidungen konkret: Zufallswerte werden ausschließlich über den browsereigenen CSPRNG erzeugt (Kapitel 19.2), und sämtliche Kurvenoperationen stammen aus `@noble/curves`, nicht aus einer selbst geschriebenen Implementierung.

Die Diffie-Hellman-Operation aus Teil III, Kapitel 14.2 findet sich unverändert wieder:

```
export const dh = (priv: Uint8Array, pub: Uint8Array): Uint8Array =>
  x25519.getSharedSecret(priv, pub);
```

Und die Argon2id-Parametrisierung aus Teil II, Kapitel 11.2:

```
export const ARGON2 = { iterations: 4, memorySizeKiB: 64 * 1024, parallelism: 1 };

export async function deriveKey(passphrase: string, salt: Uint8Array): Promise<Uint8Array> {
  return argon2id({
    password: passphrase, salt,
    iterations: ARGON2.iterations, memorySize: ARGON2.memorySizeKiB,
    parallelism: ARGON2.parallelism, hashLength: 32, outputType: 'binary',
  });
}
```

Ein letzter, sicherheitsrelevanter Ausschnitt: der Konstantzeit-Vergleich, der in Kapitel 11.3 (Duress-PIN) und Kapitel 21 (HMAC-Prüfung) verwendet wird, um sogenannte Timing-Angriffe zu verhindern — bei einem naiven Vergleich (`a === b`) bricht die Prüfung beim ersten abweichenden Byte ab, wodurch die benötigte Zeit indirekt verrät, wie viele Anfangsbytes bereits korrekt waren:

```
export function constEq(a: Uint8Array, b: Uint8Array): boolean {
  if (a.length !== b.length) return false;
  let d = 0;
  for (let i = 0; i < a.length; i++) d |= a[i] ^ b[i];
  return d === 0;
}
```

Diese Implementierung vergleicht stattdessen **immer** alle Bytes (per XOR-Verknüpfung und logischem Oder), unabhängig davon, wo eine Abweichung liegt — die benötigte Zeit ist dadurch unabhängig vom Inhalt der verglichenen Werte.

44.2a Die Base64-Kodierung: warum sie „chunked“ implementiert ist

Ein unscheinbares, aber praktisch relevantes Detail aus `primitives.ts` betrifft die Umwandlung binärer Schlüssel- und Chiffretextdaten in Base64-Text (nötig, da das in Kapitel 21.1 gezeigte Envelope-Format als JSON übertragen wird, welches nur Text, keine rohen Binärdaten transportieren kann):

```
export const b64 = {
  enc(u: Uint8Array): string {
    let s = '';
    const CH = 0x8000;
    for (let i = 0; i < u.length; i += CH) {
      s += String.fromCharCode(...u.subarray(i, i + CH));
    }
    return btoa(s);
  },
  dec(s: string): Uint8Array {
    return Uint8Array.from(atob(s), (c) => c.charCodeAt(0));
  },
};
```

Auffällig ist die Schleife in `enc()`, die die Eingabe in Blöcke von `0x8000` (32.768) Byte zerlegt, statt die gesamte Eingabe in einem einzigen Aufruf zu verarbeiten. Der Grund: Der JavaScript-Aufruf `String.fromCharCode(...u)` mit dem Spread-Operator übergibt jedes einzelne Byte als separates Funktionsargument — bei sehr großen Eingaben (etwa einem größeren Bildanhang, Kapitel 8.2) kann dies die maximale Anzahl an Funktionsargumenten überschreiten, die manche JavaScript-Engines zulassen, was zu einem Laufzeitfehler führen würde. Die „chunked“ Verarbeitung in überschaubaren Blöcken umgeht dieses Implementierungslimit zuverlässig, unabhängig von der tatsächlichen Größe der zu kodierenden Daten — ein praktisches Beispiel dafür, dass auch scheinbar triviale Hilfsfunktionen sorgfältige Randfall-Betrachtung erfordern, sobald sie mit variabel großen, potenziell umfangreichen Nutzdaten (Kapitel 44b: bis zu 2 MiB pro Envelope) umgehen müssen.

44.3 `pq.ts` — der Post-Quantum-Baustein

Der vollständige Inhalt dieses in Teil III, Kapitel 18 beschriebenen Moduls:

```
import { ml_kem768 } from '@noble/post-quantum/ml-kem.js';

export interface PqKeyPair {
  secretKey: Uint8Array;
  publicKey: Uint8Array;
}

export function newPqKeyPair(): PqKeyPair {
```

```

const kp = ml_kem768.keygen();
return { secretKey: kp.secretKey, publicKey: kp.publicKey };
}

export function pqEncapsulate(theirPublicKey: Uint8Array) {
  return ml_kem768.encapsulate(theirPublicKey);
}

export function pqDecapsulate(cipherText: Uint8Array, secretKey: Uint8Array): Uint8Array {
  return ml_kem768.decapsulate(cipherText, secretKey);
}

```

Bemerkenswert an diesem Modul ist seine Kürze: Die gesamte Post-Quantum-Erweiterung besteht aus vier kurzen Funktionen, die direkt an [@noble/post-quantum](#) delegieren — im Einklang mit dem Grundsatz aus Kapitel 13.3, keine eigene Kryptographie zu implementieren, auch nicht für ein so neues Verfahren wie ML-KEM.

44.4 [ratchet.ts](#) — Double Ratchet und X3DH-lite

Der Kern des Double-Ratchet-Zustands, wie in Teil III, Kapitel 16.2 beschrieben:

```

function kdfRk(rk: Uint8Array, dhOut: Uint8Array): [Uint8Array, Uint8Array] {
  const okm = hkdfSha256(dhOut, rk, 'RenkerVault-DoubleRatchet-RK', 64);
  return [okm.subarray(0, 32), okm.subarray(32, 64)];
}

function kdfCk(ck: Uint8Array): [Uint8Array, Uint8Array] {
  const next = hmacSha256(ck, new Uint8Array([2]));
  const mk = hmacSha256(ck, new Uint8Array([1]));
  return [next, mk];
}

```

Diese beiden Funktionen sind die direkte Code-Entsprechung der in Kapitel 16.2 gezeigten Formeln „KDF-RK“ und „KDF-CK“ — man erkennt, dass `kdfCk` aus einem Chain Key über zwei unterschiedliche HMAC-Berechnungen (mit den Konstanten `0x02` und `0x01`) sowohl den nächsten Chain Key als auch den aktuellen Message Key ableitet.

Die Verschlüsselung einer einzelnen Nachricht:

```

async encrypt(plaintext: Uint8Array): Promise<RatchetMessage> {
  if (!this.cks) throw new Error('Sendekette nicht initialisiert');
  const [next, mk] = kdfCk(this.cks);
  this.cks = next;
  const header: RatchetHeader = { dh: b64.enc(this.dhs.pub), pn: this.pn, n: this.ns };
  this.ns += 1;
  const aad = utf8.enc(JSON.stringify(header));
  const ct = await aesGcmEncrypt(mk, plaintext, aad);
  return { header, ct: b64.enc(ct) };
}

```

Bemerkenswert: Der Header (der öffentliche Ratchet-Schlüssel sowie Positionsangaben) wird als „Additional Authenticated Data“ (AAD) an die AES-GCM-Verschlüsselung übergeben (Teil III, Kapitel 13.1) — dadurch wird nicht nur der eigentliche Nachrichteninhalt, sondern auch der Header selbst gegen Manipulation abgesichert, ohne dass der Header dafür mitverschlüsselt werden müsste.

Der hybride Post-Quantum-Handshake aus Teil III, Kapitel 18.4, in seiner tatsächlichen Implementierung:

```
export function handshakeInitiator(
  myIdentity: KeyPair, theirIdentityPub: Uint8Array, theirPrekeyPub: Uint8Array,
  theirPqPrekeyPub: Uint8Array
): { sk: Uint8Array; ephPub: Uint8Array; pqCipherText: Uint8Array } {
  const eph = newX25519();
  const { cipherText: pqCipherText, sharedSecret: pqSecret } = pqEncapsulate(theirPqPrekeyPub);
  const sk = hkdfSha256(
    concat(dh(myIdentity.priv, theirIdentityPub), dh(eph.priv, theirPrekeyPub), pqSecret),
    new Uint8Array(32), 'RenkerVault-X3DH-lite-PQ-hybrid', 32
  );
  return { sk, ephPub: eph.pub, pqCipherText };
}
```

Diese eine Funktion enthält den gesamten in Kapitel 18.4 beschriebenen Hybrid-Mechanismus: Zwei klassische Diffie-Hellman-Werte (`dh(...)`, zweimal) werden mit dem Post-Quantum-Shared-Secret (`pqSecret`) zu einem einzigen `concat(...)`-Aufruf zusammengeführt und gemeinsam durch `hkdfSha256` geleitet — exakt die Formel aus Kapitel 18.4, jetzt als lauffähiger Code sichtbar.

44.5 vault.ts — At-Rest-Verschlüsselung und Duress-Modus

Die Entsperr-Logik, die in Teil II, Kapitel 10.5 und 11.3 beschrieben wurde, zeigt im Code deutlich die Reihenfolge der Prüfungen:

```
export async function unlockVault<T>(passphrase: string): Promise<UnlockResult<T>> {
  const file = readFile();
  if (!file) return { ok: false, reason: 'missing' };

  if (file.duress) {
    const dHash = await deriveKey(passphrase, b64.dec(file.duress.salt));
    if (constEq(dHash, b64.dec(file.duress.hash))) {
      return { ok: true, duress: true }; // Fake-Ansicht, echter Vault bleibt zu
    }
  }

  const kek = await deriveKey(passphrase, b64.dec(file.kdfSalt));
  let mk: Uint8Array;
  try {
    mk = await aesGcmDecrypt(kek, b64.dec(file.wrap));
  } catch {
    return { ok: false, reason: 'wrong-pass' };
  }
}
```

```

const ct = b64.dec(file.data);
const expected = hmacSha256(macKeyOf(mk), ct);
if (!constEq(expected, b64.dec(file.mac))) {
  return { ok: false, reason: 'tampered' };
}
// ... Entschlüsselung folgt erst NACH bestandener Integritätsprüfung
}

```

Zwei Details verdienen besondere Aufmerksamkeit: Erstens wird die Duress-PIN-Prüfung **vor** der regulären Passphrase-Prüfung durchgeführt — stimmt die eingegebene PIN mit dem separaten Duress-Hash überein, kehrt die Funktion sofort mit der Fake-Ansicht zurück, ohne den echten Master-Key auch nur zu berühren. Zweitens erfolgt die HMAC-Integritätsprüfung (Kapitel 10.4) **vor** dem eigentlichen Entschlüsseln der Daten — ein manipulierter Datenblob wird also erkannt, bevor überhaupt versucht wird, ihn zu interpretieren, was verhindert, dass eine Anwendung mit potenziell fehlerhaften, manipulierten Klartextdaten weiterarbeitet.

44.5a `net/client.ts` — das Envelope-Format als TypeScript-Interface

Das in Teil III, Kapitel 21.1 formelhaft beschriebene Envelope-Format ist im Code als vollständig typisiertes TypeScript-Interface festgehalten, was Tippfehler und inkonsistente Feldnamen bereits beim Kompilieren verhindert:

```

export interface Envelope {
  ct: string;
  chatId: string;
  chatKind: 'direct' | 'group';
  kind: 'text' | 'file' | 'group-key' | 'system' | 'edit' | 'delete' | 'reaction' | 'presence';
  msgId: string;
  ts: number;
  fromName: string;
  header?: { dh: string; pn: number; n: number }; // nur 1:1 (Ratchet)
  epoch?: number; // nur Gruppe (Epoch-Key)
  x3dh?: { ephPub: string; identityPub: string; pqCt: string };
}

```

Der Kommentar über der `kind`-Aufzählung im Original-Quellcode hält fest, was in Teil II, Kapitel 8.1 bereits in Worten beschrieben wurde: „`'edit' | 'delete' | 'reaction' | 'presence'` sind reine Anwendungs-Events, die wie normale Nachrichten Ende-zu-Ende-verschlüsselt über bestehende 1:1-/Gruppen-Sitzungen laufen (kein separater Server-Mechanismus nötig).“ Der Typ selbst macht sichtbar, dass `x3dh` als optionales Feld (`x3dh?`) modelliert ist — es ist nur bei der allerersten Nachricht einer neuen 1:1-Sitzung vorhanden, bei jeder weiteren Nachricht `undefined`, was der Beschreibung des Handshakes als „einmaligem“ Vorgang in Kapitel 15 entspricht.

Das `LookupResult`-Interface zeigt, welche Informationen ein Kontakt-Lookup (Kapitel 7.4) tatsächlich zurückliefert — nicht mehr als die für einen Erstkontakt-Handshake nötigen öffentlichen Schlüssel:

```
export interface LookupResult {
  userId: string;
  found: boolean;
  edPub: string | null;
  xPub: string | null;
  prekeyPub: string | null;
  pqPrekeyPub: string | null;
}
```

Auffällig: Es gibt in diesem Rückgabewert kein Feld für einen Anzeigenamen, ein Profilbild oder eine Kontaktliste — exakt im Einklang mit dem in Kapitel 7.4 beschriebenen, gezielten Lookup-Prinzip, das dem Relay keine Möglichkeit gibt, ein umfassendes soziales Netzwerk zu rekonstruieren.

44.6a `server/src/index.js` — der Relay in seiner eigenen Sprache

Auch der Relay-Server selbst benennt sein Sicherheitsmodell direkt im Quellcode, nicht nur in der Begleitdokumentation:

```
/**
 * RenkerVault Relay Server ("Zero-Knowledge"-Relay)
 * =====
 * SICHERHEITSMODELL:
 * - Dieser Server sieht und speichert NIEMALS Klartext von Nachrichten.
 * - Er kennt nur: Konto-IDs, öffentliche Schlüssel, Geräte-Metadaten
 *   und opaque, Ende-zu-Ende-verschlüsselte Envelopes (Chiffretext).
 * - Authentifizierung erfolgt passwortlos per Ed25519-Challenge-Response:
 *   der Server erhält nie eine Passphrase, nur Signaturen.
 */
```

Die konkreten Rate-Limiting- und Lockout-Parameter aus Teil II, Kapitel 10.2 finden sich als benannte Konstanten:

```
const FAIL_WINDOW_MS = 5 * 60 * 1000; // 5 Minuten
const FAIL_LIMIT = 5; // 5 Fehlversuche -> Lockout
const LOCKOUT_MS = 60 * 1000; // 60 Sekunden Lockout
const MAX_CONNS_PER_IP = 20; // Basisschutz gegen Verbindungsflut von einer IP
const AUTH_TIMEOUT_MS = 15 * 1000; // Unauthentifizierte Sockets nach 15s trennen
```

Bemerkenswert: Der In-Memory-Zustand (Teil III, Kapitel 21.3) ist explizit als solcher benannt und kommentiert — die Datenstruktur `users` speichert für jedes Konto ausschließlich eine Geräteliste und eine Warteschlange aus bereits verschlüsselten Envelopes, niemals einen Nachrichteninhalt:

```
/** userId -> { devices: Map<deviceId, DeviceRecord>, queue: Envelope[] } */
const users = new Map();
/** userId -> { fails: number[], lockedUntil: number } (Brute-Force-Erkennung) */
```

```
const authGuard = new Map();
```

Die Funktion, die eine Nachricht an alle Geräte eines Kontos weiterleitet, zeigt zugleich, wie wenig der Relay über den Inhalt entscheidet — sie prüft ausschließlich Authentifizierungsstatus und Zugehörigkeit, niemals den Inhalt der Nutzlast selbst:

```
function toUser(userId, msg, exceptWs = null) {
  const data = JSON.stringify(msg);
  for (const [ws, meta] of sockets) {
    if (meta.authed && meta.userId === userId && ws !== exceptWs && ws.readyState ===
1) {
      ws.send(data);
    }
  }
}
```

44.6b `safety.ts` — Safety Numbers im Code

Die in Teil II, Kapitel 7.5 beschriebene Safety Number wird über iteriertes Hashing berechnet:

```
export function safetyNumber(pubA: Uint8Array, pubB: Uint8Array): string {
  const [x, y] = [pubA, pubB].sort((a, b) => hex(a).localeCompare(hex(b)));
  let digest = concat(utf8.enc('RenkerVault-SafetyNumber-v1'), x, y);
  for (let i = 0; i < 512; i++) digest = sha256Bytes(digest);
  // ... aus dem Digest werden anschliessend 60 Ziffern extrahiert
}
```

Zwei Details sind hier bemerkenswert. Erstens werden die beiden öffentlichen Schlüssel vor der Berechnung **sortiert** (`[pubA, pubB].sort(...)`) — dadurch berechnen Anna und Ben, unabhängig davon, wer technisch als „erster“ oder „zweiter“ Teilnehmer gilt, garantiert dieselbe Safety Number, da die Reihenfolge der Eingabe keine Rolle mehr spielt. Zweitens wird der SHA-256-Hash 512-mal wiederholt angewendet (`for (let i = 0; i < 512; i++) digest = sha256Bytes(digest)`) — dies erschwert es einem Angreifer, der versuchen würde, gezielt Schlüsselpaare zu konstruieren, die eine bestimmte, unauffällig oder vertrauenswürdig aussehende Safety Number erzeugen (ein sogenannter Chosen-Prefix-Angriff), da jede einzelne Berechnung eines Kandidaten 512 Hash-Operationen statt nur einer erfordert.

44.6c Die `systemd-Sandbox-Härtung` im Detail

Die in Teil V, Kapitel 38.2 erwähnte `renkervault-relay.service`-Datei zeigt konkret, wie sich das Prinzip „so wenig Rechte wie möglich“ (das sogenannte „Least Privilege“-Prinzip) auf Betriebssystemebene durchsetzen lässt:

```
[Service]
User=renkervault
Group=renkervault
NoNewPrivileges=true
PrivateTmp=true
```

```

ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/opt/renkervault/server
ProtectKernelTunables=true
ProtectKernelModules=true
ProtectControlGroups=true
RestrictSUIDSGID=true
CapabilityBoundingSet=
AmbientCapabilities=
MemoryDenyWriteExecute=true

```

Jede dieser Zeilen schränkt den Relay-Prozess auf genau das ein, was er tatsächlich benötigt: `User=renkervault` sorgt dafür, dass der Prozess unter einem eigenen, unprivilegierten Systembenutzer läuft statt unter einem administrativen Konto; `ProtectSystem=strict` macht das gesamte Dateisystem außer dem explizit freigegebenen `ReadWritePaths`-Verzeichnis schreibgeschützt; die leeren Zuweisungen `CapabilityBoundingSet=` und `AmbientCapabilities=` entziehen dem Prozess sämtliche Linux-„Capabilities“ (feingranulare Admin-Rechte wie das Binden an privilegierte Netzwerk-Ports oder das Ändern von Dateibesitzern), die er für seine eigentliche Aufgabe — Envelopes zwischen WebSocket-Verbindungen weiterzuleiten — schlicht nicht braucht. Selbst wenn ein Angreifer eine bisher unbekannte Schwachstelle im Relay-Code fände und beliebigen Code im Kontext des Prozesses ausführen könnte, würde diese Sandbox-Konfiguration den erreichbaren Schaden auf das explizit freigegebene Arbeitsverzeichnis begrenzen — eine zusätzliche Verteidigungsschicht (im Sicherheitsjargon „Defense in Depth“ genannt), die unabhängig von der Korrektheit des Anwendungscodes selbst greift.

44.6d `ui/App.tsx` — die „Sitzung verbrennen“-Funktion

Als letztes Beispiel die in Teil II, Kapitel 8.7 beschriebene Burn-Funktion, die zeigt, wie die zuvor gezeigten Kryptographie- und Netzwerk-Bausteine in der Oberflächen-Schicht (Kapitel 6.1) zusammengeführt werden:

```

const handleBurnChat = (chatId: string) => {
  if (!data) return;
  const chat = data.chats.find((c) => c.id === chatId);
  if (!chat) return;
  const dropContact = chat.kind === 'direct' && chat.origin === 'real';
  if (dropContact) realChat.dropSession(chatId);
  setData((d) => {
    if (!d) return d;
    const messages = { ...d.messages };
    delete messages[chatId];
    const chats = d.chats.filter((c) => c.id !== chatId);
    if (!dropContact) return { ...d, chats, messages };
    const contacts = { ...d.contacts };
    delete contacts[chatId];
    const sessions = { ...d.sessions };
    delete sessions[chatId];
    return { ...d, chats, messages, contacts, sessions };
  });
}

```

```
});  
    setActiveChatId(null);  
};
```

Bemerkenswert: Die Funktion prüft zunächst, ob es sich um einen echten 1:1-Kontakt handelt (`chat.kind === 'direct' && chat.origin === 'real'`) — nur in diesem Fall wird zusätzlich `realChat.dropSession(chatId)` aufgerufen, die in Kapitel 44.4 gezeigte Methode aus `net/realchat.ts`, die den Ratchet-Zustand und den ausstehenden Handshake vollständig aus dem Arbeitsspeicher entfernt (Kapitel 8.7). Bei Demo-Kontakten oder Gruppen unterbleibt dieser Schritt bewusst, da dort kein echter, mit einer externen Gegenseite geteilter Sitzungszustand existiert, der „verbrannt“ werden müsste. Die anschließende Aktualisierung des lokalen Zustands (`setData`) entfernt Nachrichten, Chat-Eintrag, Kontakt und Sitzung in einem einzigen, atomaren Update — ein Zwischenzustand, in dem etwa Nachrichten bereits gelöscht, der Kontakt aber noch vorhanden wäre, kann dadurch nicht entstehen.

44.6e Typsicherheit als zusätzliches, sprachbasiertes Sicherheitsnetz

Ergänzend zu der in Kapitel 44.5a gezeigten `Envelope`-Typdefinition: Der dort verwendete TypeScript-Uniontyp bei `kind('text' | 'file' | 'group-key' | ...)` wird vom Compiler zur Übersetzungszeit erzwungen — ein `Envelope` mit einem nicht in dieser Liste enthaltenen `kind`-Wert würde den Build gar nicht erst durchlaufen. Diese Typsicherheit auf Sprachebene ist ein zusätzliches Sicherheitsnetz: Sie verhindert bereits während der Entwicklung, nicht erst zur Laufzeit, dass ein neuer Nachrichtentyp eingeführt wird, ohne an anderer Stelle im Code (etwa der in Kapitel 21.1 beschriebenen Verschlüsselungslogik oder der in Kapitel 8.2a gezeigten Tabelle) berücksichtigt zu werden. Dieselbe Absicherung greift bei den in Kapitel 44.5a gezeigten optionalen Feldern (`x3dh?`, `header?`, `epoch?`): Der Compiler erzwingt, dass jede Codestelle, die auf ein solches Feld zugreift, zuvor explizit prüft, ob es überhaupt vorhanden ist — ein Zugriff auf ein möglicherweise `undefined`-Feld ohne vorherige Prüfung wird bereits beim Kompilieren als Fehler markiert, nicht erst als Absturz zur Laufzeit sichtbar.

44.6 Warum dieser Anhang Vertrauen schaffen soll, nicht ersetzen

Dieser Code-Anhang ersetzt kein externes Sicherheitsaudit (Kapitel 16.5/25.2) — er verfolgt ein bescheidenes, aber wichtiges Ziel: Jede Leserin und jeder Leser mit Programmierkenntnissen kann selbst nachvollziehen, dass die in diesem Buch beschriebenen Verfahren tatsächlich so implementiert sind, wie sie beschrieben wurden, statt sich ausschließlich auf die Beschreibung selbst verlassen zu müssen. Transparenz auf Code-Ebene ist kein Ersatz für ein Audit, aber eine notwendige Voraussetzung dafür, dass ein Audit — oder eine informelle Prüfung durch die Leserschaft selbst — überhaupt möglich ist.

44a. Fragen zum Selbststudium

Dieses Kapitel eignet sich für Leserinnen und Leser, die ihr Verständnis der vorangegangenen Teile selbst überprüfen möchten — etwa im Rahmen einer Lerngruppe oder eines Selbststudiums im Bereich angewandte Informatik. Die Antworten finden sich jeweils in den angegebenen Kapiteln.

Zu Teil I:

- 1 Warum reicht es nicht aus, die Vertraulichkeit einer Nachricht ausschließlich durch einen gemeinsamen, unveränderlichen Schlüssel zu schützen? (Kapitel 5.7)
- 2 Was unterscheidet Client-Side-Scanning technisch von einer klassischen „Hintertür“ in einem Verschlüsselungsalgorithmus? (Kapitel 3.2, 4.4)
- 3 Nennen Sie zwei historische Episoden der „Crypto Wars“ und ihren jeweiligen Ausgang. (Kapitel 4)

Zu Teil II:

- 1 Warum verhindert die Dreiteilung des Clients in Kryptographie-, Netzwerk- und Oberflächen-Schicht bestimmte Fehlerklassen strukturell? (Kapitel 6.1)
- 2 Was passiert technisch, wenn ein bereits bekanntes Gerät sich plötzlich mit einem anderen kryptographischen Schlüssel meldet? (Kapitel 10.3)
- 3 Warum ist das Fehlen einer „Passwort vergessen“-Funktion eine bewusste Design-Entscheidung und kein Versehen? (Kapitel 37.4)

Zu Teil III:

- 1 Erklären Sie in eigenen Worten, warum zwei getrennte Diffie-Hellman-Berechnungen im X3DH-lite-Handshake mehr Sicherheit bieten als eine einzelne. (Kapitel 15.3)
- 2 Welche zwei Eigenschaften unterscheiden den Symmetric-Key-Ratchet vom Diffie-Hellman-Ratchet? (Kapitel 16.1-16.3)
- 3 Warum schützt der Hybrid-Handshake auch dann noch, wenn sich ML-KEM als fehlerhaft herausstellen sollte? (Kapitel 18.4)
- 4 Nennen Sie zwei Angreifer-Szenarien, gegen die RenkerVault explizit NICHT schützt, und begründen Sie warum. (Kapitel 23.3)

Zu Teil IV:

- 1 Was ist der Unterschied zwischen der „Chat-Kontrolle 1.0“-Interim-Regelung und dem Kommissionsvorschlag von 2022? (Kapitel 28.1-28.2)
- 2 Warum kann ein Zero-Knowledge-Relay ein DSGVO-Auskunftsersuchen über Nachrichteninhalte grundsätzlich nicht erfüllen? (Kapitel 33.2a)
- 3 Erklären Sie den Unterschied zwischen „Privacy by Policy“ und „Privacy by Architecture“. (Kapitel 33.1b)

- 4 Warum ist die Frage nach dem richtigen Mittel, nicht dem richtigen Ziel, der Kern der Chat-Kontrolle-Debatte? (Kapitel 25a.5)

Zu Teil V:

- 1 Welche drei Hosting-Wege stehen für einen eigenen Relay-Server zur Verfügung, und welche Vor- und Nachteile hat jeder? (Kapitel 38)
- 2 Warum sollte ein selbst betriebener Relay-Server keine über die Zustellung hinausgehenden Verbindungszeitpunkte protokollieren? (Kapitel 38.6)
- 3 Was ist der Unterschied zwischen dem Glossar (Kapitel 40) und dem Stichwortverzeichnis (Kapitel 46) dieses Buches?

Zusatzfragen zur Verknüpfung mehrerer Teile:

- 1 Erklären Sie, wie die in Kapitel 6.2a beschriebene „Dumb Pipe“-Analogie aus der klassischen Netzwerktechnik mit der in Kapitel 30.1a beschriebenen Unmöglichkeit serverseitigen Scannings zusammenhängt.
- 2 Welche Rolle spielt das Prinzip der geringsten Rechte (Kapitel 6.1a) sowohl in der Client-Architektur als auch in der Serverkonfiguration (Kapitel 44.6c)?
- 3 Vergleichen Sie die Begründung für AES-256 statt AES-128 (Kapitel 13.1a) mit der Begründung für den Post-Quantum-Hybrid-Handshake (Kapitel 18.4) — was ist die gemeinsame zugrundeliegende Vorsichtslogik?

44b. Tabellarischer Überblick aller kryptographischen Parameter

Eine konsolidierte Referenztabelle aller in diesem Buch genannten konkreten Parameter, zum schnellen Nachschlagen ohne erneutes Durchsuchen der einzelnen Kapitel:

Parameter	Wert	Fundstelle
X25519-Schlüssellaenge	32 Byte	Kapitel 14.2
Ed25519-Schlüssellaenge	32 Byte	Kapitel 14.3
AES-Schlüssellaenge	256 Bit (32 Byte)	Kapitel 13.1, 13.1a
AES-GCM-Nonce-Laenge	12 Byte	crypto/primitives.ts, Kapitel 44.2
Argon2id-Iterationen	4	Kapitel 11.2, 44.2
Argon2id-Speicherbedarf	64 MiB	Kapitel 11.2, 11.2a
Argon2id-Parallelitaet	1	Kapitel 44.2
Argon2id-Ausgabelaenge	32 Byte	crypto/primitives.ts
HKDF-Ausgabelaenge (Root+Chain)	64 Byte (2x32)	Kapitel 16.2
Safety-Number-Laenge	60 Ziffern	Kapitel 7.5

Parameter	Wert	Fundstelle
Safety-Number-Iterationen	512 SHA-256-Runden	Kapitel 44.6b
ML-KEM-768 Secret-Key-Groesse	2400 Byte	Kapitel 44.3
ML-KEM-768 Public-Key-Groesse	1184 Byte	Kapitel 44.3
Max. uebersprungene Message-Keys	64	Kapitel 16.4
Relay: Fehlversuche bis Lockout	5	Kapitel 10.2
Relay: Lockout-Dauer	60 Sekunden	Kapitel 10.2
Relay: Fehlversuchs-Zeitfenster	5 Minuten	server/src/index.js
Relay: max. Verbindungen pro IP	20	Kapitel 10.2, 44.6a
Relay: Auth-Timeout	15 Sekunden	Kapitel 44.6a
Relay: max. Envelope-Groesse	2 MiB	server/src/index.js
Relay: max. Offline-Warteschlange	500 Envelopes	server/src/index.js
Gruppenschluessel-Laenge	256 Bit (32 Byte)	Kapitel 17.2

45. Abkürzungsverzeichnis

Abkuerzung	Ausgeschrieben
AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Standard
CCC	Chaos Computer Club
CSA-Verordnung	Verordnung zur Bekaempfung von Child Sexual Abuse (Chat-Kontrolle)
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator
DSA	Digital Services Act
DSGVO	Datenschutz-Grundverordnung
E2E	Ende-zu-Ende(-Verschluesselung)
EDPB	European Data Protection Board (Europaeischer Datenschutzausschuss)
EDPS	European Data Protection Supervisor (Europaeischer Datenschutzbeauftragter)
EDRi	European Digital Rights
EGMR	Europaeischer Gerichtshof fuer Menschenrechte
EMRK	Europaeische Menschenrechtskonvention
EPRS	European Parliamentary Research Service
EuGH	Europaeischer Gerichtshof

Abkuerzung	Ausgeschrieben
FIPS	Federal Information Processing Standards (NIST-Normenreihe)
GRCh	Grundrechtecharta der Europaeischen Union
HKDF	HMAC-based Key Derivation Function
HMAC	Keyed-Hash Message Authentication Code
HNDL	Harvest Now, Decrypt Later
MLS	Messaging Layer Security (RFC 9420)
ML-KEM	Module-Lattice-based Key-Encapsulation Mechanism
NIST	National Institute of Standards and Technology (USA)
OTP	One-Time-Pad
PFS	Perfect Forward Secrecy
PQ	Post-Quantum
PQXDH	Post-Quantum Extended Triple Diffie-Hellman (Signal-Bezeichnung)
QRNG	Quantum Random Number Generator
TOLA	Telecommunications and Other Legislation Amendment Act (Australien)
VPS	Virtual Private Server
X3DH	Extended Triple Diffie-Hellman

45a. Deutsch-Englisches Fachbegriff-Verzeichnis

Da die meisten zugrundeliegenden Standards, RFCs und Fachpapiere (Kapitel 43) auf Englisch verfasst sind, hilft diese Gegenüberstellung beim Übergang zur Originalliteratur:

Deutsch	Englisch
Ende-zu-Ende-Verschlüsselung	End-to-end encryption
Schlüsselaustausch	Key exchange / key agreement
Öffentlicher/privater Schlüssel	Public/private key
Vorwärtsgerichtete Geheimhaltung	Forward secrecy
Nach-Kompromittierungs-Sicherheit	Post-compromise security
Sitzung	Session
Sitzungsschlüssel	Session key
Schlüsselableitung	Key derivation
Zufallswert	Nonce / random value
Authentifizierte Verschlüsselung	Authenticated encryption
Zusatzdaten (bei AEAD)	Associated data
Manipulationserkennung	Tamper detection
Einbruchserkennung	Intrusion detection
Zwangslage/Notfall-Modus	Duress mode
Sicherheitslücke	Vulnerability
Bedrohungsmodell	Threat model
Angriffsfläche	Attack surface
Lieferkette (Software)	Supply chain
Quantencomputer-resistent	Quantum-resistant / post-quantum
Datenminimierung	Data minimization
Zweckbindung	Purpose limitation
Rechenschaftspflicht	Accountability
Verhältnismässigkeit	Proportionality
Erforderlichkeit	Necessity
Aufdeckungsanordnung	Detection order
Vorratsdatenspeicherung	Data retention
Zivilgesellschaft	Civil society
Schlüsselkapselung	Key encapsulation
Geteiltes Geheimnis	Shared secret

Deutsch	Englisch
Umschlag-Verschlüsselung	Envelope encryption
Prinzip der geringsten Rechte	Principle of least privilege
Abschreckende Wirkung	Chilling effect
Zweckerweiterung	Function creep / scope creep
Seitenkanalangriff	Side-channel attack
Zeitkonstante Implementierung	Constant-time implementation
Gitterbasierte Kryptographie	Lattice-based cryptography
Perfekter Vorwaertsschutz	Perfect forward secrecy
Rueckverfolgbarkeit	Traceability
Datenschutz-Folgenabschaetzung	Data protection impact assessment
Ordentliches Gesetzgebungsverfahren	Ordinary legislative procedure

46. Stichwortverzeichnis

Ein thematischer Schnellzugriff über die im Glossar (Kapitel 40) bereits definierten Begriffe hinaus, sortiert nach Themenfeldern statt alphabetisch nach Fachbegriff:

- **Alarmsystem:** Kapitel 10 (Übersicht), 10.2 (Brute-Force), 10.3 (Geräte), 10.4 (Manipulation), 10.5 (Duress), 10.6b (IDS-Analogie)
- **Architektur allgemein:** Kapitel 6, 6.1 (Client-Schichten), 6.2 (Relay), 22.4b (Architekturvergleich)
- **Chat-Kontrolle (Geschichte/Inhalt):** Kapitel 28, 28.0 (Verfahren), 28.1-28.4 (Historie), 29 (Verordnungsentwurf)
- **DSGVO:** Kapitel 33, 33.1a (Grundsätze), 33.2a (Betroffenenrechte), 33.2b (Meldepflichten)
- **Gruppen und Kanäle:** Kapitel 9, 9.4a (Rechtevergleich), 17 (Kryptographie)
- **Hosting:** Kapitel 38, 38.4 (Tor), 38.4a (Rechtsraum)
- **Kryptographie, Grundlagen:** Kapitel 5 (Laien), 13-14 (Detail)
- **Post-Quantum:** Kapitel 18, 18.6a (Agilität), 18.6c (Signaturen)
- **Rechtsprechung (EuGH/EGMR):** Kapitel 27.3, 27.3a, 27.4
- **Sitzung verbrennen:** Kapitel 8.7, 44.6d (Code)
- **Threat Model:** Kapitel 23, 23.3a (Beispiele)
- **Vergleich mit anderen Messengern:** Kapitel 24

Kolophon: Über dieses Buch

Dieses Buch begleitet den RenkerVault-Quellcode als lebendiges Dokument, nicht als einmalig abgeschlossenes Werk — es folgt demselben Prinzip wie SECURITY.md (Kapitel 43): laufend an den tatsächlichen Stand von Code, Rechtslage und Bedrohungsbild anzupassen, statt ein einmal geschriebenes Versprechen unverändert stehen zu lassen. Die Zweiteilung in „Für Laien“ und „Für Informatiker“, die bereits die kürzere Vorgänger-Erklärung dieses Projekts prägte, wurde hier auf einen dritten Aspekt erweitert: „Für alle, die verstehen wollen, warum dieses Werkzeug in genau dieser Form entstanden ist“ — der rechtliche Teil (Teil IV), der Technik und Motivation zusammenführt, statt sie als getrennte Themen zu behandeln.

Jede technische Behauptung in diesem Buch verweist, wo immer möglich, auf den tatsächlichen Quellcode (Kapitel 44) statt auf eine bloße Beschreibung aus der Erinnerung — im Zweifel gilt: Code vor Beschreibung, Beschreibung vor Meinung.

Nachwort

Dieses Buch hat versucht, drei üblicherweise getrennte Perspektiven — Technik, Sicherheitsbegründung und rechtlicher Kontext — in einem gemeinsamen, durchgehenden Zusammenhang darzustellen. Wenn eine einzige Erkenntnis aus allen drei Teilen mitgenommen werden sollte, dann diese: Sicherheit ist kein einmaliger Zustand, den ein Produkt „hat“ oder „nicht hat“, sondern das Ergebnis einer fortlaufenden Kette einzeln nachvollziehbarer, begründeter Entscheidungen — und die Bereitschaft, jede dieser Entscheidungen samt ihrer Grenzen offenzulegen, ist selbst ein Teil dieser Sicherheit, nicht ein Widerspruch dazu.

Ebenso gilt: Die rechtliche und politische Debatte, die in Teil IV beschrieben wurde, ist zum Zeitpunkt der Lektüre dieses Buches möglicherweise bereits weitergegangen. Technik kann eine politische Debatte nicht ersetzen — sie kann nur eine der möglichen Antworten auf sie sein.

Wer bis hierher gelesen hat, kennt nun nicht nur die einzelnen Bausteine von RenkerVault, sondern auch die Begründung für jeden davon — von der Wahl einer bestimmten elliptischen Kurve (Kapitel 14.2a) bis zur Entscheidung, keine automatische Link-Vorschau zu laden (Kapitel 8.4), von der Herleitung des Post-Quantum-Hybrid-Handshakes (Kapitel 18.4) bis zur Frage, welche Generaldirektion einen Gesetzesvorschlag ursprünglich vorbereitet hat (Kapitel 28.2a). Diese Dichte an Begründungen war eine bewusste Entscheidung: Ein Werkzeug, das seine eigenen Design-Entscheidungen nicht erklären kann, verdient nach dem im Vorwort formulierten Maßstab (Kapitel 1) kein blindes Vertrauen — und ein Werkzeug, das sie erklären kann, sollte es auch tun, selbst wenn das bedeutet, ein technisches Handbuch auf die Länge eines vollständigen Buches anwachsen zu lassen.